Handwritten Notes To Text Converter

Shiven Kothari

Contents

1. Analysis **4**
1.1 Problem Definition 4
1.2 Stakeholders 4
1.2.1 Stakeholder 1: Students 4
1.2.2 Stakeholder 2: Teachers 5
1.3 Research 6
1.3.1 Competitor 1: Adobe 6
1.3.2 Competitor 2: Pen to Print 7
1.3.3 Stakeholder Survey 12
1.4 Proposed Solution 16
1.4.1 Outline of solution 16
1.4.2 Requirements 16
1.4.3 Hardware and software requirements 21
1.4.4 Limitations 22
1.4.5 Computational Methods 23
1.4.6 Success Criteria 24
2. Design **26**
2.1 Decomposition of the Problem 26
2.2 Solution Structure 27
2.3 Algorithms and Implementation 28
2.4 Usability Features 30
2.5 Variables, Data Structures, and Classes 33
2.6 Testing 34
3. Cycle 1 **36**
3.1.1 Complete Overview 36
3.1.2 Cycle 1 Overview 37
3.2 Success Criteria 37
3.3 Iterative Development Process 38
3.3.1 Home Page 39
3.3.2 Input Page 47
3.3.3 Output Page 67
3.4 Code Annotation and Naming Conventions 82
3.5 Evaluation 82
3.5.1 Changes to the project 82
3.5.2 Features to Re-iterate in Future Cycles 83
4. Cycle 2 **83**
4.1 Cycle 2 overview 83
4.2 Success Criteria 84
4.3 Iterative Development 84
4.3.1 Backend 86
4.3.2. Integration 105
4.4 Code annotation and naming conventions 112

4.5 Evaluation 113
4.5.1 Feedback 113
4.5.2 Features to Re-iterate in future cycles 114
5. Cycle 3 **115**
5.1.1 Cycle overview 115
5.2 Success criteria 115
5.3 Iterative Development 116
5.3.1 Additional features 116
5.3.2 Bug fixes 130
5.4 Code annotation and naming conventions 153
5.5 Evaluation 153
6. Evaluation **154**
6.1 Post-Development Testing 154
6.2 Stakeholder Response 160
6.3 Success Criteria Evaluation 168
6.4 Final Product Description 172
6.5 Maintenance 173
6.6 Future Developments 173
6.7 Limitations 175

1. Analysis

1.1 Problem Definition

A major problem is that students make so many handwritten notes in their exercise books. That after a while they have so many books and paper records of their school work, that it becomes unmanageable to organise or sift through their books for revision. Especially if they want to revise for a specific topic but forgot a certain quote or formula; they would have to sift through a multitude of books that can be completely unrelated to what the student wanted in the first place (e.g. having multiple books for the same subject). And then, if they want to share their work with others it can get hard to take a picture of your work without good enough lighting and perfect clarity, and this is assuming they can even read your handwriting - piling on the inconveniences of handwritten paper records on students.

Additionally, another problem that arises with handwritten work is that sometimes teachers can't read a student's essay/written piece of work; this is also applicable to students themselves who need to read old notes but can't understand or remember what they wrote. This becomes especially problematic with STEM subjects where every year they build on top of what occurred in the previous year, so the notes you made in your exercise book in the previous years are normally very important. Which is exponentially amplified in language subjects, where most of the time you use your previous notebooks as compendiums of all the phrases, grammatical structures and adjectives you've learnt. Furthermore this only further emphasises the struggles of needing to store your written work for future reference.

Therefore, as a solution I will create a free and easy to use application which uses a computer model. Where you can simply feed it a picture of all of your work and digitise all of your school notes. This makes it easier to store, share and sort your notes. I will use a convolutional neural network that will follow OCR principles. Furthermore, I will especially focus on the character recognition of cursive handwriting. Additionally when the letters can be oddly sized in random patterns, in crooked - instead of straight - lines, and just too jumbled or squashed up together. So the convolutional neural network can read ‘bad handwriting’.

1.2 Stakeholders

<u>Description of end user</u>

Overall the primary end user will be students, and the product will strive to satisfy their needs the most. However this does not limit the inclusion of secondary and tertiary consumers who can also benefit from digitising handwritten notes. Especially due to many people still writing notes and such on paper.

1.2.1 Stakeholder 1: Students

Students need an application that is flexible, convenient and user friendly; such as within the Sixth Form/College space, students are swamped with a multitude of lessons, extracurricular activities, homework and independent learning with not a lot of time. Therefore any application that can help them save time by improving the quality of their notes would be beneficial for them. In terms of requirements for the application that users would use, they would need: an easy medium to attach photos of their handwritten work. Therefore, the

application will be a website instead of a desktop application, so it is more easily accessible from a phone and can use its photo library. Which means the user doesn't need to also export the image to the desktops as most students would use their phones to take a picture of their work. Finally, if it is a phone app then there may be conflicts with the different operating systems, getting developer accounts and the ability you have when publishing - for example you must pay Apple a fee to be able to include apps in their app store. Hence, a website would be the most accessible and easy to implement medium for the user.
Next, more often than not when writing notes you're not always writing down essays. Sometimes students need to use tables, bullet points or even page dividers. So the system should aim to be able to recognise and preserve the layout of the notes it scans, with a reasonable error rate; so picking a good pre-trained convolutional neural network is key. To decrease the error rate further and reduce bias I can add my own data in the training/sample set. This can be further extended by including mathematical symbols and such in the training data, however the feasibility of this in the time constraints of the project needs to be evaluated. Finally, students would want a way to correct any errors they see in the translation before exportation, and have the exporting process be as convenient as possible. Thus the best move would be to add a correction UI, where you can have the user edit the text before exportation in a text area. Then let them choose whether to copy the text or export it as a pdf file.
Overall this lets students tackle their day to day challenges by indulging in an application that helps to digitise their handwritten notes, after doing a quick verbal survey with sixth form students many commented on their struggles with handling handwritten notes. Some responded with how they can't read their own handwriting after writing too quickly, or how for organisational purposes they would prefer to use digital notes - especially when multiple books have been compiled over the years and students need to refer back to previous notes.

1.2.2 Stakeholder 2: Teachers

Teachers can mainly use this application to read the essays of students with illegible handwriting, where with a quick and easy scan they can digitise the entire paper. This would aid with efficiency, as teachers would spend less energy trying to figure out what students are actually trying to write and more on actually how good their essay is. This can be improved by adding our own examples of student handwritten notes in the training data to help improve accuracy and remove bias. This addresses the challenges many teachers experience, I have been told from them, that generally ‘difficult to read handwriting takes up our time’ and how as a profession their time is sparse. Therefore any help that can speed up the most time consuming aspect of teaching can be extremely beneficial; a direct quote from my teacher further emphasises this: "Marking students with bad handwriting is a pain, I spend almost triple the time marking their tests compared to other students" - Miss Marchant. Therefore my application can be extremely beneficial for teachers by helping them handle students with bad handwriting (whether overall or simply during the pressure of exams).

1.22 Stakeholder 3: General Use

Finally, this handwritten notes to text converter can be used just for general use by a wide array of people, from various professions. Such as by digitising drafts written by authors or poets for easier editing, as well as digitising todo lists, action items or minutes from corporate meetings. On the other hand, it can shine in a non-professional environment as

well, to digitalise recipes, journal entries or remainders; as most often than not, writing on physical paper is always the most popular for daily life. Therefore, it can be used for just general note takers as well. For example in the profession of journalism, many journalists would quickly write down key information when in the field; however when they are back at home analysing and researching it would be heavily beneficial for all of their work to be easily organised digitally and grouped together - therefore this application would be a great benefit for them.

1.3 Research

Initial Investigation

Currently most students prefer to take notes on tablets that can store your notes digitally from the get go via tablets designed to feel like paper such as reMarkable - or simply through note apps on their regular tablets. They can also add matte screen protectors to their normal tablets to not damage the screen and to add a more natural feel when writing. However this can be extremely expensive especially if students don't already have a digital pen or tablet; as well as the fact that this would only work for those outside of lessons or those in Universities, as most schools wouldn't allow using tablets to notetake in lessons. Therefore alternatively there are quite a few options to convert handwritten notes into digital ones, though the competent examples are mostly paid options, such as Pen to Print and Adobe scan which both use OCR (Optional Character Recognition).

1.3.1 Competitor 1: Adobe

Adobe specifies that its OCR software works best when: line paper is used to keep the handwritten lines straight, it's written in capital letters to help with symbol differentiation, written slowly so it's kept as uniform as possible and finally for it to not be smudged or on crumpled paper. Therefore with a lot of these caveats a long passage of handwritten words from the classroom can lead to many inaccuracies, for example from students that would probably rush their notes to keep up in class - especially if a student has writing that's too big or in cursive, which can lead it to be quite illegible for others.

However, the main problem with using Adobe to convert handwritten records into digital text is how unclear it is to do so, and how so many steps are blocked behind a paywall.

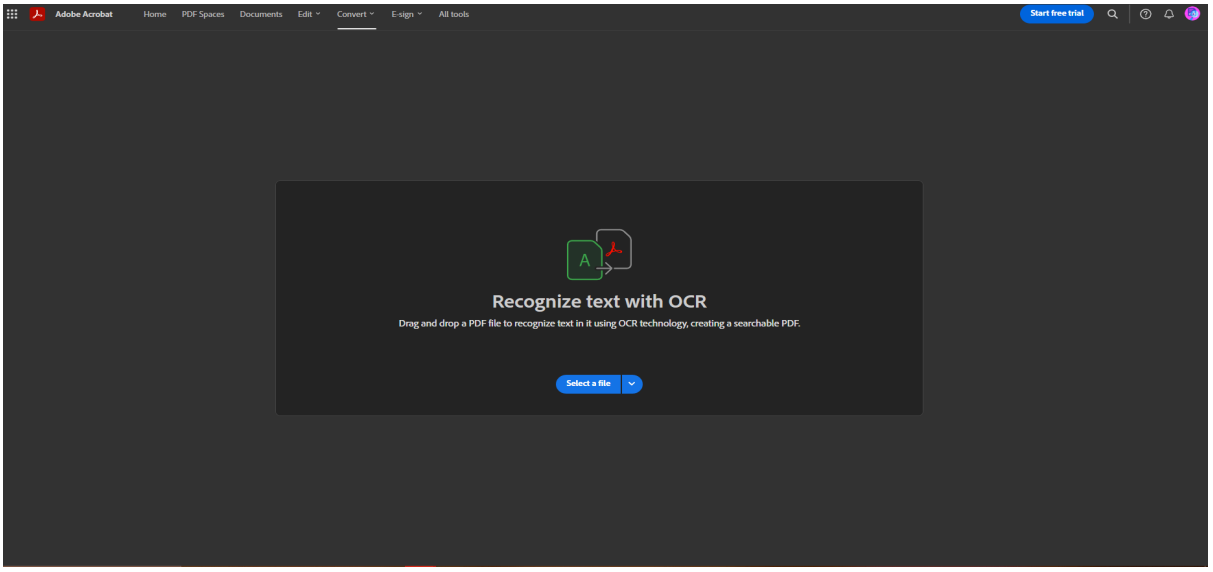


Figure: 1.3.1.1 - Adobe OCR to a PDF homescreen

Firstly, the main way to convert an image into a text is using the online ocr to pdf format shown in Figure 1.3.1.1, however this is paid and blocked behind a subscription. Furthermore, another way is to use Adobe Acrobat and use the tools options to use basic OCR; however this is mainly for digital text that has been printed and less for handwritten notes. Moreover, to actually use this feature you need to have a pdf of the image - meaning you can either pay to export a scanned image as a pdf from the Adobe Scan app or use an online pdf converter for free. Finally, the last way to use their OCR technology is to directly convert scans made in Adobe Scans, but this is also locked behind a paywall.
This continuous pile of inconveniences as well as the lack of clarity from the start on how to work these tools can lead to users quitting from the start. Especially with the ruthless use of paywalls and freemium pop ups can create a consumer unfriendly experience. On top of the fact that their technology isn't much on par compared to later examples in terms of recognising handwriting (especially in cursive) exudes to how much of an update this segment of Adobe needs.

Conclusion:

This highlights the need to create an application that is simple and concise, where users can easily follow along with how to get the end result. Consequentially emphasising the importance of making free software, where unlike in Adobe, users can access a consistent application that doesn't constantly bombard them on how they should subscribe to the website for additional features. Removing ads and paywalls to create a true user friendly atmosphere.

1.3.2 Competitor 2: Pen to Print

Pen to Print advertises and boasts of actively being able to translate handwritten cursive notes that some would deem illegible.

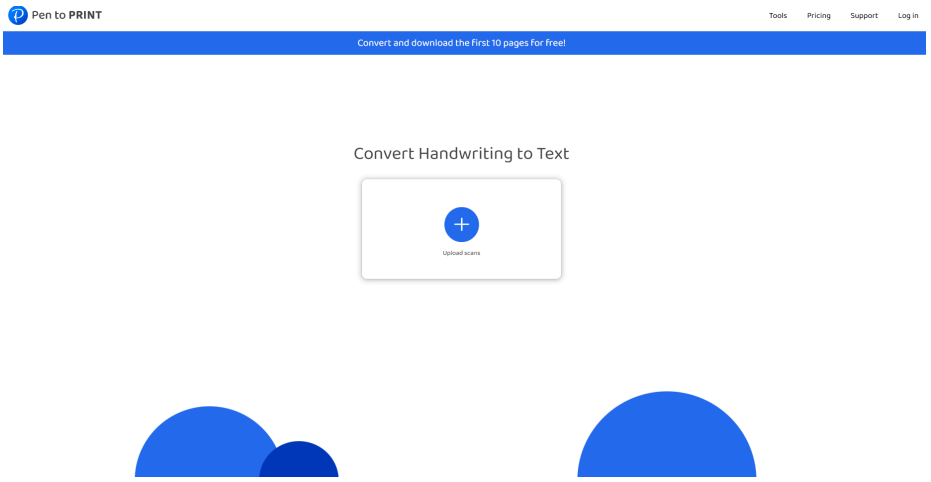


Figure: 1.3.2.1 - home screen

The home screen seen in Figure 1.3.2.1, is simplistic and highlights the main UI that made the user click on the website in the first place. Here they can drag and drop in files from their folders if on desktop or photo library if on phone. Highlighting its flexibility as most people would use their phones only to scan handwritten documents.

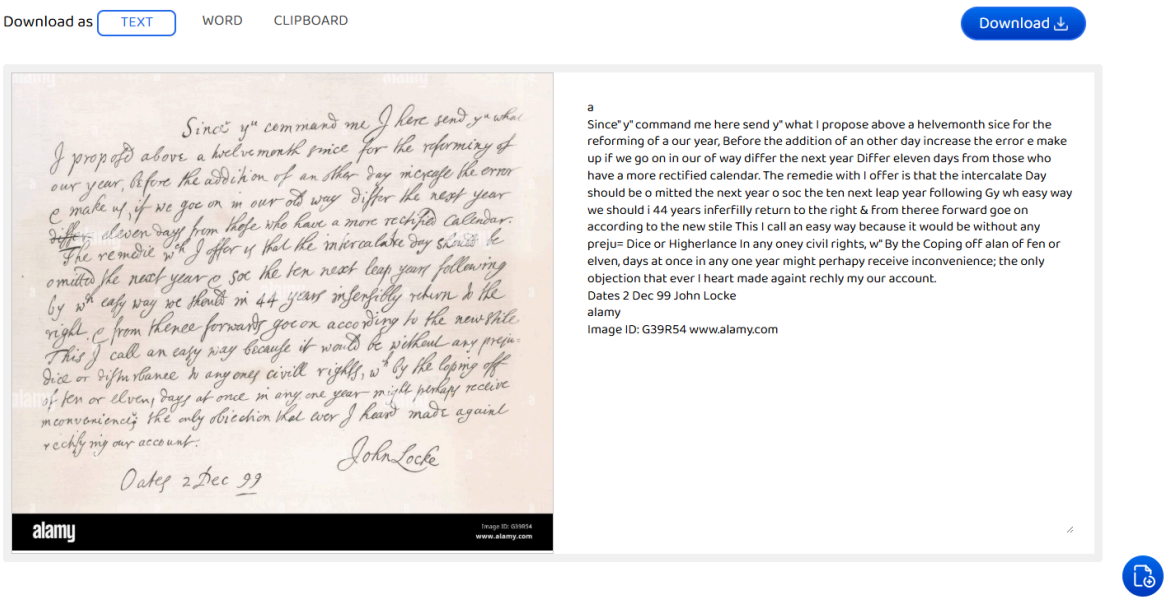
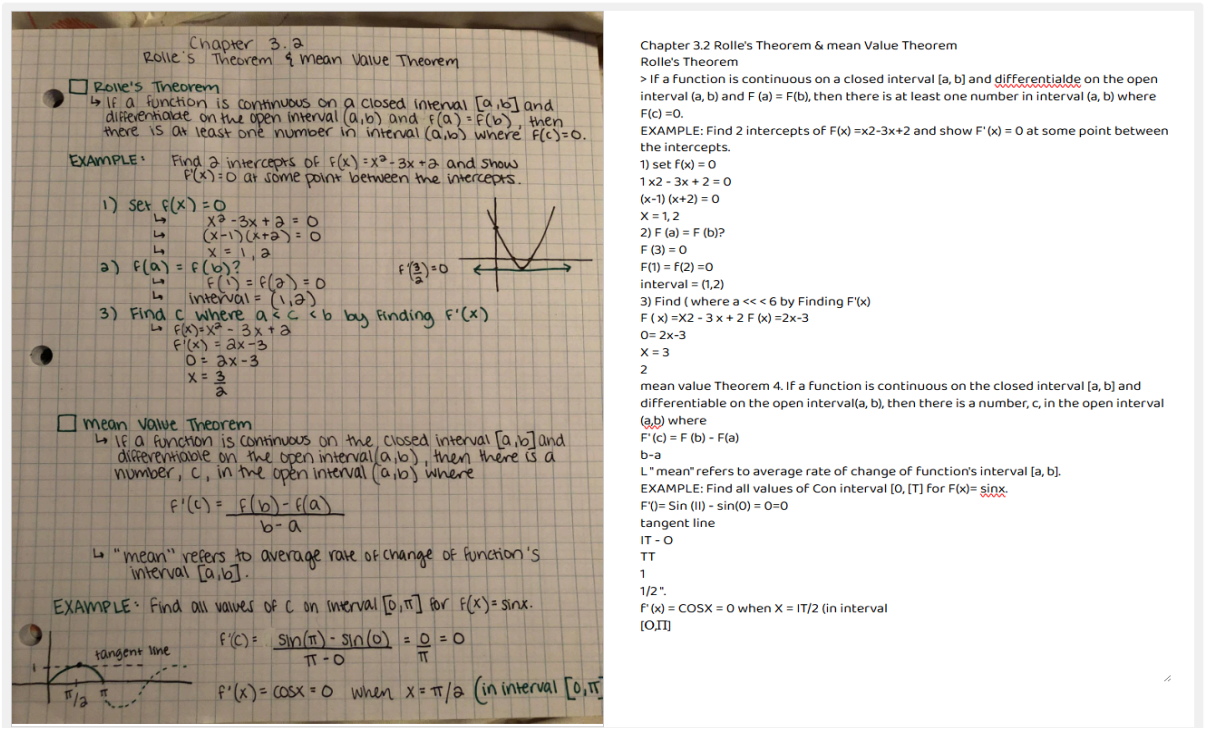


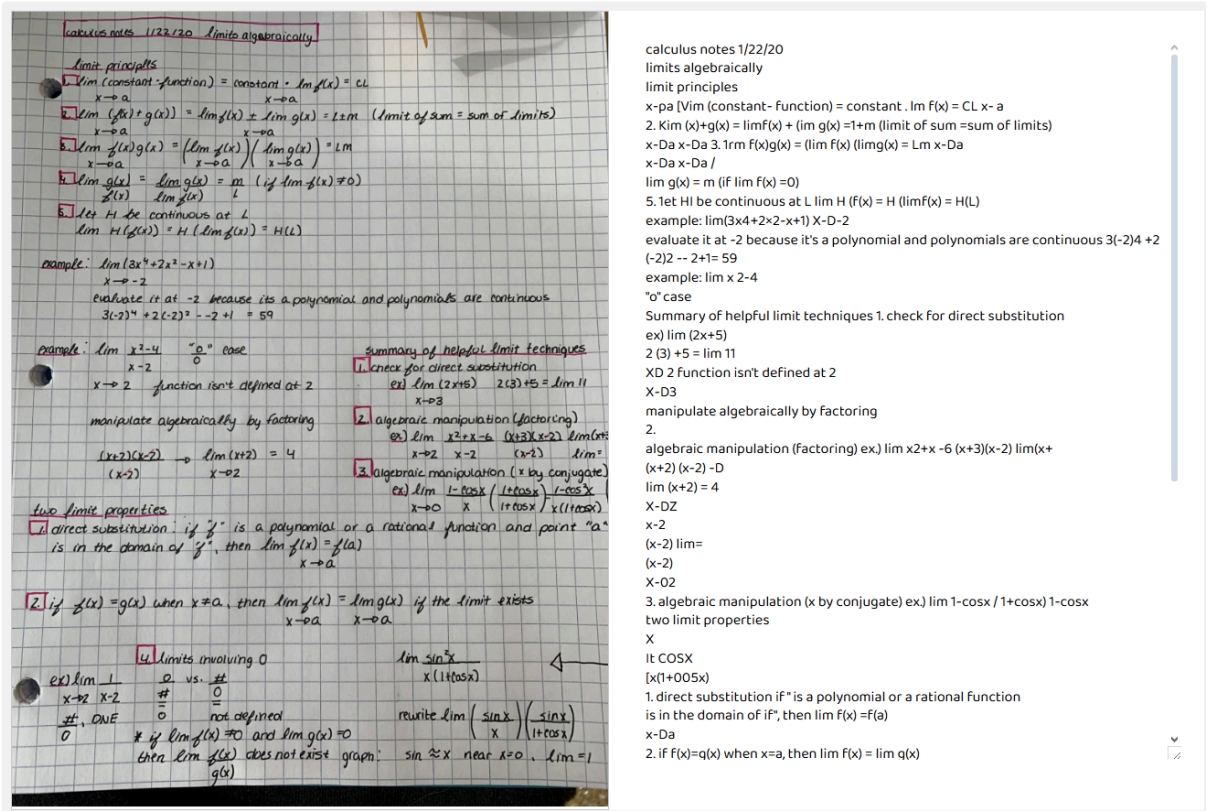
Figure: 1.3.2.2 - post processing UI

Furthermore, after processing the given image the user interface (seen in Figure 1.3.2.2) retains its simplistic design by only showing the image and the output text. Which is given in an editable text area before exportation, which can be in the form of copying the text onto the clipboard or downloading it as a word or text document.
Its design is straightforward with no unnecessary animations or noise.



Chapter 3.2 Rolle's Theorem & mean Value Theorem
Rolle's Theorem
> If a function is continuous on a closed interval [a, b] and differentialde on the open interval (a, b) and F (a) = F(b), then there is at least one number in interval (a, b) where F(c) =0.
EXAMPLE: Find 2 intercepts of F(x) =x2-3x+2 and show F'(x) = 0 at some point between the intercepts.
1) set f(x) = 0
1 x2 - 3x + 2 = 0
(x-1) (x+2) = 0
X = 1, 2
2) F (a) = F (b)?
F (3) = 0
F(1) = F(2) =0
interval = (1,2)
3) Find (where a << < 6 by Finding F'(x)
F (x) =X2 - 3 x + 2 F (x) =2x-3
0= 2x-3
X = 3
2
mean value Theorem 4. IF a function is continuous on the closed interval [a, b] and differentiable on the open interval(a, b), then there is a number, c, in the open interval (a,b) where
F'(c) = F (b) - F(a)
b-a
L " mean" refers to average rate of change of function's interval [a, b].
EXAMPLE: Find all values of Con interval [0, [T] for F(x)= sinx.
F'()= Sin (II) - sin(0) = 0=0
tangent line
IT - O
TT
1
1/2 ".
F' (x) = COSX = 0 when X = IT/2 (in interval
[O,IT]

Figure: 1.3.2.3 - Example of maths work in a linear format



calculus notes 1/22/20
limits algebraically
limit principles
x-pa [Vim (constant- function) = constant . Im f(x) = CL x- a
2. Kim (x)+g(x) = limf(x) + (im g(x) =1+m (limit of sum =sum of limits)
x-Da x-Da 3. 1rm f(x)g(x) = (lim f(x) (limg(x) = Lm x-Da
x-Da x-Da /
lim g(x) = m (iF lim f(x) =0)
5. 1et HI be continuous at L lim H (F(x) = H (limf(x) = H(L)
example: lim(3x4+2×2-x+1) X-D-2
evaluate it at -2 because it's a polynomial and polynomials are continuous 3(-2)4 +2 (-2)2 -- 2+1= 59
example: lim x 2-4
"o" case
Summary of helpful limit techniques 1. check for direct substitution
ex) lim (2x+5)
2 (3) +5 = lim 11
XD 2 function isn't defined at 2
X-D3
manipulate algebraically by factoring
2.
algebraic manipulation (factoring) ex.) lim x2+x -6 (x+3)(x-2) lim(x+
(x+2) (x-2) -D
lim (x+2) = 4
X-DZ
x-2
(x-2) lim=
(x-2)
X-02
3. algebraic manipulation (x by conjugate) ex.) lim 1-cosx / 1+cosx) 1-cosx
two limit properties
X
It COSX
[x(1+005x)
1. direct substitution if " is a polynomial or a rational function
is in the domain of if", then lim f(x) =f(a)
x-Da
2. if f(x)=q(x) when x=a, then lim f(x) = lim q(x)

Figure: 1.3.2.4 - Example of maths in a non-linear format

Furthermore, Pen to Print despite not preserving the layout of the document can still recognise basic maths symbols and even things such as capital pi and trig functions. Therefore with some basic edits you can digitise your maths notes, granted they are in a strictly linear fashion - going from top to bottom. Along with a few caveats to not trigger any

of the OCR softwares limitations such as writing fractions laterally instead of horizontally which would be easier to recognise.
However, this can raise the question of if we can split a non-linearly structured document into segments that the OCR program can process individually and then output in a way that can preserve the layout of the document. This could even be accomplished by hand selecting various segments and explicitly selecting some segments to not be included in the OCR process; to prevent what happened in Figure 1.3.2.3, where the annotations on the quadratic graph also get processed. And since the graph isn't included in the final output, it is just randomly there, wasting time during editing to delete it. This is something that should be further explored and tested during the design phase.

Summary:
Overall Pen to print is an excellent product with a bearable error rate, an exceptional ability to read cursive and 10 free pages that you can scan. While as you pay for its monthly subscriptions you can scan 100-10,000 pages per month depending on your tier, use a proper math converter and an application that can also digitise tables and forms. Though this just goes to show the need for a consistent free application that can be enjoyed by students. Furthermore, their current system of exportation and error correcting before exporting matches my initial idea to satisfy the stakeholders; therefore this only further justifies it. Additionally, they consciously stick to a simplistic motif, which makes their website ad free despite it being a huge way to monetise your application - a feature the younger demographic would appreciate.
Finally, this has also inspired me to test whether my aforementioned segmentation idea could work with my handwritten notes to text converter.

Competitor features comparison tables	
Adobe	**Pen to Print**
Massive paywalls to properly use their OCR scanning features. This would not bode well with stakeholders, especially the younger demographic of students; since the newer generation normally uphold quite a frugal attitude, especially since many would consider using an OCR software to digitise their handwritten notes as more of a ‘luxury’ than a necessity.	Exceptional at digitising cursive handwriting, with flexible rules for they can digitise. Which is perfect for both students and teachers who would both encounter cursive writing within a school setting.
Rigid rules regarding what they can digitise. Which would be extremely disadvantageous for most of my stakeholders, since teachers would need to mark students that are rushing through an essay. Consequently students are always rushing to maximise the amount of time given to them to write their essays. Therefore realistically there is no time for stakeholders to be able to	Recognises basic mathematical notation. Which would be very useful for the majority of students and math teachers (especially in the lower year groups), since most of my primary stakeholders would be studying maths.

conform to Adobe’s recommended standard for their handwritten notes.	
Unintuitive user interface, along with a very confusing process for converting handwritten notes into digital text. Overall it would just simply discourage users to use Adobe, since trying to use OCR with Adobe - despite its strict input rules - is also such a convoluted process; that many users would stop after being blocked by a paywall in the main Adobe website. Not many would try to research alternative Adobe ways to use OCR (like Adobe Scan or Adobe Acrobat's OCR features), they would simply move on.	Unable to preserve the format of non-linear documents. Which shouldn't be a problem for most of my primary and secondary stakeholders as most students and teachers need to deal with essays quite a lot. This would only be detrimental for mathematical notes, however this is made up for by its application for every other subject.
Is very restrictive on the type of input files they can receive. Since they want to use those that are already part of their own ecosystem, the user needs to constantly convert their files into PDFs. Links back to the previous point, many stakeholders would just give up after being forced to convert all of their images into PDFs.	Has an editable text field where users can edit any changes from the digitised text before exporting. This is extremely important for all stakeholders in general, since it can help them fix any errors made by the model, or edit the text in any beneficial way before properly exporting it. This saves time for the users, since they don't have to separately open each file one by one to add their edits.
Ruthless use of freemium concepts, when users attempt to use adobe to convert their documents, they fall into a rabbit hole that constantly tells them to upgrade or buy their premium package so that they can use Adobe’s OCR model to its fullest extent. This creates a very greedy environment for stakeholders, who would become exceedingly frustrated by this behaviour. Some might power through and use the application till the end, however it is very unlikely they would ever choose to use this application ever again.	Has multiple exporting options; makes the application more convenient for stakeholders by being flexible so it can cater for the various needs various users might need. Students might just want to copy the text to paste into a google document, while teachers might want a word file that they can store and refer back to later. Furthermore it accepts multiple file types as inputs, making it even more convenient for stakeholders by being flexible, so that they can satisfy users by accounting for various different use cases.
	Clean and simple user interface that scales well on all devices, useful for students that would normally interact with this application using their phones. Moreover, students would be more appreciative of a simple and intuitive design.
	Provides users with free options for a limited number of uses. This would be received well by all stakeholders, since they are mostly students with no money and normally a strong desire to not spend it on additional applications (that might be seen

	as unnecessary).

Overall Summary of Chosen Features

Therefore I shall ensure that my application is intuitive to use, and have a streamlined process that users will enjoy and understand to the degree that they feel compelled to come back to use my application again. This will provide a good counter to Adobe's convoluted mess of an ecosystem, where users will get so frustrated with Adobe that my application feels like a breath of fresh air. Furthermore, I will ensure that there are no ads, freemium features or other predatory tactics to take advantage of users - especially since I feel that many of my primary stakeholders are put off by them. Else I will definitely use the editable text field feature from Pen to Print, however I will add many more types of exportation since students and teachers will always encounter various different situations for needing different formats. This also addresses the lack of flexibility of Adobe, regarding how they solely deal with PDFs.

1.3.3 Stakeholder Survey

After reviewing the current competitors, I felt it was important to take the input of the stakeholders that will become the users of this application. To narrow down on their priorities, requests and ideal quality of life features to ensure maximum satisfaction.
This was given to a sixth form student demographic - who are the targeted end users - with 13 responses across 7 questions.

Do you use any form of handwritten paper records (e.g. exercise or note books)?

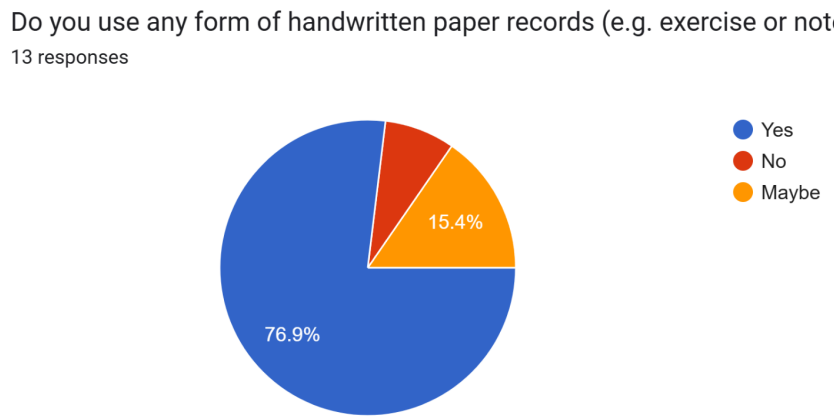


Figure: 1.3.3.1

I used this question to see if the majority of students in the sixth form worked with physical paper records and if the population was large enough to justify the creation of this application. And as expected 76.9% from Figure 1.3.3.1 do indeed use a form of handwritten paper records for their studies, while the other 23.1% would most likely be students that do a combination of BTEC subjects and/or computer science where physical notetaking isn't compulsory. Granted for lower years in a school this pie chart would most likely be completely yes as all of them partake in subjects that will need them to write. This

highlighted to me how not all students have a need for a handwritten notes to digital notes converter.

Would it be useful for you to digitise your handwritten work (and why)?

The consensus was that most respondents saw value in digitising handwritten notes, for: organisation & storage so by having everything in one place can make it easier to manage; which can be extremely useful when you are somewhere without internet, accessibility & portability so it can be accessed online without a physical copy, easier ability to edit and share it. Along with people wanting it for overall personal convenience due to disliking handwriting, where they rarely revisit handwritten notes. Furthermore, there was an emphasis on people wanting applications - which could digitise their notes - to be of great quality and to be useful even with documents such as mind maps or tables; while also being easy to use, convenient and intuitive.
However, a few in the minority preferred physical notes because: they found them easier to carry, locate, and edit manually, while digital versions only feel necessary for professional purposes. This was especially the case for STEM and maths students, who wanted to retain all the diagrams, graphs and special notation they had used in their handwritten notes. Otherwise somewhere on the fence where it was dependent on factors such as, if digital copies of their work are needed for formal/professional purposes.

Do you currently use/have used any software to digitise anything handwritten (e.g. apple or adobe scan)?

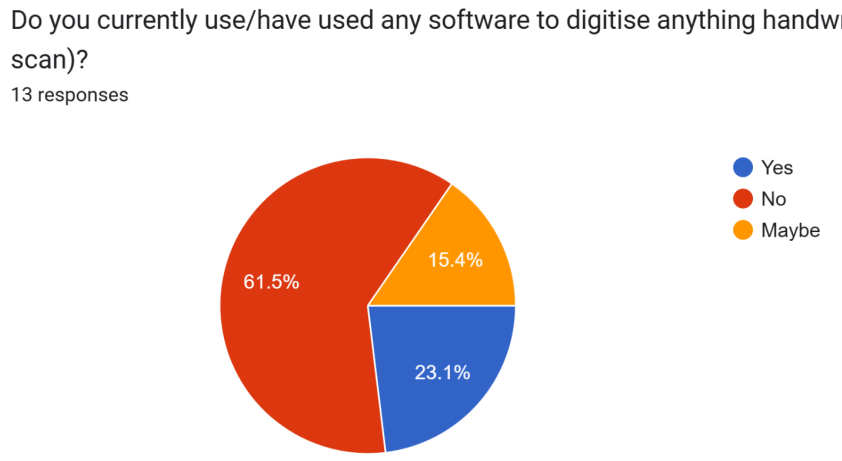


Figure 1.3.3.2

I wanted to use the question in Figure 1.3.3.2 as a precursor to see the current use of OCR software, where the majority of people (61.5%) of students don't currently use one, while 15.4% may sometimes use one for the odd job. This helps to display the current attitude towards OCR applications and how students currently don't really think much of them, which lets us analyse how we can draw them into a product that they deem worth their time and effort.

If you were to use a free application that converts handwritten notes into editable text, what features would you prefer?

The responses suggest students want a handwriting-to-text tool that goes beyond simple digitisation of words. They value features such as preserving formatting, exporting in multiple file types, ability to be uploaded to websites like google classroom and only convert text that you can highlight. Many also want spellcheck, grammar correction, and math recognition; where simplicity, speed, ease of use, and being ad-free are prioritised. Some respondents even expressed interest in Google Lens–style functionality that could provide context, examples, or links when scanning text. Overall, the emphasis is on a tool that is accurate, versatile, and user-friendly.

What medium would you prefer to access software?

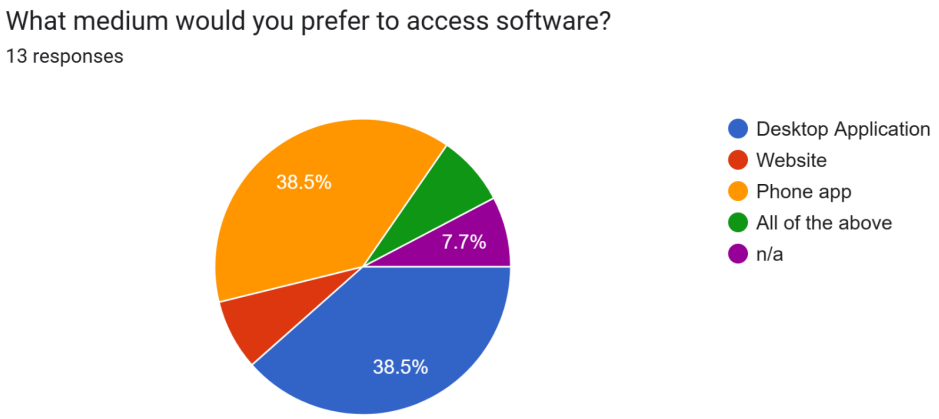


Figure: 1.3.3.3

The question shown in Figure 1.3.3.3 is another precursor example but for the next question, to help gain insight on the most popular mediums to use software. Where currently the majority lies between desktop applications and phone apps, especially since they provide the most convenient experiences for users. Where phones are extremely portable and can access anything on the internet regardless of your current location, while computers can run powerful software applications while being comfortable to use, that comes with using external peripherals and a bigger screen.

What medium would you prefer to access software where you need to attach an image of handwritten work?

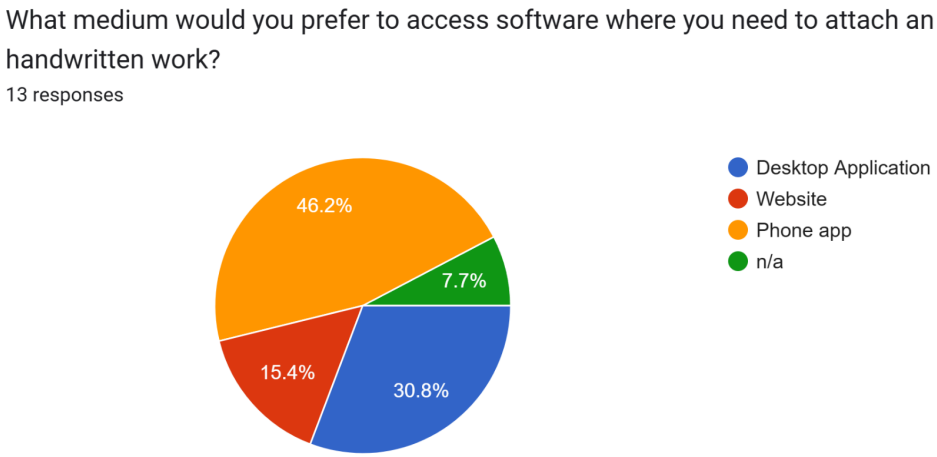


Figure: 1.3.3.4

This question then aims to see the difference between how users would prefer to use software when you need to attach an image (which is applicable to my software), here we can see in Figure 1.3.3.4 that the majority then tilts towards phones by 46.2%, as more often than not that is what is actually taking the picture. However this highlights the usefulness of websites for this software, as despite having only 15.4% of the votes, it is the only medium that can work on both phones and desktops. Where they have the convenience of being able to be used on both devices, while reducing the amount of resources that would be required to publish them on various different platforms in computers and phones.

If you had access to a software that can convert your handwritten notes into editable text on your computer, would you use it?

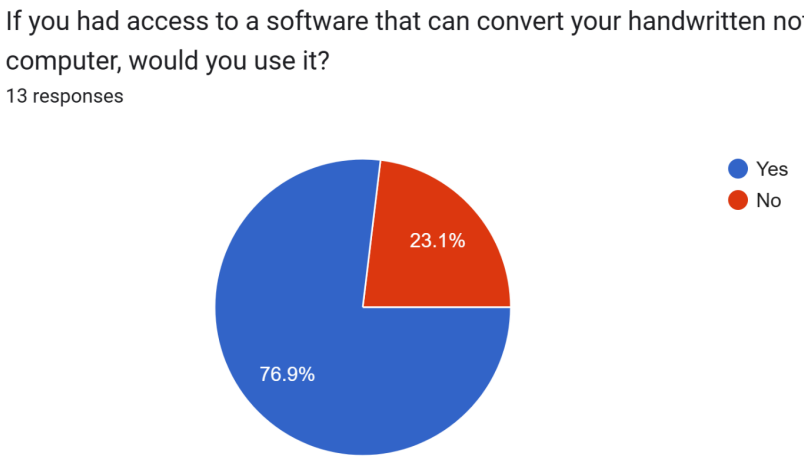


Figure: 1.3.3.5

The question in Figure 1.3.3.5, contrasts our previous question where I asked students whether they currently use any software to convert handwritten notes into computer text, where 76.9% of students would use an OCR application if they had access to it. Most likely

more so if it included all the convenient features that they further suggested, highlighting how it is mainly their access to software that limits their use of it. Further empowering my goal of needing to create a free application that can satisfy students appropriately without attempting to constantly take advantage of them.

Conclusion:
Overall this survey was essential in understanding the target audience and how to mold this software into an application that not only appeals to stakeholders but also functions to the highest of their standards.

1.4 Proposed Solution

1.4.1 Outline of solution

My product will be primarily aimed at students, to scan their class notes and then digitilise them as text that they can copy and edit/store online for free. It will use a pre-trained convolutional neural network, which will use OCR (optical character recognition) technology to convert handwritten notes to digital text. I have justified the use of a pre-trained model as it would be more beneficial for the application to host a pre-trained model that I can then further fine tune and configure with my own personal data and hyperparameters to a higher refinement. This would be more beneficial than for me to create my own model, which would be more unrefined and prone to greater error; especially due to the time constraints of this project, for the benefit of the stakeholders the obvious choice would be to use a pre-trained model.
In accordance with my stakeholder survey to make it more convenient for the user, I will attempt to implement feasible features that they have mentioned. This will be a website where the user can drop an image file and then process it to output an editable text box with the digital text from the image, where they can correct any errors before exporting their new digital notes.

1.4.2 Requirements

Essential Features	Justification & Feasibility
It is a website	I will integrate it into a website so people can use the application regardless of if they have a computer on them or not, as they can then just use their phones - which they most likely will use to take the picture with anyway; since in the survey most stakeholders would prefer to use their phone or personal computer, so a website handles each request by being accessible by all. Providing stakeholders with flexibility, so that they can use whichever device the user is comfortable in using. Therefore it is important that the user interface in the phone version of the website is scalable and appropriately formatted for use. This will be easily a feasible feature to

	implement since this is the base of the entire program, everything else simply stems off of the fact that it's a website.
You can export it as a word file, pdf, text file or onto your clipboard	Grants the user with the ability to store their handwritten paper records in any manner that suits their fancy. From attaching it to your clipboard so you can paste it into a google document which you can store in your google drive - which can let you access it on your phone - or upload onto your google classroom, to simply storing it as a word file on your local system such as your laptop for when you're on the go. This is backed by the diverse number of opinions from the stakeholder survey, since mostly all those that would like to access this application through their phones would prefer to just be able to copy it to their clipboard. While those preferring to use their desktops would like to store their processed text in the form of files. Overall being able to export plaintext in the form of files would be simple enough, but allowing for users to use the copy button - and let the website access their clipboard - might need the website to use HTTPS which would make things more complicated. Therefore while exportation to those different file types will definitely be possible, having the copy button ready for anything outside of use on my local machine might be infeasible within the project time.
System should be proficient in recognising handwriting	Since the primary function and use of this application is to convert handwritten notes to text - and was the main selling point within my stakeholder survey - since students don't really have much use for converting standard (and most likely straight digital text) into digital text. For their general life and the answers from the survey they use a lot of paper, and therefore handwritten notes, and so a website that converts those handwritten notes into digital text - whether for easier organisation, storage or transportation - is going to be a valuable resource for my primary stakeholders. Therefore, it is imperative that the OCR model I use is especially proficient in recognising handwritten notes. This would exclude many starter models that beginners would attempt to implement such as

	TesseractOCR. However in terms of feasibility, it should be moderately challenging to find a good pretrained OCR model, however I have a few leads such as through the HuggingFace website machine learning model library. I am confident that I should be able to find one within the project time.
Outputs text into an editable text field before exportation	Allows for the users to self check the text outputted by the model, where before exporting it into a file - whether directly or copying then pasting it - then opening that file then editing all the mistakes; they can merely do it on the spot before exportation. This saves time and just makes sense to help the user, especially if they are scanning many things at once. This is backed by the results of my stakeholder survey, where they want applications that are of high quality, which can enable an easy to use and convenient environment for the user. Overall this is a feature that is extremely feasible to implement within the website - well before the project completion time.
System should allow users to be able to upload multiple files (as long as their total sum is less than a predecided max size)	It saves time for users by letting them handle requests that might have taken multiple iterations before, to be completed within one iteration. Especially in cases where users want a lot of small image files to be processed, it adds more flexibility for the various cases that different stakeholders would have. Additionally, this is further supported by the criteria that stakeholders consider for a good application. Two of which is its ease of use and convenience, therefore by adding a feature that allows users to upload multiple images at a time we not only increase user satisfaction, but also the convenience they face when using the application. This provides an experience that will make stakeholders continue to come back and use this application. In terms of feasibility, it should be moderately feasible to implement within project time. I would simply need to create an array to store all the uploaded files, and constantly iterate over the stack.

Should have Features	Justification & Feasibility
Preserving formatting	A common request from the stakeholder survey was to have the very beneficial feature of being able to preserve the layout of the original document. For example if a double margin is used, or if you have used bullet points and indentations. This can help save time so that the user doesn't need to waste time formatting the output, improving the ease of use of the application. In terms of feasibility, it solely depends on the model that I use, therefore the feasibility of implementing this feature during the project completion time is still a bit uncertain.
System should be proficient in understanding irregular and cursive writing	This is extremely important to satisfy a mainly student demographic, where most of them are rushing to write their notes in the classroom, or just simply have bad handwriting. This is especially apparent with those who use cursive, as more often than not they can have irregular writing that is oddly shaped, not in a straight line and simply messy. The student survey itself includes a wide variety of individuals with a range of different handwriting types. However a lot of them do possess extremely messy and cursive writing. Therefore to satisfy the largest portion of the stakeholder population, it is imperative that the application is proficient in various writing styles. In terms of feasibility of implementing this solution before the project completion, it is again up to the actual model used - therefore the feasibility of this feature is currently uncertain.
Ability to translate non-linearly structured documents	This lets the OCR programs actually translate images like mind maps, where you can pick out each bubble as a different segment and translate it separately so that the OCR does not confuse all the text together and messes up the sequence of how to output it. Since many stakeholders said that it would be useful to digitise their handwriting, especially for non linearly structured documents such as mind maps or tables. Furthermore, it would just be a good idea to support as many different types of handwritten notes to increase the satisfaction of stakeholders. In terms of feasibility, it should be feasible for my OCR

	model to automatically recognise and infer notes that are strewn upon the page; however it is uncertain whether the model will be able to recreate the document in its original sequence - this would heavily depend on the model that's used.
A feature where you can automatically correct spelling errors	Quality of life feature that can just help out students with bad spelling or mistranslations from the model. Which can in turn reduce the error rate of the model. Many stakeholders mentioned in the google form that they want the application to be convenient, therefore by autocorrecting any mistakes they have made in their work can be a crucial feature to help reduce the amount of manual editing they might have to do before exportation. In terms of feasibility this should be moderately feasible to implement, as I would simply need to post-process the returned text from the OCR model and adjust misspelt words.

Could have Features	Justification & Feasibility
Mathematical recognition	This is more of a niche feature that if possible can be hugely beneficial to all STEM students, where it can be used to store formulae or example questions for future reference. Since most responses from STEM students - or just those that did maths - reflected a desire to be able to translate mathematical notation and other special symbols they use. Otherwise it would be quite useless for the majority of students that do STEM, especially those who only do STEM subjects. Finally in terms of feasibility, this is another aspect that is dictated by the quality of my model.
Ability to highlight text or segments, before uploading their image, to signify sections that shouldn't be processed	This can help the model ignore irrelevant text that may be annotations of a graph - which obviously won't be translated, making those annotations obsolete. Or simply help to let the user to organise their notes before digitisation, this adds greater customisation and lets the user have more control over the process. Additionally, from the stakeholder survey it is clear that they want this application to go beyond simply just digitising handwritten notes. Therefore this feature adds another layer of complexity to

	the program that increases user versatility and flexibility upon using the program. In terms of feasibility, this will definitely be a tough feature to implement before the project completion time. Since I would need to let users be able to edit their images within the upload page itself, and then personally edit the OCR model to tell it to ignore highlighted text. Alternatively I could simply black out the text instead of highlighting it within the upload page. However this would still be a big challenge, since I would need to edit and save files within the frontend itself instead of the backend (and if I first send it to the backend and then back to the frontend for review it would unnecessarily elongate the digitisation process).
Ability to draw directly into the website instead of attaching an image	This can be helpful for doing short tests with different types of handwriting without tediously taking photos. It can also be a quirky feature to let the users experiment and explore how the OCR model reacts to various forms of handwriting. Since students mentioned - in the stakeholder survey - that they would like to have an intuitive application. By implementing a feature that could let them test different styles & sizes of handwriting in the application itself, can help stakeholders understand to a greater extent the types of images that would be the most beneficial to translate using this application. In terms of feasibility, it would be easy to use a simple JavaScript Canvas and append the final version to the same input element that will send all the images back to the backend. However its inclusion might overwhelm users, since many would just want to use my website purely to convert their handwritten notes and wouldn't care about this feature. Therefore, though it is feasible to implement before the project completion time, it needs to be evaluated during my cycles.

1.4.3 Hardware and software requirements

As the application will be a website, any user wanting to use it would need to access it through the internet; there, the user’s device will be the client which will only be tasked with

loading the website resources and providing input to the website. Which will then pass it onto the server and the backend, which will have the actual model and do the inference with the given user input. Therefore for the users they would simply need a device with internet connectivity, and basic hardware (such as 4GB of RAM) to load a simple website on their browser and attach photos to it (which would barely be a few megabytes and most modern phones have 64GB or more of storage, with computers having even more) - as the frontend will be designed to be very simple and not very resource intensive in consideration of schools or children from weaker financial backgrounds that might own computers that struggle to load websites. Furthermore, as all smartphones or computers come with at least of 4GB or 8GB RAM on the cheaper end (with most smartphones being 64GB); the website is accessible to basically all students and teachers, as even if the students don't have access to a smart device, schools should be able to provide them with access to computers and internet. And as this is a lightweight website the matter of OS type, type of browser, age of the computer or screen resolution is irrelevant. While in terms of hardware the computer would need a monitor, keyboard, mouse and a device to take pictures, however most students and teachers own phones with cameras and if not most schools should own at least one webcam. In terms of hardware requirements for phones, you will need a smartphone (where the main input method would simply be touchscreen).
In terms of software requirements for the server, it would require a good GPU with high VRAM and double the VRAM for its RAM, along with sufficient cooling and power; however the stakeholders won't be affected by this and the server requirements can be further evaluated after the actual model is finalised, as the model training and inference requirements heavily dependent on the actual model and how it was built. This can either be in the form of physical hardware or the server can be run remotely from a virtual large language model server. In terms of developer requirements, I would need access to python which I can simply download for free online, and an Integrated Development Environment (IDE) as my programming environment, for this I would use Visual Studio Code and its python extension for more efficient debugging; furthermore I would use the live server extension to test the actual website itself - live. In terms of hardware requirements I would need at least 16GB of RAM for training and inference and preferably 32GB with a decent GPU with 8 or 16 gigabytes of VRAM, I know a friend willing to lend his PC that has a similar spec, so I can train and infer my model.

1.4.4 Limitations

I do not own thousands upon thousands of data that I could use as a part of the training set to train a neural network, nor the expertise to create a fully functioning machine learning model that has a low error rate compared to if I used a pretrained convolutional neural network. This would not only be more efficient by using an external model, but can give me more time to focus on the pipelining and integrating the neural network into a workable website application; where I can focus on adding additional features onto a pre-existing OCR model to prove the user experience. However, by using a pretrained model, I would bound my web application by the limits of the model; for example if it is not good at handling math notes or convoluted lighting. Which would be the case with a lot of students, those studying maths and those that just need a convenient way of taking photographs. Furthermore if it has a lot of trouble with recognising the formatting and sectioning of weirdly formatted documents. Despite being able to correctly recognise individual words, it wouldn't be able to

return coherent sentences in a suitable format; this would cause stakeholders to be extremely frustrated, after waiting so long for their processed documents, only to receive barely comprehensible scrambled notes. Some mitigation strategies would mainly revolve around researching and probing the internet for an optical character recognition model that would not face these problems. Moreover for lighting, a method - that's directly within my control - is to use OpenCV effectively to preprocess the images to the best standards to help guide the model to the most likely outcome. Other long term issues mainly revolve upon the progress of OCR models, however this can be easily fixed by just upgrading to better OCR models (since the code that does the OCR Inference should be modular, to provide for easy refactoring that improves the maintainability of my code base).

1.4.5 Computational Methods

This problem is being solved computationally, since the alternative is having stakeholders transcribe all of their handwritten notes by hand into a digital document. This is extremely time consuming and it also lacks the advantage of pattern recognition that a computational solution would hold to make the entire process more efficient. To take it one step further, if someone needed to transcribe a lot of handwritten notes but didn't have the time to, the only alternative would be to hire a transcriber to either help you out or do it by themselves. This would waste an immense amount of time and money; therefore to provide a better method to solve this problem, it is imperative to use computation methods; so that we can not only leverage pattern recognition but so that more time and money is saved.
Hence, my application will use a range of computational methods ranging from, abstraction, pattern recognition and problem decomposition.

I will be using the combination of neural networks for my optical character recognition instead of rule-based/ template-based systems, since template-based systems are an older form of optical character recognition that uses templates or a set of rules to recognise text from images. However this is mainly relevant for uniform (printed) text, such as from receipts, invoices or forms - that have a consistent layout. Though they do have a high accuracy with low processing time, they are limited with their uses within the field of handwritten recognition, which is the main point of my application. They are completely obsolete when handling images that contain handwritten text. Therefore it is better to use an optical character recognition model that relies on machine learning and neural networks. Though they have higher processing times and costs, there are plenty of free alternatives, with models that are constantly improving. Overall using CNN based OCR models as my computational method is simply the only way to move forward, by relying on their proficiency on recognising handwritten notes.

My project will utilise problem decomposition to modularise this project into three main components: the frontend website that will have the Graphical User Interface where the user can input their images of handwritten notes; which will then be fed into the backend that will actually have the optical character recognition convolutional neural network model, it will process the given images and convert them into digital text that will then be postprocessed; the final component will just be the communicator between the frontend and backend that will integrate those two ends together. Overall I will need to use object oriented programming to modularise all the components for easier integration, and implementation during web

development and convolutional network configuration.
Moreover, I need to actually preprocess the images sent over by the users, I would need to make use of many computational features such as: grayscaling, blurring and thresholding. To convert images into clear black and white images, that the OCR model can easily decipher; since the image preprocessing removes most traces of bad lighting, bad strokes and bad camera quality.
Furthermore, this application will make use of abstraction mainly inside of the neural network, where it will convert high resolution images provided by the user into low level pixel inputs for the actual neural network input nodes. This is then amplified inside a convolutional neural network - which this application will make use of - due to its use of convolutional filters. That make complex images easier to process by using a filter that recognises patterns by splitting up the image into batches, then it will reduce their file dimensions and make them smaller by pooling (combining) pixels that are equal dimensions (a square of pixels e.g. 2x2 into 1x1) into a single pixel. In functionality this abstracts the complex image into an input that is more manageable for the neural network to handle. This leads the way for the key computational method that is essential for this project, pattern recognition. This is what allows the convolutional neural network to recognise the handwriting of input images, by being trained on a broad set of training data that can then be simplified via abstraction (convolutional filters and max pooling layers) to let the neural network focus on key information. This results in greater accuracy and shorter time for the model to be trained, this is also the main reason why I must use a convolutional neural network instead of regular neural networks.

1.4.6 Success Criteria

Front end - Website

Criteria	✓
Students and teachers need a clean intuitive user interface for ease of use, so they can start using the application straight away; such as through drag and drop/attach feature for images from their photo library or folders so that the UI is especially more convenient for phone users, which most students would fall under	
To allow for flexibility, the users should be able to export their file into multiple different file types or be copied straight into their clipboard; as,students might want to paste multiple school notes into a single compiled digital document, while teachers might want a word file they can share with other teachers, overall by providing flexibility of exportation a good user experience is achieved, by accommodating to the various file types of that different users using this application might require	
Lets the user edit the digital output before exportation; to correct any simple mistakes made, or to create a finalised edit of the text before exportation, this simplifies the exportation process so that users don't need to export and then edit their text and then use it, when they can just do it in the text editor and then export the text	
Scalable onto mobile phones, should make sure the layout isn't distorted; as	

students would be more comfortable using a mobile phone, as that is most likely where their pictures are stored	
Highlight uncertain characters or words for user review; helps students or teachers as they wouldn't want to waste much time proofreading, when instead the program can show them where there would be the highest error rates	
Real-time (or near real time, roughly at least >=80% accurate) loading bars; as students prefer accurate representations of time due to normally lower attention spans	
Display error messages if there was any issues, e.g. with uploading a file - file format doesn't work; helpful for all users to determine if this was a client or server error, so they don't waste time trying to get it to work when it was a server error, or so that they don't exit the program due to impatience which is common with the younger student demographic	

Back end - Convolutional Neural Network

Criteria	✓
It auto spell checks the digital text before outputting it to roughly a >=80% accuracy; as teachers would want an emphasis on accuracy, especially if they are using the program for tests	
It displays extracted digital text clearly, preserving formatting where possible, by targeting only the sections with text and digitising them so that at least if the image is not an essay esque document, there is still some form of formatting preserved so then it's easier for the user to adjust their text	
The CNN trained on diverse handwritten datasets, including my own sample sets to reduce bias in the model, this will improve accuracy of the results leading to better user satisfaction	
The CNN model is robust when handling variations in handwriting styles, extraneous image noise, and inconsistent lighting through good training data sets; this will make the model more flexible, which will greatly increase the type of image inputs the model can process. This will elevate the user experience as users won't need to retake pictures due to bad quality, and will generally get better results due to this increased flexibility of handwriting style recognition	
It is efficient, with a low latency for faster loading times which leads to less of a wait time for users; this is extremely beneficial for younger users who have lower attention spans, and for people who want to process a larger amounts of images	
It is integrated effectively with the frontend, with clear endpoints for file upload, processing status and file retrieval so that the frontend and backend are completely in sync, to reduce the possibility of any bugs or glitches going unnoticed during testing (which then can frustrate the user causing them to lose faith in the program)	

2. Design

2.1 Decomposition of the Problem

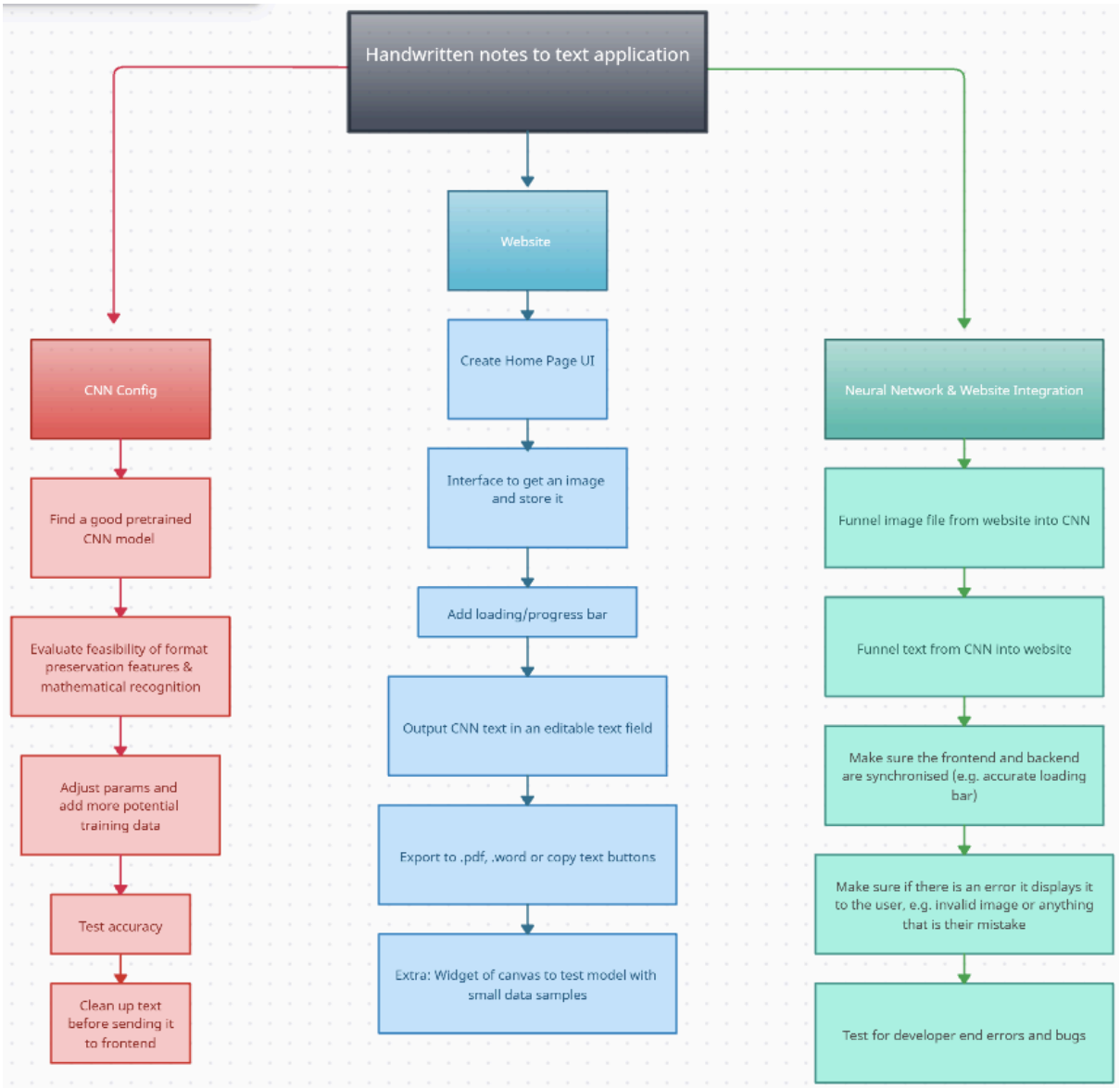


Figure 2.1

I decomposed the application into three major stages - shown in Figure 2.1: configuring the convolutional neural network, creating the website and then integrating them together. This allows me to tackle the problem with more ease and efficiency, by splitting the problem into sub problems - via logical divisions - that can be accomplished in three cycles. This also encapsulates the frontend and backend, then allowing me to focus an entire cycle on purely integrating the frontend and backend together, and then to perform some final fine tuning. In essence this also makes the entire application extremely modular, by encapsulating the

frontend, backend and integration; which consequentially increases the maintainability of this software. Where it is then easier to fix and test bugs during development due to this modular approach across the three cycles, and it's also extremely easy to modify or update code due to its modular nature (where there will be 3 components, handling the convolution neural network model, website and their integration - which can range from data communication, real time synchronisation and error display).

2.2 Solution Structure

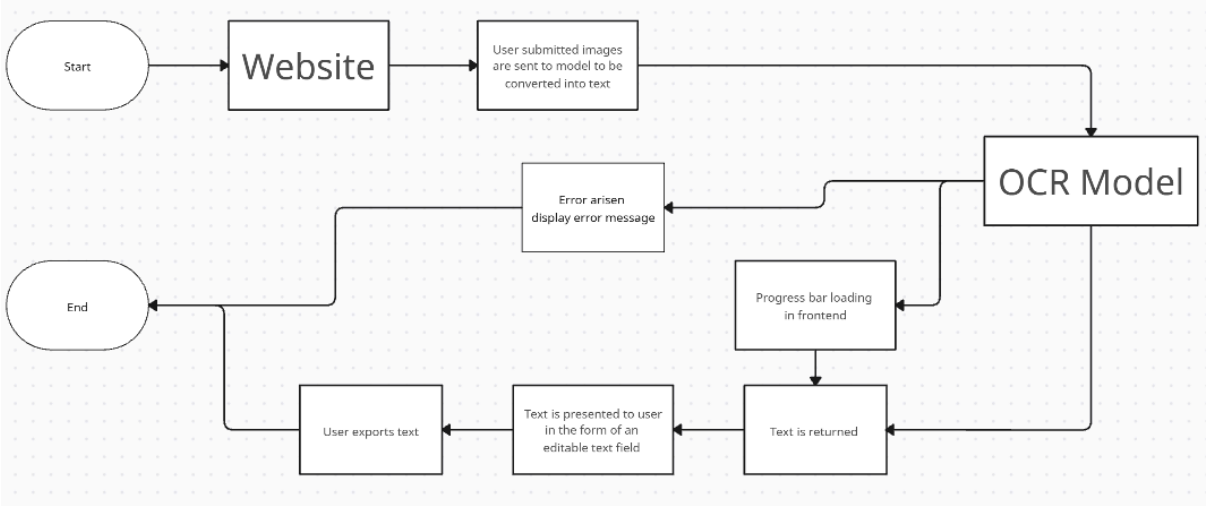


Figure 2.2

As seen from Figure 2.2, the data flow will stem from the website, where the user will upload images from their files/photo directory. Which will start a loading bar that will be near time accurate as the images get processed by the convolutional neural network model.

This will be then sent to the Optical Character Recognition model that will then preprocess the images, converting them to grayscale, denoising, normalizing pixel values, thresholding, and performing line/word segmentation or resizing so inputs are consistent, most likely through an image manipulation module such as OpenCV from python. Then the pre-trained Convolutional neural network will extract the visual features; a convolutional filter might look for edges, curves, strokes or textures. Each filter learns to recognise a different aspect of the image, and by combining the outputs of these filters, the network gains a deeper understanding of the overall image. Once the visual features are extracted, the output from the Convolutional Neural Network is further processed to segment the text into individual characters or words (these are coined as Glyphs). This step involves identifying boundaries or separating different text regions within the image or document. Accurate segmentation is crucial for enabling proper recognition in subsequent stages.
These segments (Glyphs) are passed into a sequence model like a Recurrent Neural Network or a transformer, to capture the order of characters and words. Recurrent Neural Networks are neural networks with nodes that have a memory-like component that allows the nodes to remember past information as they process new inputs.They analyse text one character at a time and consider the surrounding characters to make predictions or infer missing details. This makes them incredibly powerful as they can recognise the context of

the text, such as capturing dependencies between characters and words. In OCR especially, Recurrent Neural Networks can predict the next character in a word, based on the characters processed so far, or they can identify specific words or phrases based on the preceding text. Recurrent Neural Networks can capture these nuances and make informed predictions based on the patterns they've learned from training on large amounts of labeled data. This capability of Recurrent Neural Networks is especially valuable when dealing with handwritten text, where variations in handwriting, connected characters, or even mistakes can occur. Furthermore, their contextual understanding enhances the system's ability to handle various fonts, languages, and document layouts, making it suitable for diverse OCR applications. Then once an output tensor is given, it needs to be decoded via an activation function that returns it in text form that can be handled by humans.

Finally, postprocessing steps will be added such as spell-checking which will use the python module pyspellcheck, making it more accurate and readable. During this phase the need for a grammar checker might become apparent, however for now I will stick with a simple spell checker.

Afterwards, the Optical Character Recognition model will return the text to the frontend website, this will complete the loading bar and will output to the user a java script editable text box with the processed text, so that the user can edit it - due to any technical malfunctions or for personal reasons - before exportation. There will be an option to export it via copying it to your clipboard, else as a pdf, word or text file. This allows this application to be flexible for various stakeholders, who are in a professional environment that require words or pdf documents or just need to insert it into their current writings.

In terms of the integration of the user interface with the backend, its main interactions will simply be sending images to the backend and receiving processed text from the model that will then be displayed to the user.

2.3 Algorithms and Implementation

Image Upload and Validation Algorithm:

```
function ValidateAndUploadImage(file):
    IF file == None THEN
        RETURN "Error: No file selected"
    END IF
    IF file.type NOT IN ["jpg", "jpeg", "png", “gif”] THEN
        RETURN "Error: Invalid file type"
    END IF
    IF file.size > MAX_SIZE THEN
        RETURN "Error: File too large"
    END IF
    SEND file TO backend_endpoint("/api/ocr")
    DISPLAY progress_bar UNTIL response_received
    RETURN response_text
END function
```

Explanation: Used to validate the input files given, and clearly executes the error path when a file isn't in the form of a jpg, gif, jpeg or png. Else it will send it to the backend Optical Character Recognition model, then kick start the loading bar and output a response text to show no errors have occurred. This ensures the Convolutional Neural Network doesn't crash due to being provided with unsupported file types, ensuring that the error is caught early, and is sufficiently handled.

Image Preprocessing Algorithm:

```
function PreprocessImage(image):
    CONVERT image TO grayscale
    RESIZE image TO (128, 128)
    APPLY threshold(image)
    APPLY noise_reduction_filter(image)
    RETURN preprocessed_image
END function
```

Explanation: Used to prepare raw handwritten images to be inputted into the Convolutional Neural Network model, abstracting colour (as it's irrelevant for handwriting), resizing into a fixed size so all inputs have consistent shapes. Normalise pixel values for uniform lighting using thresholding to create a pure black and white image, and noise reduction filters out background patterns or paper texture - adding a further layer of abstraction. This ensures that the Convolution Neural Network receives clean, standardised inputs for recognition accuracy.

CNN Inference Algorithm:

```
function PredictText(image):
    LOAD trained_CNN_model("ocr_model.h5")
    preprocessed_image = PreprocessImage(image)
    cnn_features = CNN_ExtractFeatures(preprocessed_image )
    rnn_output = RNN_ProcessSequence(cnn_features)
    decoded_text = DecodePrediction(rnn_output)
    decoded_text = ActivationFunction(prediction)
    RETURN decoded_text
END function
```

Explanation: This is the entire execution phase of the Convolutional Neural Network, where it first loads the already trained model itself, and executes the preprocessing image algorithm, which is then passed into the actual Convolutional Neural Network that then does its entire process and outputs a tensor of the images extracted features that after being run through the context analysis of a recurrent neural network returns a final tensor that represents the predicted sequence of characters. Which is then decoded with an activation function such as sigmoid or softmax, this becomes decoded text that is returned to the frontend. It is necessary as it performs basically the entire backbone process of the model.

Postprocessing Algorithm:

```
function CleanText(raw_text):
    CORRECT spacing_and_linebreaks(raw_text)
    CORRECT spelling_and_grammar(raw_text)
```

```
    RETURN cleaned_text
END function
```

Explanation: This performs any needed fine tuning upon the text outputted by the Convolutional Neural Network model, cleaning the spelling, grammar, removing unwanted spaces and line breaks. This ensures that the text is cleaned and mostly accurate to a satisfactory standard for the user before being displayed on the front end. Maybe even fixing grammatical and spelling errors performed inside of the text itself that would have been a hassle for the user to fix anyway. Adding another layer to the quality of life features in this application.

Together, these algorithms form a complete pipeline for handwritten text recognition. The frontend upload and validation algorithm ensures that only the correct inputs are provided. The preprocessing algorithm standardises the image. The Convolutional Neural Network inference algorithm converts the image into a textual representation, and the post-processing algorithm refines this into readable text. Finally, the frontend integration and error-handling algorithms ensure the system is responsive, reliable, and user-friendly, completing the cycle from image input to editable text output that is ready for exportation.

In terms of tackling the implementation of these algorithms during development, I faced the problems below:

Problem	How I overcame it
Files that were dragged and dropped were not being sent to the backend	I switched from separately sending a new form with the drag and dropped files, to appending them to the input tag in the form element.
Issue with post processing, where \n would not be recognised by the editable text field	I used regex to replace the string \n with the character \n, to signify a new linebreak.
Image preprocessing would return a completely black image when used on handwritten notes (worked fine with uniform text)	I used an adaptive threshold with a gaussian blur instead of a fixed threshold that only works with uniform writing.
When the backend started the inference process, the request would get cancelled	I attempted to solve it asynchronously, using Asyncio, running it in a separate thread; but the real solution was that since the request was sent with a regular HTML form element, every time the model was being called the request timed out. So I used a background process to run the model inference after the request was completed and returned to the frontend.

2.4 Usability Features

On the side of the front end, I will stick to a very basic and simplistic User Interface, to not overwhelm the user and also allow them to get their documents digitised with no extra steps or annoying ads which commonly plague many websites on the internet. This maximises user satisfaction and enables an intuitive interface that doesn't need any extra descriptions or explanations.
The home page User Interface will simply consist of some guidance on what type of pictures the model will be the best at deciphering and also what the model is capable of. Then the user can click next and see a simple drag and drop interface where they can start straight away by uploading their handwritten notes.

Overall, I will aim to use a rounded design due to the psychological effect it normally has on users (source), where rounded shapes are seen as soft, safe and inviting while harder shapes like squares are seen as structured, rigid and attention-demanding yet sort of reliable. There is nothing wrong with using squares with sharp edges, and I will be using them time to time to not make the application seem bland or not to keep the design very repetitive; however I will mainly prioritise the use of more rounded shapes, as they provide less visual friction and are easier to process on touchscreens - such as in phones - which for a web application is extremely beneficial as most people will be using their phones only. Finally, the benefits of using softer edges appeals to our primary audience of students and children, who naturally prefer a more comfortable atmosphere.

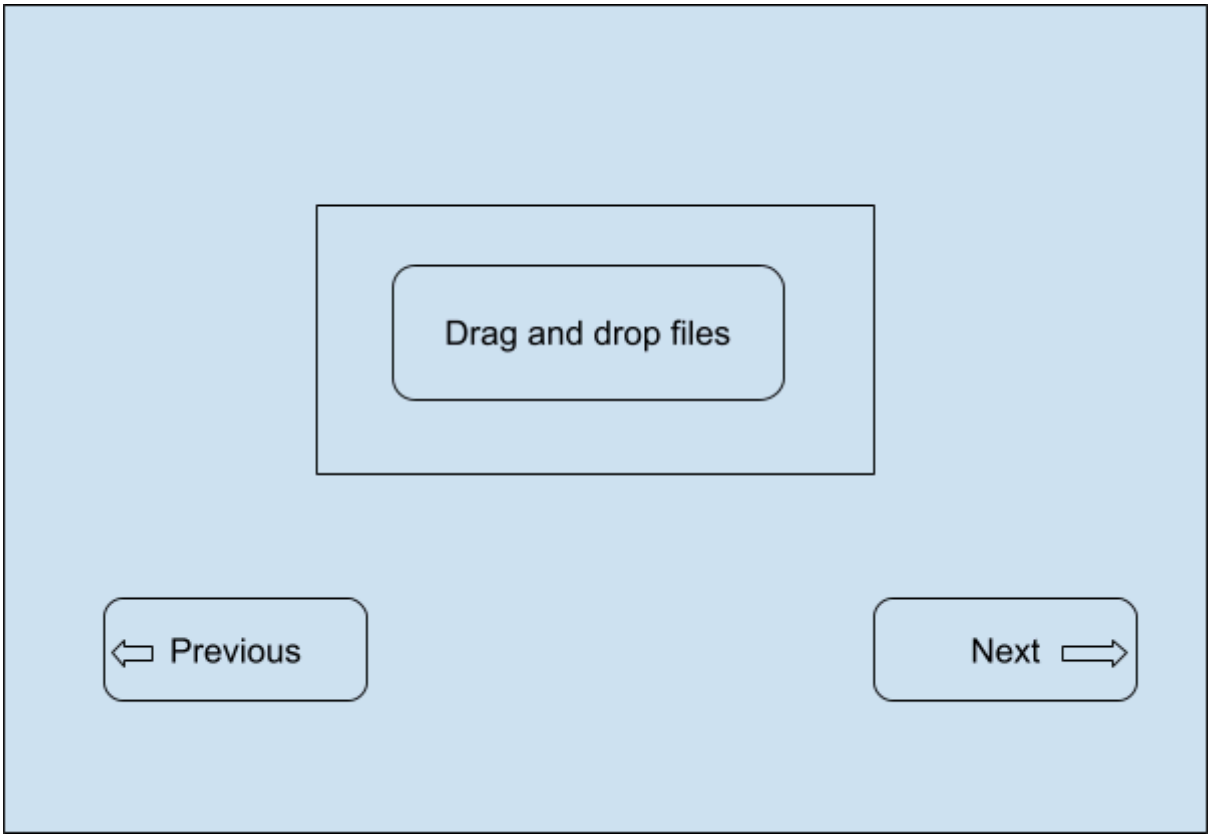


Figure 2.4.1

Figure 2.4.1 is an example of the second screen where the users can upload their images, in the actual application everything will be symmetrical and centered with a proper colour

scheme and an arial font for perfect clarity, however the basic structure and shapes will be mostly the same due to the aforementioned reasons.

Afterwards, as the user is waiting for the results of the Optical Character Recognition model, there needs to be a progress indicator. Normally this is a throbber (also known as a loading icon), however due to the size of the screen a loading bar would be more effective; it won't get lost by the eyes of the user and wont look goofy if sized up (if I were to attempt to fix the former issue when using the throbber). Furthermore, there must be a cancel process screen that will just take you back to the home page, as the program needs to have a termination feature if the user forgot to add certain images, uploaded the incorrect images or if they simply wanted to exit the process.
Furthermore, in case of an error (whether client side or simply from the model itself), there should be an error screen that displays an error message of the problem that happened. And especially if it was related to the web application (e.g. network issues or disconnections) or errors from the model (whether the images provided by the user were unsatisfactory or if the model itself had an error); which should then terminate the process and send the administer an error message of what malfunctioned with the model. This lets the users know what happened, and doesn't keep them in the dark, it keeps the users in the loop while letting the developer know if there is a problem.

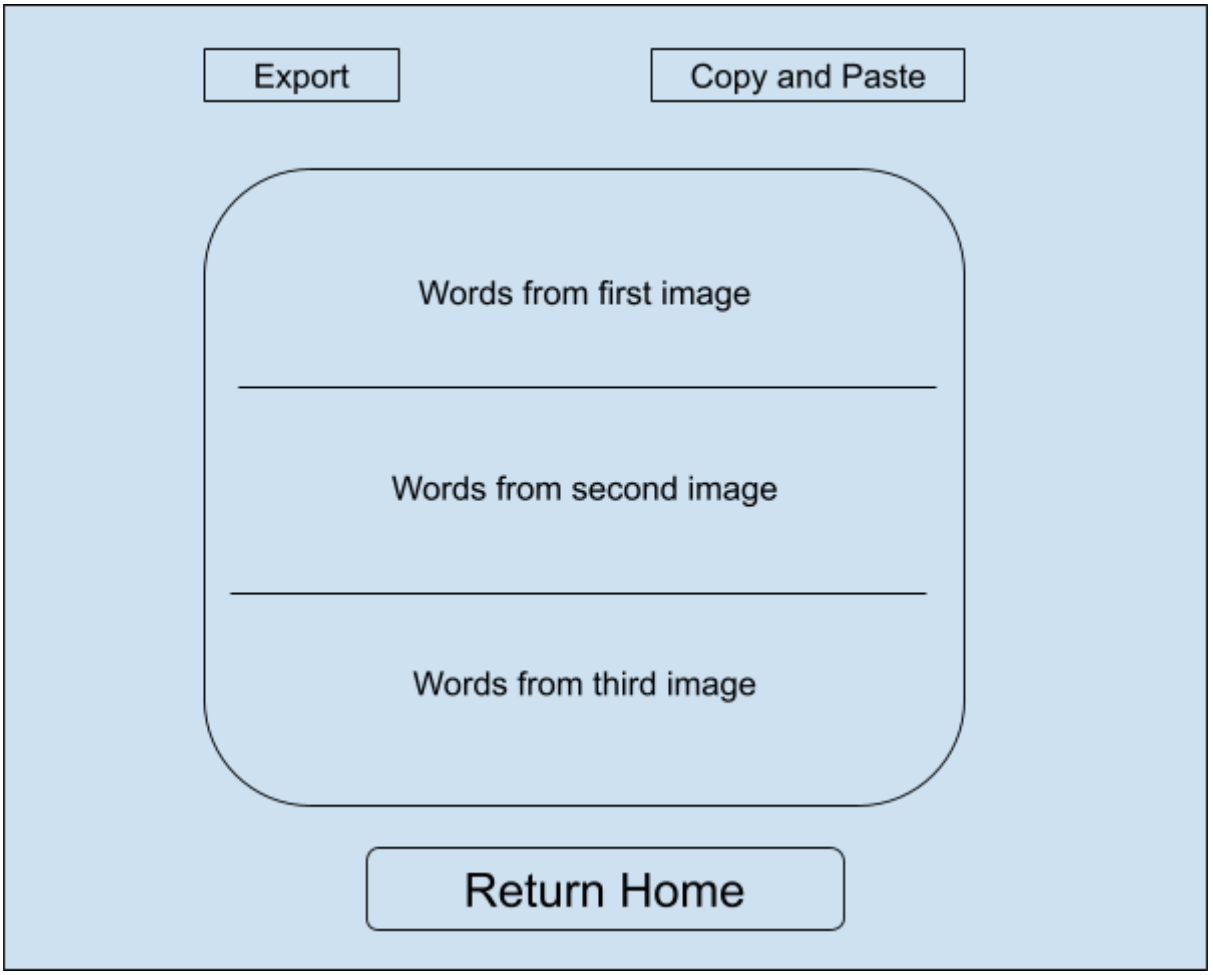


Figure 2.4.2

The Figure 2.4.2 is a rough design of the final screen that the users will see, here the user's text outputs are separated by the different images they were from. So the user can clearly identify which text belongs to which image, and it is presented in an editable text box where the benefits mentioned in 2.2 Solution Structure regarding exportation and the editable text box are highlighted here.

Overall this highlights the prioritisation of the primary stakeholders while still giving room for the satisfaction of secondary and tertiary stakeholders.

2.5 Variables, Data Structures, and Classes

Classes:

Class name	**Attributes**	**Methods**	**Purpose**
OCRModel	cnn_layers, rnn_layers, decoder, model_path, loaded	load_model(path), predict(image), cnn_extract_features(image), rnn_process_sequence(features), decode(rnn_output), activation_function(prediction)	Encapsulates the Entire CNN, RNN and activation function pipeline, keeping code modular and reusable
Preprocessor	target_size	preprocess(image), resize(image), threshold(image), denoise(image)	Handles all image preparation tasks before inference
UploadManager	uploaded_files, temp_folder, max_size, allowed_types	validate_file(file), save_temp(file), remove_temp(file)	Responsible for file validation, secure temporary storage
ExportManager	export_format_list	export_text_file(text), copy_clipboard(text), generate_pdf(text), generate_word(text)	Manages exporting recognized text (TXT, PDF, Word, copy-to-clipboard)

Variables:

Variable name	**Type**	**Purpose**	**Validation Check**
uploadedImage	File	Raw file uploaded by user	Presence and type check
fileType	String	For file type validity checks	Format and type check
fileSize	Int	Size of upload	Range check

proccessedImage	NumPy array or Tensor	Proccessed image for model	Shape check
cnn_features	NumPy array or Tensor	Feature maps output from CNN	Dimension checks
rnn_output	NumPy array or Tensor	Output sequence from RNN	Shape/length checks
decoded_txt	string	Final OCR text	Length and format check
progressStatus	Int	Progress bar state	Range check (0,100)
errorMessage	String	Stores the error faced by the user	Type check

Data Structures:

Data structure	**Format**	**Purpose**	**Justification**
NumPy array/Tensor	np.array(shape=(height, width,1))	Image pixels, CNN and RNN tensors	Required for machine learning model
List/Queue	[{imageFile1}, {imageFile2}]	Batch processing of the different images sent to the CNN model	Ordered lists, easy to implement
String	“Output text here yippee”	Final output text to frontend	Natural choice for displaying text
Dictionary/.JSON file	{“image”:”...”, “text”:”...”}	For API requests between frontend and backend	Standard data structure due to how lightweight it is, causing efficient communication between frontend and backend

2.6 Testing

Data Type	**Description & justification**
Normal data	I will input a regular image of handwritten work, and then test to see if it responds correctly to the regular form of data provided. The expected result should be a

	near to real time progress bar (which has a working option to terminate the program) that then transitions to a page where I can use the editable text box, export without difficulty and then return to the home page after pressing the button. This simply tests the entire system and how it should react if given the data expected by the Optical Character Recognition model.
Invalid data	I will input images with: no text, a text file instead of an image or a corrupted file (if I could get one). The empty image should simply work as the normal data result. As the users are warned that if they use images that don't contain any words the model will hallucinate and can then output random nonsense by trying to find imaginary patterns or simply it will return nothing. Else the model should stop and output an error. This serves as a test to see if the model will throw an error if a non-text image is provided (but due to the model architecture that is highly unlikely), but mainly to also see how errors are handled when caused by the model, and if the appropriate error message route is taken.
Extreme data	I will input an image that will contain very unusual or challenging handwritten samples, but are still valid for processing. For example, I will use images of handwritten notes that are heavily slanted, very densely written, or photographed in poor lighting conditions. This tests the model's robustness under conditions that a typical user might still realistically produce due to not reading the guidelines, however this also pushes the convolutional neural networks recognition limits as I can test its accuracy with extreme handwriting. The expected result should still follow the same process as normal data. However, the accuracy of the Optical Character Recognition output will likely decrease, and

	the returned text may contain more spelling or structure errors. This test checks whether the system continues to function smoothly (no crashes, freezes, or network timeouts) and handles unusually difficult inputs without causing exceptions or misbehaviour in the frontend or backend.
Boundary data	I will input images that test the boundaries of the program; the test focuses on the upper and lower limits of what the convolutional neural network model is designed to handle. For the lower boundary, I will use a very small image (e.g., 100×100 pixels) containing only a few characters of text. For the upper boundary, I will use a very large image (e.g., over 4 MB or 4000×4000 pixels but less than 5MB) containing an entire page of writing. The expected result for both should still be similar to the normal data process. However, the smaller image may result in missing or incorrect text since there is limited data for the model to process, while the large image may take longer to process but should not cause the application to hang or crash. This test ensures that the system's performance and responsiveness remains stable across the boundary limits of various image sizes.

For post development, I will utilise a collection of normal data that is designed to test the limits of my model. It will include images with handwritten notes of various styles, bad quality images, bad handwriting, random crosses, formatting or doodles. It will include a huge list of unpredictable handwritten data to test the boundaries of what my model can and can not transcribe. Which will be a useful metric for evaluating the effectiveness and accuracy of the current model I am using, which will in term become the prime source of evidence in regards to the future developments of my project - regarding the current model in use.

3. Cycle 1

3.1.1 Complete Overview

I will be splitting my cycles into 3 overall; cycle 1 will cover the construction of the majority of the frontend and the actual website itself. Cycle 2 will cover the construction of the backend

server and integrating the backend with the frontend to create a synchronised web application. Cycle 3 will focus on refinement and implementation of constructive feedback attained from external testers to squash any unexpected bugs and cater it more towards my stakeholders.

Additionally, I have made a github repository for this project to enable version control - so that when my codebase becomes too large for me to keep up with, any detrimental mistakes can be rolled back through Git's version control. Furthermore, it allows me to work on my project on my laptop when I am at school and my PC when I am at home; this gives me more opportunities to work on my project throughout the week. Finally, when I want to test my application by letting other users try it out, I can ask them to simply download my Github repository so that they can play with my web application from their own local PCs.

3.1.2 Cycle 1 Overview

The purpose of this cycle is to finish the entire frontend UI, upload image and export text functionality. This entails the completion of the home page, image upload page and text file export page; with all the colours and formatting completed to create a responsive cohesive layout, that aims to be approachable, intuitive and simple to use for maximum user satisfaction from the aforementioned stakeholders.
As well as being able to click & add or drag & drop various image files; additionally to be able to export the converted text as a word file, PDF, text file or simply copy it to your clipboard.

3.2 Success Criteria

Main Goal: Create all three web pages for the frontend.

Home Page:
- Title
- Button for viewing further instructions on how the application works; has hover features
- Button to start, redirects user to the Input Page; has hover features

Input Page:
- Title
- File upload section:
    - Has option to click to add image files
    - Has option to drag and drop images straight from the file explorer
    - Includes visual feedback to different user actions, such as hovering a dragged file over the drag-drop field
    - Contains an image preview of the files that will be uploaded, along with a corresponding label that displays this files name
    - Validate the files and their size to avoid bugs
    - A button to upload the images and move onto the next page; has hover features
- A button to return to home; has hover features

Output Page:

- Title
- Editable text field
- Input text box for the user to enter the name they would like to give to the file
- Export buttons; with hover features:
    - Button to copy text to clipboard
    - Button to export the text as a PDF
    - Button to export the text as a text file
    - Button to export the text into a word .docx file
- Return to home button; with hover features

3.3 Iterative Development Process

I will organise my files by having an overall frontend folder, then a folder for each web page, with their own HTML, CSS, JavaScript and any other relevant library files as seen in figure 3.3.1:

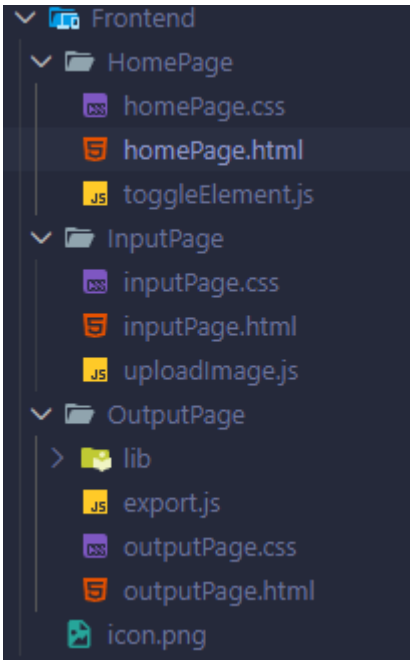


Figure 3.3.1

As I will be using HTML, CSS and JavaScript to create my frontend website, I will be using an extension called LiveServer, which launches a local development server with live reload features for every time I make a change to the actual code; along with having dynamic pages.
It is an extremely useful tool that I have chosen as live reloading saves time by removing the need to manually reload my webpages so I can instantly see the effects and impacts of my code, making debugging faster & much more smoother - enabling more efficient development cycles. Finally, it also simulates a real web server environment, helping ensure the website behaves correctly before deployment.

Furthermore, I have this code at the top of all my HTML files:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">
```

So that the website is properly scaled on different platforms, devices and monitor sizes and so that the website itself is just more compatible with a greater number of various device models.

3.3.1 Home Page

```
<link rel="icon" href="/Frontend/icon.png" type="image/
x-icon">
<title>
    Notes to Text converter!
</title>
```

Figure 3.3.1.1

I started off by adding an icon and title for the actual tab at the top of the web page as shown in figure 3.3.1.1. I sourced the icon image from:
https://www.hiclipart.com/free-transparent-background-png-clipart-hpihm

I did this because it adds more weight and character to the actual webpage itself, as without it, the top tab would feel empty and so would make it simply frustrating to organise. As many people group tabs together and navigate their browsers primarily through the tab bar, therefore having an expressive tab bar is essential for it to be memorable. Furthermore, this can be further broken down by looking at our primary stakeholders - students and teachers - who normally have a multitude of tabs open for their work, therefore a fleshed out tab bar is key.
A picture of the implementation of figure 3.3.1.1 is shown below:

```
body{
    background-color: black;
}
```

Figure 3.3.1.2

I started off with the web page by first turning the background of the homepage to be black, as shown in figure 3.3.1.2. As darkmode/dark pages are often the most preferred by younger generations, furthermore it is especially useful when people might be visiting the website at night. As there will be less of a glare on their eyes and they will take less time to actually focus on the page. Therefore this decision will be especially beneficial as most stakeholders

will most likely be working after dark and will want an application that is easy on the eyes on first launch. The result of figure 3.3.1.2 is shown below:

```
<h1>
    <div class="box">Notes to Text Converter</div>
</h1>
```

Figure 3.3.1.3

In figure 3.3.1.3 I have started off by creating a div, using the box class (inside of the heading 1 tag), this will contain the title of the homepage.
It is further styled with CSS shown in figure 3.3.1.4:

```
.box{
    background-color: dimgray;
    width: 100%;
    height: 200px;
    border: none;

    /*Creates a flex box and centres it*/
    display: flex;
    align-items: center;
    justify-content: center;
}
```

Figure 3.3.1.4

In figure 3.3.1.4, the box classes background colour, width, height and border attributes have been defined; then the display is set to flex so a flex box is created (a flexbox creates a layout where it is easier to arrange objects vertically or horizontally in a “flexible” - responsive - manner) which I can then use to centre the box’s contents by setting align-items and justify-context equal to centre.

```css
h1 {
    color: white;
    text-align: center;
    font-size: 70px;
    font-family: Georgia;
    font-weight: bold;
}
```

Figure 3.3.1.5

Furthermore the actual text is stylised, as shown in figure 3.3.1.5, to be white to contrast the black background, aligned to be in the centre as it is the title and one of the main elements of the page, therefore it has also been boldened and has a large font size.
Moreover, I have chosen the title (and all other text in the entire website) to be in the Georgia font, a serif typeface; as “Georgia is the quintessential serif typeface for reading onscreen. That’s what it was designed for, and that’s what it excels at.” - Adobe Fonts.
Personally, it looks clean, elegant and simple which make it perfect for this web application.

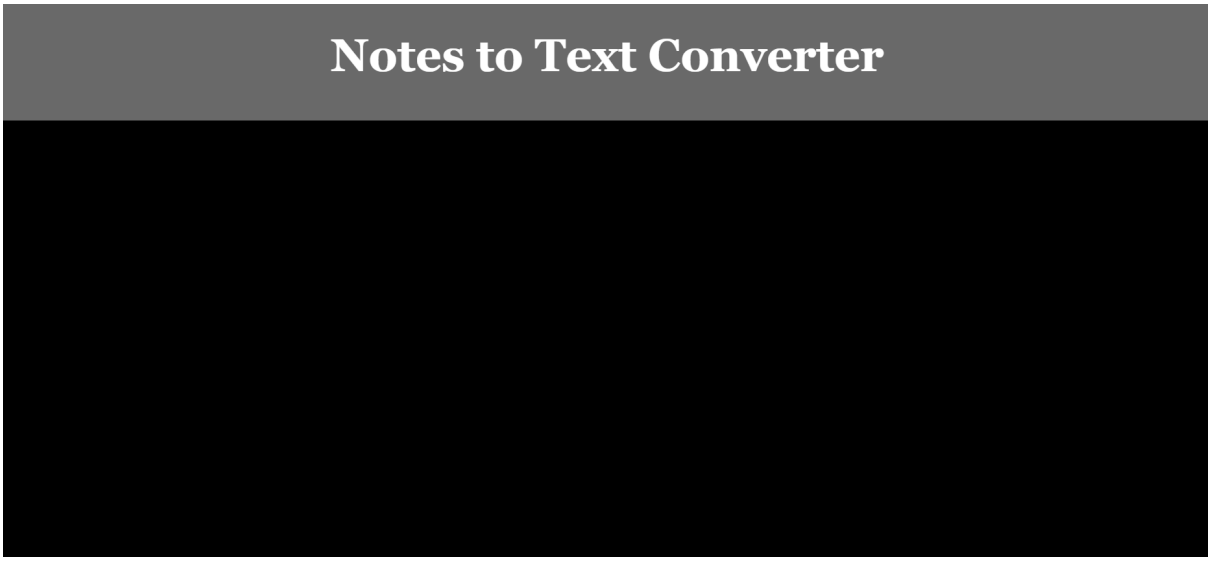


Figure 3.3.1.6

The reason why I have created a box element to hold the title of the homepage (as seen in figure 3.3.1.6) is so that the website is sectioned off, and focus is drawn immediately to the title as the user opens the page. As aforementioned the user interface needs to be simple yet intuitive, especially as it is aimed towards school children with shorter attention spans, information must be communicated effectively and clearly. This box element achieves just that, it separates the title from the other elements to draw attention towards itself.

```html
<button class="button startButton">Start</button>
```

Figure 3.3.1.7

Next as shown in figure 3.3.1.7, I created a simple button with the class button and name startButton, this will aim to redirect the user to the next page when they want to continue forward and start the actual notes to text conversion.

I am using the button class, by making a button template in the CSS - shown in figure 3.3.1.8 - as the base styling for all buttons on the home page:

```css
.button{
    border: none; /*removes any visible border around element*/
    color: white; /*sets text colour as white*/
    padding: 16px 32px; /*adds space inside of the button to make it larger and easier to click*/
    text-decoration: none; /*removes anything like underlines*/
    display: block; /*turn it into a block to make it easier to centre and manipulate as it takes up its own horizontal space*/
    font-size: 25px; /*sets font size*/
    margin: 4px 2px; /*adds sapce outside the element*/
    transition-duration: 0.4s; /*sets the transition dration when hovering*/
    cursor: pointer; /*changes mouse pointer into hand icon to show its clickable*/
    background-color: darkgray; /*changes background colour of the button to dark grey*/
    font-family: Georgia;
}
```

Figure 3.3.1.8

Figure 3.3.1.8 follows the same conventions as the title to retain the continuity and theme of the website, which will ensure all other buttons created in the homepage will automatically have a set default theme, base padding, margin, font size, font family and colour scheme; allowing me to further improve each button individually without needing to constantly redefine the elements needed for the base theme. This improves the modularity of my code and removes any redundant code in my program.

The basic button template which will give a button that looks like this:

Furthermore, to add an element of visual interactiveness, I added figure 3.3.1.9:

```css
.button:hover {
  background-color: whitesmoke;
  color: rgb(0, 217, 255);
}
```

Figure 3.3.1.9

This makes it so that if the cursor is hovering over the button, all buttons that are part of the button class have their background colour and text colour changed to a cyan and white smoke colour respectively, and this entire transition lasts 0.4s from when the user first hovers over the button - as predefined in the button class:

Show Start Button Hover (Homepage).mp4

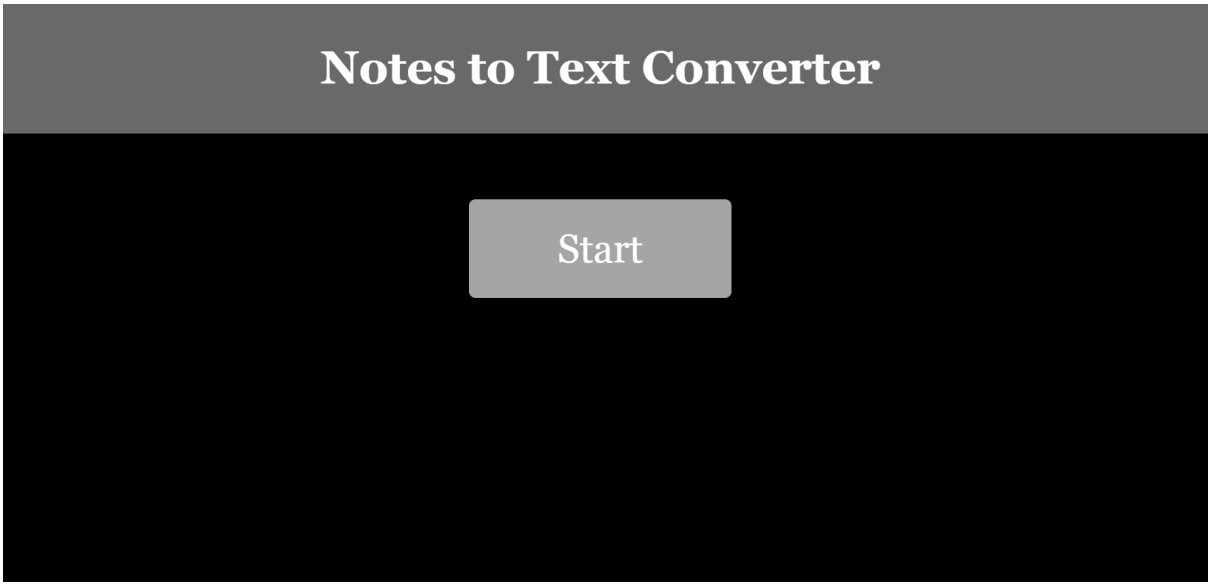
By having the button visually react to the user's cursor the website feels more alive and dynamic and less static, this makes the website seem more approachable and will therefore

appeal more towards the majority of stakeholders, especially to the younger demographic in the student stakeholders who could appreciate a website that reacts to their actions.

```
.startButton{      You, 23 minutes ago • Uncommitted changes
    width: 400px;
    height: 150px;
    border-radius: 10px;
    margin: 0 auto; /*centres the button horizontally*/
    margin-top: 100px;
    font-size: 60px;

}
```

Figure 3.3.1.10

For the start button, in figure 3.3.1.10 I customised its dimensions, rounded its edges in accordance with the previous ideas in my design section regarding softer and more rounder buttons and interface. I centered it horizontally, increased the font size to be almost as large as the title and shifted it to be much lower than the title. Due to the fact there are only 3 main elements, because of this simplistic design, in the Home web page each element will need its own space and attention - so they need to be proportionally large and separated into almost 3 separate sections of the homepage.
The results of the work on the start button can be seen below:



Next, I added a button - as seen below in figure 3.3.1.11- that will give instructions to users on how to properly use the application, along with further guidelines on what they should do to ensure the model outputs the most clear and concise results.

```
<button class="button instructionsButton">Further Instructions<br>⬇</button>
```

Figure 3.3.1.11

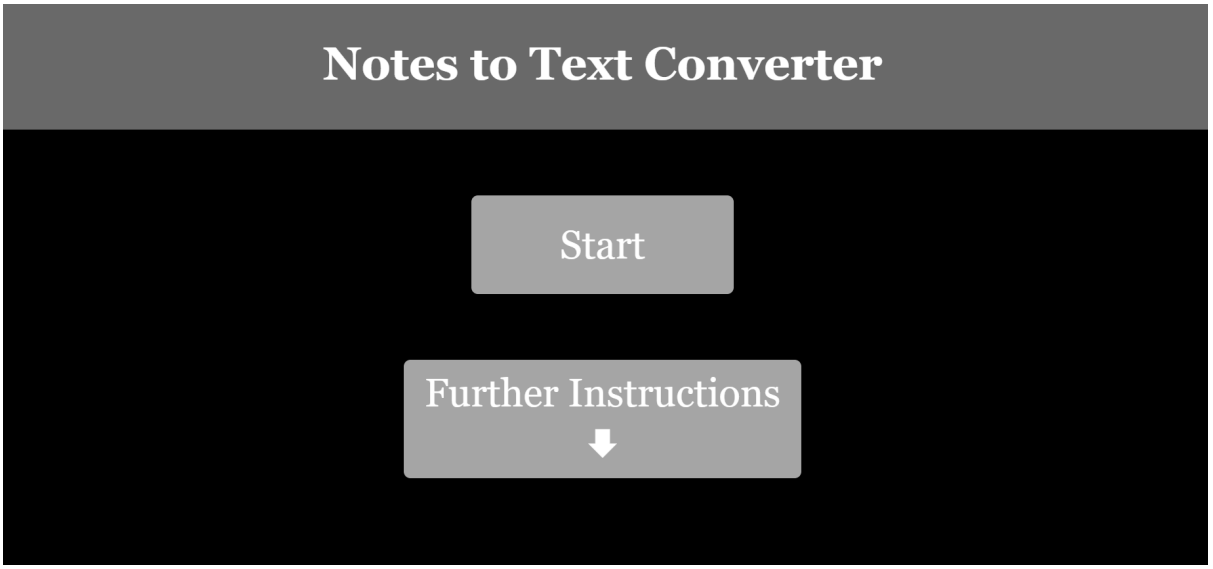
Figure 3.3.1.11 like figure 3.3.1.7 creates a button using the button class, but now with the name instructionButton. Therefore, since it's part of the button class it will contain the original button classes base features, including the hover functionality; this inherently boosts the modular aspects of my codebase and ensures the lack of unnecessary repetition of code.

```
.instructionsButton{
    border-radius: 10px;
    margin: 0 auto; /*centres the button horizontally*/
    margin-top: 100px;
    font-size: 60px;
}
```

Figure 3.3.1.12

In the individual CSS of the instruction button shown in figure 3.3.1.12, It maintains most of the same functionality and style from the button class, however it does not need a resize compared to the start button. However, both the start button and instructions button have the same font size, border radius, margin-top values and horizontal centering; so though it seems to make sense to simplify them by adding them onto the base button class. Yet, I want to keep the button class as it is, in case I need to add more buttons when I am refining the layout of the home page, which might not be centered or that large.

The result of figure 3.3.1.12 is shown below in the current homescreen image:



Now to display the actual text instructions, I will make the Further Instructions button act as a toggle that can toggle the instructions text off and on.
First I created a div, as seen in figure 3.3.1.13, to contain the actual text that needs to be displayed:

```
<div id="text-to-toggle">
    After taking pictures of your handwritten notes simply
    attach them, let them be proccessed and then export the
    digital text
    <br>
    <br>You can send a maximum of 5MB worth of images
    <br>
    <br>Make sure they have clear lighting and are not very
    pixelated
</div>
```

Figure 3.3.1.13

Figure 3.3.1.13 stores the text in a div container with the identifier: text-to-toggle, so we can find and manipulate it within CSS and JavaScript from the HTML file.

In the CSS as seen in figure 3.3.1.14, I attempted to style the toggled text:

```
.textToToggle{
    border: 2px red;
    color: white;
    font-family: Georgia;
    font-size: 50px;
    text-align: center;
}
```

Figure 3.3.1.14

However when I refreshed the website, the homepage would just look exactly the same as the current homepage image shown above; so I thought that maybe it was invisible compared to the background so I gave it a red border - though this resulted in the same result. I then deleted all the css, and saw that the text was there (just black but as my current background colour is black it was hidden), meaning it wasn't actually being stylised by my CSS code. This made me realise beforehand when I was working with classes I only used the . then name syntax, so I did the same with textToToggle as shown in figure 3.3.1.14. This caused the text to not conform to the style I dictated in the CSS, as this - technically a logic error - occurred as the div has an id attribute and not a class attribute, and if you want to reference an identifier you need to use # instead of a period. So while the syntax was not causing any errors, the text wouldn't stylise as I was trying to style a non-existent class in the div rather than calling it via its identifier.

I fixed the code to include the hash as shown in figure 3.3.1.15:

```css
#text-to-toggle{
    color: white;
    font-family: Georgia;
    font-size: 55px;
    text-align: center;
    margin: 4px;
    padding: 20px;
    display: none;
}
```

Figure 3.3.1.15

In the CSS shown in figure 3.3.1.15, I merely added the basic style that has been the current theme of the website and centered it, with a large (but not larger than the title) font size for it's text. Notably I set the display value to none, so that this div doesn't actually appear on screen for now. Rather in the JavaScript file I will manipulate the text to appear when the button is clicked.

```javascript
const divContainer = document.querySelector("#text-to-toggle"); //assigning the html div into this div container to manipulate it
let isClicked = true;

let ToggleDisplay = function(){
    if (isClicked){
        divContainer.style.display = "block";
        isClicked = false;
    }else{
        divContainer.style.display = "none";
        isClicked = true;
    }
}
```

Figure 3.3.1.16

Firstly, in figure 3.3.1.6, the file gets a reference to the actual div using the query selector to search for its identifier. Then it sets a basic boolean value of isClicked to be true, which when the button is pressed starts the: if true set display equal to block and set isClicked to false. This displays the text and reverses isClicked, so next time the button is pressed the: else sets the display equal to none, and sets isClicked to true, hiding the text and enabling an endless cycle of being able to click the button to toggle the instructions text through the simple boolean variable of isClicked
Finally, this function needs to be attached to the actual button, so as seen in figure 3.3.1.17 I iterated over figure 3.3.1.11, by attaching the function ToggleDisplay to the onclick attribute of the button. So the button calls the function every time it is clicked.

```html
<button onclick="ToggleDisplay()" class="button instructionsButton">Further Instructions<br></button>
```

Figure 3.3.1.17

Show Instruction function (Homepage).mp4

Overall, the instruction button exists to complete the homepage, the homepage's purpose is to introduce the user to the web application, it doesn't start off right away with what you need to do and throw you straight into the program. It starts off simple, with a simple title, a set of instructions and then the opportunity to actually use the program.

It lets the user understand and acclimate to the web application - before they actually start to engage with it - to understand what they need to do, and what they can expect out of the application itself. The reason for the instruction button to be a toggle is so that the website isn't too cluttered from the start when the user clicks on and loads into the homepage. It retains the simplistic aim for the website, while still being informative and an adequate guide that users can easily follow - ensuring there are no compromises regarding the quality or quantity of the information.

Overall, for the last addition to the Home Page of this website as shown in figure 3.3.1.18, I iterated over what I had done in figure 3.3.1.7 and added the file path to the input page's HTML file to the start button by adding the local file path to the location.href attribute of the button; therefore once it is clicked, the user will be redirected to the next page, the input page.

```
<button class="button startButton" onclick="window.
location.href='/Frontend/InputPage/inputPage.
html'">Start</button>
```

Figure 3.3.1.18

Show Start Button Functionality (Homepage).mp4

3.3.2 Input Page

To maintain continuity within the website the input page also has the same code as the home page that relates to the inclusion of the icon and title for the tab bar, as shown in figure 3.3.1.1. Along with the black background code shown in figure 3.3.1.2, creating a black canvas exactly like the one at the start of the home page construction, for the input page to follow a relatively consistent set theme.

```
<main>
    <h1>Upload Image</h1>
</main>
```

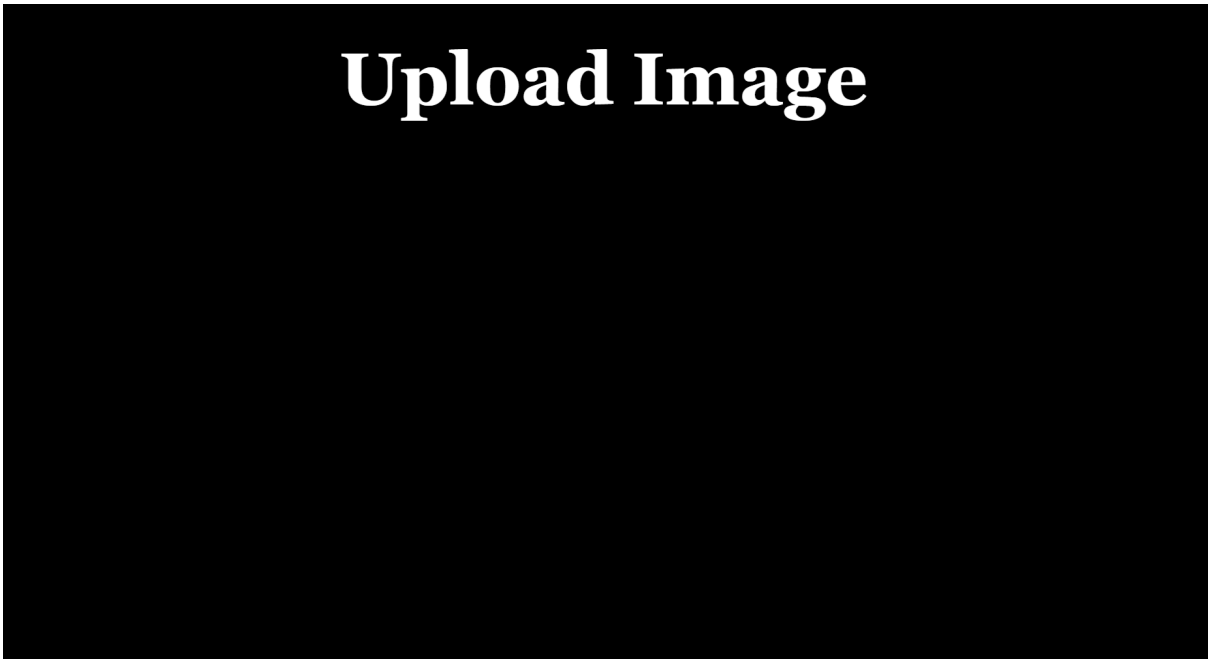
Figure 3.3.2.1

Similar to figure 3.3.1.3, figure 3.3.2.1 just sets up the title of the input page, however it is not separated by a box like the home page. As I want all the elements of the input page to be unified and not split up, because the user is now actually starting to use the application, so everything needs to be seen as a singular cohesive section. This includes the: title, upload file section and process button. So there is no point in sectioning the title for the input page.

```css
h1{
  color: white;
  text-align: center;
  font-size: 90px;
  font-family: Georgia;
  font-weight: bold;
  margin-top: 50px;
  margin-bottom: 100px;
}
```

Figure 3.3.2.2

I have styled figure 3.3.2.1 using the CSS from figure 3.3.2.2, using the same theme from the homepage for continuity. But I made this title larger, with greater margin sizes compared to 3.3.1.5 and I then boldended the title, due to the absence of the box. Since I still want the title to be clear, loud and noticeable without sacrificing the unity of the components.
The result of figure 3.3.2.2 is shown below:




```html
<div class="file-upload" id="drop-zone"> </div>
```

Figure 3.3.2.3

Next, for the upload section of the input page, as shown in figure 3.3.2.3 I created a div with the class file-upload and identifier drop-zone. So I could think of all the elements that would be required for an upload section, and then add all the elements needed for it. This includes: an upload button, drag and drop box, error message display, progress bar for image upload & processing time and an area to preview images before they are uploaded. And since all of them are in the file-upload div class, I can manipulate them all together and simply group

them all together to make it easier to manipulate the group itself and then the individual divs that contain their own elements - creating a useful modular environment.
This is the main structure of the file-upload div, to firstly group all elements in a single div class and then individually create a div container for each element so that they can then group up and store their own elements. Overall this nested div structure decomposes the entire problem, by having an overall drop-zone where I can easily separate each sub problem and deal with them individually.

```
<div class="file-upload" id="drop-zone"> <!--Overall div for the file-upload section-->
  <input id="fileInput" type="file" accept="image/*" multiple/> <!--Creates the input form, for multiple image files-->
  <div id="file-preview"></div> <!--Creates a DIV where I can append all the file previews-->
  <div id="progress-bar"></div> <!--Creates a DIV where I can manipulate a progress bar-->
  <div id="error-message"></div> <!--Creates a DIV where I can display error messages-->
</div>
```

Figure 3.3.2.4

After listing the various elements I wanted in the upload section I improved on figure 3.3.2.3, turning it into figure 3.3.2.4, first I added the actual input element that will take any type of file as an input, this is further refined by telling it to only accept image files and then that it can take many inputs at once - so it can accept multiple files to upload, to help save time for users that would need to process many files that all together are less than the accepted max size. Next is another div called file-preview, this will be the container that will contain the file previews of the files the user has uploaded. I continued to use this same logic to create the progress-bar and error-message divs, where they will all be separate containers that will contain and show their respective elements. Using the format of grouping all of these elements in a single div, that represents all uploading operations, makes everything easier to handle and manipulate; especially by having more individual divs inside of the overall container. Everything is properly decomposed into individual steps and criterias that all need to be fulfilled to ensure a proper file-upload section, overall this reduces the complexity of the problem - making it easier to tackle and solve.

```
.file-upload{
  width: 50%;
  font-family: Georgia;
  font-size: 25px;
  display: block;
  padding: 20px;
  border: 2px dashed white;
  border-radius: 8px;
  transition: border-color 0.3s;
  gap: 15px;


  /*To centre elements in the div*/
  display: flex;
  justify-content: center;
  align-items: center;

  /*To centre the div itself*/
  margin: 0 auto;
}
```

Figure 3.3.2.5.1

In figure 3.3.2.5, I then styled the overall div class of file-upload. The attributes of the div I have set at each line: firstly, I set the width of the div to be 50% of the screen as it is the main mechanism of this page so it needs to be big and have plenty of space for the file names and images for the file preview, or the error messages and loading bar; then I set a medium font-size as it doesn't need to be too big due to the file previews; it is displayed as a block so it is its own separate element on its row; it has 20px of padding so it doesn't feel like that it's elements are all smushed together despite it's large dimensions; it then has a dashed border to signify that you can also drag and drop files in this inferred drop zone; then it is slightly rounded due to the previous designs I created in the design section; it will have a transition of the border colour - of 0.3 seconds - which will change the colour when you hover an image over the drag and drop section for the user to have more visual interactivity in the website; and then finally it'll have a gap of 15px - so that the box has a little gap between the bottom and the where the input form is - to show the user that something will happen in that space (having predefined space for the image previews) so it feels natural when they upload their images, which are then being displayed as previews so they can double check they have appended the right files. In the next section I have added the display flex code (as seen before in figure 3.3.1.4) to ensure all elements inside of the div are centred, while in the next section the margin: 0 auto ensures the actual div itself is centered.

Finally, in preparation for when I add drag and drop functionality I added more visual interactiveness as shown below in figure 3.3.2.5.2, which adds the class dragover in file-upload's class list. So that when a file is being hovered over the drop zone, the border turns from white to a lighter blue, and the background colour becomes the lightest blue with an opacity (alpha) of 0.1 to make it a little bit see through, giving a tint effect.

```
.file-upload.dragover {
  border-color: rgb(96, 96, 244);
  background-color: rgba(0, 123, 255, 0.1);
}
```

Figure 3.3.2.5.2

However, this won't result in a visible output until I start writing the javascript code.

Therefore the results of figure 3.3.2.5.1 are shown below:

This shows the upload box, though the “choose files" button is not centered it is also not even compatible with the theme of the website. It looks old, bland and thoughtless, while other elements around it seem to blend well together. Furthermore, the problem is that if I were to go to the css of #fileInput and attempt to change the text colour and etc. it wouldn’t change it as it is a file input form, so as shown below in figure 3.3.2.6 if I change the input tags colour it only changes the selected file name’s colour and not the button itself (this text is also responsible for its weird centering)

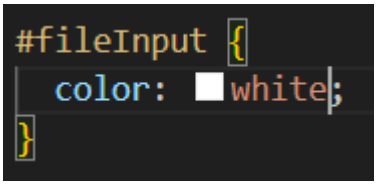


Figure 3.3.2.6

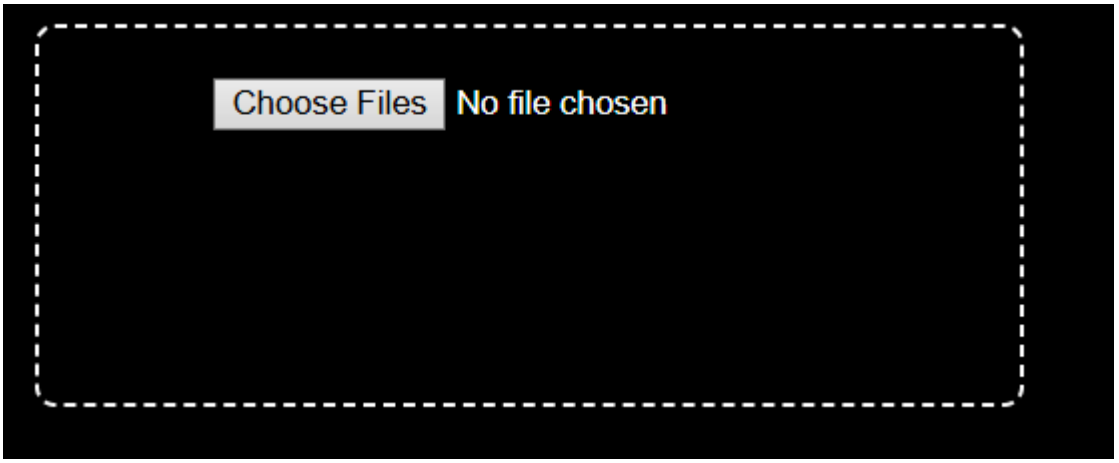


Figure 3.3.2.7

The result of figure 3.3.2.6 is shown above in figure 3.3.2.7, only the chosen file text colour is changed; and overall it is aesthetically unpleasant, especially within the theme of my website.

Therefore to make sure all elements conform to the predefined style of the website I decided to just discard this input form UI and create my own. I started off by removing the display of the current UI as shown in figure 3.3.2.8.

```
#fileInput {
  display: none; /*Hides the ugly upload button so we can add our own custom UI*/
}
```

Figure 3.3.2.8

I then iterated over figure 3.3.2.4 and added a label to the HTML elements to act as the "Choose Files" text from the previous UI, as shown in figure 3.3.2.9.

```
<div class="file-upload" id="drop-zone"> <!--Overall div for the file-upload section-->
  <input id="fileInput" type="file" accept="image/*" multiple/> <!--Creates the input form, for multiple image files-->
  <label for="fileInput">Select Images or Drag & Drop</label> <!--Creates a caption for the file input-->
  <div id="file-preview"></div> <!--Creates a DIV where I can append all the file previews-->
  <div id="progress-bar"></div> <!--Creates a DIV where I can manipulate a progress bar-->
  <div id="error-message"></div> <!--Creates a DIV where I can display error messages-->
</div>
```

Figure 3.3.2.9

Then as seen in figure 3.3.2.9 I added the label tag, which uses the for attribute to attach the tag to fileInput, this will actually show the text "Select Images or Drag & Drop" on the input form button - replacing the previous "Choose Files" button into a customisable text. Furthermore, this custom text is also more effective at appealing for our stakeholders, as it is clear and concise on how and what the users need to do. It tells them they have the option to drag and drop files - a functionality not all websites include, meaning that this could be a feature they can appreciate due to its added convenience - and that they need to input images files specifically. This would avoid mistakes such as user selecting pdfs or word files that have an attached image to it, which would result in an error message and frustrate users as they would need to try again (which takes them out of the flow of using the application and can increase chances of them closing the tab and moving on).

```css
.file-upload label {
  display: inline-block;
  padding: 10px 20px;
  background-color: rgb(96, 96, 244);
  color: white;
  cursor: pointer;
  border-radius: 5px;
  transition: background-color 0.3s;
}

.file-upload label:hover {
  background-color: blue;
}
```

Figure 3.3.2.10

Next in the CSS code seen in figure 3.3.2.10), I stylised the label tag, I made it an inline block as I want it to be inline with any other elements I add to the custom file input element and not be by itself in a separate row. I then added some basic padding, made it blue as a placeholder colour, with white text to keep in theme; moreover since it is a button, I made the mouse become a pointer if you hover over it, then I made it a bit rounded due to the aforementioned reasons in my design, and I then added the basic transition time for when you hover over the label. This is then shown in the second function where the background colour becomes a darker blue when you are hovering over it for greater visual interactiveness.

The current custom button looks like:
Show Hovering Upload Button (Input Page).mp4

```css
#file-preview {
  margin-top: 20px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

#file-preview img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  border-radius: 4px;
  margin: 0 auto;
}
```

Figure 3.3.2.11

After stylising the file upload button, I decided to stylise the file previews that will be displayed (as seen in figure 3.3.2.11). I added 20px to the top of the margin so that the pictures are evenly spaced apart from each other if multiple are added, and also it provides

space for the first preview against the file-upload label. Then I created a flex box by setting display equal to flex and making the flex direction equal to column, this means that all items that are appended to the flex box are listed downwards as a column rather than side by side in a single row. As a list (column) of all of the files you have appended not only looks and feels more natural to see, but it is also easier to implement as you can simply append a huge amount of images vertically down the web page and extend the web page proportionally to the number of images. This would only increase the amount the user would need to scroll and would not negatively affect the website at all; however if it was listed horizontally then the images would need to be squished if too many were added, else using a mix of rows and columns adds unneeded layers of complexity to allow the website to function dynamically. Therefore the use of a column listing ensures that all information is retained and not lost through forced compression, while also looking natural. Finally, the gap property defines a gap between the flex box columns; this works on top of the margin attribute to make sure each file preview has its own space of 30px.
The next function customises the actual image that is appended to the flex box itself, I set the dimensions to be a square, while the attribute object-fit: cover ensures that the original file preview fits into the square by: ensuring the aspect ratio is preserved. While, when the image is fit to fill the container any excess parts of the image are cut, this properly preprocesses the image into a proper format before it is shown on the website. Furthermore the border radius is set to 4 pixels so it is rounded by a bit, and the margin is set at 0 and auto so that the image itself is going to be centred in the file upload box.

Before I started to work on the javascript so that users could actually upload files, I realised that since the backend isn't finished yet I have no actual way to estimate or utilise the progress bar currently. Also, that it made more sense for it to be outside of the overall upload-file div and it should probably be between the upload-file div and the upload images/proccess images button. So I reiterated over the HTML code and moved the progress bar div outside of the div as shown in figure 3.3.2.12.

```
<div class="file-upload" id="drop-zone"> <!--Overall div for the file-upload section-->
  <input id="fileInput" type="file" accept="image/*" multiple/> <!--Creates the input form, for multiple image files-->
  <label for="fileInput">Select Images or Drag & Drop</label> <!--Creates a caption for the file input-->
  <div id="file-preview"></div> <!--Creates a DIV where I can append all the file previews-->
  <div id="error-message"></div> <!--Creates a DIV where I can display error messages-->
</div>

<div id="progress-bar"></div> <!--Creates a DIV where I can manipulate a progress bar-->
```

Figure 3.3.2.12

Finally, I started off the javascript for the upload file section by first calling all of my HTML elements and storing them as constants so that they can't be redeclared or updated, as I only want to manipulate their attributes & properties and not influence the main instantiating HTML code, in this script (as seen in figure 3.3.2.13):

```
// creates variables for all the html elements so I can manipulate them
const dropZone = document.getElementById('drop-zone');
const fileInput = document.getElementById('fileInput');
const preview = document.getElementById('file-preview');
const errorMessages = document.getElementById('error-message');
```

Figure 3.3.2.13

Then, I created a new variable called fileStack shown in figure 3.3.2.14.1, to create a stack that will contain all the uploaded files, and will be the data container used by the frontend to send all the user uploaded files to the backend.

```
let fileStack = [];
```

Figure 3.3.2.14.1

```
// Enable drag and drop functionality with visual feedback
dropZone.addEventListener('dragover', (e) => {
  e.preventDefault(); //prevents the website to handle the drag and drop event normally so we can handle the event ourselves
  dropZone.classList.add('dragover'); //activates the css class element dragover to change the colour of the button when the user drags over a file
})
```

Figure 3.3.2.14.2

I started off by starting to add the drag and drop features, as shown in figure 3.3.2.14.2, I added an eventlistener to the drop-zone div. It is then told to listen for the event “dragover” (when something is dragged over it) and store this event as e. Then once the event is actually detected I will prevent the default handling of a drag and drop event (which for a website is to open the dropped file in a new tab) so that the file can actually interact with my website - instead of defaulting to interacting with the web browser as a separate webpage. Finally, I enable the previously seen dragover CSS attribute of the dropZone (figure 3.3.2.5.2) to change the colouring of the drop box itself - this basically replaces a buttons hovering feature but for this drag and drop box; this is so that I can maximise all opportunities to include visual interactiveness to appease to my primary stakeholders.

```
dropZone.addEventListener('dragleave', () => {
  dropZone.classList.remove('dragover'); //removes the dragover element when nothing is being dragged over the drop zone
})
```

Figure 3.3.2.15

Furthermore, since I added dragover to the class list of the drop zone in figure 3.3.2.14.2 I need to remove it once there is nothing being dragged over the drop zone (i.e. The user contemplates dragging over a certain file, but changes their mind and instead of dropping it they simply put it back in their file explorer), this is what it implemented in figure 3.3.2.15. When the dropZone hears that the “dragleave” event has occurred, it will remove the dragover attribute from the classList, so that the file hovering colour change is reversed.

```
dropZone.addEventListener('drop', async (e) => { //when item is dropped it will execute, it is async so the rest of the website works asynschrously while the file is being uploaded, e is the event object
  e.preventDefault(); //prevents the broswer from accidently opening the file
  dropZone.classList.remove('dragover'); //removes the dragover class property as the user has dragged and dropped the file
  await HandleFiles(Array.from(e.dataTransfer.files)); // calls the dropped files, converts it into a js array and sends it to HandleFiles func, await makes it so that everything waits for the file upload to finish before executing anything else
})
```

Figure 3.3.2.16

Next for figure 3.3.2.16, we listen for the event when the user actually drops the file into the drop zone. Firstly we prevent the default reaction of the website as we did before in figure 3.3.2.14.2. Then we make this an asynchronous function as I don’t want the entire website to hang if the files aren't handled correctly, this makes it so that all the website features work and are interactable while all the files are being handled. This decouples the actual file management from the regular website and reduces the number of dependencies the website relies on so it can immediately respond to user actions. Moreover, the drop event adds

another case for removing the dragover attribute, since when the actual file is dropped on the drop zone we don't need the hovering colour change. Finally, in the last line I use the await keyword to freeze the function for now so that the Handlefiles() function can be executed before this function ends while the rest of the application can still be interacted with in a different thread. So in the actual handleFiles function, I pass in the events target files, convert it into a JavaScript array and send it as a parameter to the function which will handle these files.
This completes all the logical operations needed for how the drag and drop box interacts with the users inputs.

```
//when user uses file picker
fileInput.addEventListener('change', async (e) => {
  await HandleFiles(Array.from(e.target.files)); //same logic from drog and drop, but when it uses file picker
})
```

Figure 3.3.2.17

Now on the other hand, if the user decides to use the file picker (I implemented figure 3.3.2.17), we simply need to wait for the event: "change", then send the converted files (which are converted into a JavaScript array) as the parameters for HandleFiles function and then just wait for the function to execute. As there is no default case to look for or any visual changes to reset or set.

Below shows the tested result of adding the upload feature, which includes the drag and drop function and file selector function:
Upload file (Input Page)
Moreover, as seen in the video file above you can see that when you use the file picker you can also attach images in your clipboard, which is a feature that users on computers would highly appreciate.

```
//handles the file preprocessing and preview
async function HandleFiles(files) {
  //preproccess
  preview.innerHTML = ''; //resets any previous html preview content
  errorMessages.innerHTML = ''; //resets any previous error messages
  fileStack = []; //reset current file stack to avoid confusion with previous attempts

  //Validate, append & induce preview
  for (const file of files) { //loop through every file in the file array inputed to the function
    const isValid = await ValidateFile(file); //bool isVaild is the result of called function that will validate files, use await since we want this to be finsihed before moving on
    if (isValid) {
      fileStack.push(file); //appends it into the files stack
      const reader = new FileReader(); //lets the script read the contents of the file to get the preview
      reader.onload = (e) => CreatePreview(e.target.result, file.name); //when file reader finsihes, callback func creates preview with the data url and filename
      reader.readAsDataURL(file); //start reading the file as a data url, which lets it access the file type, encoding and actual file data, this kick starts reader.onload
    }
  }
}
```

Figure 3.3.2.18

In the actual HandleFiles function (as seen in figure 3.3.2.18), I made it asynchronous as I want to decouple the file handling from the actual website. Then I first preprocessed my file preview and error messages html elements by removing any previous unnecessary preview values or error messages in their HTML, and I even reset the current filestack to avoid any mix ups with previous attempts. Then for every file in the inputted parameter files, it will loop through each file and: validate them by calling an external ValidateFiles function (which freezes the HandleFiles function due to the use of await, to wait until the validation is complete before continuing with the function); then if it is valid it'll push the file to the actual file stack; create a constant variable called reader which will have a new file reader to let the

function gain access to the contents of the file, so it can make it into a preview; then the onload property will contain an event handler that will be executed when the onload event is called when content is read with readAsDataURL in the next line, this lets the reader have access to the actual file data and kick starts the onload event which will fire up the create preview function and will pass through the image source and file name to the preview function.

```
//validates files before adding them to the file stack
async function ValidateFile(file) {
  const validTypes = ['image/jpeg', 'image/png', 'image/gif'];  //set of acceptable image types
  const maxSize = 5 * 1024 * 1024 //5 MB limit in bytes
  let isVaild = true; //base bool so that if nothing is wrong it simply returns true

  //checks file type
  if (!validTypes.includes(file.type)) {
    ShowError(`${file.name} is not a valid file type`);
    isVaild = false;
  }

  //checks file size
  if (file.size > maxSize) {
    ShowError(`${file.name} is too large and exceeds the 5MB limit`);
    isVaild = false;
  }

  //returns if valid or not
  return isVaild;
}
```

Figure 3.3.2.19

The first function that is called in HandleFiles() is ValidateFile (as seen in figure 3.3.2.19), it is also an asynchronous function, due to the aforementioned reasons, that takes a file as a parameter. The function starts off by creating a list of all the different image types the website will ‘support’ or accept, right now it has the obvious JPEG and PNG files along with GIFs (but these will be further evaluated after the backend is actually realised). Now since the image size is stored in bytes I have an integer constant maxSize value of 5MB (in bytes) for each file, with a max number of files most likely being set after the backend is made so that I can actually check the limitations of the model later on. Finally for the last variable I simply have a boolean value that is set to true, and if any validation checks fail it will be set to false so that then the entire function returns false, else it will output true to signify that the file is the right size and file type. Now in the actual if statements, if the file type isn't the correct one or if the file size exceeds the maximum file size, it will call the ShowError() function and pass on the error message as a parameter then set isValid = False as it failed a test.

```
//shows error message
function ShowError(message) {
  const error = document.createElement('div');  //creates div container to contain the error
  error.textContent = message;  //the error message equals the given parameter
  errorMessages.appendChild(error); //appends error as a child into the actual error messages element in the html
}
```

Figure 3.3.2.20

The ShowError() function (as seen in figure 3.3.2.20) has a message string as its parameter, and defines a div container for this error message, and sets the divs content equal to the parameter, then it adds this div which contains the error message to the actual errorMessages div; this creates a chain of error messages that are in their own separate div being appended to this overall div container that will contain all the error messages, this creates an effect similar to the column list of preview images by having a list of individual error messages pop up beneath the previous one - and in this case mechanically it is extremely useful to help with separating error messages from each other while also giving the user the ability to discern which message is the most recent one (acting as almost like a log).

Figure 3.3.2.21

After finishing the ShowError() function I wanted to further stylise the actual appended error messages, so I added to my CSS file as shown in figure 3.3.2.21. I first set the display to be a block since I want each error message to stack on top of each other for greater readability. Then I made the text colour be red to clearly indicate to the users that this text is a consequence of something going wrong, especially since red is the signature colour for errors making it the best colour to use in this case. Finally, I added some gaps between each error message by having a margin-top of 10 pixels, this differentiates each message and further improves readability.

Below is the result of testing whether the max size limit would work:
Show Max Size Error Message (Input Page).mkv
While this video file shows the result of trying to drag and drop a non image file (this is not tested with the file picker as the file picker will only let the users select image files as that is the supported type):
Show File Type Error Message (Input Page).mkv

```
//creates preview
function CreatePreview(source, id) {
  const container = document.createElement('div'); //creates the actual div container where the preview will be shown
  container.className = 'preview-item'; //name the div class preview item, as it only stores a single preview item

  const img = document.createElement('img');  //creates an image
  img.src = source; //it sets the img source equal to the inputted image URL, so that when its appended to the html it knowns what the image actually is
  img.alt = id; //if the image cant be shown for some reason the id acts as the alt text thats shown if that occurs

  const fileName = document.createElement('span'); //create span for the lable of the image
  fileName.textContent = id;  //displays image name by setting the span equal to the image id

  //append img and span to the container and then append the container as a child to preview (actual html element) so it can be displayed
  container.appendChild(img);
  container.appendChild(fileName);
  preview.appendChild(container);
}
```

Figure 3.3.2.22

The final function called in HandleFiles - as shown in figure 3.3.2.22 - is the CreatePreview function which takes in the files source and the files id (name). It first creates a div container, then calls its class preview item - since the div will only store a single image preview. Then we create an image element in the actual HTML, then set its source URL equal to the inputted source param to let the HTML knows the contents of the image and so that it actually knows what to display; then we set the images alt text equal to it's id, so if the image fails to render in any way the file name is displayed. Next we need to actually label the uploaded image, so we create a span HTML element and set its text content equal to the image's id. Finally, the actual image and its span (fileName) is appended to the container which is then appended to the preview div in the main HTML as its child. This decomposes the main preview field div into multiple separate divs that will contain and display a single image image and name, listing each image as a separate div in a column fashion - due to the preconstructed flexbox in figure 3.3.2.11.

```
#file-preview span{
  /*for the image text displayed below the text*/
  font-family: Georgia;
  color: white;
  font-size: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

Figure 3.3.2.23

After creating the CreatePreview() function I realised I hadn't stylised the span element itself, so in the CSS I appended figure 3.3.2.23. Firstly I followed the basic themes of my website and set its font family, colour and size; then I centered it using a flex box as basically all of my elements are centred due to its aesthetic appeal.

When I tested the images previews, my result looked like this:

The select button and the image preview were skewed, and I was getting a logic error where even though the image preview and select buttons were separate flexboxes they were still grouping together.

I fixed this by reiterating the code in figure 3.3.2.5.1:

```
.file-upload{
    position: relative;
    width: 50%;
    display: block;
    padding: 20px;
    border: 2px dashed #ccc;
    border-radius: 8px;
    transition: border-color 0.3s;

    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 15px;      You, now • Uncommit

    margin: 0 auto;
}
```

Figure 3.3.2.24

The image preview was on the side of the upload button but not below it, the problem was that I used a flex container on the parent file-upload so its child file-preview has its flex box affected and grouped by the parent flex container. So I needed to make them not group with each other as a row, hence I used flex-direction: column to make the file previews go down and list down a column. This still has the parent and child div's flexboxes grouped together, but now the elements in the parent flex box in figure 3.3.2.24 are displayed as items in a

column, which means I can manipulate the children as if they were separate blocks that I could centre and such.
The result is shown below:

For images with short names this looked sufficient, however when I tested further with images that contained longer names, the image became uncentered whilst the text remained unchanged - as seen below:

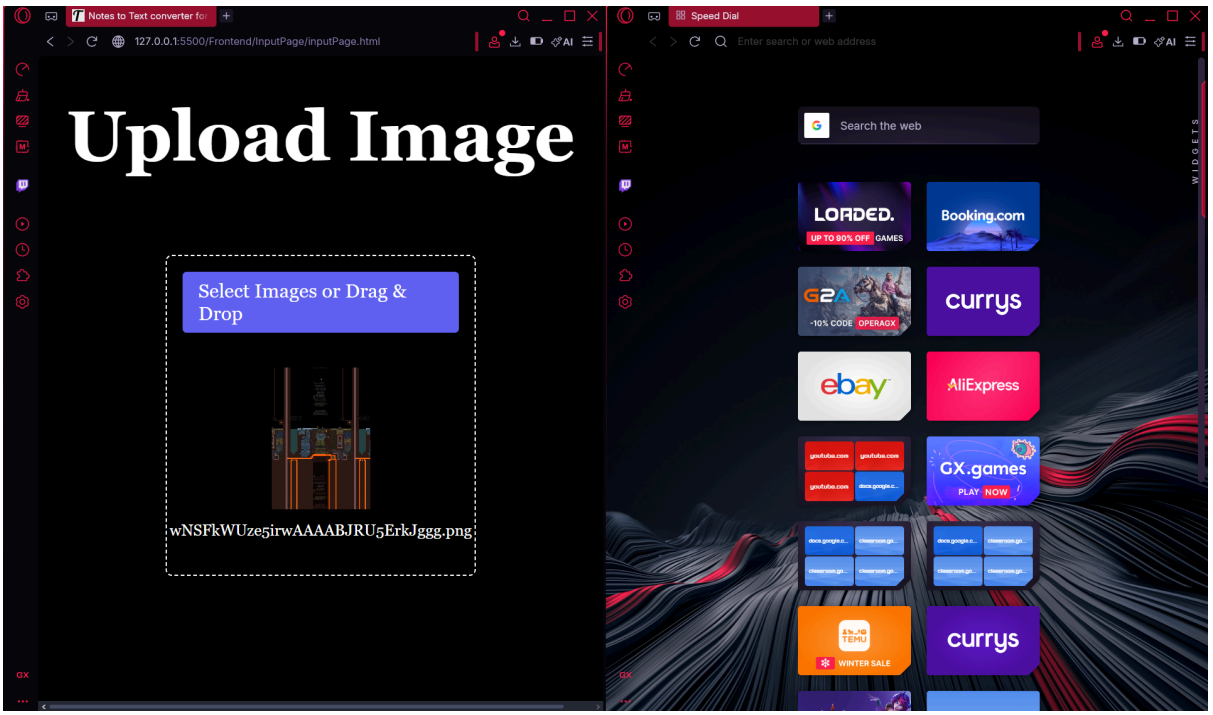
I fixed this bug by improving upon figure 3.3.2.11 as seen below in figure 3.3.2.25:

```css
#file-preview img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  border-radius: 4px;
  margin: 0 300px;
}
```

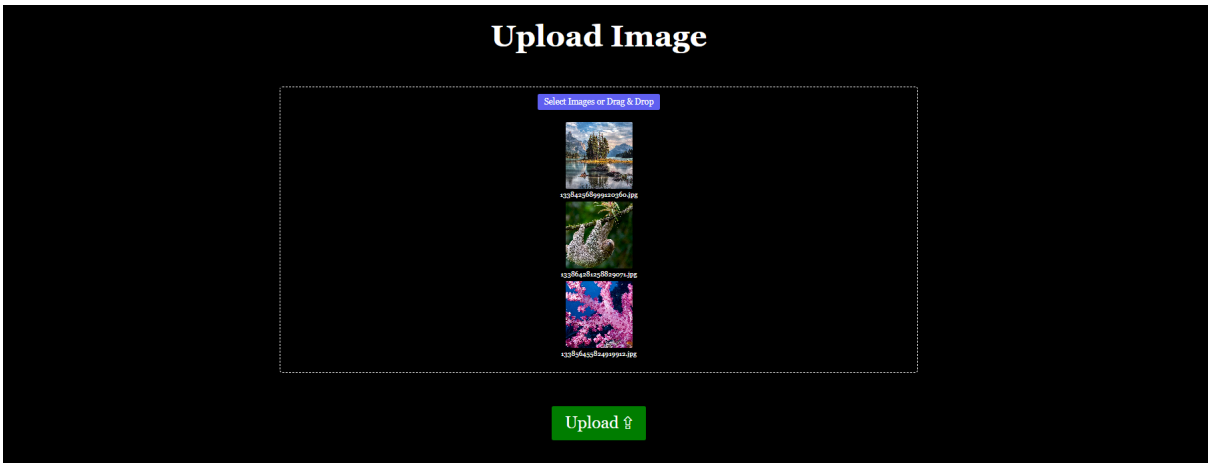
Figure 3.3.2.25

I replaced the auto in margin: 0 auto; with 300px, by having an insanely large number (really anything above 300 works) as the side margins cause the image to automatically become centered. Since it causes both sides of the image to be spaced 300px away from the border, but since the length of the margin exceeds the length of the border the image just goes to the center by going the max margin away from the border at each side equally. This was the result of some trial and error working with the margin values to achieve this solution, this is shown in the picture of the result below:

Additionally, even in split screen this solution holds (passing another test in proportions) as shown below:



Additionally, it also works perfectly when multiple images have been selected (the image below is a zoomed out version) as seen below:



Next, I added an upload button to the web page HTML as seen below in figure 3.3.2.26:

```
<button id="uploadButton">Upload ⇧</button>
```

Figure 3.3.2.26

When the user clicks on the upload button the aim is for it to then send all their uploaded files to the backend for processing, while the frontend is still on the Input page with a progress bar that is roughly synchronised with the progress occurring within the backend.

```css
#uploadButton {
  display: block;
  padding: 20px 40px;
  background-color: green;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.3s;
  font-size: 50px;
  font-family: Georgia;

  margin: 100px auto;
}

#uploadButton:hover {
  background-color: darkgreen;
}
```

Figure 3.3.2.27

In the CSS I stylised the button from figure 3.3.2.26 using the code in figure 3.3.2.27. I first set its display to equal block, so that it is a separate element below the file-upload box; then I set its padding to create some blank space between the text and the actual button so it looks larger. Then I made the text white, made the button more rounded, let the cursor become a pointer when it hovers over the button (since it is a button, it acts as an indicator that the button can be pressed), kept the usual transition duration for when the cursor is hovering over the button and kept the same Georgia font family - overall most of these were just standard configurations to keep the button in theme. Then I made the font size much bigger, and centered it horizontally, but placed it lower vertically from the file-upload box; because this upload button is a major part of this page, it needs to be big and green as defined in the CSS. I want it to be a bit more vibrant and something that seems highlighted by the website, which is why it is green for now, with its hovering colour being a darker shade of green.
In the actual implementation of the website, it looks like:

Finally, I wanted to add a button for the user to use to return back to the homepage (which would be very useful if the user wanted to return to the homepage to view the application instructions).

```
<button class="button button1" onclick="window.location.href='/Frontend/HomePage/homePage.html'">⬅ Return To Home</button>
```

Figure 3.3.2.28

As seen in figure 3.3.2.28 I started off by adding a button element in the HTML, with a href link to the file location of the home page HTML file. I added a ⬅ symbol to appear more friendly to the user by adding more character to my buttons.

```
.button{
  border: none;
  display: block;
  color: white;
  padding: 16px 32px;
  font-size: 25px;
  margin: 0 auto;
  transition-duration: 0.4s;
  cursor: pointer;
  background-color: darkgray;
  font-family: Georgia;
}

.button:hover {
  background-color: whitesmoke;
  color: rgb(0, 217, 255);
}
```

Figure 3.3.2.29

Since this button links back to the homepage I wanted it to have the same button theme as the home page, so I appended the button and button hover style to the Input Page style sheet as seen in figure 3.3.2.29. Most of the attributes shown in figure 3.3.2.29 are similar to figure 3.3.2.8 and figure 3.3.2.9, since I want this button to be a similar theme to the home page so that the user can just subconsciously associate that button with the home page. While for the input page I just modified some padding and the margin since it is a new webpage with different elements of varying dimensions.

The return to home button was tested and it works perfectly as shown below:
Show return to home button (Input Page).mkv

The addition of this last button completed all of the UI elements for the Input page, as shown below:

Since I had added the upload button to the frontend, I realised that there needs to be another validation check for ensuring an actual file is selected before the upload process occurs, since we don't want to send the backend an empty stack of files that might cause a multitude of errors or bugs within the model. Therefore in the JavaScript I added another function called ValidatePresence(), shown in figure 3.3.2.30.

```
//checks if the file stack is empty before upload, if so returns an error message so the website doesnt process an empty set of files
function ValidatePresence() {
  if (fileStack.length == 0) {
    ShowError('Please select the files to upload');
  }
}
```

Figure 3.3.2.30

In figure 3.3.2.30 I first check if the file stack is empty, if it is then I show an error to inform the user that they haven't actually uploaded any image files to the website.
After defining the function, the next step is to link it to when the upload button is clicked. So I added a reference to the upload button from the HTML file for in JavaScript by appending to figure 3.3.2.13, as shown below in figure 3.3.2.31

```
// creates variables for all the html elements so I can manipulate them
const dropZone = document.getElementById('drop-zone');
const fileInput = document.getElementById('fileInput');
const preview = document.getElementById('file-preview');
const errorMessages = document.getElementById('error-message');
const uploadButton = document.getElementById('uploadButton');
```

Figure 3.3.2.31

Then I added an event listener to the upload button in figure 3.3.2.32, so it will listen for when the button is clicked and execute the ValidatePresence() function.

```
uploadButton.addEventListener('click', ValidatePresence); //assigns the upload button the validatePresence function on click, so if no file is selected, it shows an error
```

Figure 3.3.2.32

So I then tested the result of what would happen if there was no file attached to the webpage, shown in the video below:

Validate Presence Error Message (Input Page).mkv

If there are no files attached then any error will be produced, which stacks if they continuously reattempt it. But it then disappears if any image file is uploaded.

3.3.3 Output Page

As I have expressed previously within the start of the documentation of the Input page, I have started out by setting out a black background colour, tab icon and name to continue the theme of the website.

Furthermore, I started off by adding a heading to the output page (as seen in figure 3.3.3.1) in a similar fashion to the title of the input page since the output page also needs a sense of unity between all the various elements that it will hold.

```
<h1>Converted Digital Text</h1>
```

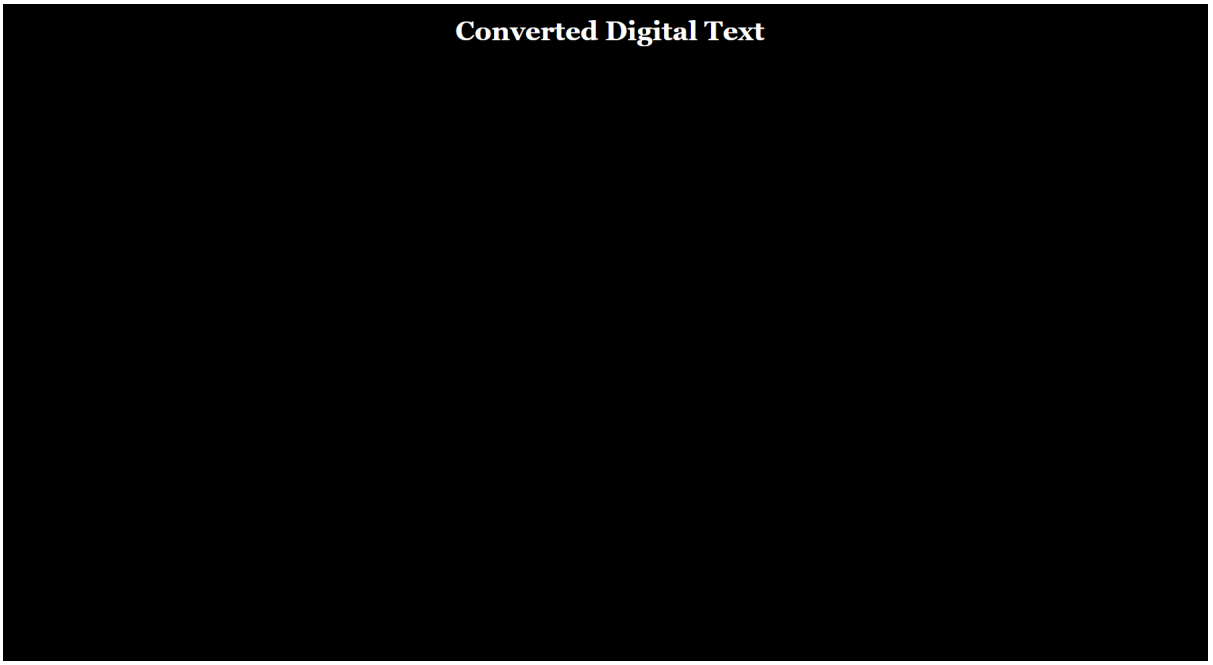
Figure 3.3.3.1

Then I stylised figure 3.3.3.1 using the CSS from figure 3.3.3.2 to centre the title and change its colour and font to match the predefined style. However, in this case I did not increase the font size to give it the extra space like I have given to my previous titles; this is because in the output page, the majority of the screen needs to be dedicated to the editable text field where the converted text will be stored and displayed for the user to edit if and when necessary. Therefore I have not inflated the size of the heading, to instead give room for the text field while letting the title just blend in with the text field.

```
h1{
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-family: Georgia;
}
```

Figure 3.3.3.2

The result is shown below:




```
<textarea id="textToExport" autocorrect="on">Lorem ipsum dolor sit, amet consectetur
  adipisicing elit. Enim porro doloremque nobis dolore amet eum reiciendis libero obcaecati
  deleniti, corporis maiores neque natus quis sequi deserunt perspiciatis quisquam eaque
  pariatur!
</textarea>
```

Figure 3.3.3.3

Next in figure 3.3.3.3, to create an editable text field to display the processed text, I decided to use a textarea element which has its autocorrect attribute set to on for a greater quality of life for the user, and I have defined it with an identifier called textToExport. Currently I have the basic Lorem ipsum placeholder text as a placeholder until cycle 2 is finished and the backend is integrated with the frontend.

```
textarea{
    width: 60%;
    height: 500px;
    display: block;
    margin: 0 auto;
    margin-bottom: 25px;
    font-family: Georgia;
}
```

Figure 3.3.3.4

Then I stylised the text area in the CSS in figure 3.3.3.4, I first set its width to equal 60% of the user's entire screen and its height to be 500px; not only because it is the main part of the output page but I also want it to be big and broad enough that it can contain as much of the converted text as possible before the user needs to scroll. So the user feels content enough with the result of the application, especially after waiting for the website to finish processing

their images. Next, I set its display as block since it needs to be a separate element on it's row, then I centred it using margin, then I added 25 pixels to the bottom margin to give some space between the buttons and the text field, then I made it so that the text in the text area is set to the Georgia font.

Editiable Text Field (Output Page).mp4

The result of the implementation of the output screen is shown in the recording above, here the text field can be extended downwards quite a bit with its own scrollbar - making it quite useful for longer texts.

```
<div class="export-buttons-container">
<button class="button copyB" id="copyB" title="Copy to clipboard">📋 Copy</button>
<button class="button docxB" id="docxB" title="Download as a Word (.docx) file">📄 Export .docx</button>
<button class="button pdfB" id="pdfB" title="Download as a PDF">📕 Export PDF</button>
<button class="button txtFileB" id="txtB" title="Download as a text file">📩 Download .txt</button>
</div>
```

Figure 3.3.3.5

Additionally, after creating the editable text area I focused on creating the major export buttons that will actually transfer the text content of the text area as a downloadable file or something that the user can copy to their clipboard. In figure 3.3.3.5 I started off by defining an overall div with class name export-buttons-container to store and group up all my export buttons. Then one by one I created buttons with the overall class buttons and an abbreviated ID of their respective exportation methods (e.g. copyB for copy button). Then I edited their title attribute to add text to the tooltips that would be displayed if the user hovered over a button, this provides users with extra instructions on how to process if anything at all is unclear to them at any point in the output page - this increases the quality of life for the user by giving them extra guidance. Finally in the actual text content of the buttons I added emojis for each export type from this website:https://emojidb.org (using the list of emoji results from when I searched up pdf and document in the website), in my opinion this adds even more character and personality to the website. While also adding more clarity to the buttons themselves to further improve the users experience.

```
.export-buttons-container{
    display: flex;
    align-items: center;
    justify-content: center;
}
```

Figure 3.3.3.6

I then centred the div containing all of the buttons by using a flex box and then aligning the items and justifying the content as seen in figure 3.3.3.6.

```css
.button{
    transition-duration: 0.4s;
    cursor: pointer;
    font-family: Georgia;
    width: 100px;
    height: 40px;
    border-radius: 5px;
    margin-right: 20px;
}
```

Figure 3.3.3.7

Afterwards, I stylised the overall button class for the export buttons as seen in the CSS code of figure 3.3.3.7. The use of the button class is to simply define the attributes of all the buttons instead of having to define it for all of them, which decreases any redundancies in the codebase, and makes it easier to simply modify the button group as a whole.
In terms of defined attributes, I defined some basic attributes, such as the transition duration to the regular 0.4 seconds, to turn the cursor into a pointer when hovering over the button, the font family to Georgia font, the basic dimensions (which are lower than my previous buttons in the Home and Input page because I want the editable text field to be the main focus of this page, however the buttons will have their own appeal through their hovering colours), a slightly curved border radius and finally I have added 20 pixels of space on their right margin so that they are evenly spaced out from each other - since they will all be in a single row in the flexbox.

```css
.copyB:hover{
    background-color: blueviolet;
}

.docxB:hover{
    background-color: blue;
}

.pdfB:hover{
    background-color: red;
}

.txtFileB:hover{
    background-color: yellow;
}
```

Figure 3.3.3.8

As seen in figure 3.3.3.8 I have set the copy button's hover colour to be blue violet, this is because I want every button's hover colour to be more vibrant and reflective of its export type. As aforementioned, having elements display even minimal visual reactiveness is very important to me and crucial for the overall design of the website; therefore I have set the

copy to clipboard button to be set to blueviolet (basically light purple) as the other buttons have more clear colours that can be associated with, while the copy to clipboard button is more miscellaneous so (light) purple is a good fit as it is a more unique colour. Furthermore, I have set the hover colour of the .docx button to be blue. As the logo and theme of Microsoft Word is blue, so it seems sensible and reasonable for all users to associate blue with the export to word button. Next, for similar reasons the hover colour for the PDF button is set to red due to Adobe Acrobat's colour theme of red, this links in with my reasons from choosing blue as the colour for the .docx button as it is a sensible but primarily recognisable choice for that particular file type. Finally for the text file button I quite arbitrarily chose the colour yellow as the hover colour, it is already a quite vibrant colour and white - for blank text files - seemed to be bland (and conflicting as it is the predefined background colour for all the buttons). Therefore I chose yellow as the highlighted colour.

The result of creating the export button UI is shown in the video below:
Show export buttons (Output Page).mkv
Above, the layout of the export buttons seems ordered and not bugged, furthermore the tooltips themselves are appearing perfectly and don't obstruct much of the actual buttons themselves.

Now it is time to actually implement the exportation process in the JavaScript of the Output Page.

```
const textArea = document.getElementById("textToExport");
const copyButton = document.getElementById("copyB");
const docButton = document.getElementById("docxB");
const pdfButton = document.getElementById("pdfB");
const txtButton = document.getElementById("txtB");
```

Figure 3.3.3.9

I start off by defining all the variables I will be calling from the HTML in figure 3.3.3.9, by making a reference to the editable text field and all the exportation buttons.

```
//when user updates the text area it updates the content text
textArea.addEventListener("input", (e) => {
    textArea.textContent = e.target.value;
    console.log("Live Text: ", textArea.textContent);
})
```

Figure 3.3.3.10

But before I actually start exporting the text in the editable text field, I need to constantly update the text content of the text area (since for now whatever the user types into that text field will not actually change the textContent attribute of the text area element). So I added an event listener to the text area that will listen for an input event, and when the event occurs

it will change the textContent attribute to equal the target events value (what the user has typed in to update what the overall string will become). Everytime this occurs, I have added a console.log() message to notify me of the new message (this was used to debug the text area and test if the event listener was actually working).

As seen below, in the website live log the text areas content is being constantly stored and updated.
Show debugging of text area (Output Page).mkv

```
//Copy to clickboard
async function copyToClipBoard(){
  await navigator.clipboard.writeText(textArea.textContent);
  console.log(textArea.textContent);

}
```

Figure 3.3.3.11

After clearing up the text areas textContent, I created the asynchronous function shown in figure 3.3.3.11, as I don't want the website to stall if an error occurs while the textContent is being copied to the users clipboard (plus the process of overwriting the clipboard with the text content is a simple process where I can use the await keyword), while exporting as a file would need to be dedicated process that happens synchronously with the rest of the websites elements. So, I used await (since it's asynchronous) and called the navigator (the user) and wrote the textContent text into their clipboard. Then I used the console.log to log the text content when the copy button is used, so I can test if it is actually working.

I then modified figure 3.3.3.5 as seen below, to include the onclick attribute of the HTML button so I could add the coptToClipboard() function to it.

```
<button class="button copyB" id="copyB" onclick="copyToClipBoard()" title="Copy to clipboard"> Copy</button>
```

Below, I have tested the copy button on my local device and it works stunningly within the console log and the actual copying to the clipboard.
Show Copy button functionality (Output Page).mkv

```
//export as a .txt file
const exportAsTxt = () =>{  //use arrow func as we want to "return" the file to the user implicitly, instead of utright returning it
    const link = document.createElement("a");   //create HTML link element which will be attached to the button
    const file = new Blob([textArea.textContent], {type: "text/plain"}); //get text and add it to the plain-text blob (file like object to represent data not native to js
    link.href = URL.createObjectURL(file);  //add the blog to the href attribute of the link, by creating a URL link that points to a given file (blob in this acse)
    link.click();   //so that the file is saved on the local computer
    URL.revokeObjectURL(link.href); //remove the file link at the end to reset the downloadable text file, ready for the next time they edit the editiable text field
}
```

Figure 3.3.3.12

Then in figure 3.3.3.12 I started to work on the exportation to a text file, I define the exportAsTxt function as an arrow function as I want to return the file to the user implicitly (since it's a file they will be downloading) rather than outright. I start of by creating a HTML

link element, then I defined a variable called file which will store a blob - which is a file like object used to represent non-JavaScript native data such as this text file - that will have the contents of the textArea and have its type be text/plain, which is a plain text file. I set the links href attribute equal to the blob file (which is converts the blob into a URL which the href can direct the user to); then I call the method link.click() so the file is saved on the local computer, and then at the end I reset the text export button by removing the original file link so that this function can be called again with a new text content in the editable text field. This basically creates a new link element and a new blob - which will store the text file - and convert the blob into a link attached to the link element so that when the link is clicked it will download the file locally, and then remove the URL in preparation for a new text content.

So then I simply added the exportAsText() function to my buttons onclick event as seen in the modified code for figure 3.3.3.5, shown below:

```
<button class="button txtFileB" id="txtB" onclick="exportAsTxt()" title="Download as a text file">  Download .txt</button>
```

Then I tested the function and got the result as shown below:
Export as text file test (Output Page).mkv

Now this wasn't what I wanted, the file itself is stored as a blob text file as we can see in the URL of the video above, but it is being opened as a file in the browser rather than something being downloaded as a separate text file.
So after some googling, I realised I actually need to also change the links download attribute to give the file a name so the computer will know it is a downloadable link. Which means I need to actually let the user add the name of the file they want to download; which in retrospect feels obvious as it gives the user direct control over the exportation process so they don't need to change the name of the file manually in their file explorer after every download.

```
<label for="exportFileName">File Name: </label>
<input type="text" id="exportFileName">
```

Figure 3.3.3.13

So then I created a label and an input form in figure 3.3.3.13. The labels for the attribute is linked to the labels ‘exportFileName’ identifier so it knows that it's a label for the text input element - and I have just added a clear indication for the user to enter their chosen file name. And of course I have set the labels type to be a text form, as the name of the file will just be a short text input.

```css
#exportFileName{
    margin: 0 auto;
    margin-top: 5px;
    display: flex;
    align-items: center;
    justify-content: center;
}

label{
    margin-top: 20px;
    color: white;
    font-family: Georgia;
    display: flex;
    align-items: center;
    justify-content: center;
}
```

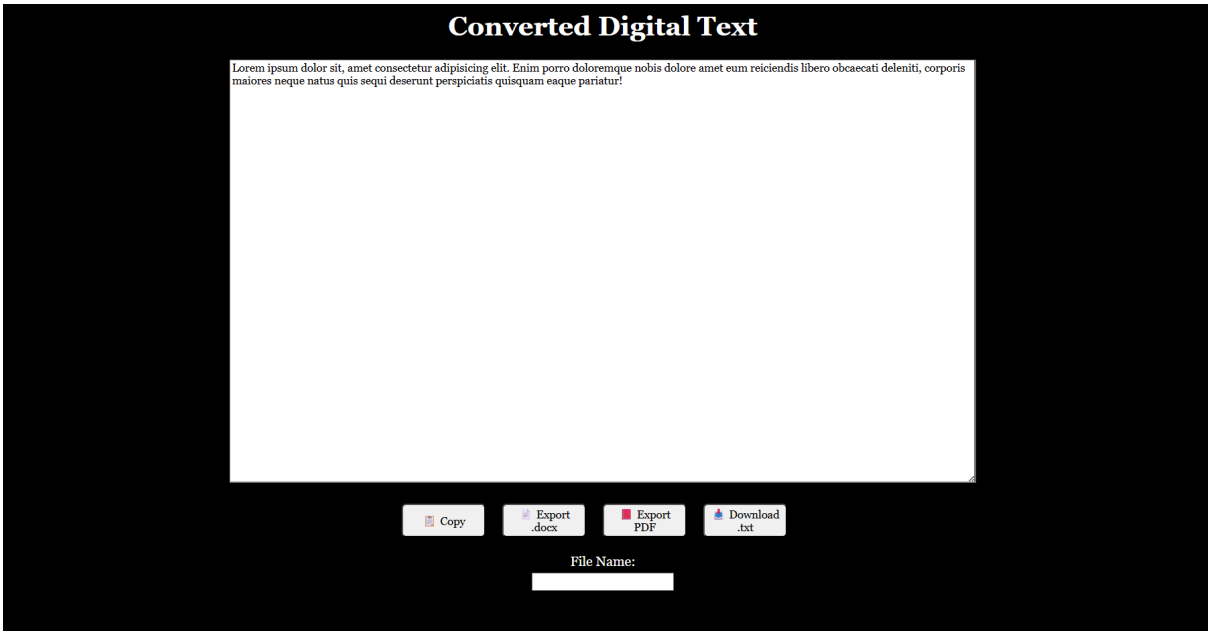
Figure 3.3.3.14

Then in figure 3.3.3.14 I have stylised figure 3.3.3.13, by first centering the input box, giving it some space from the label (by 5 pixels), then I have margin: 0 auto; and a centered flexbox at the same time because for some reason it won't centre the input box unless both are there (even though it's not in any divs). I don't know why but it works and isn't too redundant so it is fine.

Proof of centering failure:
Centering Input Form Nightmare (Output Page).mkv

Then in terms of the label style, I have added a buffer between it and the export buttons just to give it some space; then I have just set some basic attributes, like setting it's colour to be white, and it's font to be Georgia, and then just centering it using a flexbox.

It gives a result of the following:



So then finally in the JavaScript, I have appended figure 3.3.3.9 by adding another reference to the input field for the files name, as seen in figure 3.3.3.15:

```
const textArea = document.getElementById("textToExport");
const copyButton = document.getElementById("copyB");
const docButton = document.getElementById("docxB");
const pdfButton = document.getElementById("pdfB");
const txtButton = document.getElementById("txtB");
const fileName = document.getElementById("exportFileName")
```

Figure 3.3.3.15

```
//export as a .txt file
const exportAsTxt = () =>{  //use arrow func as we want to "return" the file to the user implicitly, instead of utright returning it
    const link = document.createElement("a");   //create HTML link element which will be attached to the button
    const file = new Blob([textArea.textContent], {type: "text/plain"}); //get text and add it to the plain-text blob (file like object to represent data not native to js like text files)
    link.href = URL.createObjectURL(file);  //add the blog to the href attribute of the link, by creating a URL link that points to a given file (blob in this acse)
    link.download = `${fileName.value}.txt`; //adds the name of the file to the download attribute so it knows this is a downloadble link
    link.click();   //so that the file is saved on the local computer
    URL.revokeObjectURL(link.href); //remove the file link at the end to reset the downloadable text file, ready for the next time they edit the editiable text field
}
```

Figure 3.3.3.16

Additionally, I have also modified the code in figure 3.3.3.12 as seen in figure 3.3.3.16 to include the modification of the link's download attribute to set the files name (and in extension file type).

Below is the final text for the export as text button, it flawlessly exports the modified text content with the correct file name; and as an added layer of protection against errors, as seen in the video when I did a presence check on the file's name it did not stop the functionality and rather just exported the text as txt.txt. In my opinion this is more beneficial for users than outputting an error, especially if users don't particularly care about what their files are named, and if they are or if they simply didn't notice the file name form. As they can then easily fix it without getting broken out of the flow of the application with an unnecessary

error. Therefore I hope to repeat this when the user wants to export a file as a word document and a PDF.
Export as text file functionality (Output Page).mkv

However, there is no straightforward way to export a PDF in JavaScript, so I used an external library called jsPDF; using an external library to convert my text into a PDF not only saves me time on writing my own script, testing and then implementing it but also provides me with a predefined codebase that will be more efficient and up to a greater professional standard compared to my A-Level Computer Science programming skills. So I followed the documentation and added an external script link to my HTML code in figure 3.3.3.17 using a content delivery network (CDN) hosted by cloudflare to link the github repository to my codebase.

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/4.0.0/jspdf.umd.min.js"></script> <!--Importing JS to PDF library-->
```
You, 11 minutes ago • Uncommitted changes

Figure 3.3.3.17

```
const {jsPDF} = window.jspdf;  //reference the global object so i can call upon the source library
// Default export is a4 paper, portrait, using millimeters for units
const doc = new jsPDF();
```

Figure 3.3.3.18

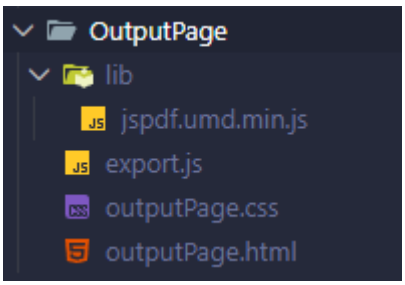
In figure 3.3.3.18, I have implemented the function txtToPDF that will convert the text content into a PDF. I started off by defining a jsPDF variable that will reference the global jspdf object as a global window object. Then a new jsPDF() element to actually create the PDF - here I mainly followed the basic implementation documentation that was given on their Github repository to write this code.
Then as I did with the exportAsTxt(), I modified figure 3.3.3.5 and added this function to the PDF buttons onclick attribute as seen below:

```
<button class="button pdfB" id="pdfB" onclick="txtToPDF()" title="Download as a PDF">  Export PDF</button>
```

However when I ran the code in figure 3.3.3.18, when I clicked the export to PDF button, nothing happened; so after double checking if anything was wrong with my button not calling the function (where as before I used a console.log to check if it was working, and it was not the problem) I then thought if the script was loading correctly. So I went into the network section of the inspect element of my website and I was given the below error:

jspdf.umd.min.js	(failed) net::ERR_BLO...	script	outputPage.html:29	0.0 kB	585 ms

This meant that the network wasn't allowing the CDN to load the script, this was probably due to the firewall blocking it directly. So I needed to use an alternative solution, as I saw it inefficient to find a method to enable the CDN to work on not only my firewall but also all my client browser's firewalls. Therefore I simply copied the only file I needed (jspdf.umd.min.js, as seen in the actual network log it's the only file being called) and made a local library to store that file, as seen below:



And then modified figure 3.3.3.17 into figure 3.3.3.19:

```
<script src="lib/jspdf.umd.min.js"></script> <!--Importing JS to PDF locally stored library-->
```

Figure 3.3.3.19

This solution is not only more time effective but also causes no compromise between the quality of the produced PDF.

Moreover, when I ran the code in figure 3.3.3.18 again, I got the following error:

```
Uncaught TypeError TypeError: Cannot destructure property 'jsPDF' of 'window.jspdf' as it is undefined.
```

So I assumed, since I used the documented code for calling the file through a CDN script I am calling jspdf incorrectly. So after sifting through the jsPDF codebase I scrapped figure 3.3.3.18 into figure 3.3.3.20 as seen below:

```
//export as a pdf
function txtToPDF(){
    const doc = new window.jspdf.jsPDF(); //creates a new PDF document
    doc.text(textArea.textContent, 10, 10); //supply the text content to the PDF and write it at position 10,10 -
    so top left corner of the page
    doc.save(`${fileName.value}.pdf`);  //define PDF file name
}
```

Figure 3.3.3.20

Here I define a new jspdf.jsPDF() to actually create a new PDF document in the client window, then I append the textArea's content into this document and set the appended texts base position to be 10,10 (top left corner of the document); and finally save this document in the users browser with their given file name.

I tested figure 3.3.3.20 as seen below:

Export to PDF bugged (Output Page).mp4

At first everything seems to be going swimmingly, however after I actually open up the PDF all of the text is just appended to a singular line…

So I went back to figure 3.3.3.20 and modify the code - into figure 3.3.3.21 - to split the text area's content into separate lines that I will then append into the document.

```
//export as a pdf
function txtToPDF(){
    const doc = new window.jspdf.jsPDF(); //creates a new PDF document
    const lines = doc.splitTextToSize(textArea.textContent, 180); //split the doc into lines, each line is 180
    units long (basic PDF size), so each line is split by 180 units
    doc.text(lines, 10, 10);  //add the text to the doc at position 10,10 - so at the top left corner of the PDF
    doc.save(`${fileName.value}.pdf`);  //define PDF name
}
```

Figure 3.3.3.21

So I created a new line variable in figure 3.3.3.21 and split the textArea's content into lines - each being 180 units - for the doc. Then I write the lines in instead of the text content directly.

This results in the video below, where everything seems to go according to my original vision:
Export to PDF fixed functionality (Output Page).mp4

```
<script src="https://unpkg.com/docx@7.1.0/build/index.js"></script> <!--Importing JS to Word library-->
```

Figure 3.3.3.22

Similarly I have also used an external library in figure 3.3.3.22 for the txtToDocx function - for the aforementioned reasons when I was talking about jsPDF. Here I have used: https://github.com/dolanmiu/docx, I followed the documentation to add the script to my project.

```
//export as a word docx
function txtToDocx() {
  console.log("BUTTON CLICKED") //check if the button is linked to the func
  const doc = new docx.Document({ //create a new word document
    sections: [ //specficy the doc attributes
      {
        properties: {}, //no additional properties needed for our needs
        children: [
          new docx.Paragraph({  //create a new paragraph element thats a child of the overal document
            children: [
              new docx.TextRun(textArea.textContent), //then append the actual text content as a child of the paragrapgh
            ]
          })
        ]
      }
    ]
  });

  docx.Packer.toBlob(doc).then((blob) => {  //use the docx packer to create a doc blob to store the doc file so it can then be downloaded
    const link = document.createElement("a"); //create a new link element
    link.href = URL.createObjectURL(blob);   //convert the document blob into a link that will point to the downloadable docx file
    link.download = `${fileName.value}.docx`; //set the document name to tell broswer its downloadable
    link.click();   //to save file on local computer
    URL.revokeObjectURL(link.href);   //to reset the button for a new text content
    console.log("Document created and downloaded!");  //debug to check if the blobification actually ocurs
  });
}
```

Figure 3.3.3.23

Then I just looked through the documentation mainly and a little of the codebase for figure 3.3.3.23. Where I have a console.log message for when a button is clicked, then I create a new document that has no extra properties, a paragraph child which contains the textArea's content. Then using a blob - akin to what occurred with exportAsTxt - to create a link that is linked to the URL of the 'blobified' document, which is saved to the users computer (with link.click() method) and then resetting the URL for the next button press. And at the end I have just added another console log to make sure the end of the doc to blog function occurs.

I then modified figure 3.3.3.5 and added the function to the docx button by defining the onclick attribute.

```
<button class="button docxB" id="docxB" onclick="txtToDocx()" title="Download as a Word (.docx) file">📄 Export .docx</button>
```

This gave me the perfect result in the video below:
Export as docx functionality (Output Page).mp4

Finally to finish off the final webpage in my frontend, I need to add a button so the user can return back home and then repeat this entire cycle again if they wish to process more apps.

```
<button id="returnButton" onclick="window.location.href='/Frontend/HomePage/homePage.html'">Return to Home</button>
```

Figure 3.3.3.24

So I made figure 3.3.3.24 and appended to the end of my HTML file. I set its href link to take the user back to the homepage HTML and named the button returnButton.

Then I stylised the button in the CSS in figure 3.3.3.25

```
#returnButton{
    width: 250px;
    height: 80px;
    border-radius: 15px;
    display: block;
    margin: 0 auto;
    margin-top: 30px;
    background-color: darkgray;
    transition-duration: 0.4s;
    cursor: pointer;
    font-family: Georgia;
}

#returnButton:hover{
    background-color: whitesmoke;
    color: rgb(0, 217, 255);
}
```

Figure 3.3.3.25

I set its dimensions to be larger than 2 export buttons as it will be a singular button in a row at the very end of the webpage. I made it more curved than the other buttons since it's on it's own and it's bigger so I need to make it more curved as compensation; finally I set it's display to be a block so it's alone on its row, then I centre it with the margin, then I space it away from the input field above it to give it some space, afterwards I simply set some of the basic attributes for the rest of the return button and make its hover qualities equal to that of the home page to thematically link the button to the homepage.

The return home button works amazingly and all the buttons in the frontend all link in with each other:
Return home button functionality (Output Page).mkv

Once I finished the entire output page I went over everything and checked to make sure everything was commented if necessary and all naming conventions were followed. However when I looked at the JavaScript code for the Output page all functions were in camel case instead of pascal case. So I changed all function names using the inbuilt ctrl+F2

control to change all references of the functions and converted them into pascal case, as seen below:

```
//Copy to clickboard
async function CopyToClipBoard(){
    try{
        await navigator.clipboard.writeText(textArea.textContent);
        console.log(textArea.textContent);
    } catch (err){
        textArea.textContent = err
        console.error("Failed to copy: ", err);
    }
}

//export as a word docx
function TxtToDocx() {
  console.log("BUTTON CLICKED") //check if the button is linked to the func
  const doc = new docx.Document({ //create a new word document
    sections: [ //specficy the doc attributes
      {
        properties: {}, //no additional properties needed for our needs
        children: [
          new docx.Paragraph({  //create a new paragraph element thats a child of the overal document
            children: [
              new docx.TextRun(textArea.textContent), //then append the actual text content as a child of the paragrapgh
            ]
          })
        ]
      }
    ]
  });

  docx.Packer.toBlob(doc).then((blob) => {  //use the docx packer to create a doc blob to store the doc file so it can then be downloaded
    const link = document.createElement("a"); //create a new link element
    link.href = URL.createObjectURL(blob);   //convert the document blob into a link that will point to the downloadable docx file
    link.download = `${fileName.value}.docx`; //set the document name to tell broswer its downloadable
    link.click();   //to save file on local computer
    URL.revokeObjectURL(link.href);   //to reset the button for a new text content
    console.log("Document created and downloaded!");  //debug to check if the blobification actually ocurs
  });
}


//export as a pdf
function TxtToPDF(){
    const doc = new window.jspdf.jsPDF(); //creates a new PDF document
    const lines = doc.splitTextToSize(textArea.textContent, 180); //split the doc into lines, each line is 180 units long (basic PDF size), so each line is split by 180 units
    doc.text(lines, 10, 10);  //add the text to the doc at position 10,10 - so at the top left corner of the PDF
    doc.save(`${fileName.value}.pdf`);  //define PDF name
}

//export as a .txt file
const ExportAsTxt = () =>{  //use arrow func as we want to "return" the file to the user implicitly, instead of utright returning it
    const link = document.createElement("a");   //create HTML link element which will be attached to the button
    const file = new Blob([textArea.textContent], {type: "text/plain"}); //get text and add it to the plain-text blob (file like object to represent data not native to js like text files)
    link.href = URL.createObjectURL(file);  //add the blog to the href attribute of the link, by creating a URL link that points to a given file (blob in this acse)
    link.download = `${fileName.value}.txt`; //adds the name of the file to the download attribute so it knows this is a downloadble link
    link.click();   //so that the file is saved on the local computer
    URL.revokeObjectURL(link.href); //remove the file link at the end to reset the downloadable text file, ready for the next time they edit the editiable text field
}
```

```
<div class="export-buttons-container">
<button class="button copyB" id="copyB" onclick="CopyToClipBoard()" title="Copy to clipboard">📋 Copy</button>
<button class="button docxB" id="docxB" onclick="TxtToDocx()" title="Download as a Word (.docx) file">📄 Export .docx</button>
<button class="button pdfB" id="pdfB" onclick="TxtToPDF()" title="Download as a PDF">📕 Export PDF</button>
<button class="button txtFileB" id="txtB" onclick="ExportAsTxt()" title="Download as a text file">📥 Download .txt</button>
</div>
```

Afterwards, since I have tested all of this on my own local machine I got a few of my friends to test this website live - since I am using a live server. And when they tested it live everything would work except for the copy button, but while it worked for me it did not work at all for anyone else that was using a separate computer to copy and paste.

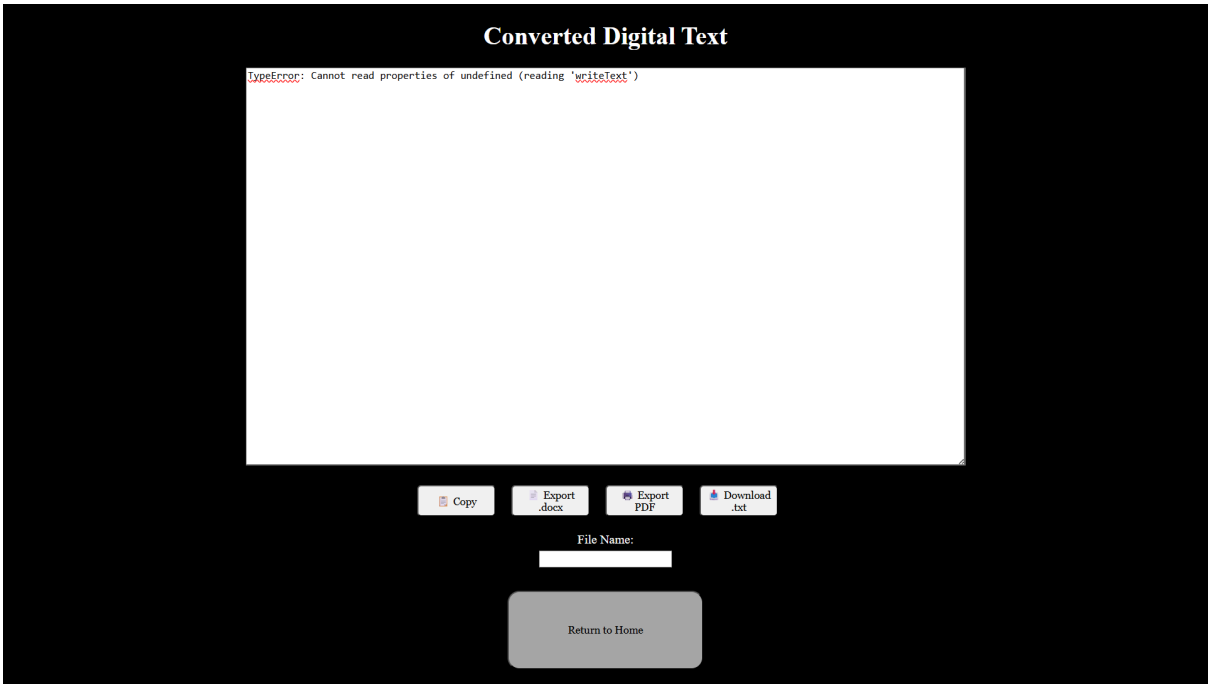
So I went back to my JavaScript function in figure 3.3.3.11 and used a try except loop to output the error if one occurs when the user tried to use the copy button, as seen below in figure 3.3.3.26:

```
//Copy to clickboard
async function copyToClipBoard(){
    try{
        await navigator.clipboard.writeText(textArea.textContent);
        console.log(textArea.textContent);
    } catch (err){
        textArea.textContent = err
        console.error("Failed to copy: ", err);
    }
}
```

Figure 3.3.3.6

In figure 3.3.3.6 if an error occurs then it will set the text content equal to that error and console error it (the reason that I am using the text content as my medium to show the error as well is because for some reason my tester couldn't inspect elements on the computer they were using at the time - probably because it was a school computer).

This had the result below:



After a few google searches, the most likely cause for this would be that the navigator requires a secure origin, therefore since my code is running over HTTP localhost which has no SSL certificate and not HTTPS it won't let this 'random' website change the clients clipboard. Therefore it is probably a good idea to get a local SSL certificate, however this is a feature that I will reserve for exploring in a later cycle - as the current priority is to get the website's frontend and backend to be functional.

Lastly, when I asked another friend to test my website on their firefox browser it produced extremely inconsistent results - disregarding the current issue with the copy button - when

they attempted to export the text area's content as a PDF or word document, it only worked properly after a few tries in a seemingly random pattern. Though the exportation as a text file worked perfectly, sometimes exported PDFs or word documents would be completely blank and empty as seen in the video below:
Firefox not working.webm

However when I tested the website with other users in any other chromium browser it worked perfectly fine, so I think it's merely a problem between firefox and my application. Therefore I will attempt to find a fix for this in a later cycle, however since it doesn't look too feasible to fix I should strive to at least add a warning for those using firefox that it might take a few tries to properly download your converted text. Especially since the issue lies with the inherent structure of firefox not being a chromium browser, therefore this may become a limitation of web browsers for firefox users.

3.4 Code Annotation and Naming Conventions

Annotating code is extremely important for any codebase, so I have striven to comment on everything that would be hard to understand if I came back and tried to integrate a new feature - or if another developer wanted to build upon my project. Therefore, in extension having a consistent naming convention is also key; it allows new developers to understand my codebase with more ease. And even helps me with remembering what everything is, therefore in terms of naming conventions I have named all divs using train case (e.g. this-is-a-div), all variables (even constants) and file names in camel case and all functions in pascal case. I have left most of my constants in my JavaScript files in camel case instead of all capitals, since they are normally references to external HTML elements and not data values. Therefore I will be changing the value of their attributes but overall I don't want anything to overwrite or influence the actual html elements stored within them.
In conclusion a codebase with good explanatory comments and a basic naming convention can vastly improve the future maintainability of the system; as it can help not only the original developer with remembering certain details, but it's also an extremely crucial step to ensure new developers understand the system - to allow them to contribute or improve upon the application with greater quality and depth.

3.5 Evaluation

3.5.1 Changes to the project

Overall as I have been developing my application, I have strayed quite a bit from my original design. For example my 3 cycles included creating the website, configuring the CNN model and then integrating them together. But now it seems more sensible for cycle 1 to be dedicated to creating the frontend, cycle 2 to be finishing the backend and integrating it with the frontend and then for cycle 3 just work on some user feedback and necessary changes that I could not accomplish in the previous cycles.
Otherwise, in terms of usability features and just the general design of my website I have defined in my design section, have remained relatively the same. It's just my

draft examples of how I imagined my web pages to look like that have changed, since while I am developing I realise that I am missing a feature that I didn't include originally but need to include for everything else to work properly. Such as the output webpage and the layout of its export buttons, which originally were gonna be a drop down list after the user clicks export, but now is in a row with all the options - which also just makes more sense. Otherwise, when I needed to export the text as a file I would need the user to actually give it a name before it could be downloaded so I added another input form on the output webpage. Other features just came due to the extent of what my code could do inherently, such as being able to use images in the clipboard to upload images when you use the file picker. Finally, in the actual success criteria of this cycle I added the feature of being able to preview files before uploading them, though this wasn't in my original success criteria it seemed like a sensible and beneficial addition so I also added it.

3.5.2 Features to Re-iterate in Future Cycles

In cycle 1 I was not able to make sure everything has been refined and perfected on the frontend side of things, for example I need to get my website a SSL certificate so that users can actually use the copy button if I am to continue to use local host to host my website for others - since it's only working on my local machine - and also warn non-chromium browser users (e.g. Firefox users) on the inconsistency the application may pose for them when they wish to export files. Furthermore, after my backend is finished and I want to test the efficiency of the CNN model and how many files it can manage etc. (basically I want to test its limits) so I can then refine how many megabytes worth of files you can send to it at once and update any of the further instructions if necessary. This is where cycle 3 will come in, it will be the prime to work on issues I could not accomplish in previous cycles, and just optimise the website application even further - as otherwise it is merely a cycle to work on user input and feedback to make sure the website is catered towards the targeted stakeholders.

4. Cycle 2

4.1 Cycle 2 overview

The main goal is to first create a backend that will receive the images from the website, store them, preprocess them, then infer them through my pretrained machine learning OCR (optical character recognition) model and then finally return the processed text to be displayed in the text area in the Output page. Overall due to the amount of times I will need to switch between working on the frontend and then backend to implement these features, there aren't very clear points on where I can section off everything - compared to cycle one where I could easily split everything in terms of each singular webpage.

4.2 Success Criteria

Main Goal: Create Backend and integrate it with frontend

Frontend:

- Loading Bar:
    - Create loading bar for the wait period between input and output page
    - Make it accurate by syncing it with the backend process to know when the OCR model has completed processing
- Dependencies with Backend:
    - If no files are attached cancel the file transfer to the backend to prevent an internal server error
    - When the processed text is returned to the frontend, set it equal to the text content of the output page

Backend:

- Image preprocessing with OpenCV:
    - Greyscalling
    - Blurring
    - Threshold
- Inference with an OCR model:
    - Pick a suitable OCR model
    - Download it to run it locally
    - Funnel in the preprocessed image to be inferred
- Return processed text back to the frontend

4.3 Iterative Development

Now that I need to create a backend and also integrate it with the frontend as well, I used some rough flow diagrams and checklists to generalise what I needed to accomplish and how to implement these features in a manner that would satisfy both the frontend and the backend. This created a more digestible plan that I could more easily implement and follow through with by decomposing all of the various dependencies and endpoints.

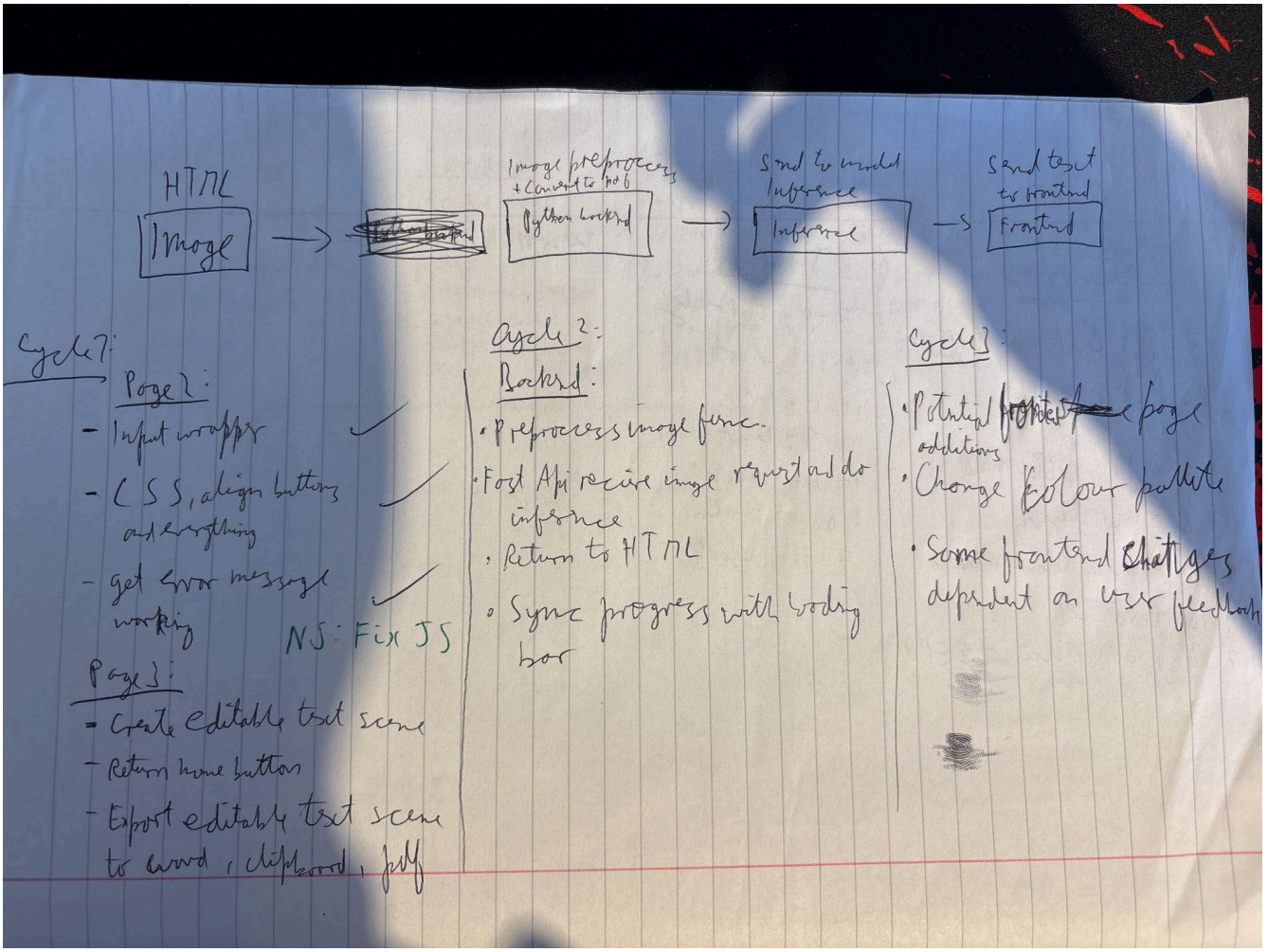


Figure 4.3.1

In figure 4.3.1 I simply made an overall checklist for all cycles to envision what features and functionalities need to be implemented and when, by thinking of the overall flow of the program as seen above. This helped me visualise what I needed to accomplish for cycle 2, and led me to create a more indepth plan as seen in figure 4.3.2.

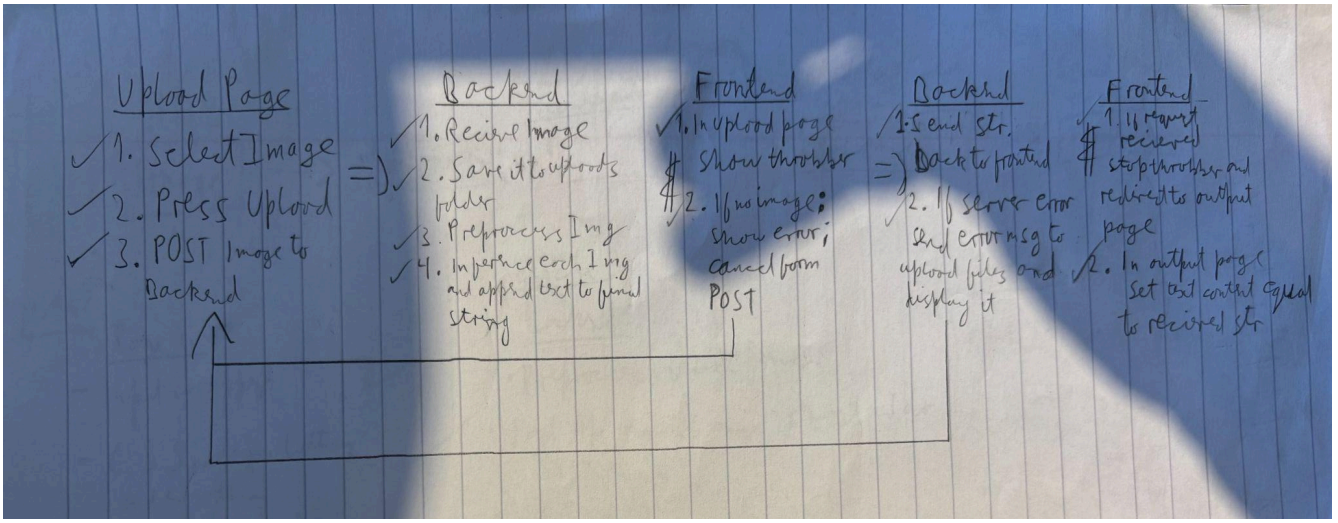


Figure 4.3.2

Here I focused mainly on the backend frontend integration side of things, where I start of by selecting an image and then uploading it to the backend server through POST, the backend then processes it while the frontend shows an loader to show the user that their request is being processed. Then if the file stack was empty it returns the users back to the first stage with an error message, else a string of the final processed text is returned to the frontend that then stops the loader and then shows the user the output page with their processed text - and if there is an internal server error it returns them back to the upload page.
This is the planned architecture of my application, visualised through a rough flow diagram.

4.3.1 Backend

```
PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> python -m venv .venv
```

Before starting the programming of my backend server, I initialised a virtual environment (as seen above) for my project. I used a virtual environment since it is an extremely useful tool in managing and handling various external python module dependencies and any other conflicts different modules may have with another. Overall, it is an extremely useful form of best practice that ensures that all third party libraries required are all encapsulated in their own internal environment. This is extremely important in my case as well since I want to locally run a huge large-language-model for OCR operations which will require many third party libraries to be run locally on my machine. Therefore this justifies the use of a virtual environment, to ensure my entire backend server runs without any conflicts within libraries.

```
PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> .venv/Scripts/activate
```

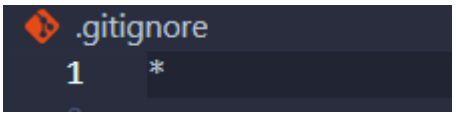
I then activated my virtual environment using the command in my project terminal shown above.

```
(.venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> pip install "fastapi[standard]"
```

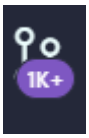
Next I needed to decide on what backend server I would like to use, there were mainly three that originally popped into my head: Flask, FastAPI and Django. So I evaluated the qualities I needed to pick the best option out of these three; excluding Django was extremely easy

since it is mainly used for larger web applications and in my opinion had a huge learning curve that could make me waste more time learning the framework itself, instead of focusing on the optimisation and quality of my website. Furthermore, it lacked the asynchronous support I was looking for and was much slower than FastAPI. But at least it had some support, while flask has no native support for asynchronous work which is crucial for my work with all the features regarding: file uploads and the huge processing time the backend would need before it can respond to my frontend website's request. So Flask was easily removed from my list. This left FastAPI, which has excellent native asynchronous support and looked much more simpler & easier to implement compared to Django; overall it is a speedy and efficient framework that can allow me to focus more on creating the best website for my stakeholders, rather than focusing on figuring out how to actually implement the features I want. So I installed the fastapi library in my virtual environment as shown above.

Since I have been using Github for various reasons I have previously listed, I need to add a .gitignore file since I have included a virtual environment in my project. This makes sure not all 1-4 thousand files of all my installed modules and dependencies are sent back to my main repository, which would create just an unnecessary heap of useless memory. Furthermore, since I am using an external pre-trained OCR model I might need to use an external API to run the model before I do so fully locally. This would require me to use an API key in my environment variable to access that API, so if I publish my virtual environment then the public would have access to my personal and private API key. Therefore I have added the git ignore file shown above.



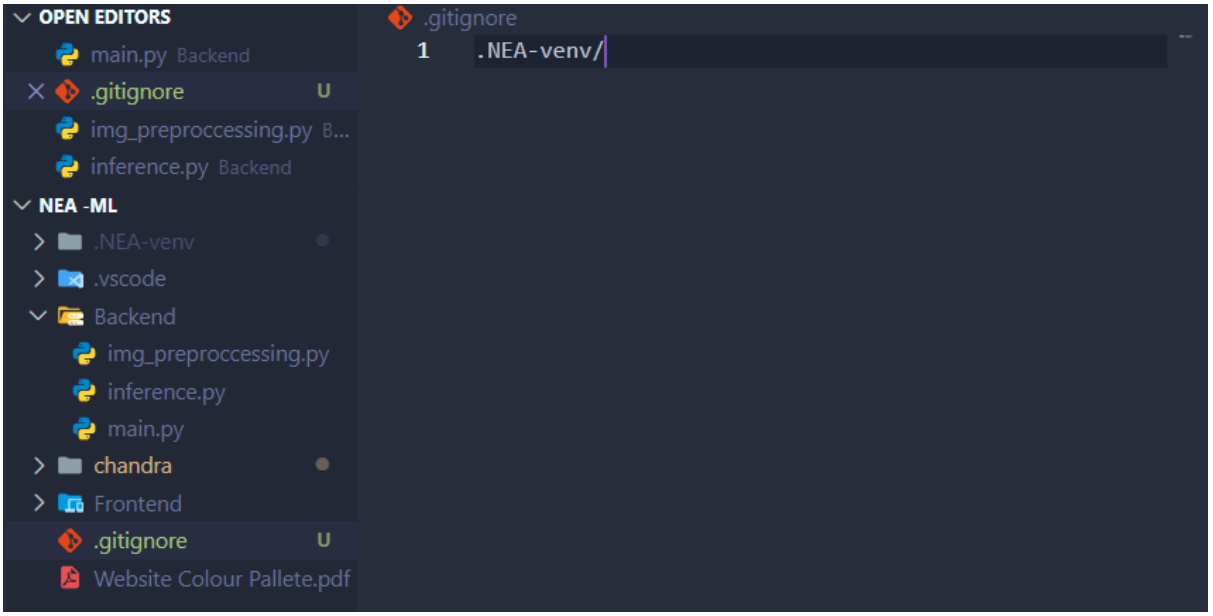
Inside of it, as seen above, I have added a star to tell git to ignore all types of files that are normally on its list of ignored files. However this did not work and I still had a thousand changes to commit on my main git as seen below:



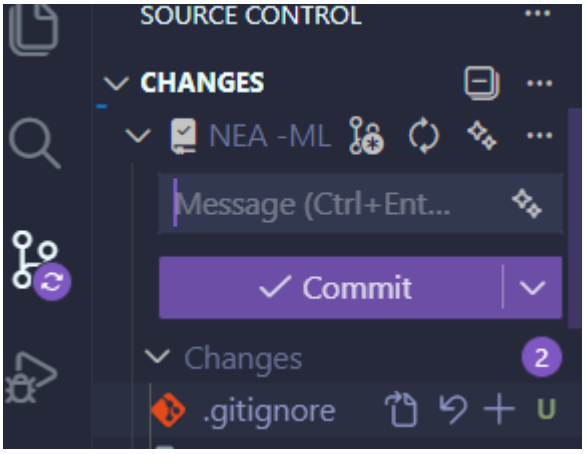
So, as seen below, I removed the star and added the specific .venv/ component to specify that it should target my virtual environment.

However this posed the same result and my virtual environment was still present in my git commits.

I then realised this was the case because I had already committed the originally initialised virtual environment - since you can't use git ignore on any files already committed to the git repository - and these changes were the additions of extra libraries I had downloaded.
So I deleted the entire virtual environment and created a new one in the root directory of my project and reinstalled all of the necessary modules and added the following command to the git ignore file:



This proved to be fruitful as the gitignore worked and the repository no longer recognised the virtual environment as something that it needed to append into my main repository. As seen below:



However when I tried setting up a virtual environment on my laptop I got this error:

I then figured out that this was because windows 11 was being weird (my home PC is windows 10 which proved beneficial for this case). So I needed to grant the virtual environment special permission in the same terminal to run and execute it's python scripts, before I could use it, so I used:

```
PS C:\Users\lords\Coding Stuff\NEA--ML\Backend> Set-ExecutionPolicy Unrestricted -Scope Process
```

To give the virtual environment its necessary permission.

Then I had to use activate.ps1 specifically to initialise my virtual environment, so that the virtual environment worked on my windows 11 device:

```
PS C:\Users\lords\Coding Stuff\NEA--ML\Backend> .NEA-venv\Scripts\activate.ps1
```

I then installed python multipart so that my FastAPI backend server could handle multipart data (files) that I have sent from the frontend back to the backend:

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> pip install python-multipart
```

```python
#to import FastAPI backend and upload file functionality
from fastapi import FastAPI, UploadFile

app = FastAPI() #instantiate the fast api object

#decorator to use the HTTP POST protocol into the defined path in the param
@app.post("/uploadfile/")
async def Create_Upload_File(file: UploadFile):  #async funtion thath will take in the file from the frontend
    return {"filename": file.filename}   #return its name to the server for now
```

Figure 4.3.1.1

I created a main.py file in my backend folder and created a basic FastAPI server as seen in figure 4.3.1.1. I first imported FastAPI itself and UploadFile - which would let FastAPI receive files from the frontend - then I initialised FastAPI using the identifier app. Next I used the decorator @app.post() to use the HTTP method post (which lets the backend receive data from the frontend) to receive the uploaded files from the user. I set the parameter equal to /uploadfile/ so that when the user sends their images the HTML knows that the endpoint of the path for this post is uploadfile, and sends the files to this uploadfile endpoint on the website to the backend. Now for the function itself, it is obviously asynchronous so that the frontend doesn't freeze when the user sends their request - rather it will show a progress bar to signify to the user that everything is working as intended. It then has UploadFile as a param, telling the website that when it sends a POST method that the server expects a file object to be sent and passed into the Create_Upload_File function, and then for now it will simply return a JSON object that shows the sent files name.

```html
<div class="file-upload" id="drop-zone"> <!--Overall div for the file-upload section-->

  <form method="POST" action="http://localhost:8000/uploadfile/" enctype="multipart/form-data">

  <input id="fileInput" type="file" name="file_upload" accept="image/*" multiple/> <!--Creates the input form, for multiple image files-->
  <button id="uploadButton" type="submit">Upload ⇪</button>
  <label for="fileInput">Select Images or Drag & Drop</label> <!--Creates a caption for the file input-->

  </form>

  <div id="file-preview"></div> <!--Creates a DIV where I can append all the file previews-->
  <div id="error-message"></div> <!--Creates a DIV where I can display error messages-->
</div>

<div id="progress-bar"></div> <!--Creates a DIV where I can manipulate a progress bar-->      Shiven K, 3 days ago • Improved maintainability

<button class="button button1" onclick="window.location.href='/Frontend/HomePage/homePage.html'">⬅ Return To Home</button>
```

Figure 4.3.1.2

Then to actually link the frontend and the backend together I modified the HTML code for the Input page in figure 4.3.1.2, I improved upon figure 3.3.2.9 and encapsulated the entire file input element inside of a form tag. I defined the form tags attributes to include a POST method to signify to the website to send the contents of this form back to the backend, then I set the action attribute equal to the URL of the backend (including the /uploadfile/ endpoint of the Create_Upload_File() function), then I set the name attribute of the input tag inside of the form to equal the name of the Create_Upload_File() functions parameter, so that the form knows that the result of the input tag is the piece that needs to be inputed to the function, finally I set the enctype - data encoding of the contents of the form being sent by POST - of the forms data equal to multipart form data, this is used because the contents of my form are files, which require a multipart enctype.

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> pip install uvicorn
```

Next to actually test my backend frontend functionality I needed to run it all on a proper server, so I decided to use Uvicorn servers, they are extremely swift ASGI (Asynchronous Server Gateway Interface) servers that are the main go to for FastAPI developers. Therefore I downloaded it as shown above.

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> uvicorn Backend/main:app --reload
```

Then after looking at its basic documentation I attempted to start the server, with a --reload at the end to make sure if any code is changed in the frontend or backend the server automatically reloads itself.

```
INFO:     Will watch for changes in these directories: ['C:\\Users\\Shiven Kothari\\Desktop\\Shiven\\Coding stuff\\NEA -ML']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [12044] using WatchFiles
ERROR:    Error loading ASGI app. Could not import module "Backend/main".
```

However when I ran it (guiding it to the Backend folder which contained the main python file) it could not ‘import’ it. As seen above.

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> cd Backend
```

Therefore, as seen above, I decided to just go to the Backend directory where the main file is and then run the unicorn server command.

Which resulted in a success, as shown below:

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\Shiven Kothari\\Desktop\\Shiven\\Coding stuff\\NEA -ML\\Backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [6196] using WatchFiles
INFO:     Started server process [17860]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

This also gave me the local port the backend will be hosted at - as seen in line 3 - this is what I used to guide my HTML form to the correct backend endpoint as shown in figure 4.3.1.2.

Then I pressed on the live server extension button and tested the website. This resulted in the picture below, which showed the upload button being engulfed in the drop-zone box, this was since I needed to include the submit button inside of the form itself in figure 4.3.1.2 for it

to work. But obviously the form itself is inside of the overall dropzone div, causing this box to be attached to the overall drop zone.

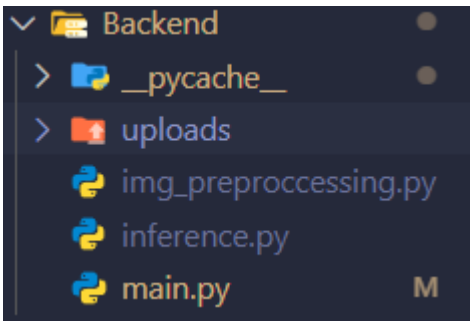
```
<form id="uploadFile" method="POST" action="http://localhost:8000/uploadfile/" enctype="multipart/form-data">

<input id="fileInput" type="file" name="file_upload" accept="image/*" multiple/> <!--Creates the input form, for multiple image files-->
<label for="fileInput">Select Images or Drag & Drop</label> <!--Creates a caption for the file input-->

</form>

<div id="file-preview"></div> <!--Creates a DIV where I can append all the file previews-->
<div id="error-message"></div> <!--Creates a DIV where I can display error messages-->
</div>

<div id="progress-bar"></div> <!--Creates a DIV where I can manipulate a progress bar-->

<button id="uploadButton" type="submit" form="uploadFile">Upload ⇪</button>      You, now • Uncommitted changes
```

Figure 4.3.1.3

So I then iterated over figure 4.3.1.2 into figure 4.3.1.3 and moved the button outside of the div and set its form attribute equal to the forms ID, so that it could be separated from the drop zone div but still be connected to the overall form.

This resulted in the following: Showing form input file BASIC (1).mp4
It resulted in the upload button to be correctly positioned and have a successful integration between the frontend and backend, however it returned a JSON with an empty filename - which I even retested by renaming my files from large numbers into a simple hi. However when I used the normal file picker it worked perfectly fine and the backend received the files name and displayed it normally; therefore for now I will simply continue working on the file picker until it is complete and then attempt to debug and fix the drag and drop functionality later on as it is not an essential feature.

Next I created a new folder that will contain and locally store all of the uploaded images for the backend:



Next I imported the pathlib library for easier file handling as seen in figure 4.3.1.4. And then defined a constant to hold the file path for the uploads folder so that all files can be transferred and stored into that folder when received.

```python
#to store files in the server
from pathlib import Path

#const has the directory that the uplaoded file will go to
UPLOAD_DIR = Path() / 'uploads'
```

Figure 4.3.1.4

Then I started the process of letting the server actually read the contents of the file sent to the server by iterating over figure 4.3.1.1, as seen below in figure 4.3.1.5

```python
#decorator to use the HTTP POST protocol into the defined path in the param
@app.post("/uploadfile/")
async def Create_Upload_File(file_upload: UploadFile):  #async funtion thath will take in the file from the frontend
    data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
    save_to = UPLOAD_DIR / file_upload.filename #save a file in the predefined directory with the files name
    with open(save_to, 'wb') as f: #write to the files bytes
        f.write(data)   #write the read data, so we have completley stored this file in our backend
    return {"filename": file_upload.filename}   #returns its name to the server for now
```

Figure 4.3.1.5

As shown in figure 4.3.1.5 I first initialised the data variable that will first read the file received by the function - which is accomplished asynchronously using the await keyword so that the file reading can be accomplished separately from everything else, in case a file is particularly too large. Then I create the file path of where the file will be stored, which is stored within the identifier save_to by appending the inputted files name with the upload directory file path. Next, I actually save the file by opening this newly created file at save_to using wb (write bytes) so I can completely write in the submitted files data content into this new copy at the uploads folders.

Uploaded file Video: Uploaded file to the backend (2).mp4
The result above spectacularly showed the result of a singular file upload, but for the purposes of this website I need the user to be able to upload many files at the same time. So below I attempted to send through multiplex files at the same time.

Failed to upload multiple files Video:
Show that i cant uploadmultiple files to backend(3).mp4

The video above displays the backends inability to upload more than the first file it receives from the form.

```python
#decorator to use the HTTP POST protocol into the defined path in the param
@app.post("/uploadfile/")
async def Create_Upload_File(file_uploads: list[UploadFile]):  #async funtion that will take in a list of files from the frontend
    for file_upload in file_uploads:    #iterate over each file to save it to our directory
        data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
        save_to = UPLOAD_DIR / file_upload.filename #save a file in the predefined directory with the files name
        with open(save_to, 'wb') as f: #write to the files bytes
            f.write(data)   #write the read data, so we have completley stored this file in our backend
    return {"filename": [file_upload.filename for f in file_uploads]}   #returns a list of file names of all uploaded files
```

Figure 4.3.1.6

I then modified the code of figure 4.3.1.5 as shown in figure 4.3.1.6 to incorporate the use of lists, so that when I send over all of my files it views it as a list of files rather than only expecting a single file and ignoring all others. I started off by defining the file_upload parameter to become the: file_upload**s** parameter which is now expected to be a list of uploaded file objects. Then I used a for loop to iterate over every file in the given list and then save it into the uploads directory.

```html
<input id="fileInput" type="file" name="file_uploads" accept="image/*" multiple/> <!--Creates the input form, for multiple image files-->
```

Figure 4.3.1.7

I then modified the original figure 4.3.1.3 into figure 4.3.1.7 by changing the name of the parameter it corresponds to match the new identifier for it.
Attempt of using multiple files: Show 422 error (4).mp4

I then received the 422 error when I attempted this:
{ "detail": [{ "type": "missing", "loc": ["body", "file_uploads"], "msg": "Field required", "input": null }] }

However when I then tried this again the next day it magically disappeared, since this was an error between the transmission between the frontend and the backend; I will blame my internet for now for sending the POST request in a weird form that seemed unprocessable by the server. Unfortunately I can't fully debug this to understand since I never looked at the network logs of the website when this error occurred.

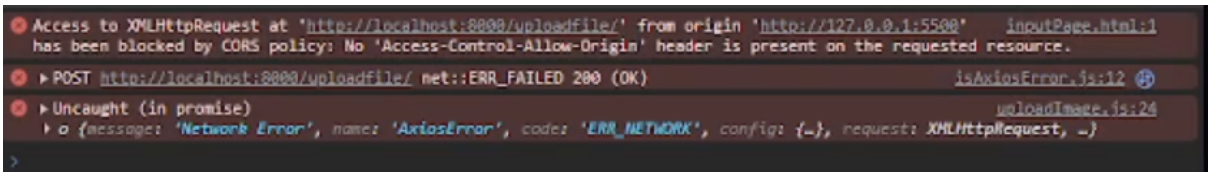
```javascript
//handles the file preprocessing and preview
async function HandleFiles(files) {
  //preproccess
  preview.innerHTML = ''; //resets any previous html preview content
  errorMessages.innerHTML = ''; //resets any previous error messages
  fileStack = []; //reset current file stack to avoid confusion with previous attempts

  //Validate, append & induce preview
  for (const file of files) { //Loop through every file in the file array inputed to the function
    const isValid = await ValidateFile(file); //bool isVaild is the result of called function that will validate files, use await since we want this to be finsihed before moving on
    if (isValid) {
      fileStack.push(file); //appends it into the files stack
      const reader = new FileReader(); //lets the script read the contents of the file to get the preview
      reader.onload = (e) => CreatePreview(e.target.result, file.name); //when file reader finsihes, callback func creates preview with the data url and filename
      reader.readAsDataURL(file); //start reading the file as a data url, which lets it access the file type, encoding and actual file data, this kick starts reader.onload

      const form = new FormData();
      form.append("file_uploads", file);
      await axios.post("http://localhost:8000/uploadfile/", form);
    }
  }
}
```

Figure 4.3.1.8

Now that the file picker's upload to the backend is sufficient, for now I started to work on how to fix the drag and drop functionality by iterating over my HandleFiles() function in the Input pages javascript as seen in figure 4.3.1.8. I figured out that since I have a custom drag and drop feature, drag and dropped files are treated differently from the file picked ones. The ones used by the file selector are part of the form that is defined in figure 4.3.1.3, while the drag and dropped images are only stored in handle files so I need to send them to the backend separately. I did this by defining a new form variable that used the Formdata data type. I then pass in the name of the POST function in the backend and the actual file contents together as a key and value pair respectively (in JSON format), this is then manually posted to the backend server using axios - an inbuilt javascript library that can make HTTP requests.

Results: Drag and drop upload functionality (five).mp4
The result of figure 4.3.1.8 above made it so that the website can't handle multiple drag and dropped files, however as a silver lining it has fixed singular drag and dropped files.

The errors showed in the video above were:



It started off with a message from CORS blocking my axios POST request.

```python
#middleware for different port communication
from fastapi.middleware.cors import CORSMiddleware

#const has the directory that the uplaoded file will go
UPLOAD_DIR = Path() / 'uploads'


app = FastAPI() #instantiate the fast api object

#allowed middle ware
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],    #for simplicity allow any origin      You, 5 minute
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Figure 4.3.1.9

The reason for the CORS error is due to the different ports of the origin and endpoint that are part of this axios POST request. Therefore to handle the conflicting ports I added a middle manager as seen in figure 4.3.1.9, from the FastAPI documentation I followed, I first

imported the middleware from the main fastapi and then defined the middleware to accept basically any origin (as signified with the star)

Final working video of multiple drag and drop upload:
Solve multiple file functionality(6).mp4

The result above completely fixed the drag and drop function of the website.
However currently it just automatically sends the files to the server without letting the user click the upload button.

```
async function uploadFiles() {
  if (fileStack.length != 0){
    for (const file of fileStack){
      const form = new FormData();
      form.append("file_uploads", file);
      await axios.post("http://localhost:8000/uploadfile/", form);
    }
  }
}
```

Figure 4.3.1.10.1

So, as seen in figure 4.3.1.10.1, I removed it from the handle files function and instead created a new asynchronous function that will use the filestack to send the files to the backend if the file stack isn't empty, and if it is they will be served with an error message so they can just try again then.

```
dropZone.addEventListener('drop', async (e) => { //when item is dropped it will execute, it is async so the rest of the website works asynschrously
while the file is being uploaded, e is the event object
  e.preventDefault(); //prevents the broswer from accidently opening the file
  dropZone.classList.remove('dragover'); //removes the dragover class property as the user has dragged and dropped the file
  await HandleFiles(Array.from(e.dataTransfer.files)); // calls the dropped files, converts it into a js array and sends it to HandleFiles func, await
  makes it so that everything waits for the file upload to finish before executing anything else
  uploadFiles();
})
```

Figure 4.3.1.10.2

Then in figure 4.3.1.10.2, I call the uploadFiles to the drag over function, so that the function can be called and executed when the files are dragged and dropped. Furthermore, making sure uploadFiles is called after HandleFiles makes sure that the file stack is not empty automatically, however the presence check in figure 4.3.1.10.1 is still best practice.

Then as seen below in figure 4.3.1.11 I then added the function to the HTML code of the Input page so that when the upload button is clicked it will then use the POST protocol to send the files to the backend.

```
<button id="uploadButton" type="submit" form="uploadFile" onclick="uploadFiles()">Upload ⇪</button>
```

Figure 4.3.1.11

While testing this I found a bug where if I drag and drop multiple files I would get an internal server error from the server: SHOW INTERNAL SERVER ERROR (BACKEND).mkv

Since it's internal server error I need to catch any errors I made in my server code and return them to figure out what specifically is causing the problem inside the server, so I added a try and except block, as seen below in figure 4.3.1.12, to catch and raise it:

```python
#decorator to use the HTTP POST protocol into the defined path in the param
@app.post("/uploadfile/")
async def Create_Upload_File(file_uploads: list[UploadFile]):  #async funtion that will take in a list of files from the frontend
    try:
        for file_upload in file_uploads:    #iterate over each file to save it to our directory
            data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
            save_to = UPLOAD_DIR / file_upload.filename #save a file in the predefined directory with the files name
            with open(save_to, 'wb') as f: #write to the files bytes
                f.write(data)   #write the read data, so we have completley stored this file in our backend
    except Exception as e:
        return {f"{e}"}
    return {"filenames": [f.filename for f in file_uploads]}   #returns a list of file names of all uploaded files
```

Figure 4.3.1.12

This gave me a permission error, meaning something is wrong with the file handling:
SHOW PERMISSION ERROR (BACKEND).mkv
Either the server doesn't have access to the uploads folder and I can't write to it or something is wrong with the process of instantiating the file into the uploads folder. I could not figure out the cause of this issue within a reasonable time frame, therefore due to the time constraints I have I chose to leave this for cycle 3, where I could then have a reasonable idea on how to fix this issue due to the ample amount of time provided within cycle 3 - especially since the main file picker works for now, so I have decided that it's better to continue forward.

Now that I have a system that receives and then stores those files for the backend to access for later processing, I need to create the modules that will do this further processing.
The first thing that needs to be done is to preprocess the images funneled into the backend before I send them to the actual OCR model. Therefore to preprocess these images I need to use OpenCV, it holds extremely optimised algorithms for real-time computer vision, image processing, and machine learning. So for my application it is perfect, and as seen below I installed OpenCV for my backend server.

```
(.NEA-venv) PS C:\Users\lords\Coding Stuff\NEA--ML\Backend> pip install OpenCV-python
```

Then I created a separate python file purely for image preprocessing and wrote the IMG_Preprocess function, as shown below in figure 4.3.1.13.

```python
import cv2
      Shiven K, 7 hours ago • Image preproccessing stuff …
def IMG_Preproccess(filePath):
    #load the image
    img = cv2.imread(filePath)

    #greyscale it
    greyImg = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    #apply otsu thresholding algorithm
    threshImg = cv2.threshold(greyImg, 0, 255, cv2.THRESH_BINARY_INV | cv2.
    THRESH_OTSU)[1]

    #use distance transform to focus on the words
    distImg = cv2.distanceTransform(threshImg, cv2.DIST_L2, 5)

    #Normalise image, so its either black or white
    distImg = cv2.normalize(distImg, distImg, 0, 1.0, cv2.NORM_MINMAX)
    distImg = (distImg * 255).astype("uint8")

    cv2.imwrite(filePath, distImg)

IMG_Preproccess(r'C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA
-ML\Backend\uploads\yeet.png')
```

Figure 4.3.1.13

In figure 4.3.1.13, I first import OpenCV and then define a function that takes in the file path of the image it needs to process as its given parameter. Then I instantiate the image by using OpenCV to read it from its filepath, then I apply a greyscale to it to convert the images RGB colours into shades of grey. I need to do this to reduce the complexity of the image by eliminating its colour data while retaining the images intensity information (what lets people actually discern various objects from each other in the image), furthermore it is required to pass in an image into a thresholding or distanceTransform method for it to be grayscaled. Then I pass in the greyscaled image into a threshold method, the method uses a threshold value which is compared to each pixel in the image. If the pixel is less than the threshold value then the pixel is set to 0 (as defined in the second argument for the method), else if it's larger it is set to the max value allowed in the function (in this case it's the maximum, black as it is the third argument for the method). This particular thresholding algorithm uses Otsu's threshold, which unlike previous thresholding algorithms does not arbitrarily choose a threshold value, rather it automatically determines the most optimal threshold value; it is passed in as an extra value in the fourth parameter. This returns a list of the threshold value and the actual image, we want the image so we do [1] to gain the image and store it in the threshold variable. Then the distanceTransform method passes in the thresholded image into it, it figures out the distance between each white pixel to the nearest black pixel to suppress noise and provide greater line segmentation, this is extremely useful for OCR algorithms. Furthermore I used a five by five mask for it to see thicker strokes and smoother curves. I then normalise this image, this adjusts the range of pixel intensity values in the image to increase contrast. Then to save this image with OpenCV I convert the current image - which is being stored as float32 since we processed it so many times - into a uint8 image so I can then overwrite the previous image into this new processed image.
To test this function before integrating it into my final web application, I have called it at the bottom with a custom file path to call the image attached to it.

```
[Running] python -u "c:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend\img_preproccessing.py"
Traceback (most recent call last):
  File "c:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend\img_preproccessing.py", line 1, in <module>
    import cv2
ModuleNotFoundError: No module named 'cv2'

[Done] exited with code=1 in 0.091 seconds
```

However as soon as I ran my code I got a module not found error as seen above. It seemed that my python file could not be debugged inside of my virtual environment no matter what I tried. This error made it extremely hard to test features and simply made it hard to debug in general, therefore to make it easier to debug I deleted my virtual environment. Since there are no clashing dependencies anyway, it's best to just download them all locally - especially since I can just uninstall them later if need be - because I would much prefer this compared to the alternate debugging nightmare that would have arisen otherwise.

Video of it failing with IMG_preprocessing with handwritten notes:
IMG_PRE_FAIL(7).mp4
Granted the handwritten note is in MS paint, it still failed completely when given a non digital text image to process.

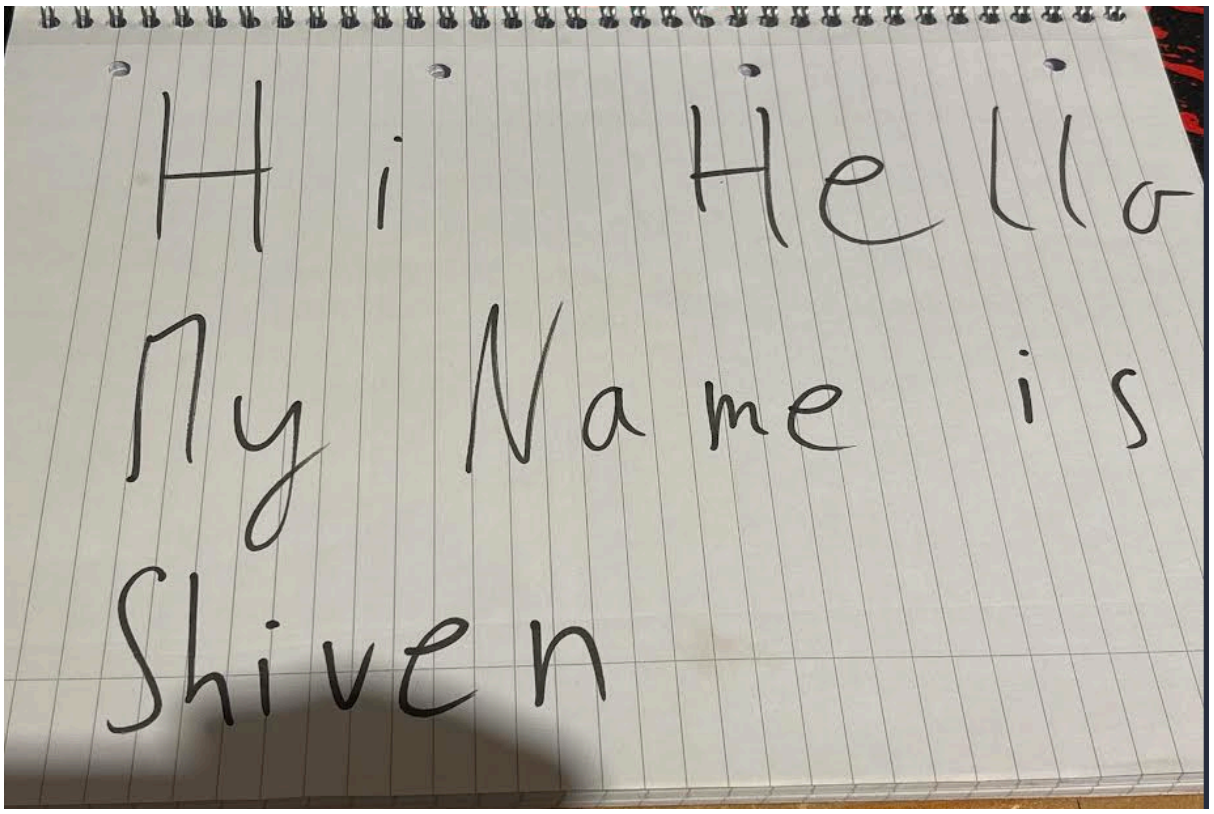
```python
def IMG_Preproccess(filePath):
    #load the imagez``
    img = cv2.imread(filePath)

    #greyscale it
    greyImg = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    #apply gaussian blur to handle bad strokes to make everything look
    uniform
    blurImg = cv2.GaussianBlur(greyImg, (5, 5), 0)

    #Apply adaptive threshold
    threshImg = cv2.adaptiveThreshold(blurImg, 255, cv2.
    ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 31, 10)

    cv2.imwrite(filePath, threshImg)

IMG_Preproccess(r'C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA
-ML\Backend\uploads\y3.png')
```

Figure 4.3.1.14

I realised the reason it had not worked beforehand was because using an Otsu threshold and a distanceTransform is only useful for the uniformed digital text, and not my main priority, which is handwritten notes. Therefore I looked more deeply into how to effectively preprocess an image of handwritten text. So I improved upon figure 4.3.1.13 with figure 4.3.1.14, here I first used a Gaussian blur - with a five by five mask size, which blurs the image incrementally within that mask - to smooth the image and remove any external noise that could hinder the final processed image. Then I applied an adaptive threshold instead, while the original threshold uses a global threshold value for the entire image, the adaptive threshold separates the image into regions and chooses a threshold value for each region, making it adaptive. This is extremely useful when an image has a very convoluted lighting or for areas of handwriting that are more desperate than others, this allows for the entire image to be represented well without there being any errors due to small regions causing terrible

compromises for the overall image. I then overwrote this new processed image from its old position.

It worked successfully as it turned this image:



Into this:

Next I attempted to integrate this successful function from its module into the main backend as seen below in figure 4.3.1.15

```
#img preproccess set up
from img_preproccessing import IMG_Preproccess
```

Figure 4.3.1.15

```
for file_upload in file_uploads:    #iterate over each file to save it to our directory
    data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
    save_to = Path().cwd() / 'uploads' / file_upload.filename #save a file in the predefined directory with the files name
    with open(save_to, 'wb') as f: #write to the files bytes
        f.write(data)   #write the read data, so we have completley stored this file in our backend
        counter += 1
    #img preproccess
    IMG_Preproccess(save_to)      You, now • Uncommitted changes
```

Figure 4.3.1.16

Then as seen in figure 4.3.1.16 I call the IMG_Preproccess function inside of the for loop, to process each image as they are being stored.

Video of successful integration: Show_Pipeline_imgPreprocess_fromWeb(9).mp4

Next the final external module I need to create is the inference python file for my chosen OCR model. During my research many basic OCR programs such as TessarectOCR came up, however most of these were too old and only focused on basic uniformed text recognition. I wanted to utilise a model that is recent, efficient, effective and made for

handwritten character recognition. Other names such as GPT-4V normally dominated most websites, being propped up as the go to OCR model for all my needs; however its proprietary software status pushed me away from it. Realistically I would like an open sourced model which I could be free to play around with to the utmost of my ability, time and imagination. This is when I stumbled upon the ChandraOCR model in the huge collection of pre-trained large language models in the HuggingFace.ai collection, one that is open source, completely handwriting supportive and a recent addition to the OCR roster that seemed incredibly promising. Therefore I cloned the Chandra github repository, created a folder called output to store the processed file, and did a basic command in the CLI to process a dummy image to check if it was working properly with the correct dependencies and such using a local model, since I did not feel it was worth it to pay to use the virtual large language model server. I set the method to hf so it would process it locally using the HuggingFace model:

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend> chandra uploads\yeet.png ./output --method hf
```

It produced the following error:

```
Found 1 file(s) to process.

[1/1] Processing: yeet.png
  Processing pages 1-1...
  Error processing yeet.png: Torch not compiled with CUDA enabled

Processing complete. Results saved to: output
```

This signified that the pytorch library had not been properly installed with the correct CUDA toolkits that abided with the GPU I had. So I uninstalled my current installations of all pytorch modules, as shown below:

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend> pip uninstall torch torchvision torchaudio -y
Found existing installation: torch 2.10.0
Uninstalling torch-2.10.0:
  Successfully uninstalled torch-2.10.0
Found existing installation: torchvision 0.25.0
Uninstalling torchvision-0.25.0:
  Successfully uninstalled torchvision-0.25.0
```

Then I went to their website and chose the current link that reflected my Nvidia GPU and installed it.

```
(.NEA-venv) PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend> pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```

This proceeded elegantly, however since my computer is not powerful enough it could not process this request locally. However this won't be of any consequence; since if I were to deploy it I would then need to use a server that would call to the chandara datalabs API or run it on a powerful enough machine with a GPU that had roughly 20GiB of virtual RAM and normal RAM combined. But this is more of an issue for the actual deployment, currently during the development and evaluation phase I am simply going to use my friend's computer to run my model locally. This is an excellent example of the benefits of Github, it will help me test my entire web application through the help of my friend who has better hardware than mine.

```
Found 1 file(s) to process.

[1/1] Processing: yeet.png
  Loaded 1 page(s)
  Processing pages 1-1...
  Error processing yeet.png: CUDA out of memory. Tried to allocate 18.60 GiB. GPU 0 has a total capacity of 6.00 GiB of which 717.69 MiB is free. Of the allocated memory 4.21 GiB is allocated by PyTorch, and 38.20
 MiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting PYTORCH_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See documentation for Memory Management  (h
ttps://pytorch.org/docs/stable/notes/cuda.html#environment-variables)

Processing complete. Results saved to: output
```

After testing Chandra through the command line interface I created a new inference python file as seen below in figure 4.3.1.17.

```python
from chandra.model import InferenceManager
from chandra.input import load_image

def Inference(filePath):
    #init inference manager and its model type
    manager = InferenceManager(method="hf")
    #init img
    img = load_image(r'Backend\uploads\y21.jpg')
    #init the images with their src at the file path
    images = load_image(r'Backend\uploads\y21.jpg')
    #generate result using the manager class from the chandra OCR
    results = manager.generate(images)
    #return markdown verison of it
    return results[0].markdown

Inference()
```

Figure 4.3.1.17

As seen in figure 4.3.1.17, I first imported the inference manager so I could direct the model to where it should be running - which is in the local machine - then I imported load_image so I can correctly load my image into the model. I then defined the inference function, which holds the filePath parameter that will be inputted into the function to direct the model to which file it needs to process (currently it is not used since I want to test this function by itself first, so for now a direct file path is going to be used). I initialised the models manager, then the image itself using load_image with an image that is stored in this directory. Then I used the generate method to generate the processed output and return it in markdown form.

When I ran it, it proclaimed that the image object being passed into the model was not iterable, as shown below:

```
  File "C:\Users\Shiven Kothari\AppData\Local\Programs\Python\Python311\Lib\site-packages\chandra\model\hf.py", line 18, in generate_hf
    messages = [process_batch_element(item, model.processor) for item in batch]
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'Image' object is not iterable
```

Therefore to make it iterable I wrapped the image inside of a list so that it could process it correctly as seen in the improved code in figure 4.3.1.18.

```python
from chandra.model import InferenceManager
from chandra.input import load_image

def Inference(filePath):
    #init inference manager and its model type
    manager = InferenceManager(method="hf")
    #init img
    img = load_image(r'Backend\uploads\y21.jpg')
    #init the images with their src at the file path
    images = [load_image(r'Backend\uploads\y21.jpg')]
    #generate result using the manager class from the chandra OCR
    results = manager.generate(images)
    #return markdown verison of it
    return results[0].markdown

Inference()
```

Figure 4.3.1.18

It then provided me with the given error below, stating that the image does not have a defined prompt attribute:

```
  File "C:\Users\Shiven Kothari\AppData\Local\Programs\Python\Python311\Lib\site-packages\chandra\model\hf.py", line 52, in process_batch_element
    prompt = item.prompt
             ^^^^^^^^^^^
AttributeError: 'Image' object has no attribute 'prompt'
```

Therefore I attempted to fix the code in figure 4.3.1.18 by refining it into figure 4.3.1.19, by changing the prompt attribute of the image with a prompt to create digital text from the handwritten notes.

```python
from chandra.model import InferenceManager
from chandra.input import load_image

def Inference():
    manager = InferenceManager(method="hf")
    img = load_image(r"Backend\uploads\y21.jpg")
    img.prompt = "Transcribe all handwritten text"
    results = manager.generate([img])

    if results:
        print(results[0].markdown)

Inference()
```

Figure 4.3.1.19

However I still got the exact same error as before. This led me to use a different approach, which led to another iteration of figure 4.3.1.19 into figure 4.3.1.20.

```python
from chandra.model import InferenceManager        You, yesterday • Started
from chandra.model.schema import BatchInputItem
from chandra.input import load_image

def Inference():
    #init inference manager and its model type
    manager = InferenceManager(method="hf")
    #init img
    img = load_image(r"Backend\uploads\y21.jpg")
    #init batches of the input, with image src, and prompt equaling ocr
    batch = [BatchInputItem(image=img, prompt_type="ocr_layout")]
    #generate result
    results = manager.generate(batch)
    #return markdown verison of it
    return results[0].markdown
```

Figure 4.3.1.20

In figure 4.3.1.20, I straight up used batches directly, instead of using load_images as a middle man to configure my files into a batch, and also I figured out the whole deal with the prompt was its prompt type, and to specify that I wanted to use its OCR capabilities - and not to treat it as a LLM chatbot.

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 9.00 GiB. GPU 0 has a total capacity of 6.00 GiB of which 994.10 MiB is free. Of the allocated memory 3.94 GiB is allocated by
PyTorch, and 28.80 MiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting PYTORCH_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See
documentation for Memory Management  (https://pytorch.org/docs/stable/notes/cuda.html#environment-variables)
```

It then led to the same result as when I used the CLI, therefore so far it is working as expected.
Finally to end it off I then started the integration of the inference of the model with the actual main python file.

```python
proccessedText = 'yeet'
```

Figure 4.3.1.21

Firstly, as seen in figure 4.3.1.21 I defined a variable that should hold an empty string, which should then be equal to the output of the processed text from the OCR model. However since my device can't run it, for now it is stored with a placeholder of yeet.

```python
#inferenece
proccessedText = Inference(save_to)

#send back to user
return proccessedText
```

Figure 4.3.1.22

4.3.2. Integration

Then, as seen in figure 4.3.1.22, inside of the for loop in the main file I set the processed text to equal the inferred text (which has the current files path passed into it)

```
const response = await fetch("http://127.0.0.1:8000/uploadfile/", {
  method: 'POST'
});
const text = await response.text();
textArea.textContent = text;
```

Figure 4.3.2.1

Then in the Output page's javascript I added figure 4.3.2.1, which will fetch the returned text file from the backend, then convert it into normal text and then set the text content of the text area into that text.

```
<button id="uploadButton" type="submit" form="uploadFile" onclick="window.location.href='/Frontend/OutputPage/outputPage.html'">Upload ⇪</button>
```

Figure 4.3.2.2

Then in figure 4.3.2.2 I added a link to the output page's HTML so that when the user clicks on the webpage it would take them to the output page with their processed text.

Video of it failing: ATTEMPTED OUTPUT PAGE REDIRECT - FAILED (BACKEND).mkv
It failed since the submit button was sending the file to fastapi and also sending them to the output page, so the fastapi post is prioritised and cancels the redirect to the output page.

So I first removed the changes I made in figure 4.3.2.2:

```
<button id="uploadButton" type="submit" form="uploadFile">Upload ⇪</button>
```

So I thought that the backend could redirect the user instead, so maybe the server could return a redirect response and processed text. But before I can implement that I need to firstly implement a progress bar for a transition between the input and output pages.

After finishing all of the Backend processing functions, I have chosen to change my original plan of implementing a loading bar and rather to replace it with an eternal throbber that will keep throbbing indefinitely until the backend process has been completed. It will still be animated and will act as a replacement for the loading bar, to provide the users with some reassurance that the application hasn't hung up on them and everything is going according to plan. Furthermore if I wanted to implement a loading bar I would most likely need to time multiple attempts of using the backend with various different types of inputs; not only would this be extremely tedious and in my opinion a waste of time - especially due to the limited

amount of times I can run the application, since I will be enlisting a friend of mine for help with the debugging and testing of it - but even if implemented won't be perfect, and could potentially frustrate stakeholders by taking longer than expected and being frozen at 99 percent or even at 100 percent. Therefore a throbber seems like the best option for a loading transition from the input and output page.

```
<div class="loading-wheel"></div>
```

Figure 4.3.2.3

As seen in figure 4.3.2.3 I started off by replacing the progress bar with a loading wheel div.

Next as seen below in figure 4.3.2.4, started to style the loading wheel in the input pages CSS. I gave it square dimensions, put it in a flexbox and centered it. Next I set its transition opacity and visibility to last 0.7 seconds, then I added loading-wheel-hidden to the loading-wheel class. Instantiating the base state of the loading wheel to be completely hidden, since I only want to reveal it during the page transitions while the user is waiting for the process to finish.

```
.loading-wheel{
  width: 200px;
  height: 200px;
  display: flex;
  justify-content: center;
  align-items: center;
  margin: 0 auto;
  transition: opacity 0.75s, visibility, 0.75s;
}

/*to hide the loader when the proccess is done*/
.loading-wheel-hidden{
  opacity: 0;
  visibility: hidden;
}
```

Figure 4.3.2.4

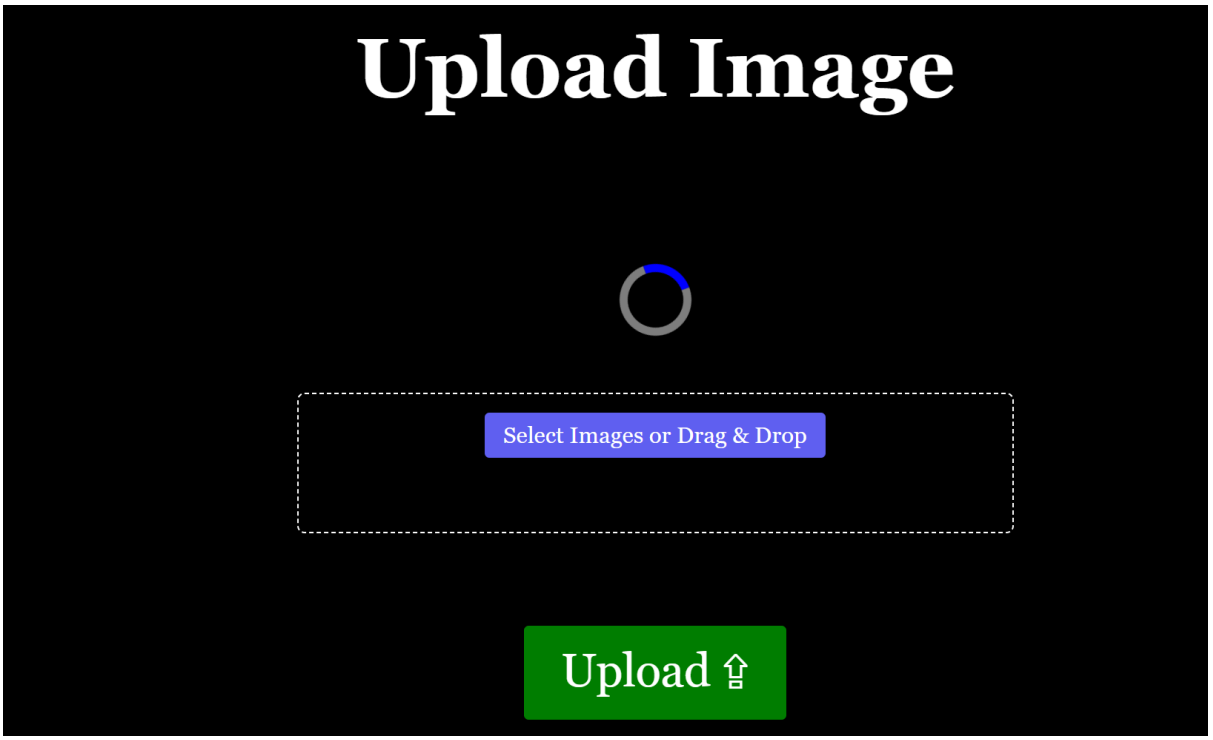
```
/*after the throbber is initalised this stuff is added so that the keyframes can be used to animate it*/
.loading-wheel::after{
  content: '';
  width: 60px;
  height: 60px;
  border: 9px solid grey;
  border-top-color: blue;
  border-radius: 50%;
  /*sets the keyframes to occur in 0.75s intervals, using ease it eases the two key frames together and then just keep going forever*/
  animation: loading 0.75s ease infinite;
}

@keyframes loading {
  from{
    transform: rotate(0turn);
  }
  to{
    transform: rotate(1turn);
  }
}
```

Figure 4.3.2.5

Next, as seen above in figure 4.3.2.5, I used the after keyword to initialise additional settings after the throbber is originally initialised so I can animate. I set its new dimensions, set content to ‘’ so that nothing else comes after the loading wheel, thick border with its top part being blue, and it being extremely curved to turn it into a circle with a 9px border radius. Then I will load the loading keyframes to animate, using the ease and infinite keywords to smoothen the movement and enable this loading-wheel to spin infinitely. Next I actually set the keyframes to repeat from 0 to 1 turns in 0.75 second intervals.

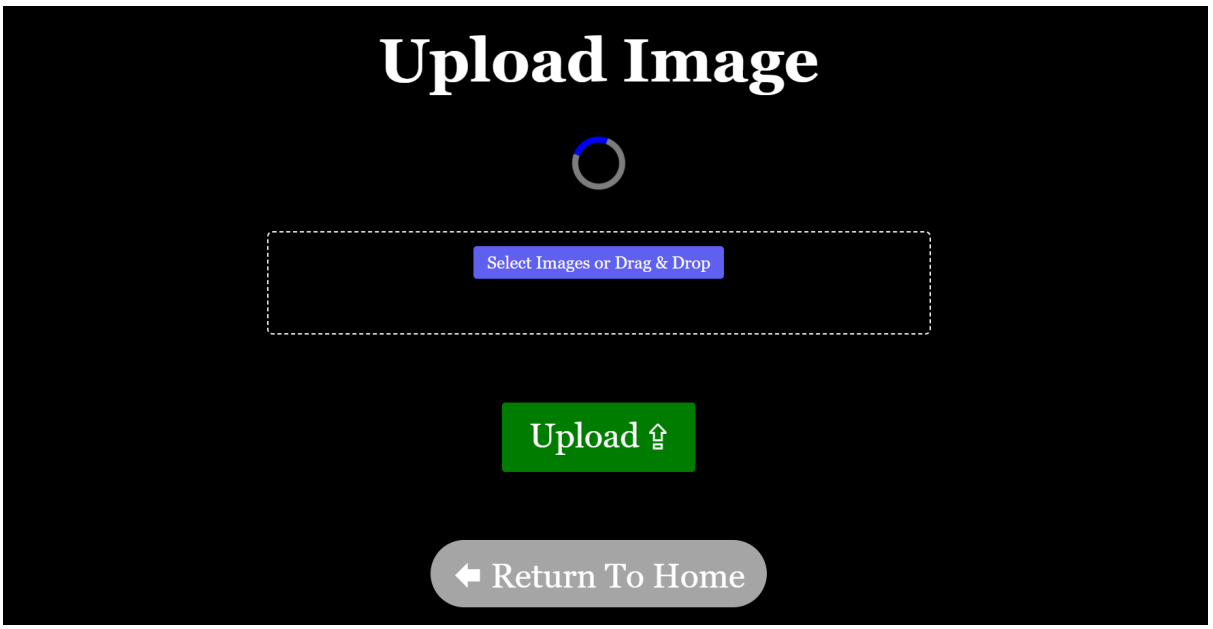
Video of the throbber working: THROBBER THROBBING(1).mkv

Currently I don't like where the throbber is so I will move it to be above the drop-zone so it is not too low:




```css
h1{
  color: white;
  text-align: center;
  font-size: 90px;
  font-family: Georgia;
  font-weight: bold;
  margin-top: 50px;
  margin-bottom: 0px;
}
```

Then as seen above I have reduced the margin of the title so it is closer to the title.



I like this better, so now I will edit it's JavaScript so it's only shown when you click the upload button, and that when a response is received the throbber is removed and you are redirected to a new page.

Before that I will start of with the throbber hidden:

```
<div class="loading-wheel loading-wheel-hidden"></div>
```

By calling the hidden class.

Then in the JavaScript I start of by initialising the loadingWheel by firstly referencing it from the HTML:

```
const loadingWheel = document.querySelector(".loading-wheel");
```

Also, I am using a query since it's a class and it is not an identifier.

```
function ShowThrobber(){
  loadingWheel.classList.remove("loading-wheel-hidden")
}

function HideThrobber(){
  loadingWheel.classList.add('loading-wheel-hidden');
}
```

Figure 4.3.2.6

Next in figure 4.3.2.6, I initialised basic functions to either show or hide the throbber by removing or adding the loading-wheel-hidden class from the overall loading-wheel classlist.

```
async function WaitUntillResponse(){
  return new Promise(resolve => setTimeout(resolve, 2000));
}
```

Figure 4.3.2.7

Furthermore, to simulate the wait of waiting for a response I have created an asynchronous function in figure 4.3.2.7, it will for now wait for 2 seconds and then continue. This will for now be used as a timer, and later will be replaced with the true fetch code to fetch the return data from the backend.

```
uploadButton.addEventListener("click", async (e) =>{
  if (ValidatePresence()){
    uploadButton.disabled = true; //prevent double clicks
    ShowThrobber();
    WaitUntillResponse()

    window.location.href = 'http://127.0.0.1:5500/Frontend/OutputPage/outputPage.html';
    HideThrobber();
    uploadButton.disabled = false;
  }
})
```

Figure 4.3.2.8.1

Finally in figure 4.3.2.8.1 I added an event listener to the upload button that will execute when the button has been clicked. It will then run the Validate presence subroutine to ensure that everything is valid and working as intended. Then it will disable the upload button to prevent any double clicks, then it will show the throbber, wait for 2 seconds, then send the user to the output page, hide the throbber and enable the upload button.

Result: SHOW_WEB_NULL_ERROR.mp4 and sometimes:
ATTEMPTED OUTPUT PAGE REDIRECT - FAILED (BACKEND).mkv

As seen above when I tried to execute my code it returned null to my website - though it also just sometimes froze it, I don't know why yet so hopefully this will go away when I fix those cases of the null error.
Which were occurring because in my main.py currently I am not returning anything back to the frontend after the post is completed, so using the fastapi.responses I will redirect users to the output page if all is valid and no errors arise. Else I will send them back to the input page to try again.

```
#import response types
from fastapi.responses import RedirectResponse
```

Figure 4.3.2.8.2

In figure 4.3.2.8.2 I imported a redirect response from fastapi.responses, this allows me to redirect the user to any webpage on the website.

```
    return RedirectResponse(url='http://127.0.0.1:5500/Frontend/OutputPage/outputPage.html'), proccessedText
except Exception as e:
    return RedirectResponse(url='http://127.0.0.1:5500/Frontend/InputPage/inputPage.html')
```

Figure 4.3.2.9

I then implemented this in figure 4.3.2.9, by returning the redirect response and the processed text. Else if there is an error then it will send them back to the input page, though the normal response is to just send the user to the backend output page with the error message; I believe it's worth it to trade that with this seamless transition back to the Input page if they made a mistake. Since more often than not it would be a user error rather than a server error, making me lean towards simply overriding the base response to instead send users back to the input page.

Video result: FAILED_TO_REDIRECT_USERS.mp4

It failed to redirect the users and just returned a response as a JSON instead of sending it back to the frontend. Also the error message repeated twice.

To combat this bug I fixed the code of figure 4.3.2.9 in figure 4.3.2.10:

```
#decorator to use the HTTP POST protocol into the defined path in the param
@app.post("/uploadfile/")
async def Create_Upload_File(file_uploads: list[UploadFile]):  #async funtion that will take in a list of files from the frontend
    counter = 0
    try:
        for file_upload in file_uploads:    #iterate over each file to save it to our directory
            data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
            save_to = Path().cwd() / 'uploads' / file_upload.filename #save a file in the predefined directory with the files name
            with open(save_to, 'wb') as f: #write to the files bytes
                f.write(data)   #write the read data, so we have completley stored this file in our backend
                counter += 1
            #img preproccess
            IMG_Preproccess(save_to)

            #inferenece
            #proccessedText = proccessedText + Inference(save_to) + '\n'    #INFERENCE DO IT LATER
        return RedirectResponse(url='http://127.0.0.1:5500/Frontend/OutputPage/outputPage.html', status_code=303)
    except Exception as e:
        return RedirectResponse(url='http://127.0.0.1:5500/Frontend/InputPage/inputPage.html', status_code=303) #CHOSE THIS RATHER THAN RETURNING AN ERROR MESSAGE
```

Figure 4.3.2.10

Firstly I realised that I needed to do a separate HTTP request back to the frontend to deliver the final processed string. Furthermore since all the image preprocessing, inference and processed string manipulation is occurring in a for loop each inference should be appended to the final string. Otherwise in terms of the redirect, I need to specify status code 303, this indicates to the client browser that it should redirect to the URL in the location header instead of doing its normal process and returning the url as a JSON output.

In terms of the HTTP request to send information back to the frontend I will be using a GET request as shown below in figure 4.3.2.11:

```
@app.get("/returnfile/")
async def Send_back_file():
    if proccessedText != '':
        return proccessedText
```

Figure 4.3.2.11

In figure 4.3.2.11 I define the GET request which will send information back to the frontend, I have defined a new endpoint of /returnfile/, and it will return processedText if it is not empty.

```
async function WaitUntillResponse(){
  //return new Promise(resolve => setTimeout(resolve, 2000));
  await fetch("http://127.0.0.1:8000/returnfile/");
}
```

Figure 4.3.2.12

Therefore since I now have a way to send back a response to the frontend I re-iterated over figure 4.3.2.7 in figure 4.3.2.12. By using fetch at the defined file path to get the get request. This will ensure that when the function is called whenever it will receive its answer the rest of the function will need to wait for it - therefore it becomes a dynamic timer that keeps waiting until the processed text is filled.

Finally to remove the double error messages I need to remove figure 3.3.2.32, since in figure 4.3.2.8.1 the ValidatePresence() is already being called, so having it called twice is redundant. Therefore since we need to have this function in figure 4.3.2.8.1 it makes sense to keep it there.

Video: SHOW_URL_REDIRECTION.mp4

However the final piece that doesn't fit is the fact that the lorem ipsum place holder text has not been replaced with yeet yet.
This is because we need to get the export.js to now receive this final string.

```
async function SetTextContent() {
  const response = await fetch("http://127.0.0.1:8000/returnfile/");
  const text = await response.text();
  textArea.textContent = text;
}

document.addEventListener("load", SetTextContent())
```

Figure 4.3.2.13

As seen in figure 4.3.2.13, I have created a new asynchronous function that will wait for a response from the GET request at returnfile, it will then convert it into readable text for the JavaScript and then set the text content of the text area equal to this text content. Finally an event listener is added to the document itself, so that when the output page has been loaded, it immediately starts the SetTextContent() function.

Finally, I noticed a logic error I accidentally included in main.py regarding figure 4.3.2.11, where it seems erroneous to return the final processed text exactly when it becomes not

empty, as that doesn't account for the fact that each for loop iteration appends to the text one by one, which does not leave this string empty while still being an incomplete process. Therefore it makes more sense to include an independent global boolean variable for this instead, modifying figure 4.3.2.11 into figure 4.3.2.14:

```python
isReady = False

#decorator to use the HTTP POST protocol into the defined path in the param
@app.post("/uploadfile/")
async def Create_Upload_File(file_uploads: list[UploadFile]):  #async funtion that will take in a list of files from the frontend
    counter = 0
    try:
        for file_upload in file_uploads:    #iterate over each file to save it to our directory
            data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
            save_to = Path().cwd() / 'uploads' / file_upload.filename #save a file in the predefined directory with the files name
            with open(save_to, 'wb') as f: #write to the files bytes
                f.write(data)   #write the read data, so we have completley stored this file in our backend
                counter += 1
            #img preproccess
            IMG_Preproccess(save_to)

            #inferenece
            global proccessedText
            #proccessedText =+ Inference(save_to) + '\n'    #INFERENCE DO IT LATER
        global isReady
        isReady = True
        return RedirectResponse(url='http://127.0.0.1:5500/Frontend/OutputPage/outputPage.html', status_code=303)
    except Exception as e:
        return RedirectResponse(url='http://127.0.0.1:5500/Frontend/InputPage/inputPage.html', status_code=303) #CHOSE THIS RATHER THAN RETURNING AN ERROR MESSAGE
      You, now • Uncommitted changes

@app.get("/returnfile/")
async def Send_back_file():
    if isReady:
        return proccessedText
```

Figure 4.3.2.14

Here I define a global boolean variable isReady and modify it at the end of the for loop, so the get request is sent at the very end when everything is processed. Furthermore, to change the values of these functions within the local scope of Create_Upload_File() function I must use the global keyword.

Finally the test footage of the entire website so far (excluding the use of the Chandra model):
PREFINAL_FINAL_C2.mp4
It works seamlessly, though you can't see the loading wheel very much, hopefully when the application runs with the Chandra model it will also work seamlessly as a component that is integrated smoothly within the frontend and backend.

Now since I will be using my friend's hardware to test my application I currently need to wait for a time when they are available to test it fully with the entire website. So far they could test the inference model itself however I will move onto cycle 3 for now while I wait for them to finish testing the website with the machine learning model.

4.4 Code annotation and naming conventions

I have regularly annotated most of the final code that I deem unintuitive or not very obvious due to the various reasons I have mentioned in 3.4 - the code annotations and naming convention section of cycle 1. Furthermore in terms of naming conventions, I have made all python functions follow Pascal Case but with an underscore between each word so that they also follow snake_case. Furthermore in accordance with best practice I have kept all my constant values in all capital letters and my variables in the basic camel case. In terms of file names I haven't imported that ‘as’ any abbreviated name since there is only one subroutine

in each file; therefore it's best to just import those functions directly. Otherwise I have also named my files using snake_case.

4.5 Evaluation

The greatest change I have indulged in from my original design is changing the progress bar into a loading wheel. However as I have stated before, I feel it is the best choice for the best user experience. Else I have mostly implemented all of the features I wanted for this cycle.

4.5.1 Feedback

Since my cycle 3 is mostly focused on further refinement and adhering to stakeholder opinions (as well as fixing any model integration bugs after my friend finishes testing it locally), I have conducted a basic interview with **7** people after letting them play around and test my website. This is so in my third cycle I can amplify my current web application into one that would definitely appeal to my target demographic; and since that is primarily students all of those interviewed are current Year 13 Sixth Form Students.

<u>Summary of their responses:</u>

Mr. Joe Woodall:

- Looks good, not the most visually appealing colour scheme in the world (it could be more coherent and have a singular colour palette throughout), not a big fan of the font; looks functional, upload button seems too confusing as it seems like it is the image selector itself - could be changed to process image button. Commented that it would help with their regular drama schoolwork, letting him organise his handwritten essays or works digitally.

Miss Agnes Radek:

- Boring colour scheme, not visually appealing - maybe a more colourful palette would be appropriate. Converted text is too small, add in a zoom button for those not as tech literate enough to manually zoom in. Rudimentary but effective functionality, web page format is well designed overall - everything is clearly conveyed to the user. The ability to edit the translated text is extremely helpful in correcting any errors in the converted text and works as an intuitive way for the user to add their own input; making the process more efficient.

Mr. Rhodri Chow:

- Found it very intuitive to use, would prefer a lighter colour scheme and would generally be fine with using it if the time arises. The further instructions seem clear, to the point and helpful for the user to navigate through the website. I loved the overall design of the website though, I love how everything reacts to what I do and it just has a very chill vibe to it.

Mr. Burney Feng:

- Doesn't like Georgia font, suggested making it a lighter colour scheme, right now it seems personally unapproachable and the white in the output page contrast too heavily with the black making it a nightmare if users were to use this in the night. Else, the functionality seems normal, nothing out of the ordinary and they can tell what they need to do at each stage. As a senior prefect of the diversity team they have a bigger workload than most so they commented that this application would be very useful for their day to day life for organisation or just basic studying.

Miss Charlotte Prock & Miss Avani Riat:

- It is too dark and it's not a very inviting colour scheme, they would suggest to keep it minimalist but also add a little bit of pazzazz or character to it. Feels very easy to use, and easy to work out how to use; it is extremely user friendly, doesn't bombard you with any unnecessarily long instructions and “you can get right to it”. Miss Prock especially loved the font immensely, she said it felt very nice and inviting, it was homely yet simultaneously formal.

Mr. Dogu Ozmen:

- To quote: “please change that font”, wasn't an especially big fan of the font, otherwise commented that the program itself was very impressive, it looks to be a powerful tool that seems easy to use. They would definitely use it themselves if the opportunity were to ever arise.

4.5.2 Features to Re-iterate in future cycles

Feature to add	Justification	Accomplishability
Add a local SSL certificate to the website	To allow for non-local clients to be able to use the copy button. Also it simply just adds to the overall robustness of the website by making it secure, giving it the lock icon next to the website URL - which can lead to people trusting the website more greatly.	Possible to add a local SSL certificate however current POST and GET requests are HTTP, so there might be a possibility that I would need to replace them with HTTPS compatible requests.
Zoom feature in the editable text area	Adds greater visibility for images with a lower amount of word content, is especially helpful for those that are less tech literate and was directly suggested by an interviewed student.	Currently unknown regarding its complexity, it will be key to add two separate buttons for zooming in and out however I would need to actually work on it in cycle 3 to evaluate the feasibility of implementing this on the editable text field itself.
An improved and more cohesive colour palette	It was commented upon by almost all interviewed students that they disliked the colour palette. Therefore as the primary stakeholders for this application it seems as the best course of action to follow their advice and lean into a palette that is lighter, brighter and has more character.	Extremely easy to accomplish, I can generate a random colour palette and easily replace all of the colour elements in the website.

Improve the font of the entire website.	Despite my best efforts to pick out a font that would appeal to my target user demographic; they all wanted it to be changed - except for one amazing individual who loved the font, but sadly it is imperative to follow the opinion of the majority of those interviewed, as it should be the most accurate representation of the opinions of my primary stakeholders.	Extremely easy to implement, I will just need to research more on a font that is well received by many and simply change the font values for everything in the website.
Change the name of the Upload Button into a more appropriate name such as Process Image.	One interviewed student expressed concerns regarding the clarity of this button, and in retrospect I agree.	Extremely easy to accomplish, I simply need to change the name of that button.
Need to fix drag and drop file transfer to the backend.	Current drag and drop doesn't work, it doesn't send the files back to the backend.	Currently unknown the accomplishability of this, since this is a bug that I am attempting to solve.

5. Cycle 3

5.1.1 Cycle overview

The aim of this cycle is to complete previous tasks that I was not able to complete in previous cycles, as well as implementing more features to cater towards my primary stakeholders by using the ideas given to me by the survey I did at the end of cycle 2. Overall this cycle will focus on the refinement and polishing of the web application.

5.2 Success criteria

Main Goal: Fix previous cycle bugs, implement features and improvements suggested by my target demographic.

<u>Bug Fixes:</u>

- Fix the drag and drop bug, where it would return a permission error if anyone tried to upload drag and dropped images
- A SSL certificate to enable HTTPS requests so users can use the copy button, also just adds an extra layer of trust and assurance from the users regarding the website
- Fix any issues with the model integration if they arise when my friend tests the model

<u>New Features to implement:</u>

- New website colour scheme

- New website font
- New zoom in and out features for the text area in the output page

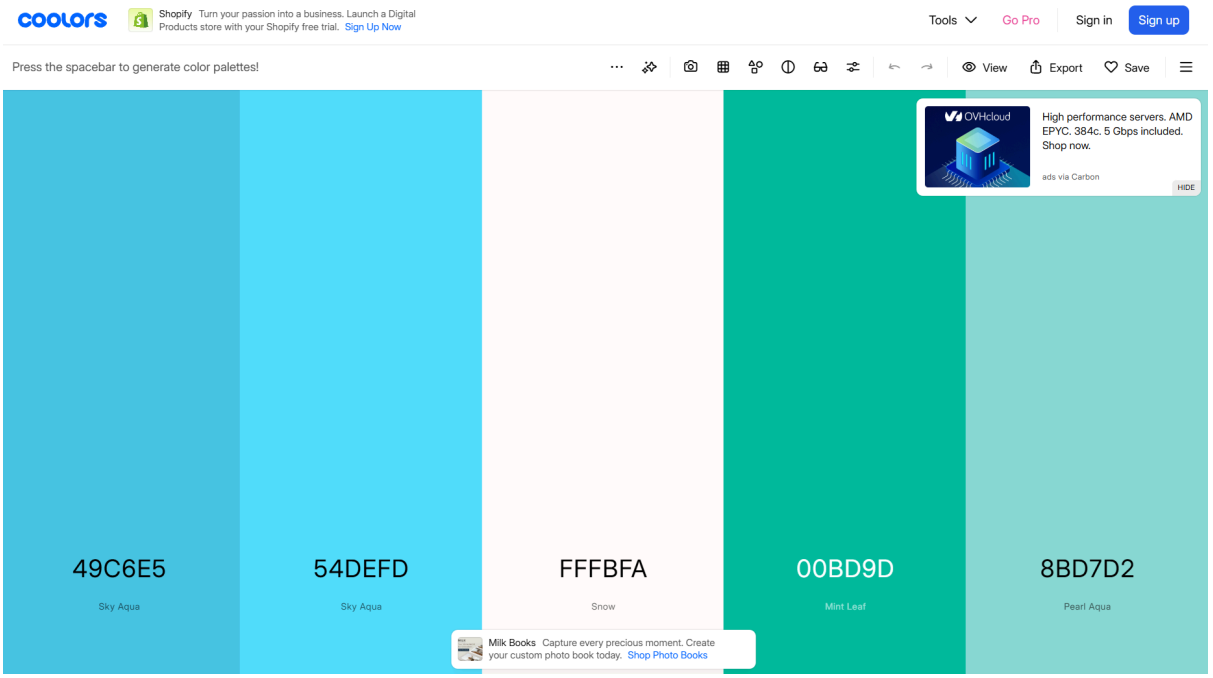
5.3 Iterative Development

5.3.1 Additional features

Colour scheme change:

To decide on a more appealing colour scheme I decided to check out random colour palette generators. Since I was told to make every web page have a singular cohesive colour palette for consistency, I decided using a colour palette would be useful. Therefore I used coolors to generate a random colour scheme.

Generated this colour scheme:



I love this colour scheme and started to implement it to all the CSS of my different web pages. My original plan is to make all the text white, have Pearl Aqua as the background and use Mint Leaf to highlight certain parts of the website. Otherwise I will be using the different shades of Sky Aqua for my buttons to make them pop out from the Pearl Aqua background.

```css
h1 {
    color: white;
    text-align: center;
    font-size: 70px;
    font-family: Georgia;
    font-weight: bold;
}

body{
    background-color: #8BD7D2;
}

.box{
    background-color: #00BD9D;
    width: 100%;
    height: 200px;
    border: none;
```

Figure 5.3.1.1

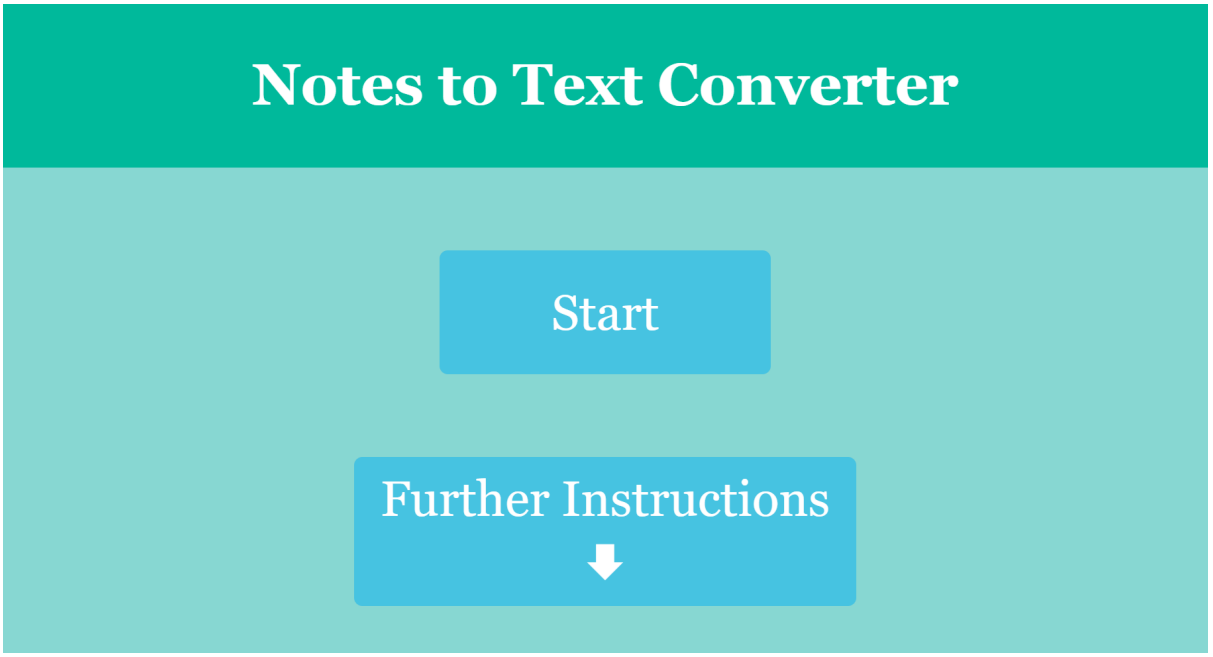
I started off by first changing all the Home Page CSS, as seen in figure 5.3.1.1. I first kept the heading colour to white but I changed the background to pearl aqua and made the box have a mint leaf colour, this is in support of my previous point, as this will let the box stand out against the pearl aqua background with its darker tones.

```css
.button{
    border: none; /*removes any visible border around element*/
    color: white; /*sets text colour as white*/
    padding: 16px 32px; /*adds space inside of the button to make it larger and easier to click*/
    text-decoration: none; /*removes anything like underlines*/
    display: block; /*turn it into a block to make it easier to centre and manipulate as it takes up its own horizontal space*/
    font-size: 25px; /*sets font size*/
    margin: 4px 2px; /*adds sapce outside the element*/
    transition-duration: 0.4s; /*sets the transition dration when hovering*/
    cursor: pointer; /*changes mouse pointer into hand icon to show its clickable*/
    background-color: #49C6E5; /*changes background colour of the button to dark grey*/
    font-family: Georgia;
}

.button:hover {
  background-color: #54DEFD;
  color: whitesmoke
}
```

Figure 5.3.1.2

Next in figure 5.3.1.2 I changed the buttons background colour to the darker sky aqua colour while setting the hover colour to the light version. I did this so that it's easier to discern the visual effect after the users interact with the button, furthermore since most of the people in my survey preferred the use of lighter and vibrant colours it makes more sense for the hovering colour to be bright.

Result:



homePage.css M | inputPage.css M | outputPage.css

Frontend > InputPage > # inputPage.css > .file-upload

```css
body{
  background-color: #8BD7D2;
}

h1{
  color: white;
  text-align: center;
  font-size: 90px;
  font-family: Georgia;
  font-weight: bold;
  margin-top: 50px;
  margin-bottom: 0px;
}

.button{
  border: none;
  display: block;
  color: white;
  padding: 16px 32px;
  font-size: 25px;
  margin: 0 auto;
  transition-duration: 0.4s;
  cursor: pointer;
  background-color: #49C6E5;
  font-family: Georgia;
}

.button:hover {
  background-color: #54DEFD;
  color: whitesmoke
}
```

Figure 5.3.1.3

Next for the input page I had a similar idea in terms of design, with a white text title, pearl aqua background with the same combo of dark and light sky aqua buttons to continue the consistency of my website and appeal to those in my userbase who want a more consistent theme throughout the frontend.

Next as seen below in figure 5.3.1.4 I have added more specific colour choices for the Input page, firstly for the entire border of the drop-zone I have set it to mint leaf - for a similar reason why I set the box in the home page to mint leaf as well - so that it stands out against the pearl aqua background while also simultaneously not clashing with the background, rather contrasting against it well. Finally I have left the Return to Home button in the same generic colour form as the home page for consistency.

```css
.file-upload{
  width: 50%;
  font-family: Georgia;
  font-size: 25px;
  display: block;
  padding: 20px;
  border: 2px dashed #00BD9D;
  border-radius: 8px;
  transition: border-color 0.3s;
  gap: 15px;


  /*To centre elements in the div*/
  display: flex;
  flex-direction: column;
  align-items: center;

  /*To centre the div itself*/
  margin: 0 auto;
}

.file-upload.dragover {
  border-color: rgb(96, 96, 244);
  background-color: rgba(0, 123, 255, 0.1);
}

#fileInput {
  display: none; /*Hides the ugly upload button so we can add our own custom UI*/
}

.file-upload label {
  display: inline-block;
  padding: 10px 20px;
  background-color: #49C6E5;
  color: white;
```

Figure 5.3.1.4

Furthermore, as seen at the end of figure 5.3.1.4 and the start of figure 5.3.1.5, I have further modified my CSS code to set my custom Select Images button's background to the current button style. Since I have removed the original file input button - due to its inability to conform to my stylistic designs - instead of editing a button, I need to modify the values of the label inside of the drop-zone.
Additionally, in terms of the colour of the error message I have kept it the same, since it is an error it needs to stand out and it will especially stand out in the present colour scheme in contrast to the blues and greens compared to the previous black background.

```css
.file-upload label:hover {
  background-color: #54DEFD;
}

#file-preview {
  margin-top: 20px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

#file-preview img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  border-radius: 4px;
  margin: 0 500px;
}

#file-preview span{
  /*for the image text displayed below the text*/
  font-family: Georgia;
  color: white;
  font-size: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
}

#error-message {
  display: block;
  color: red;
  margin-top: 10px;
}
```

Figure 5.3.1.5

However in figure 5.3.1.6 I needed to stray away from the shades of sky aqua as my button colours, since having three buttons in a row have the same aesthetic may seem lazy, unimaginative and simply not a good style to look at. Therefore to have some variety I have the upload button to have the leaf mint colour with a regular green hover colour. This breaks the chain of sky aqua buttons and adds some more variety to the colours for the input page; and also highlights the process button as the main button of attention - since it is in the middle of the two buttons and is the only one coloured differently - which is exactly what I want the users attention to draw to, since I want them to continue using the application and process their images.

```css
#uploadButton {
  display: block;
  padding: 20px 40px;
  background-color: #00BD9D;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.3s;
  font-size: 50px;
  font-family: Georgia;

  margin: 100px auto;
}

#uploadButton:hover {
  background-color: green;
}
```

Figure 5.3.1.6

```css
/*after the throbber is initalised this stuff is added so that the keyframes can be used to animate it*/
.loading-wheel::after{
  content: '';
  width: 60px;
  height: 60px;
  border: 9px solid grey;
  border-top-color: blue;
  border-radius: 50%;
  /*sets the keyframes to occur in 0.75s intervals, using ease it eases the two key frames together and then just keep going forever*/
  animation: loading 0.68s ease infinite;
}
```

Figure 5.3.1.7

For the same reasons as the error message, I haven't really changed any of the aesthetic details of the throbber in figure 5.3.1.7. Since the blue and grey components just seem natural, basic working components, especially since there is less of a contrast between everything due to the removed previous black background.

Result:

So far, I feel that reiterating over the website to change the colour palette was the best move. As all of the button hovering has improved substantially, moreover the website overall seems much softer, brighter and more vibrant.

```
body{
    background-color: #8BD7D2;
}

h1{
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-family: Georgia;
}
```

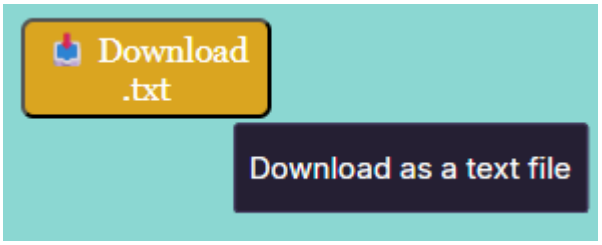
Figure 5.3.1.8

Finally for the Output page I have started off by simply changing the background colour to match the previous pages and keeping the title text to be white, as seen in figure 5.3.1.8.

```css
.copyB:hover{
    background-color: blueviolet;
    color: white;
}

.docxB:hover{
    background-color: blue;
    color: white;
}

.pdfB:hover{
    background-color: red;
    color: white;
}

.txtFileB:hover{
    background-color: goldenrod;
    color: white;
}
```

Figure 5.3.1.9

However when I then tested the website with the new background colour, I felt that when the user hovered over each export button, keeping the text colour black felt too bland. So I implemented the code in figure 5.3.1.9, which made it so that the text colour changes to white when hovered over. However after testing it, the colour of the download as text file button felt too bright - making it harder to read with such a light yellow background. Therefore I swapped to a more golden colour as seen below, it made the text more readable while letting me adhere to visual practices that I deem the best to implement.

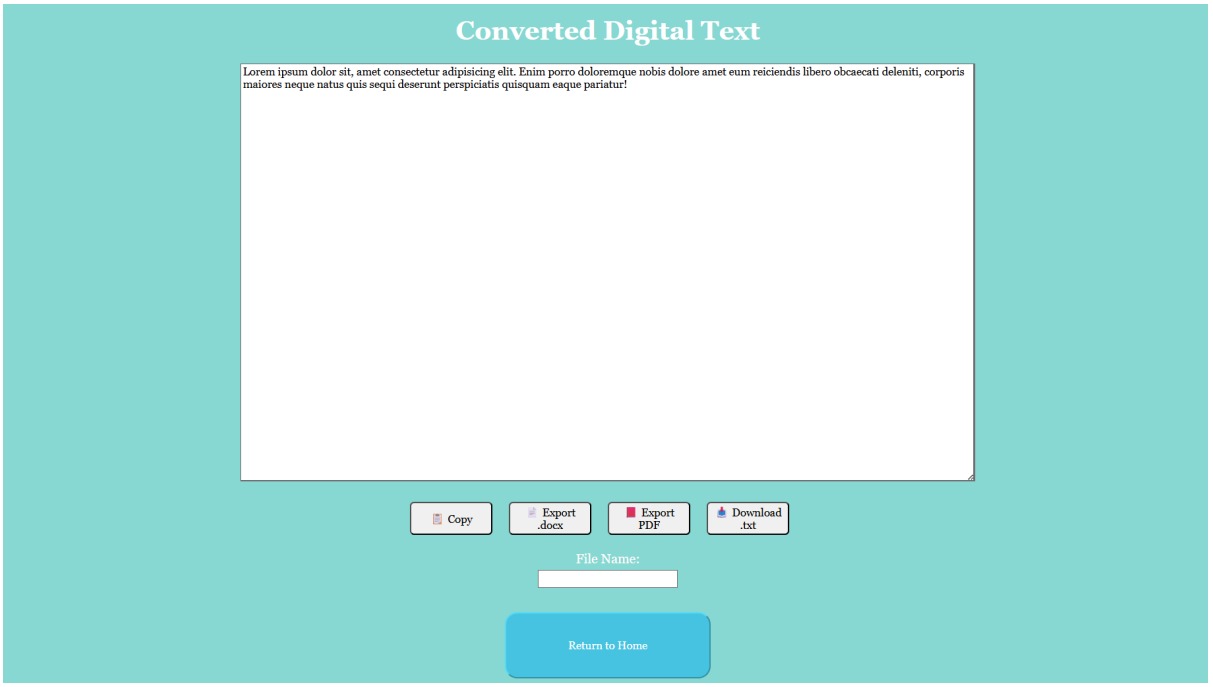


```css
#returnButton{
    width: 250px;
    height: 80px;
    border-radius: 15px;
    border-color: #54DEFD;
    display: block;
    margin: 0 auto;
    margin-top: 30px;
    background-color: #49C6E5;
    color: white;
    transition-duration: 0.4s;
    cursor: pointer;
    font-family: Georgia;
}

#returnButton:hover{
    background-color: #54DEFD;
    color: whitesmoke;
}
```

Figure 5.3.1.10

Finally, as seen above in figure 5.3.1.10, I changed the colour of the return button, but I used the darker sky aqua for its resting colour but used the lighter version for its hovering and as its border. It not only adds variety to the website by using the colour palette with more combinations of the different colours but also the lighter border colour gives it a sort of bolden look. Making it feel special after the user has spent so much time reading instructions, uploading images and waiting for them to be processed.

Result:




```
<button id="uploadButton" type="submit" form="uploadFile">Proccess Image ⇧</button>
```

Figure 5.3.1.11

Then in the input page's HTML, seen above in figure 5.3.1.11, I modified the upload buttons text content to display Process Image instead of upload due to a complaint by an interviewed student. Which in reflection makes everything more clear, effectively informing the user the difference between pressing the select images button and the upload button.

Result:

Then, I started my search for a better font. Overall, mostly everyone thought that the original Georgia font looked too basic and was reminiscent of older web pages. Therefore I needed to use a font that would appeal to the younger demographic, something that seems comfortable and fresher. So after a few recommendations from my interviewees I looked at the Comfortaa font. I feel that it fits as an adequate font for my user interface as it looks clean, soft, and has a highly legible rounded aesthetic that would match most of my rounded elements in the frontend.

```
@import url("https://fonts.googleapis.com/css2?family=Comfortaa:wght@300..700&display=swap");

h1 {
    color: white;
    text-align: center;
    font-size: 70px;
    font-family: Comfortaa;
    font-weight: bold;
}
```

Figure 5.3.1.12

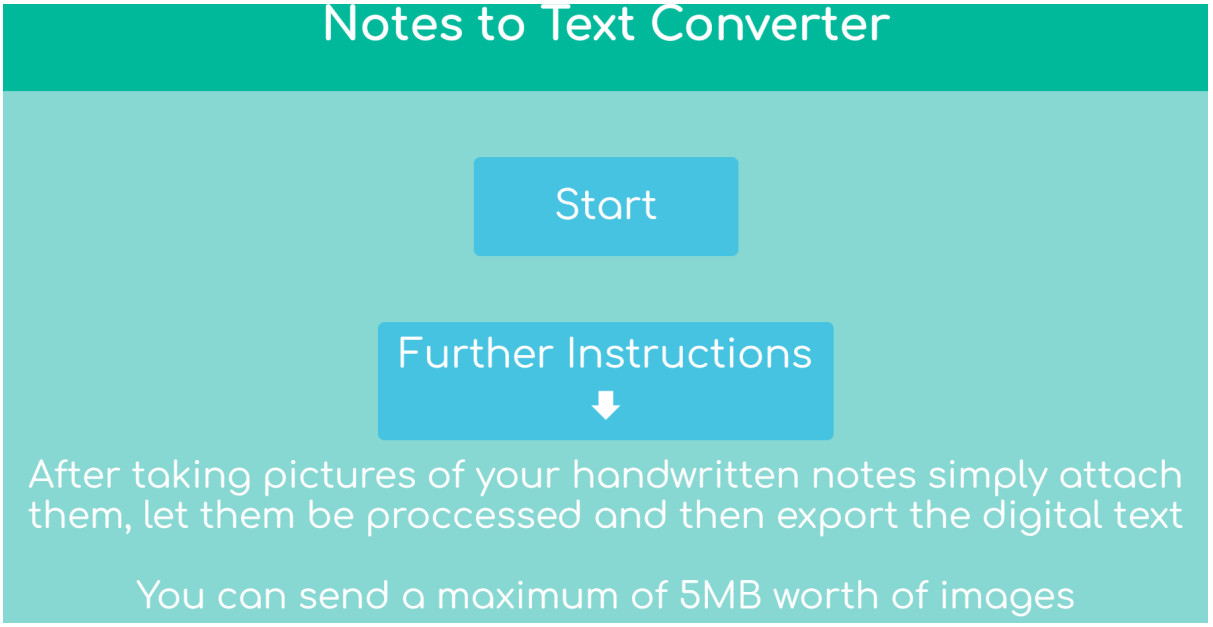
In figure 5.3.1.12 I imported a URL of Comfortaa from google fonts, this will be added to the top of each of my CSS files. Then I started to add it to each component in my website

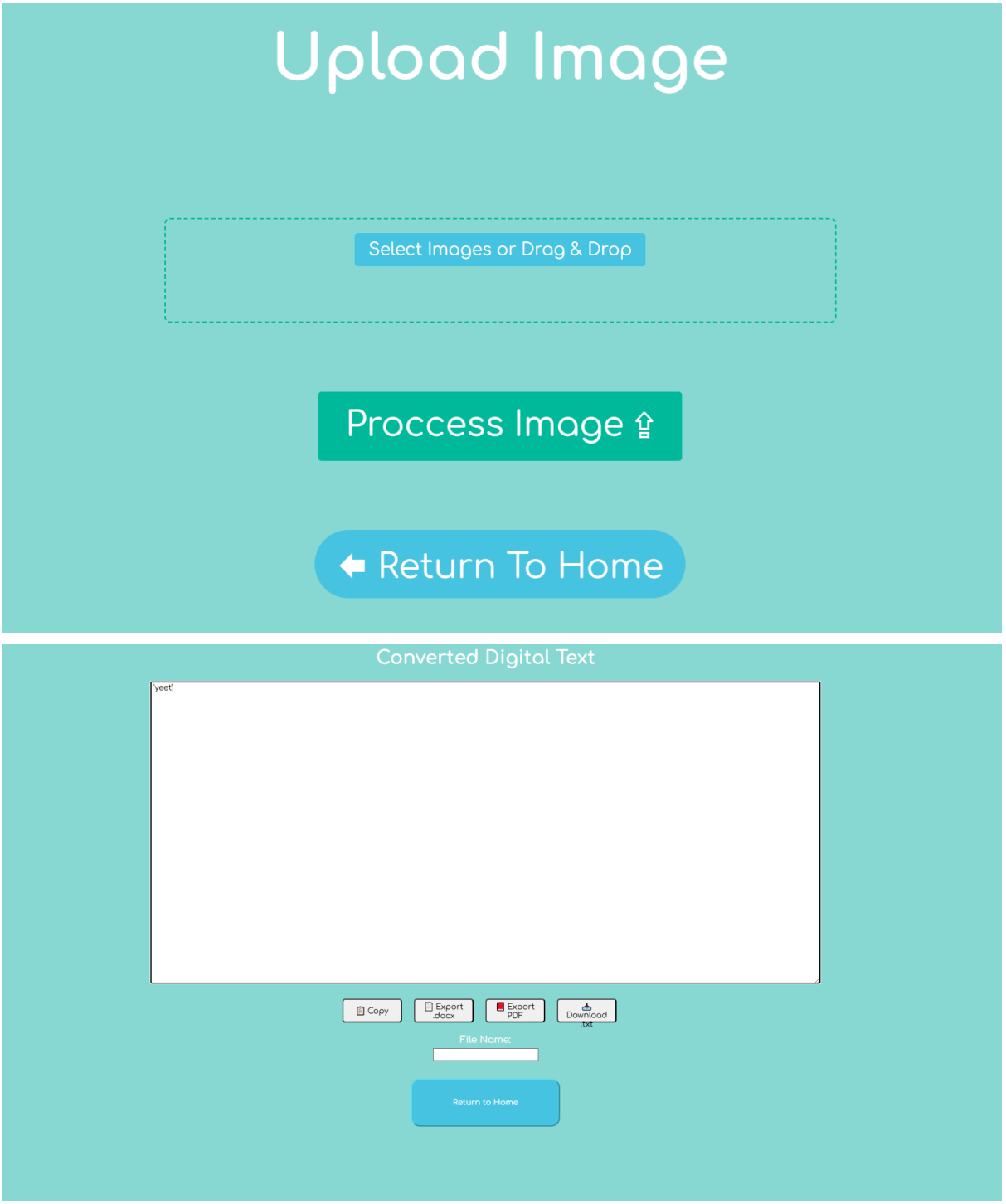
```
font-family: Comfortaa;
```

Figure 5.3.1.13.1

Next, as seen in figure 5.3.1.13.1 I then amended each reference of the font-family: Georgia with Comfortaa.

The result of using the Comfortaa font are shown below:



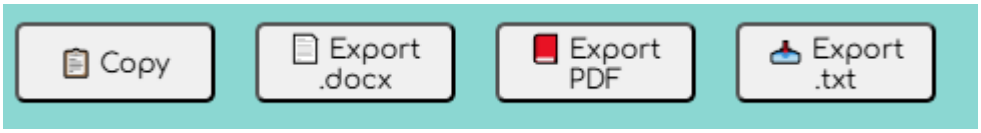


As seen in the picture of the output page above, it seems that the download button's text is overflowing due to my new UI changes. I will fix it by changing its text content, as seen below in figure 5.3.1.13.2.

```
<button class="button txtFileB" id="txtB" onclick="ExportAsTxt()" title="Download as a text file">📥 Export .txt</button>
```

Figure 5.3.1.13.2

Now not only does the export to a text file button match the other export file buttons, but it also looks much cleaner in the new UI without overflowing itself. Providing the result below, which saved me the pain of having to resize every single button to fix the overfill bug.



Result of all the UI changes: Show_new_colours.mp4

Afterwards, I started to work on the Zoom features.

```
<label id="zoomLabel">Adjust
Text Size</label>

<div class="zoom-buttons">
  <button id="zoomIn">+</button>
  <button id="zoomOut">-</button>
</div>
```

Figure 5.3.1.14

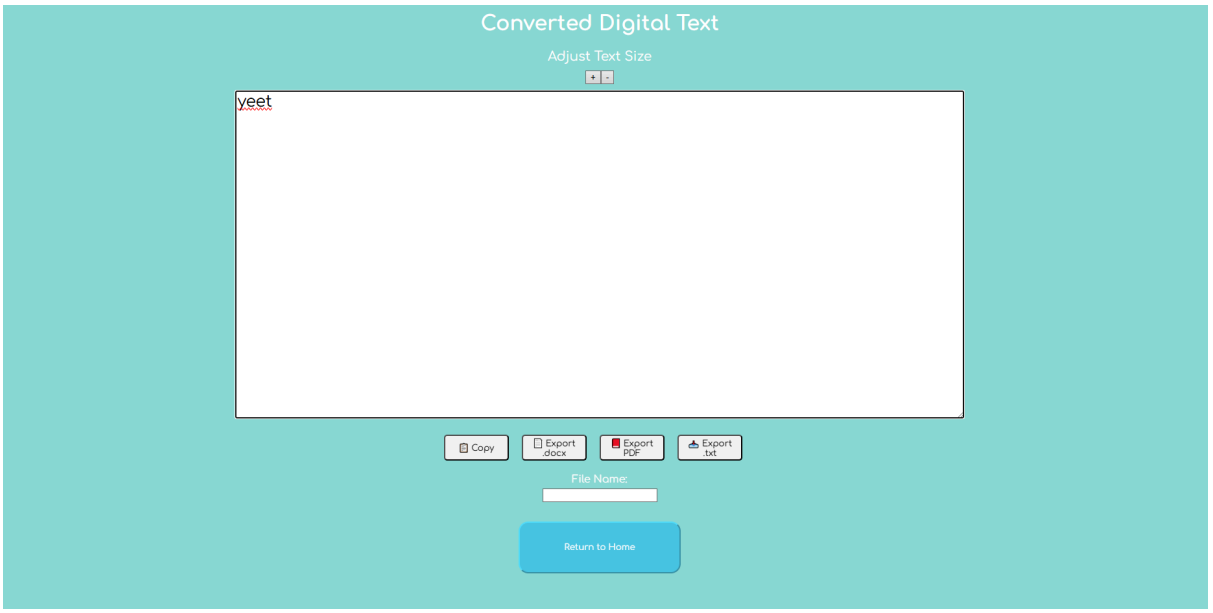
I started off by adding a label identified as zoomLabel, and a div class called zoom-buttons that will contain the zoom buttons in my output page HTML - as shown above in figure 5.3.1.14. The label will act as the instruction to clarify why there are two buttons with a plus and minus sign. Furthermore, the zoom-buttons div will have two buttons named zoomIn and zoomOut, they will have a simple plus and minus sign since the label can clarify the use of the buttons while the buttons themselves can remain simple.

Then as seen below in figure 5.3.1.15, I adjusted the CSS of the output page by creating a flex box for the two buttons so that they are treated as a singular entity in regards to centering and position. I then set position to relative, centering them and then adding some vertical margin space between them and other elements. Then for the actual label I keep it relatively small at 20px since it is only an instructional label, and set its font family. I purposefully didn't set the cursor to become a pointer when hovering over the button, since they are smaller, more simple utility buttons, the basic cursor seems more appropriate than the pointer.

```css
.zoom-buttons{
    position: relative;
    display: flex;
    justify-content: center;
    align-items: center;
    margin: 10px auto;
}       You, now • Uncommitted cha

#zoomLabel{
    font-family: Comfortaa;
    font-size: 20px;
}
```

Figure 5.3.1.15

This results in the following UI:



As you can see in the result above I have not really customised the style of the zoom in and out buttons themselves, this is because I want those buttons to match the export buttons; so it groups all of these buttons together to almost wrap the editable text field. I want them to overall seem a little plain, so without even reading the label above the buttons, it's clear to the user that those buttons are related to the text field and that it's obvious what their function is. And the reason I want to keep it plain is so that these buttons have a more minimal styling (just simple text labels with light borders), to create a stark contrast against the background. This makes them feel crisp and intentional, since they are simply utility buttons it gives a sense of form over function in an unobtrusive way.

```javascript
const increaseButton = document.getElementById("zoomIn")
const decreaseButton = document.getElementById("zoomOut")
```

Figure 5.3.1.16

Finally to make these buttons functional I added the contents of figure 5.3.1.16 to the JavaScript file export.js that is attached to the output page. I first call the two buttons via their identifiers.

```
function UpdateFontSiz(increment) {
  let currentSize = getComputedStyle(textArea).fontSize; //use let so I can reassign this var, and reads current actual style being used
  currentSize = parseInt(currentSize); //converts xpx into just x

  if (currentSize <= 1){  //if less than or eq to one keep it at one, extreme case
    textArea.style.fontSize = '1px';
  }
  else if (currentSize >= 100){ //if greater or eq to 100 keep it at 100, extreme case
    textArea.style.fontSize = '100px'
  }
  else{
    textArea.style.fontSize = `${currentSize + increment}px`;  //else change the sizeD
  }
}
```

Figure 5.3.1.17

Next as seen in figure 5.3.1.17 I created a new function called UpdateFontSize() that takes in a value as the parameter increment. This function first defines a variable that stores the actual size font of the current style, but since it is from CSS it will be in the form e.g. 20px, so I need to parse it so it's stored as an integer. Then if the current size of the font is less than one, the text ‘s font size will be limited at one; this procedure is also reciprocated for the polar extreme case - where if the size is greater than a 100, it will cap the size to a 100. Else everything should be working as it should, so the font size of the text area gets incremented by the value of the increment parameter.

```
increaseButton.addEventListener("click", () => {
  UpdateFontSiz(1);
})

decreaseButton.addEventListener("click", () => {
  UpdateFontSiz(-1);
})
```

Figure 5.3.1.18

So then in figure 5.3.1.18 I added the function shown in figure 5.3.1.17 to an event that's attached to both the increase and decrease button. So that when they are clicked the font size is updated by either 1 or -1.

Result of my code, shows the zoom feature works perfectly: Zoom_TextArea.mp4

5.3.2 Bug fixes

After adding all of these features my friend tested the individual inference model itself to make sure that Chandra was working and was fully functional:
Testing_inference_model.mkv

Overall, though it proved to take a while to process the image it provided an extremely accurate result; and if the computer wasn't also dealing with recording this process it would normally only take 20-30 seconds. This could be further improved with dedicated hardware or a virtual large language model server that could be implemented before and during deployment. However with my current budget and expertise it is still the best option to simply test the server locally for now.

After testing the individual inference module itself, when they attempted to test my website and my models integration with it; they finally came back to me and told me there was an error for when the model returned a value back to the main code base.
It was the result of uncommenting the line in figure 5.3.2.1.

```
#inferenece      You, 4 days ago • Finished in
global proccessedText
proccessedText =+ Inference(save_to) + '\n'
```

Figure 5.3.2.1

Where the Inference model should be a returned string.

I recreated it locally on my computer by having the Inference() function return a simple string without calling the Chandra model: model_integration_error.mp4
So the image is sent to the backend and preprocessed for the OCR model, but the page is not redirected to the output page, furthermore there was a cancelled request from the output.html, and on top of that the throbber itself isn't shown while it is being loaded.

```
.loading-wheel-visible{
  visibility: visible;
}
```

Figure 5.3.2.2

To fix the throbber issue I modified the original procedures that actually enabled and disabled it. Firstly I created a whole new class for the throbber that sets its visibility to be visible as seen in figure 5.3.2.2.

```
function ShowThrobber(){
  loadingWheel.classList.remove("loading-wheel-hidden");
  loadingWheel.classList.add("loading-wheel-visible");
}

function HideThrobber(){
  loadingWheel.classList.remove("loading-wheel-visible");
  loadingWheel.classList.add('loading-wheel-hidden');
}
```

Figure 5.3.2.3

Then in Figure 5.3.2.3 I modified figure 4.3.2.6 so that when ShowThrobber is called if loading-wheel-hidden was in the class list it is removed before loading-wheel-visible is added, I then repeated this in vice versa for HideThrobber.

The resultant video of a perfected throbber: show_throbber_working.mp4

Furthermore after carefully overloading the server with so many upload button requests I was able to slow it down enough so that I could read the cancelled request in the network console that pointed towards the outputPage.html. And it showed that it was occurring in line 172 of the code of figure 5.3.2.4 below:

```
166 uploadButton.addEventListener("click", async () =>{
167   if (ValidatePresence()){
168     uploadButton.disabled = true; //prevent double clicks
169     ShowThrobber();
170     WaitUntillResponse();
171
172     location.assign('http://127.0.0.1:5500/Frontend/OutputPage/outputPage.html');
173     HideThrobber();
174     uploadButton.disabled = false;
175   }
176 })
```

Figure 5.3.2.4

I believe that since I was basically attaching two different HTTP requests to the upload button simultaneously - even without the minor timer at line 170 to offset the second request - so neither one is cancelled. However after reflecting upon this code I realised that it was completely redundant and unnecessary and that I could simplify it to the new & improved version in figure 5.3.2.5 below.

```
uploadButton.addEventListener("click", async () =>{
  if (ValidatePresence()){
    ShowThrobber();
  }
})
```

Figure 5.3.2.5

Basically the only core components I needed was calling ValidatePresence() to validate the input and return an error message if it is invalid and then simply show the throbber for the rest of the input pages life cycle. Since in main.py I already return a redirect response to the output page, so I can already eliminate the source of the original cancelled request I got during testing - which also means I don't need to call WaitUntillResponse() and wait until I get a response from the get request from the backend, since I can just display the throbber until the backend is done and once it's done then it'll return the redirect response by itself. And when the page gets redirected it will reset the entire page so it will do the job of HideThrobber() automatically, finally the double click of the Upload button is also automatically prevented due to the HTTP request. Therefore I can cut down to the code left in the figure above.

Next the main goal is to figure out which error is occurring in the server itself, since beforehand in figure 4.3.2.10 I set the except condition to simply return the users back to the input page due to my confidence in my program - which I need to disable for now due to my short sight on the account of further errors happening in the backend server. Therefore I appended figure 4.3.2.10 with the temporary figure 5.3.2.6:

```
except Exception as e:
    #return RedirectResponse(url='http://127.0.0.1:5500/Frontend/InputPage/inputPage.html', status_code=303) #CHOSE THIS RATHER THAN RETURNING AN ERROR MESSAGE
    return {f"{e}"}
```

Figure 5.3.2.6

So then when I tested the program again I was met with this returned error:

Which meant that my string handling syntax was incorrect, this led me to correct the code in main.py that handled the string handling of the processedText variable to the code in figure 5.3.2.7, as seen below.

```
global proccessedText
proccessedText += Inference(save_to) + '\n'
```

Figure 5.3.2.7

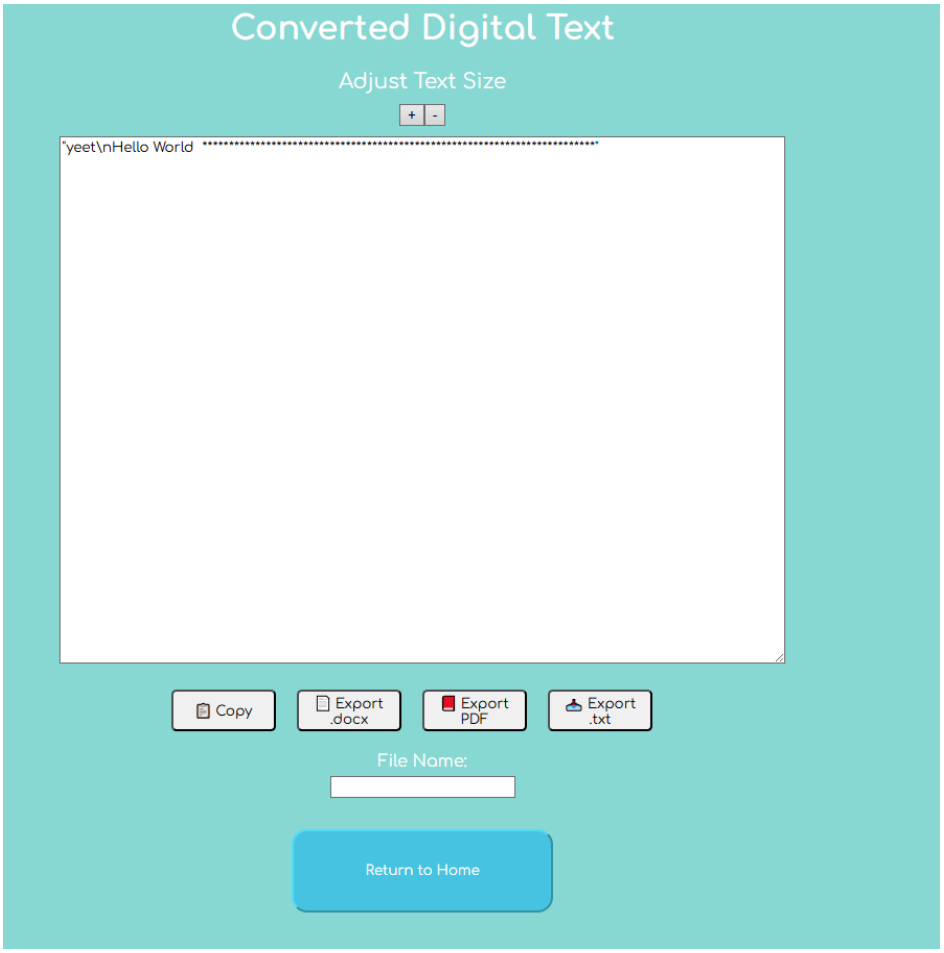
This resulted in this output in the main website pipeline:
Show_model_integration_test1.mp4
The video above showed a massive success regarding the integration between the frontend and backend, the only current problem is the backslash n not being recognised properly. I deduced that since I was passing this entire string to become the new value of the text area's text content, a new line wasn't being entered since \n is exclusively for python string handling - so the text area doesn't recognise it has to do something special when it sees the \n.

```
proccessedText += f"\n{Inference(save_to)} **********************************************************************"    #INFERENCE DO IT LATER
```

Figure 5.3.2.8

So I implemented a more rudimentary solution for the problem in figure 5.3.2.7, by adding a bunch of stars and spaces as seen in figure 5.3.2.8. So that if multiple images are used they are separated by stars. This is even more beneficial for an editable text field since the users can simply use their own judgement and remove stars and add their own separations. Therefore this method - that might seem a little crude - can become a valuable solution for this problem by working alongside the editable text area; though I wish I could implement a proper and more sophisticated method there are other more important things to accomplish for my cycle 3 that are better uses of the limited time I have allocated for this cycle.

Result of using the stars is shown below:

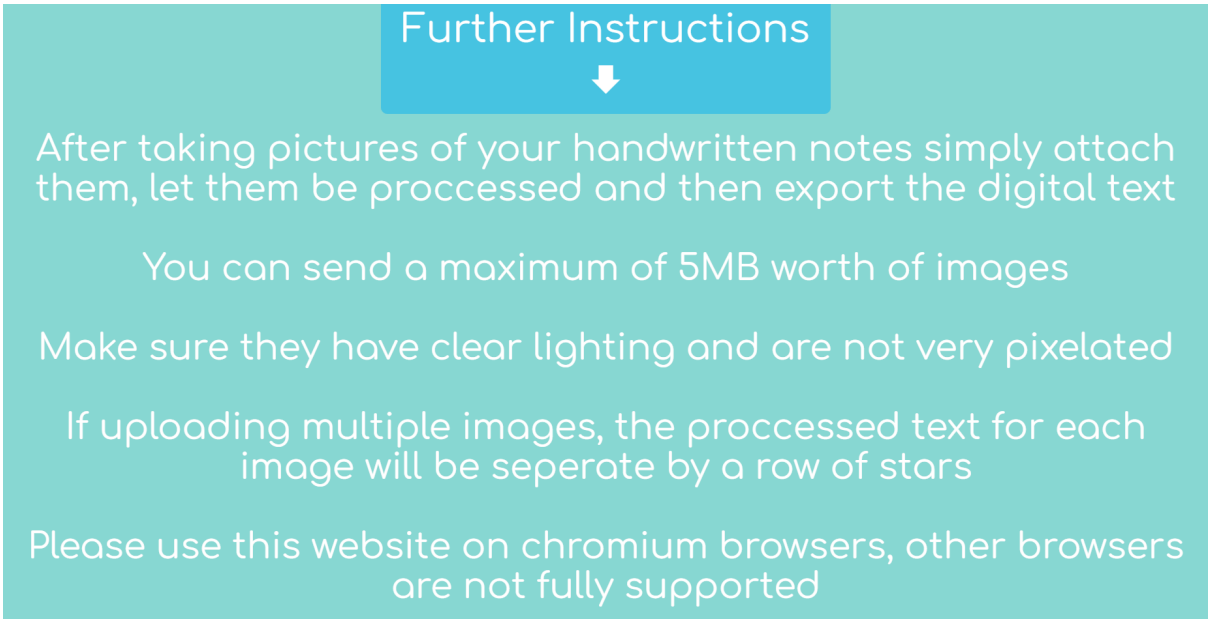


The stars look too small compared to the text box but this is at max size, if the user is using a phone or a smaller monitor it should be fine - and even now it doesn't look too bad, though a few more stars and spaces might be sufficient.

```
<div id="text-to-toggle">
    After taking pictures of your handwritten notes simply attach them, let them be proccessed and then export the digital text
    <br>
    <br>You can send a maximum of 5MB worth of images
    <br>
    <br>Make sure they have clear lighting and are not very pixelated
    <br>
    <br>If uploading multiple images, the proccessed text for each image will be seperate by a row of stars
    <br>
    <br>Please use this website on chromium browsers, other browsers are not fully supported      You, now • Uncommitted change
</div>
```

Figure 5.3.2.9

Finally to actually inform the users on the meaning of why there are stars, I have added further instructions to my original instructions in the home page, as seen above in figure 5.3.2.9. Additionally, due to my firefox issue within cycle 1, I have added another warning to inform the users of the lack of support for non-chromium browsers - such as firefox.



This concludes the completion of all of the problems I had from the model integration tests; so hopefully afterwards I can send my friend the new code to test it with the actual Chandra model.

So then I started to work on the next bug, which caused a permission error whenever I tried to upload files that were dragged and dropped in. Which is probably the result of me adding custom functions to help create my drag and drop functionality.

```
async function uploadFiles() {
  if (fileStack.length != 0){
    for (const file of fileStack){
      form.append("file_uploads", file, file.name);
    }
}
}      Shiven K, 4 days ago • Added readme and attempted
```

Figure 5.3.2.10

I first started off by removing the axios request from the original code in figure 4.3.1.10.1, since beforehand it is simply sending two requests at the same time, one from the form in the input page html and one from here through axios; so I removed the axios post and just let the form do its thing. By instead appending the files as data to the form, so that when the submit button is pressed it should simply just send the files to the backend - axios should not be required. Furthermore, this ensures that the upload button does not become obsolete since uploadFiles() is then called in figure 4.3.1.10.2, which is as soon as the files are dropped into the box. In conclusion figure 5.3.2.10 ensures that the values of the file stack are added to the form element, in preparation for them to be sent when the upload error is sent.
However this shouldn't actually affect the progress of the error itself - as the permission error is most likely the product of an internal server issue - but at least this improves the efficiency and quality of my code.

And sure enough as seen below I am still getting the same error:

This should mainly be happening because there is an error when handling the saving of the uploaded files onto the backend server's uploads folder.

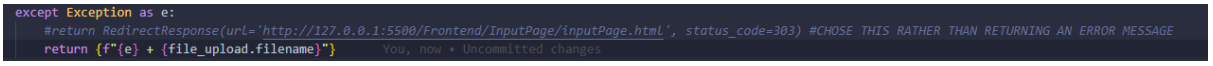
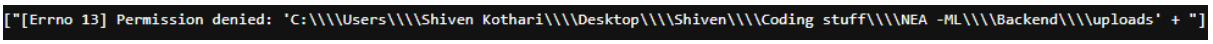


Figure 5.3.2.11

Therefore to further debug this issue I implemented figure 5.3.2.11, which should return any errors that the server receives, and also sends back the file uploads name - to check if the file name is being passed into the function correctly.

Which led to this error response:



Which returned an empty string in place of where the file's name is supposed to be, so the current extent of this error is that the server isn't actually receiving a file name. So when it tries to save the file to the uploads folder there is no value for it to save it as.

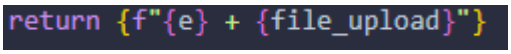
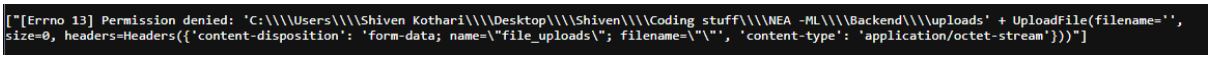


Figure 5.3.2.12

Additionally to make sure that it was only the name that was the current issue in the file handling loop, I modified figure 5.3.2.11 into figure 5.3.2.12, which will return the details of the actual file that is uploaded.

Which resulted in this unexpected response:



Which means that just no file at all is being held inside of the file_upload variable - since it's size is 0 bytes and literally every other value for the file is empty - which leads me to the conclusion that the files that are dragged and dropped into the drop zones are not being properly appended to the form.

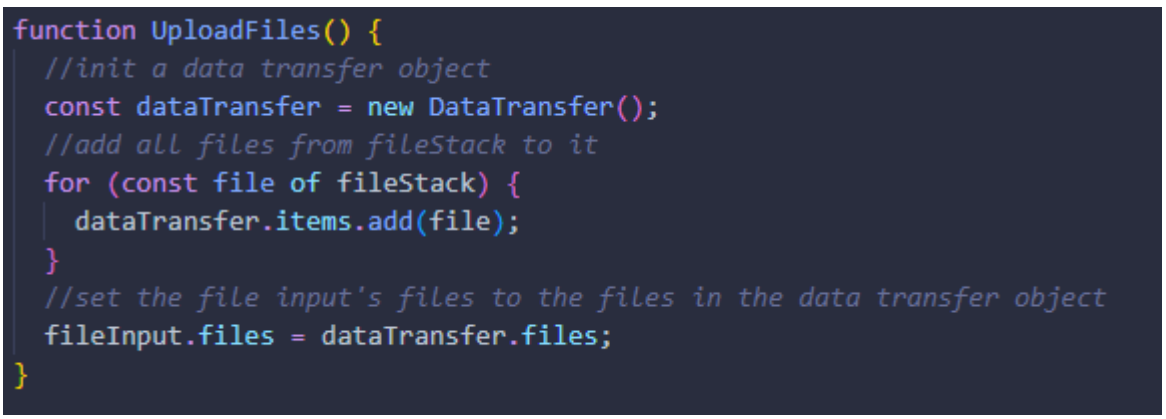


Figure 5.3.2.13

I then figured out that the entire reason that the files in the file stack weren't being appended to the form was that they are not supposed to be sent to the form itself, they need to be transferred to the fileInput element that is a child of the overall form. That is the place where all the files need to be stored, as the form automatically sends the contents of the fileInput element to the backend. Therefore as seen above in figure 5.3.2.13, I first created a new data transfer object - which will be the middle man of sorts to transfer my files from the file stack to the file input element - then I appended every file in the file stack into the new data transfer object using a basic for loop; and then I finally set the fileInput's file value to become that of the various files stored in the data transfer object. Afterwards, as seen in the aforementioned figure, I have also fixed a maintainability issue that I noticed while debugging this function - which is the name of the function being in camel case instead of pascal case. Therefore I have fixed this error and modified UploadFiles to follow the proper naming conventions.

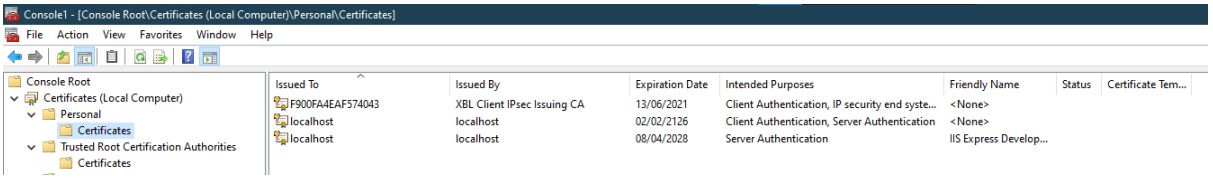
Video result: Show_model_integration_test2.mp4
Drag and drop makes sure that the wrong file type isn't accidentally uploaded via HandleFiles which ensures files of the right size and file type are accepted.

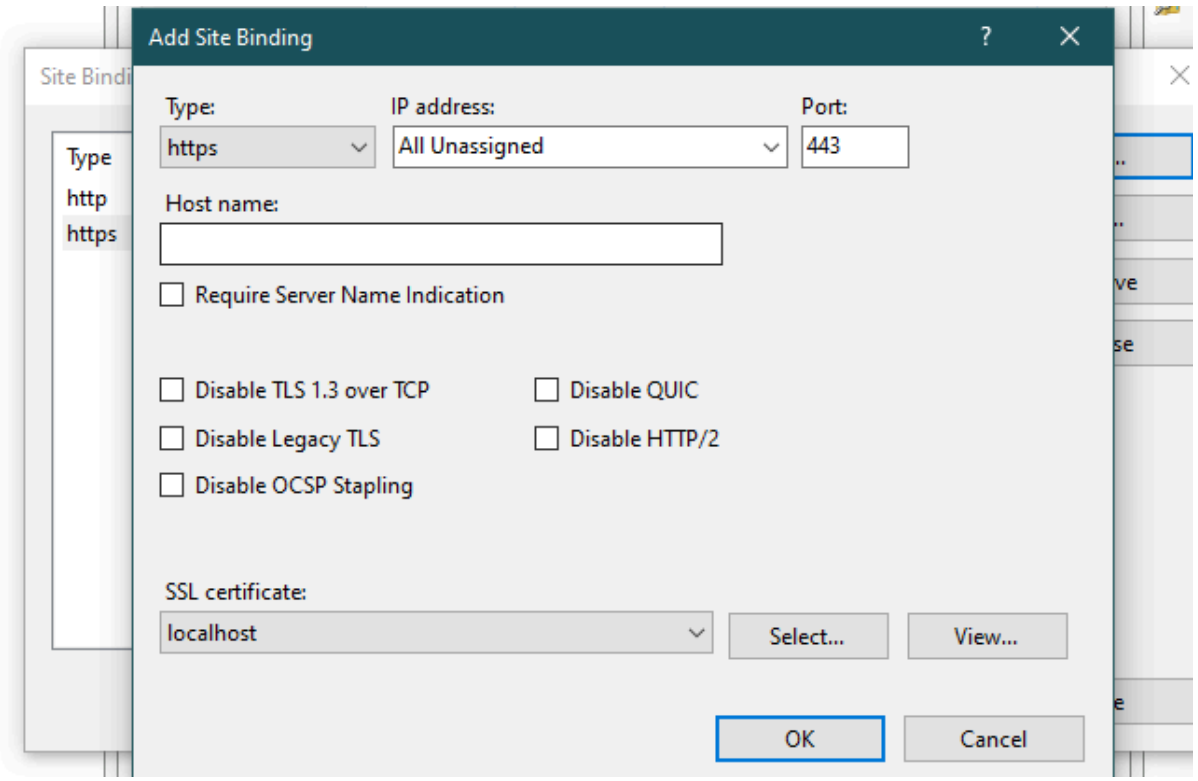
The next order of operation is now to implement an SSL certificate, to not only allow other users to use the copy button when I host it locally, but also to simply add another layer of security and trust from the users.
I first attempted to get an actual public certificate from cloudflare, however I then realised that my current website is simply being hosted locally using my live server extension. To get a public certificate I need a domain name and I would need to host my website on a dedicated hosting service or on my own raspberry pi (or an alternative piece hardware) - both of which would be quite expensive. Therefore for my current needs I have decided to implement a local self signed certificate for my device, this would still allow me all of the benefits I need for all users connecting to my website from the same network.

I started off by going into windows powershell and creating a new local self signed certificate that should be valid for the next 100 years.

```
PS C:\Windows\system32> New-SelfSignedCertificate -DnsName "localhost" -CertStoreLocation "cert:\LocalMachine\My" -NotAfter (Get-Date).AddYears(100)


   PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\My

Thumbprint                                Subject
----------                                -------
10E058C7B280F3826B39D5D06C8A52CF13A13D05  CN=localhost


PS C:\Windows\system32>
```

I then went to my console roots and added the local host certificate to the trusted root certification authorities.





Then I added the local host SSL certificate to the site bindings in the internet information service manager.

Next I needed to create a SSL key which I could attach to my project to allow for HTTPS, for this I needed to install a special openssl library to enact this change for my Windows device.

```
PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML> git clone https://github.com/openssl/openssl.git
```

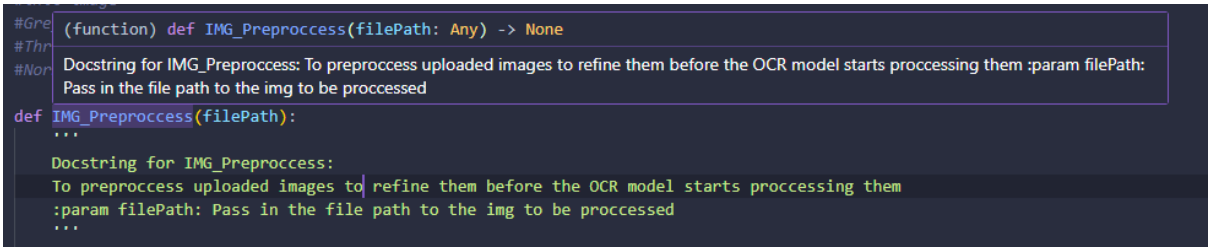
However at this point when I attempted to generate a private key:

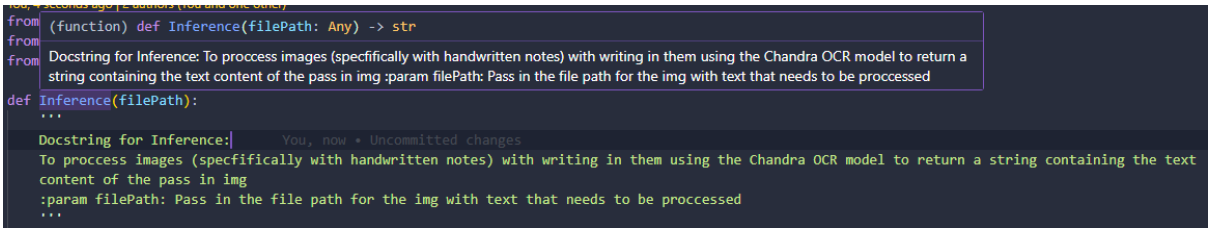
```
openssl genrsa -aes256 -out localhost.key 2048
```

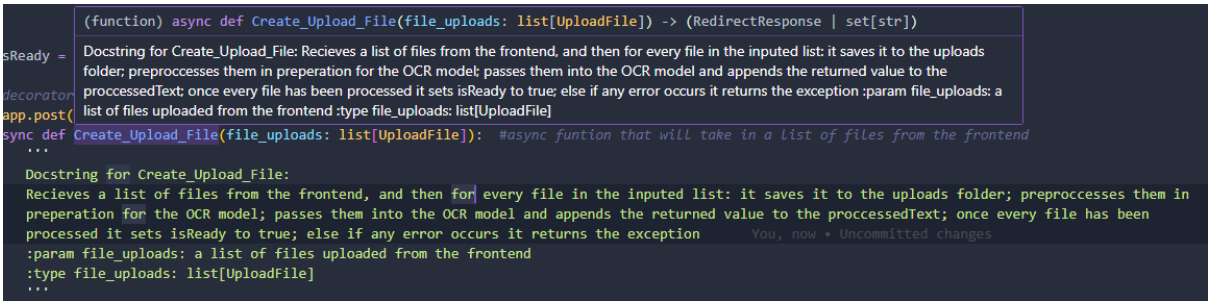
I got a command not found error, where it didn't recognise the openssl command. After this I quickly scanned the next steps I would need to actually enable a self signed certificate for the Live server extension; it would be a lengthy proceeding of linking my new self signed certificate with the local host key - which I am struggling to initialise right now anyway - and then I would need to configure the live server extension for this. In retrospect it seems worthless to have a local certificate, where only people connected to the same network as my computer can access it (on top of that having the entire website being run on just my own personal computer). It seems to be worth it more to finish the entire web application and then run it on a cloud server or on a server with a separate dedicated piece of hardware. Therefore due to my time limitations and the possible growth opportunities in public SSL certificates, I have scrapped the idea of implementing a local SSL certificate.

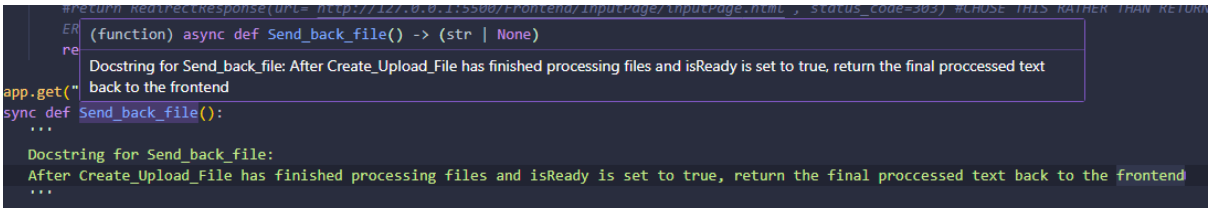
Next, I wanted to ensure I had taken the greatest possible measures to make sure my code was maintainable and readable, whether for the benefit of future contributors or for my future self. Therefore for all of my python functions I custom Docstrings to add proper descriptions for my functions for future maintainability.

They are seen below:









I personally feel the use of Docstrings is an extremely effective strategy that can convert obvious intent and functionality from the time of a function's inception, into corporeal descriptions. Which can save time by simply telling the developer every detail required, what the function does, the data type it expects and what the parameter passed into actually needs to be. This is a form of best practice that can truly transform the maintainability of a code base, by saving time and headaches with concise descriptions of functions. This is also extremely invaluable in my program code, which can lead to long wait times during debugging or testing as the inference model starts processing each batch of images.

After adding Docstrings, I received word from my friend that after trying to use the application with multiple images - the program would instantiate the Chandra model twice (probably due to calling it every time I want to infer an image, without waiting for the image before it to finish up, or because the model before it didn't properly shutdown), and after loading up it would just sit there and do nothing after even 2 hours of waiting. Even after processing a single image, it would take 10 minutes and then nothing would happen (since they used Uvicorn through the terminal, as I did, and were not paying attention to the network console, there was no way to tell what happened), my hunch was that it was an issue related to the network and how the backend was handling the various POST & GET requests. Also I received word that he was not going to be able to help me with testing the

Chandra model, which bore even more bad news, since my computer is nowhere near powerful enough to do so.

I decided to tackle this entire tangle of problems one by one. So I first attempted to recreate the problem myself as seen below:
CANCELLED_MODEL_REQUEST.mp4
As soon as the model starts to load, the request gets cancelled and the model sort of just freezes or stops.

```python
if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

Figure 5.3.2.14

Firstly, before tackling the situation I decided to make it easier to debug I appended figure 5.3.2.14 at the end of my main.py file, this would fun the uvicorn server every time I ran the program - so I could actually use VS Code's debugging features - which meant I didn't have to constantly run everything from the terminal, increasing my quality of life. Of course, as I did within the terminal command, I ensured reload was enabled and set the port to the basic 8000 backend port.

```python
import gc #garbage collector
gc.collect()

import torch
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    torch.cuda.synchronize()
```

Figure 5.3.2.15

Firstly as seen in figure 5.3.2.15, I added to the inference file by adding a cleanup sequence at the end of the entire inference process. Firstly I have imported the garbage collector library to attempt to reclaim memory occupied by objects that are no longer in use by the program. Since the main concern was that all the memory - when more than one file was being processed - was being eaten up, so to try and free up some space I have first imported the garbage collector class. Next I have imported a torch, then if cuda is available (it is enabled), it will then empty the cuda cache (to free up memory) and then synchronise the cuda systems, which will ensure that all cuda processes are finished and not abruptly halted before it ends.

This doesn't change anything and leads to the model simply freezing before it can even finish loading any of its weights. Sometimes it freezes at 100 percent, sometimes at 8/70:

```
Loading weights: 100%|          | 750/750 [00:33<00:00, 22.46it/s, Materializing param=model.visual.pos_embed.weight]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fa
ster downloads.
```

```python
from transformers import pipeline

#init pipeline
pipe = pipeline("image-text-to-text", model="datalab-to/chandra")

#disable cuda
torch.cuda.is_available = lambda: False
```

Figure 5.3.2.16

So - as seen in figure 5.3.2.16 - I went back to the HuggingFace documentation page of the Chandra model (from datalab) to see if there was an alternative way to call the model - since I wanted to test if this was the reason for the model freezing on me. This was accomplished by calling the transformers (made by HF) library directly to get the pipelining function, to call the Chandra model directly (with the first param being the input output types of the data - the text in the images into only text). I then disabled cuda since I have decided to run all of this on my CPU, since I physically can't run this on my GPU due to the lack of VRAM, I thought to run this very slowly (but still run it) until the process finishes. Furthermore, hopefully by having the model load all of its weights and such at the start when I activate the server, will hopefully root out the cancelled error message as a consequence of loading the model when it is called.

```python
messages = [
{
    "role": "user",
    "content": [
        {"type": "image"},
        {"type": "text", "text": "Extract the handwritten notes in the image into text"}
    ]
},
]
```

Figure 5.3.2.17

Then - as seen in figure 5.3.2.17 - I defined a message parameter that would be a JSON that would be a ‘message’ to the model, telling it to expect an image and to return text with the prompt to extract the text from the image.

```python
proccessedText += f'{file_upload.filename}' + pipe(image=save_to, text=messages)[0]["generated_text"]
```

Figure 5.3.2.18

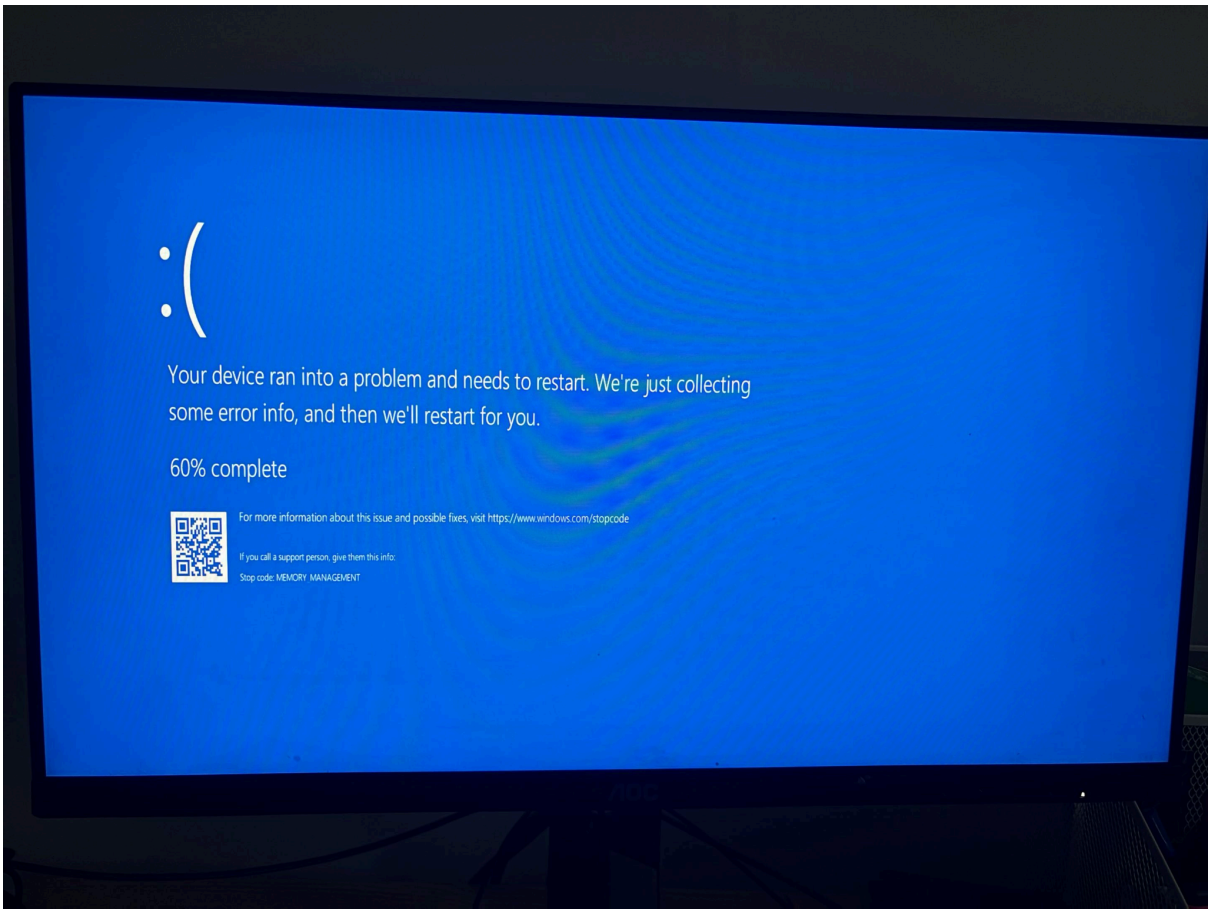
Then in figure 5.3.2.18, I replaced my previous Inference() function call with the new pipeline reference. For the first parameter: I have inputted the image to send to the model - by referencing it's file path - and then sending the message as the second parameter as the models instructions. From there I have immediately split up its response so I can single out the returned text. Since it comes back as a list, I take the response’s 0 index and use the key: ‘generated_text’ to get the text value from the JSON - python dictionary.

```
#init pipeline
pipe = pipeline("image-text-to-text", model="datalab-to/chandra", device="cpu")
```

Figure 5.3.2.19

Then I set the device type to CPU, as seen above in figure 5.3.2.19, as mentioned above I have lost my tester. So I need my own way to do so, so I need to use my own CPU.

However this attempt simply resulted in the same as before. And even lead to a few instances of the following:




```
#init pipeline
pipe = pipeline("image-text-to-text", model="datalab-to/chandra", device="cpu",
                dtype=torch.float16,
                low_cpu_mem_usage=True)
```

Figure 5.3.1.20

Therefore, I then attempted to reduce the actual complexity of the Chandra OCR model (as seen above in figure 5.3.1.20) - so my computer would not blow up like the 4th of July, as seen above in blue - this would definitely reduce the accuracy of results, however this would allow for my application to actually become complete. Hence, it is a compromise I am fine

with indulging in. So with torch I used a lesser data type for representing all the floating point numbers, and I set the low_cpu_mem_usage variable to be true.

This resulted in absolutely no changes - excepted for no blue crashing screens - and instead I was met with the continued barrage of terminal messages seen below:

```
File "<frozen importlib._bootstrap>", line 1178, in _find_and_load
File "<frozen importlib._bootstrap>", line 1149, in _find_and_load_unlocked
File "<frozen importlib._bootstrap>", line 690, in _load_unlocked
File "<frozen importlib._bootstrap_external>", line 936, in exec_module
File "<frozen importlib._bootstrap_external>", line 1032, in get_code
File "<frozen importlib._bootstrap_external>", line 1130, in get_data
```

```
Loading weights: 100%|          | 750/750 [00:33<00:00, 22.46it/s, Materializing param=model.visual.pos_embed.weight]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fa
ster downloads.
```

```
Loading weights:   0%|                                  | 1/750 [00:00<?, ?it/s, Materializing param=lm_head.weight]
  WARNING   WatchFiles detected changes in 'main.py'. Reloading...
Loading weights:  18%|| | 137/750 [00:50<00:44, 13.80it/s, Materializing param=model.language_model.layers.12.mlp.gate_pro
  WARNING   WatchFiles detected changes in 'main.py'. Reloading...
Loading weights:   8%| | 60/750 [00:04<00:45, 15.22it/s, Materializing param=model.language_model.layers.5.mlp.gate_proj.
  WARNING   WatchFiles detected changes in 'main.py'. Reloading...
Loading weights:   7%| | 50/750 [00:02<00:44, 15.68it/s, Materializing param=model.language_model.layers.4.mlp.up_proj.we
  WARNING   WatchFiles detected changes in 'main.py'. Reloading...
```

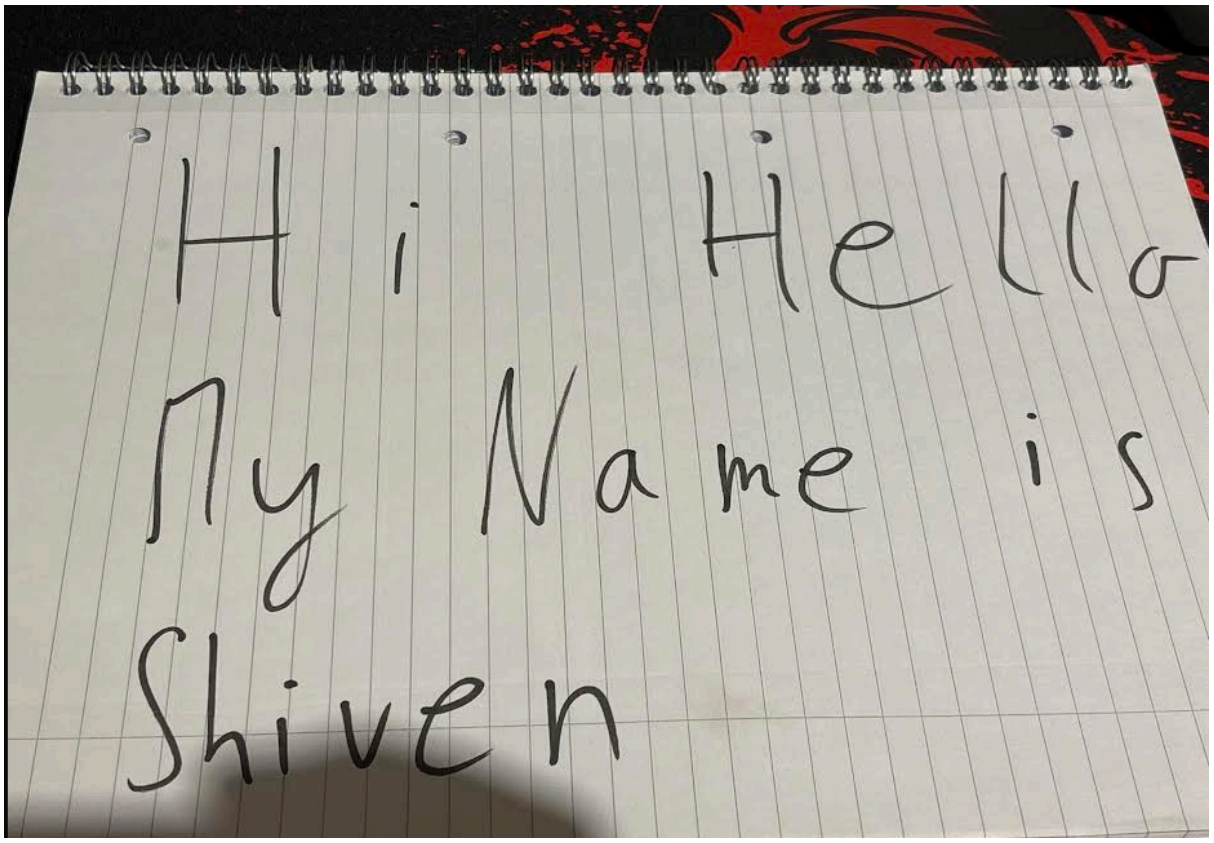
Chandra constantly froze up on me, it was now becoming unreliable, completely unrealistic to work with and not allowing me to even come close to the original bug at all. This made me realise how using the CPU or even using it at lower memory versions of Chandra is still extremely unrealistic, since this model was made on top tier GPUs to provide the best OCR for all forms of text. It isn't designed with the idea of someone being able to run it on a 6 year old CPU - which in retrospect seemed naive but the only path forward in those circumstances. However I first reasoned to myself to search for any other possible alternative.

```python
import easyocr
from pathlib import Path
#init directory of inm
dir = Path().cwd() / 'uploads' / 'y21.jpg'
#init easyOCR reader, lang = english
reader = easyocr.Reader(['en'])
#call readtext method and direct it to the dir, ensure its str
transcribedText = reader.readtext(str(dir))
#join then print the str of the results by only getting the text so [1], for each detection made for each
#line of the img it transscribbed
print('\n'.join([detection[1] for detection in transcribedText]))
```

Figure 5.3.1.21

So I then attempted to find other OCR machine learning models that were at least competent enough to perform basic handwriting character recognition. Therefore I tested out easyOCR in a separate python test file (as seen above in figure 5.3.1.21), easyOCR was a low intensive model that seemed adequate enough for uniform text and the best least-intensive model for handwritten notes. So I imported easyocr and pathlib (to get to the path of the image I wanted to process), I first initialised the directory of the image I wanted to transcribe

- using path as the base path to the Backends folder - then I initialised the easyOCR reader (with english as the language of choice). Then I set transcribedText to be the returned value of calling readtext with easyOCR, when I pass in the image seen below:



Then I join the resultant string of each line of each line of detection made within the image. This would split each detection - so each word - by a new line and then print it. Which resulted the output seen below, in a few mins:

```
He
Na
h &
Shiven
```

Though the time of processing was astoundingly minute, the accuracy was extremely disappointing.

Though I had seen a few other OCRs that could have been a solution to this problem - such as PaddleOCR - I was not convinced that they would be worth any time, or would lead to any sort of a fruitful result. Then after all these failures, I changed my perspective and instead of trying to find good machine learning models that I could simply run on my own computer; I decided to try and find any models I could use for free that were being hosted on the cloud. A few prospects popped up, using virtual large language model (vllm) servers instead of locally running HuggingFace models, or using special GPUs from Google Colab. However none of them were very welcoming for the financially incapable; this led me to think of using an external API key of a current large language model (LLM) such as ChatGPT, Claude or Gemini. Models that have had millions if not billions of pounds invested in them, with sufficient expertise in transcribing handwritten notes. Therefore I then narrowed down my

choices to Gemini, since it is the only one with a free tier. Furthermore, this also allowed me the opportunity to increase the possibilities of further maintenance and scalability. Originally the plan was to incorporate a dedicated server with its own dedicated hardware to run the Chandra model in the future when I would deploy the app. However, now instead of having to deal with painstakingly updating my hardware and manually configuring all of the dependencies of Chandra - or even other better models that may pop up in the future - I could simply upgrade to the current best model of Gemini (or another large language model). Having a model update instead of a hardware update, each time I wanted to improve my application. Finally, it will also make the Inference and Image preprocessing python files completely obsolete; since it will have its own process on performing optical character recognition on handwritten notes, and I don't want to interfere with that.

I started off by first installing gemini through python within my terminal, as seen below:

```
PS C:\Users\Shiven Kothari\Desktop\Shiven\Coding stuff\NEA -ML\Backend> pip install google-genai
```

```
#import geminai integration
from google import genai
#import to pass in img from file path into gemini readable img
from PIL import Image
```

Figure 5.3.1.22

Then as seen above in figure 5.3.1.22, I imported genai itself and Pillow to read the saved images so I can actually pass them into gemini.

```
API_KEY = ''

client = genai.Client(api_key=API_KEY)
```

Figure 5.3.1.23

Furthermore, I then went to the actual google ai studio to create a dev account where I could create my very own API key to use for this project. As seen above in figure 5.3.1.23, I created a constant (which will hold my API key) and then I instantiated the Gemini client using my API key.

```
img = Image.open(save_to)

#OCR W/ Gemini
transcribed_text = await client.models.generate_content(
    model="gemini-2.0-flash",
    contents=[img, 'Transcribe the all the text in this image into plain text.']
)

global proccessedText
proccessedText += f'{file_upload.filename}'  + transcribed_text.text + '                    ************************************************************'
```

Figure 5.3.1.24

Moreover I then implemented figure 5.3.1.24 into the POST Create_Upload_File() function in main.py, I first attempted to test all of this when the use case only involves a single image. So, I opened the image using a pillow from the file path it was saved to originally and then

called the generated content function from the client models. I then chose to use the gemini 2.0 flash model as a test subject for now, then I set the contents of the request to be composed of the Pillow opened image and the prompt to transcribe all the handwritten notes into plain text. This is wrapped within the await keyword, so that it is asynchronous and the request waits for the response of the API before cancelling the request.

Though this should have removed all of the variables and complexity of running a model locally and sending it requests, sending the API request to Gemini still bears the same problem. Everything works if the Gemini API call is excluded, but the request gets cancelled out when it is called - but the request is still received by Gemini, though no output is received since the request is cancelled, so nothing is returned.

```
#to avoid the thread blocking
import asyncio
```

Figure 5.3.1.25

Therefore to try and make the request stop cancelling, I first attempted to use the Asyncio library to stop any thread blocking the API request might be causing due to its synchronous nature - which is wrapped asynchronously using await - I imported Asyncio as seen above in figure 5.3.1.25.

```
#wraps synchronous api call into a async I/O wrapper that prevents blocking the event loop
loop = asyncio.get_event_loop()
#OCR W/ Gemini
transcribed_text = await loop.run_in_executor(   #calls to gemini in new thread pool
    None,  #use default thread pool executor
    lambda: client.models.generate_content(
        model="gemini-2.0-flash-exp",
        contents=[img, 'Transcribe all the text in this image into plain text.']
    )
)
```

Figure 5.3.1.26

Then in figure Figure 5.3.1.26, I iterated over figure Figure 5.3.1.24 by defining an Asyncio event loop that will wrap the synchronous api call into an asynchronous wrapper to prevent the thread blocking. I then execute the generate content function in an awaited Asyncio loop, using the default thread pool executor, using lambda to define the generate_content function.

This did not work, and resulted in the same original error of a cancelled network request.

```
from fastapi.concurrency import run_in_threadpool
```

Figure 5.3.1.27

The next attempt that was recommended from stack overflow was to use fastapi's concurrency section to run the api call in a completely different thread to the main program,

in hopes that the request isn't cancelled. Therefore as seen above in figure 5.3.1.27, I imported run_in_threadpool from the concurrency module in fastapi.

```
#define the gemini api call as a subroutine so it can be run in a different thread
def OCR():
    return client.models.generate_content(
        model="gemini-2.0-flash-exp",
        contents=[img, 'Transcribe all the text in this image into plain text.']
    )

#OCR W/ Gemini - use run_in_threadpool to run the api call in a diff thread
transcribed_text = await run_in_threadpool(OCR)
```

Figure 5.3.1.28

Then I modified figure 5.3.1.26 into figure 5.3.1.28 by encapsulating the contents of the generated content function within the OCR function, then I will simply run this function asynchronously in a different threadpool.
However this also resulted in the same manner, as seen before in the CANCELLED_REQUEST_MODEL video. The request was being cancelled and even extending the timeout attribute of the server would only lead to a stage of eternal stasis.

```
#to import FastAPI backend and upload file functionality
from fastapi import FastAPI, UploadFile, BackgroundTasks
```

Figure 5.3.1.29

My next approach was rather than having the request wait for the api reference to finish, instead have the request end and return a value to the browser while processing the api call in the background. Therefore I reiterated over figure 4.3.1.1 into figure 5.3.1.29 and imported BackgroundTasks from fastapi so I could call Gemini in the background.

```
def GeminiAPICall(img): #pass in pillow opened img
    global isReady, proccessedText

    #API call to gemini
    response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents=[img, 'Transcribe all the text in this image into plain text.']
    )

    #append response to proccessed text
    proccessedText = response.text
    #signal that get request can be used
    isReady = True
```

Figure 5.3.1.30

I then defined GeminiAPICall() outside of Create_Upload_File() in figure 5.3.1.30, which will have access to isReady and processedText. Here it will first make a request to the gemini api, then it will set the processedText variable equal to the plain text response of the model and set isReady to true. Therefore in essence while the background task is being processed and the user is waiting in the output page, the editable text field will still receive the request but while the user is waiting in the output page.

```python
async def Create_Upload_File(file_uploads: list[UploadFile], background_tasks: BackgroundTasks):
```

Figure 5.3.1.31.1

I first define a Background task parameter in figure 5.3.1.31.1, FastAPI will then create the object of type BackgroundTasks by passing it in as a parameter - so that I can then just add functions to the list of tasks and they will run after a response is sent back.

```python
try:
    for file_upload in file_uploads:    #iterate over each file to save it to our directory
        data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
        save_to = Path().cwd() / 'uploads' / file_upload.filename #save a file in the predefined directory with the files name
        with open(save_to, 'wb') as f: #write to the files bytes
            f.write(data)   #write the read data, so we have completley stored this file in our backend
        #img preproccess
        #IMG_Preproccess(save_to)

    #Open img - single img test
    img = Image.open(Path().cwd() / 'uploads' / file_uploads[0].filename)

    # Schedule background processing
    background_tasks.add_task(GeminiAPICall, img)
```

Figure 5.3.1.31.2

Then I called GeminiAPICall() in figure 5.3.1.31.2, firstly I open the first image sent over in the list using pillow - since currently I am only testing one image to make sure this works - and then I add the GeminiAPICall() function to the task list of the background task, and img as the param for the GeminiAPICall() function.

```
(function) async def Create_Upload_File(
    file_uploads: list[UploadFile],
    background_tasks: BackgroundTasks
) -> RedirectResponse
```

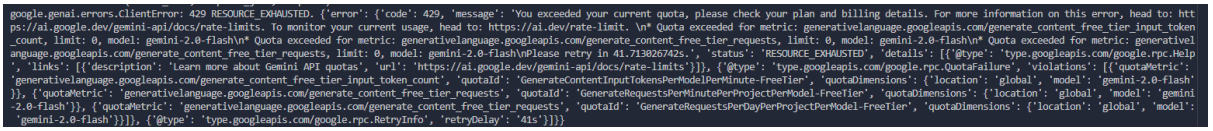
Docstring for Create_Upload_File: Recieves a list of files from the frontend, and then for every file in the inputed list: it saves it to the uploads folder; preproccesses them in preperation for the OCR model; passes them into the OCR model and appends the returned value to the proccessedText; once every file has been processed it sets isReady to true; else if any error occurs it returns the exception :param file_uploads: a list of files uploaded from the frontend :type file_uploads: list[UploadFile] :param background_tasks: creates a background_task for calling the api, ignore during HTML form referencing

```python
async def Create_Upload_File(file_uploads: list[UploadFile], background_tasks: BackgroundTasks):  #async funtion that will take in a list of files from the frontend
    '''
    Docstring for Create_Upload_File:
    Recieves a list of files from the frontend, and then for every file in the inputed list: it saves it to the uploads folder; preproccesses them in preperation for the OCR
    model; passes them into the OCR model and appends the returned value to the proccessedText; once every file has been processed it sets isReady to true; else if any error
    occurs it returns the exception
    :param file_uploads: a list of files uploaded from the frontend
    :type file_uploads: list[UploadFile]
    :param background_tasks: creates a background_task for calling the api, ignore during HTML form referencing
    '''
```

Figure 5.3.1.31.3

Finally to ensure that the codebase is up to date, I iterated over the previous docstring description for Create_Upload_File() to include a description for the new background_task parameter. As seen above in figure 5.3.1.31.3.

Single img test first result:



The error above hinted to me that I should use a different model of Gemini, since the current Gemini 2.0 flash is being overloaded by the number of requests it is being sent. Therefore I need to choose an older model that can handle more requests - in my current free tier.

```python
def Gemini_API_Call(img): #pass in pillow opened img
    global isReady, proccessedText

    #API call to gemini
    response = client.models.generate_content(
    model="gemma-3-27b-it",
    contents=[img, 'Transcribe all the text in this image into plain text.']
    )

    #append response to proccessed text
    proccessedText = response.text
    #signal that get request can be used
    isReady = True
```

Figure 5.3.1.32

So as seen above in figure 5.3.1.32, I went back to Gemini's list of models and chose an older model with a much higher request per minute rate than the 2.0 flash model. Which is the 3.27b model. I also fixed a previous naming convention and implemented a snake case to the name of the Gemini_API_Call() function.

Result: GEMINI_INTEGRATION_TEST(1).mp4
Which resulted in a near perfect result, though the markdown of the original response is still noticeable (e.g. being able to see the line breaks, \n), also the text in the editable text field only outputs null and the user needs to reload the page before they can see the response.

```javascript
async function SetTextContent() {
  const response = await fetch("http://127.0.0.1:8000/returnfile/");
  const text = await response.text();

  if (text == 'null'){
    textArea.textContent = 'Please Wait, We are proccessing your image(s)...'
  }
  else{
    textArea.textContent = text;
  }
}

document.addEventListener("load", SetTextContent())
```

Figure 5.3.1.33

Therefore to first fix the null problem within the editable text field, I implemented figure 5.3.1.33 in export.js. Here I first engage in a simple if loop that checks if the text is null, if so it replaces null with a waiting message (which sort of replaces the throbber with this waiting message instead), then once it isn't equal to null it will simply set the text content to be the response text from Gemini.

Video: GEMINI-INTEGRATION-TEST2-NORELOAD.mp4
Here not only do I not need to reload, but null is nowhere to be seen.

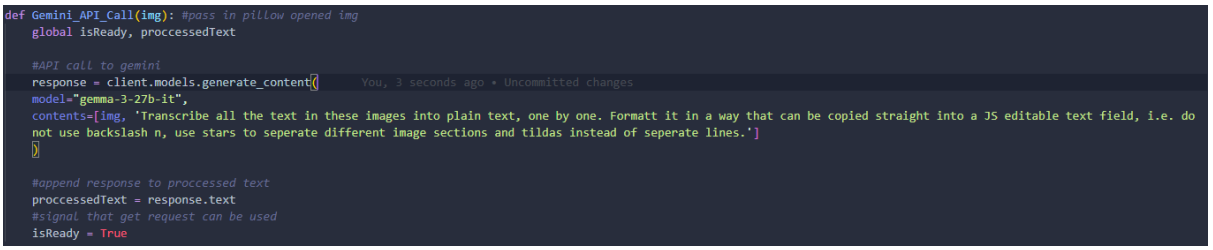


Figure 5.3.1.34.1

Furthermore, to remove the fluff from the text I made my Gemini prompt more detailed so that it wouldn't be plagued with a bunch of \ns. As seen above in figure 5.3.1.34.1

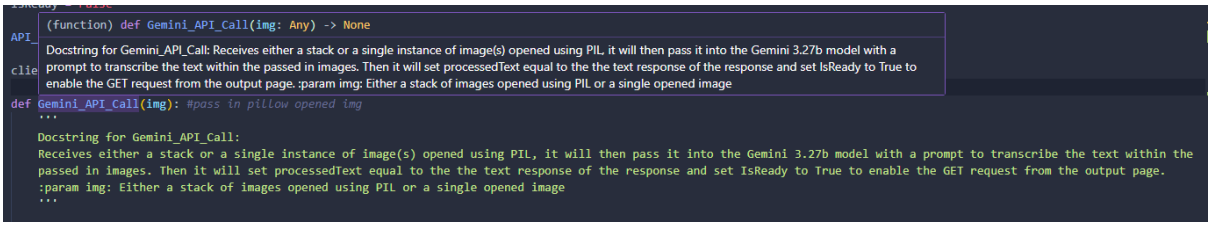


Figure 5.3.1.34.2

Then to follow along with my previous attempts at improving the future maintainability of this codebase, I added a docstring description for this function, as shown above in figure 5.3.1.34.2.

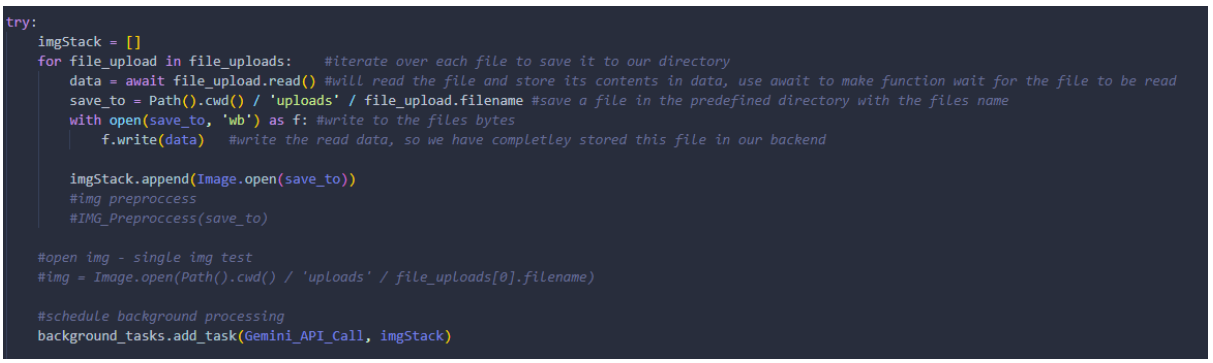


Figure 5.3.1.35

Finally since my test with single images led to a successful outcome I then attempted to implement a method for handling the event of multiple being sent to the backend. Therefore as seen in figure 5.3.1.35 I first created an imgStack and appended all of the pillow opened images into it, and simply passed that in as the parameter for the GEMINI_API_Call() function.

Video: GEMINI-INTEGRATION-TEST3-MULTFILES.mp4
Though the result is extremely successful, the markdown has simply just become worse.

```
async function SetTextContent() {
  const response = await fetch("http://127.0.0.1:8000/returnfile/");
  const text = await response.text();

  if (text == 'null' || text == 'Please Wait, We are proccessing your image(s)...'){
    textArea.textContent = 'Please Wait, We are proccessing your image(s)...'
  }
  else{
    textArea.textContent = text;
  }
}
```

Figure 5.3.1.36

Moreover in figure 5.3.1.36, I first attempted to fix the problem of needing to click reload every time you wanted to check the progress of whether a result has been achieved or not. Therefore within the original if statement I have also set the condition that if the text is 'Please Wait…' then it won't default to the else case - which would have accepted any other form of text that is not null.

Video: GEMINI-TEST4-AUTORELOAD.mp4
This fixes the problem of needing to click reload to get your final result.

```
contents=[img, 'Extract only the text from the image(s) provided. Output ONLY the transcribed text with no explanation, no labels, no markdown.']
```

Figure 5.3.1.37

Now that the auto-reload feature has been implemented, I need to fix the prompt I am sending to the Gemini model - as seen above in figure 5.3.1.37.

Video: GEMINI-INTEGRATION-NO-LABELS.mp4
Though the result led to no extra labels or explanations, it would lead to a problematic result if it couldn't show to users a clear separator to signify which parts of the response belongs to which files they had inputted. Therefore I will need the prompt to split up the different responses for each image. Furthermore I need to figure out how I can get the editable text field to recognise that when it sees a \n that it needs to break to a new line.

```
contents=[img, "Extract only the text from the image(s) provided. Output ONLY the transcribed text with no explanation, no markdown. Just section each image's content with
a row of dashes equal to the number of characters in the largest sentence."]
)
```

Figure 5.3.1.38

Firstly as seen in figure 5.3.1.38, I iterated over figure 5.3.1.37 to make it so that each section is split up by a row of dashes that match the longest sentence in the response; so that the entire response looks uniform and formatted.

```
<div id="text-to-toggle">
    After taking pictures of your handwritten notes simply attach them, let them be proccessed and then export the digital text
    <br>
    <br>You can send a maximum of 5MB worth of images
    <br>
    <br>Make sure they have clear lighting and are not very pixelated
    <br>
    <br>If uploading multiple images, the proccessed text for each image will be seperated by a row of dashes
    <br>
    <br>Please use this website on chromium browsers, other browsers are not fully supported
</div>
```

Figure 5.3.1.39

Then I changed the further instructions section in the HomePage HTML, to update the separator from stars to dashes. As seen above in figure 5.3.1.39.

```
async function SetTextContent() {
  const response = await fetch("http://127.0.0.1:8000/returnfile/");
  const text = await response.text();

  let processedText = text.replace(/\\n/g, '\n'); //use regex to replace every backslash n with \n

  if (processedText == 'null' || processedText == 'Please Wait, We are proccessing your image(s)...'){
    textArea.textContent = 'Please Wait, We are proccessing your image(s)...'
  }
  else{
    textArea.textContent = processedText;
  }
}
```

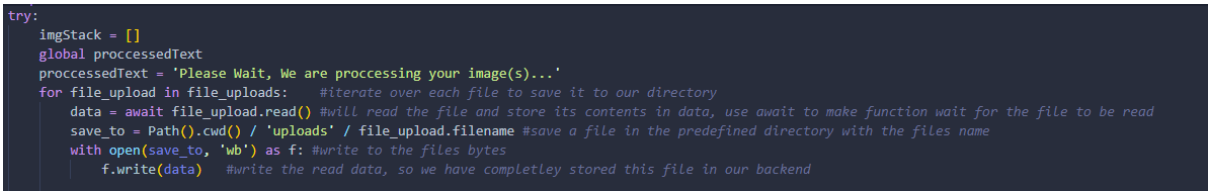
Figure 5.3.1.40

Finally, to fix the backslash n logic error that the editable text field was facing I iterated over figure 5.3.1.36 by implementing figure 5.3.1.40. Here I use regex to postprocess the processedText by replacing every '\n' string with the '\n' character. I first define a regex expression by using /…/ and then I define the \n with \\n (since a single backslash means escape) with a g at the end for global, so it replaces each \n substring with the \n character. The difference here is that originally it sees the backslash and the n as two different characters that are just appended together, but I need it to recognise it as the newline character - with string length 1 - so we replace it with the character \n not the string \n.

Video: GEMINI-TEST-FIXED-FORMAT.mp4
This resulted in the perfect outcome, everything was formatted correctly and the result was extremely accurate.

Second test: GEMINI-TEST-GOOD-REFRESH-TEXT.mp4
To then test the robustness of my program I inputted five different test images into the program (which took a while since this was my 30th api call, so the model was quite exhausted with my free tier access). However this presented another array of bugs, for example since this request took substantially longer than my other tests, the model took way

longer to respond back to the output page. Therefore calling an if statement at the start of when the output page loads proves to be insufficient, causing users to manually reload the page themselves. Furthermore it seems that the processedText needs to be reset every time a new request is made, since previously received responses aren't cleared from memory.



```
try:
    imgStack = []
    global proccessedText
    proccessedText = 'Please Wait, We are proccessing your image(s)...'
    for file_upload in file_uploads:    #iterate over each file to save it to our directory
        data = await file_upload.read() #will read the file and store its contents in data, use await to make function wait for the file to be read
        save_to = Path().cwd() / 'uploads' / file_upload.filename #save a file in the predefined directory with the files name
        with open(save_to, 'wb') as f: #write to the files bytes
            f.write(data)   #write the read data, so we have completley stored this file in our backend
```

Figure 5.3.1.41

Therefore in figure 5.3.1.41 I have done the final modification for the main.py file, which is to simply set processed text to equal to the waiting message - which will obviously be overwritten when a response is received from the Gemini model.

Third test: Doodle Fail Gemmini.mp4
Finally, for my last test I simply inputted the five images again but with the doodled file first - which had a few words in it - when my model had almost reached its maximum capacity, to test how it handled requests when it was at its limits. It resulted in the outcome seen above in the third test, the model completely froze for a good while before outputting a bunch of dashed lines and nonsense. This shows the dip in accuracy within the Gemini model as the limits of my free tier are reached, it probably tried to analyse the huge doodle - which also contained a few overlapping words in it.

Final test of cycle 3: FINALMENTE.mp4
Finally after a fresh new day with the first request of the day, the result was phenomenal, the processedText variable is appropriately reset, the accuracy is adequate but the overall limitation I faced this entire cycle was the lack of time to build a proper auto reloader to automatically update the state of the editable text field.

5.4 Code annotation and naming conventions

For the most part I simply adhered to the previously defined naming conventions within cycle 1 and 2, since cycle 3 was mainly set around finalising the integration between the frontend & backend while also implementing additional changes. This was the same with the importance of code annotation, I have continued to utilise it consistently to ensure that whenever I look back at an old piece of code I can recognise what it does immediately. To ensure that this was maximised - which consequentially also boosted the future maintainability of this project - I added Docstring descriptions for all of my python functions, to make my code more readable and increase the overall maintainability of the codebase. These will surely help any future developers or myself in the future with understanding how the backend functions.

5.5 Evaluation

Overall I met most of the targets I set at the start of cycle 3, I implemented an improved colour scheme, font family, fixed the drag and drop functionality, added zoom buttons for the

output page, attempted to implement a self signed SSL certificate but then realised the disadvantage outside the pros, and finalised the model integration issues by realising the futility of using the Chandra model especially after I couldn't physically run it locally; leading me to find solace with the idea of using of Gemini model 3.27b. I definitely feel that having a brief stakeholder survey at the end of cycle 2 was monumental for a lot of my achievements within cycle 3; by using the AGILE methodology - by receiving constant user input throughout the cycles - I truly believe that this application is catered to my stakeholders to the best of my ability.

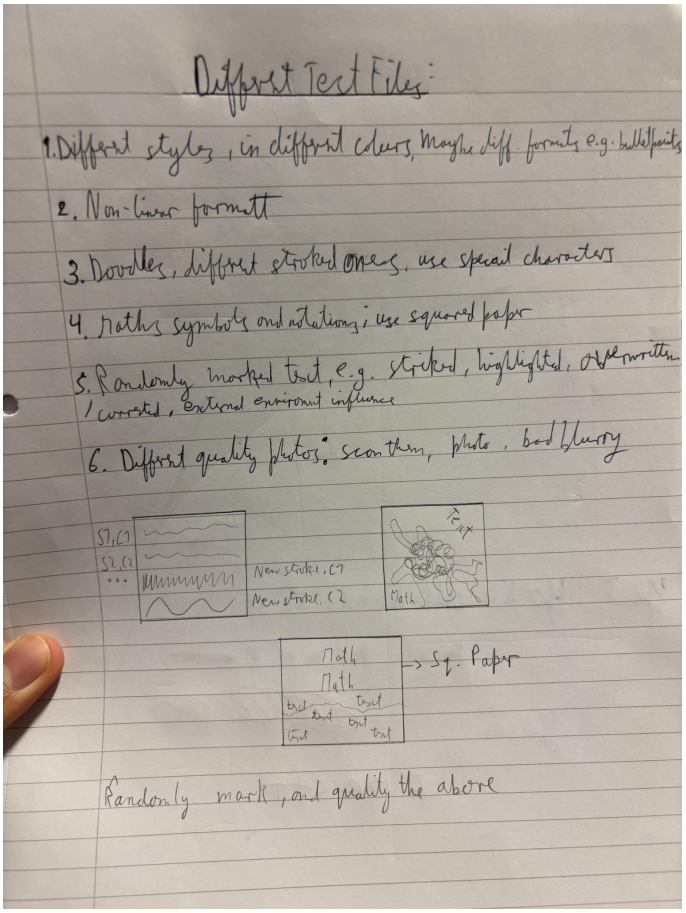
6. Evaluation

6.1 Post-Development Testing

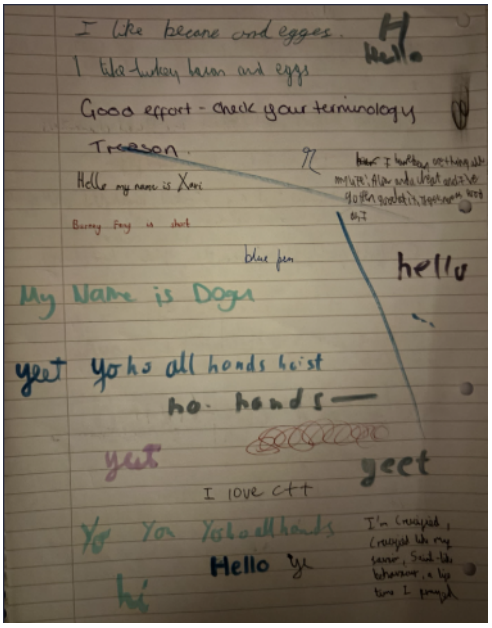
Test Type	**Evidence & Description**
Image with no text	NO-TEXT-TEST.mp4 This results in a **success** as the model simply returns a wordless response, since there are no words in the file.
Invalid data (non image file)	NOT-IMG-TEST.mp4 This results in a **success** since the website rejects the file, refusing to append it to the overall form and therefore signals the user to input the correct file type.
Extremely small image	SMALL-IMG-TEST.mp4 I inputted an image of the word ‘evaluation’ which was very small (412 bytes). This resulted in a **success**, despite taking a bit longer than usual - which can be justified due to the abnormal size of the inputted file - the model still returns an accurate text response.
Extremely large image	UPPER-BOUNDAR-TEST.mp4 I inputted a 4.95MB file of a dot (largest that I could get to be close to five megabytes) and the model handled it effortlessly, resulting in a **success.**
Presence check & Robustness in preparation of repeated action	presence-check.mp4 This resulted in a **success**, as it outputted an error message asking the users to actually input a file, and the repeated action of spamming the upload button resulted in no unexpected behaviour.
Out of bounds check (exceeding five megabytes)	out-of-bounds-test.mp4 This resulted in a **failure**, when I inputted

	individual files bigger than five megabytes or a stack of them - it accepted it all. Overall this doesn't have drastically negative impacts on the application for an average stakeholder, however can be a massive security flaw to allow people to completely overload the model.
Different image file types	PNG-AND-JPG-FILE-TEST.mp4 This resulted in a **success**, both jpeg and png files are accepted, and in the future more file types can be supported by the model.

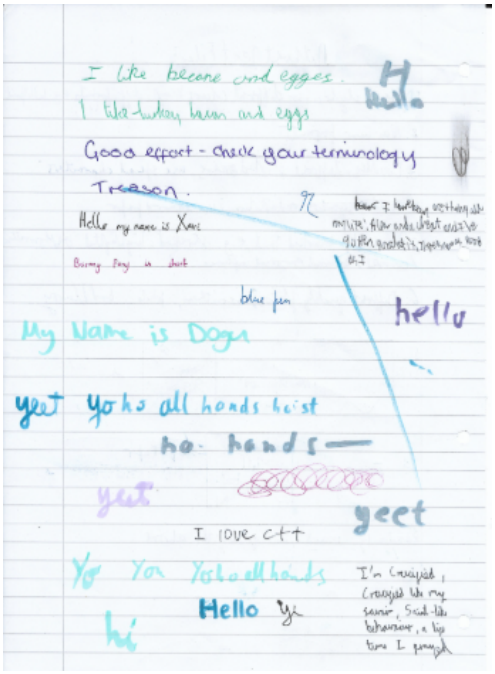
The final type of testing focuses on the actual functionality of the model and its capability to handle various types of valid inputs. To summarise a list of topics I wanted to test for, and how to maximise each criteria within the least amount of images; I created a rough draft shown below:



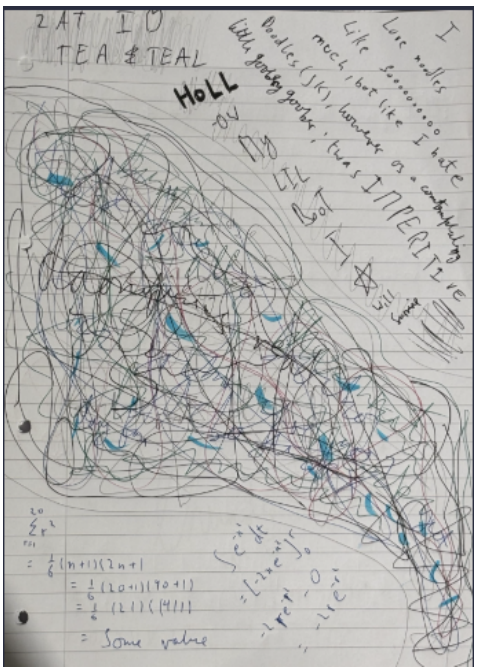
Finalised normal inputted data testing:



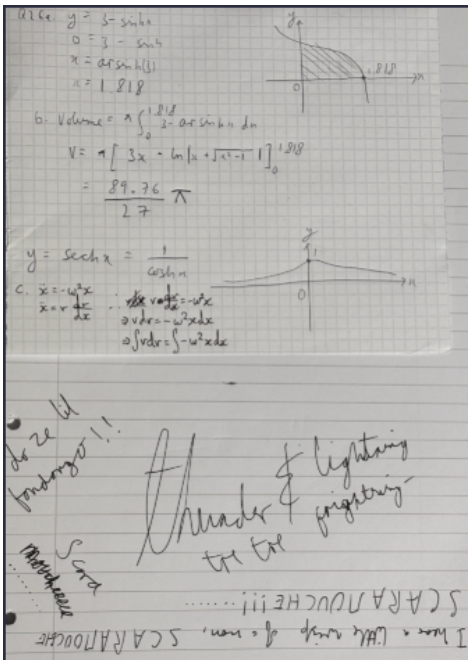
Bad lighting image (BLI), with a huge bunch of nonsense words, in various different colours, styles of handwriting, strokes, types of writing equipment, structured non linearly and with some random lines to see how the model would handle that. This also works as a bad lighting test.



Good lighting image (GLI), is exactly the same picture as the previous one, but scanned using a printers photoscanner, it has the complete optimal lighting and perfect clarity. It is useful to put the same image in the polar opposite spectrums of quality to ensure that a fair result and a fair comparison can be drawn at the end.

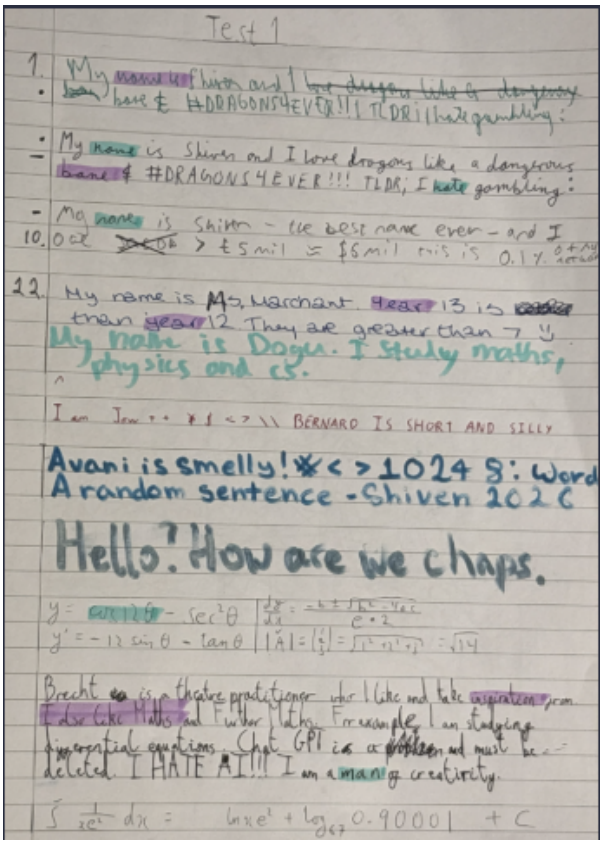


Doodle image (DI); next is an image that has both math and words scattered around it in a completely non linear format, it has slanted text, bolden text, and a huge doodle in the middle - that also contains some words inside of it - this is to test the models ability to ignore the doodle, and recognise not only normal maths but also slanted maths & slanted text, and even text that has a shaded background or text that flows in a column like manner.



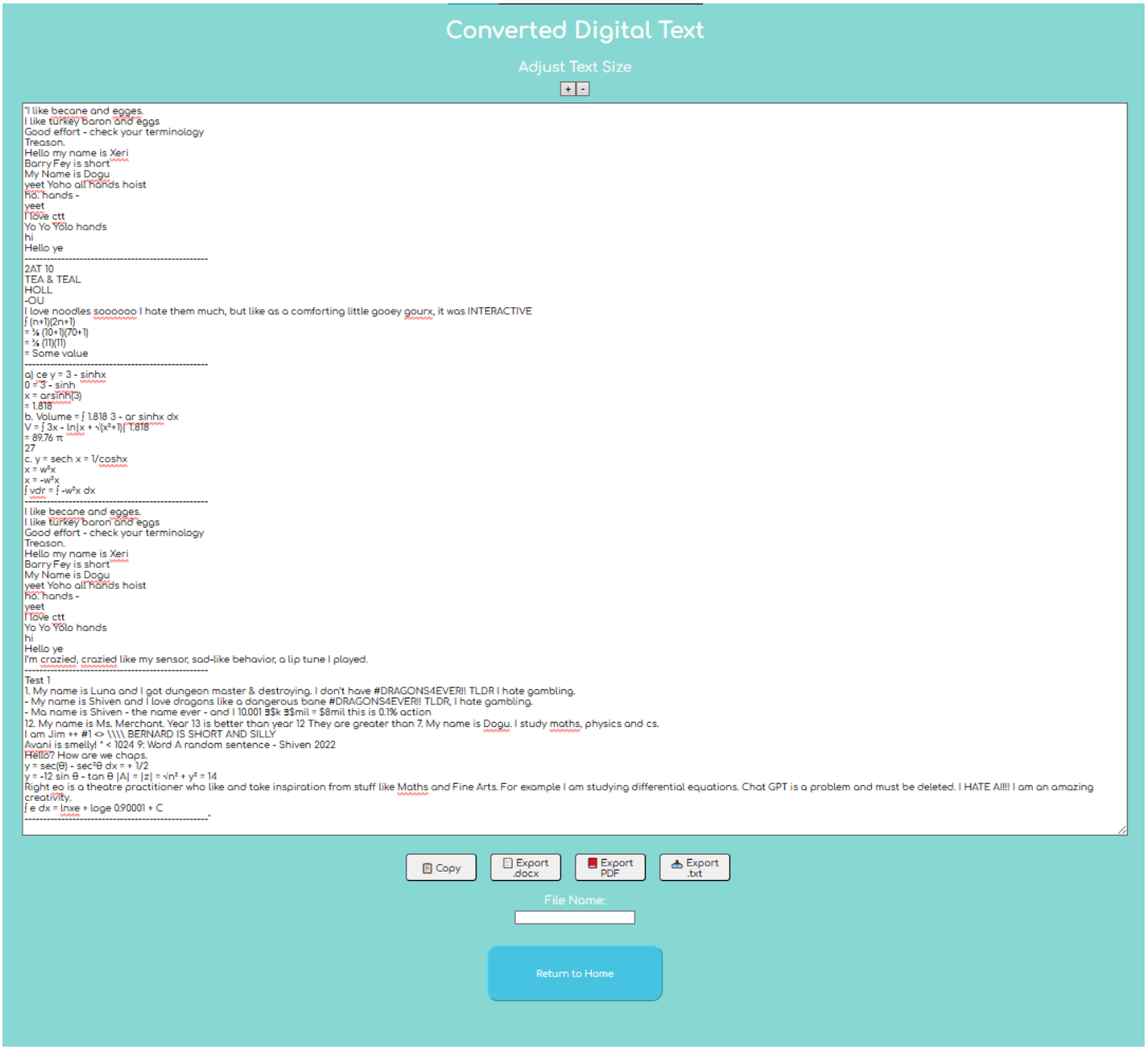
Nonlinear and math image (NMI); finally as a control, to ensure that the huge doodle does not obscure the results of the nonlinearly structured text and to test the models ability to recognise simple maths; I have this image which contains labelled graphs - which the model will hopefully ignore - with linearly structured maths with a square paper background (to simulate a typical image of what the model would expect a maths student to enter).

Moreover, it contains irregular text at the bottom, upside down text, different sized text and overall text that has no uniform format.



Style image (SI); the final test image actually tests the very core of the optical character recognition model, and its ability to recognise a wide variety of characters. Here every sentence is written by a different person, with a different handwriting and style, with different colours, different types of pens - ink pens, highlighters, pencils, ballpoint pens - with variations with the text size,. Additionally, it contains random dashes, bullet points, numbered points, crossed out words, a variety of special symbols, highlighted words, words overly corrected to mimic a student writing an essay. This is the complete test of the basic functionality of this model.

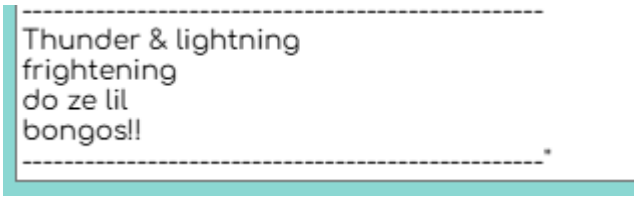
Result:



Overall the model is extremely successful at translating BLI when compared to the translation of GLI. So far in the BLI response it has completely ignored the right hand side, with the bad handwriting (the spider looking one being an example that I myself can not read accurately), while GLI sort of attempted to translate the bottom right text. Showing the advantages of having better lighting is not detrimental to an accurate result but obviously helps to provide a more accurate one. Overall GLI and BLI resulted in sufficiently translated outputs. In terms of the result of DI, the result was quite satisfactory in terms of most of the text translation regarding the top half of the image; it sufficiently ignored the doodle and could translate most of the uniformly slanted text but not the snaking column of text above the doodle, highlighting the limits of the model - which in truth is quite adequate. In terms of maths, it was completely hopeless at translating non linearly structured maths, however most notes from a maths classroom should not be structured as seen in the DI test - so in practice hopefully it should be better. Furthermore, it was quite extraordinary to see such an accurate response for NMI, the maths was reasonably correct, the figures and drawings

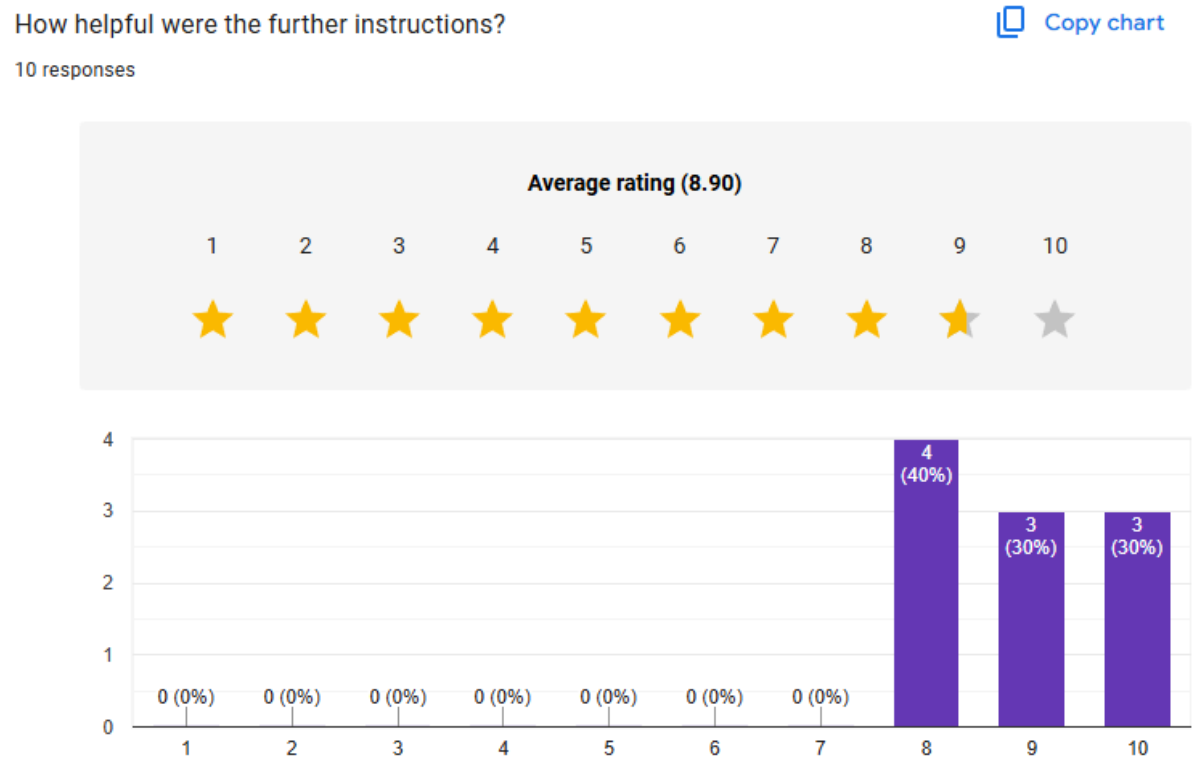
were ignored. However the text at the bottom was too chaotic for the model to reasonably translate. Finally for the SI test image, the model was sufficiently accurate at transcribing the handwritten notes in this test. However it did not attempt to ignore crossed out notes, this is something that should be included within the prompts so that only necessary content is transcribed. Furthermore the mathematical translation was accurate as it could be with the given image quality (of a basic phone) but it can not handle nonlinearly structured maths, such as when a double margin is used.

However some of this needs to be taken with a slight grain of salt, since a lot of the time there can be quite a bit of variation within the model. For example in a second test run, the model can actually translate some of the upside down text in NMI:



6.2 Stakeholder Response

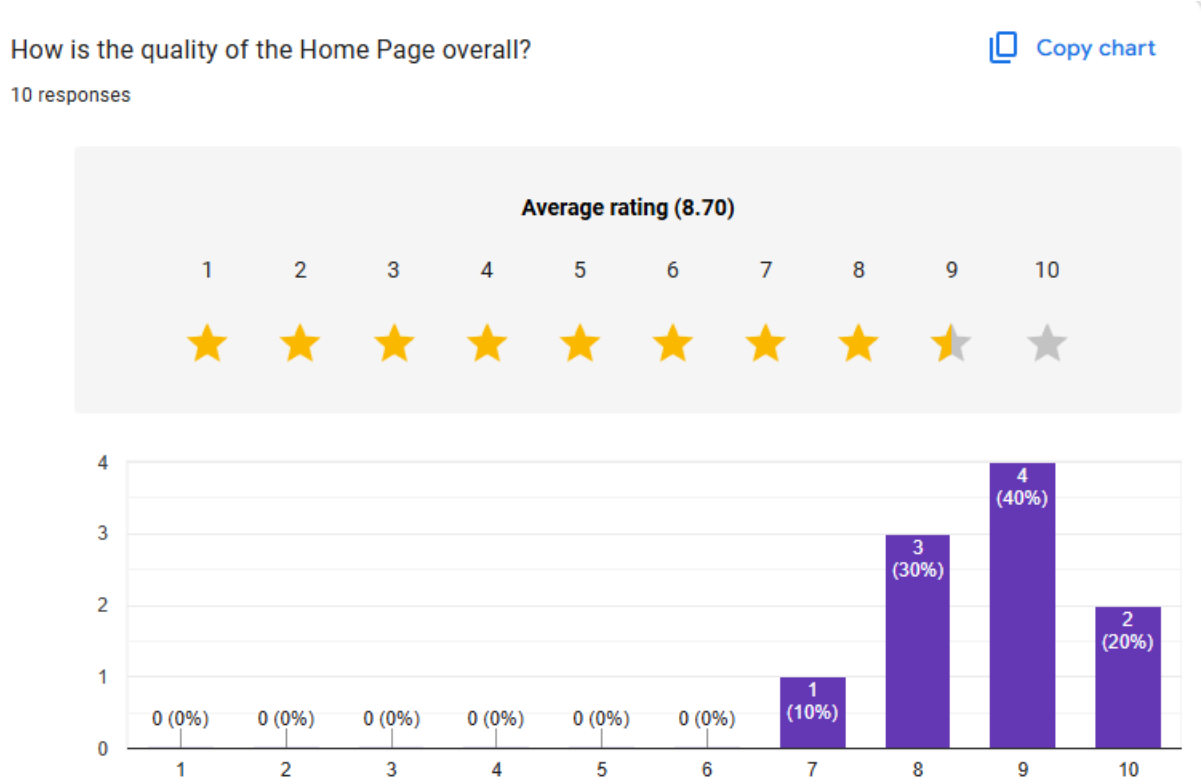
After completing my project, I created a google form for my stakeholders (those part of the Grey Court Sixth Form) to fill out after testing out my application. To get their opinions on the project:



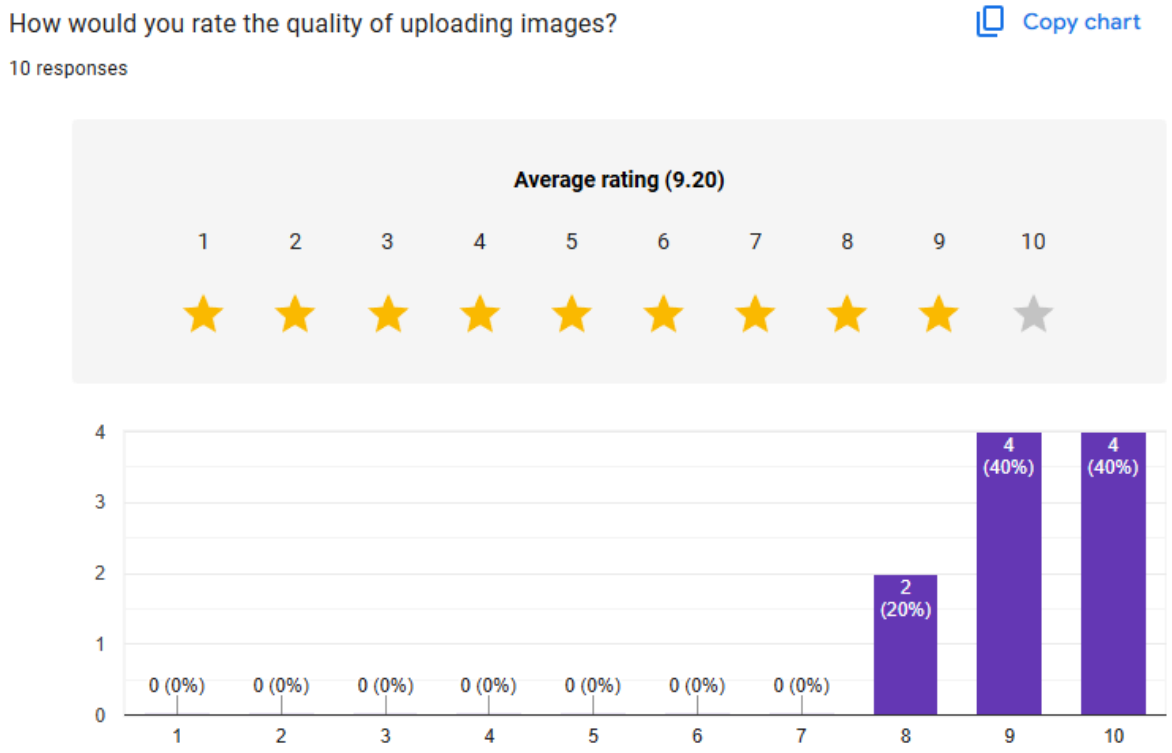
My first question asked for their opinions on how helpful the further instructions were, most were pleased with its description and level of detail. However some griped at the lack of a mention on what different types of file types the application would accept. And some were confused with the terminology used - to quote ‘what is a chromium browser’ - however after

being informed on what the terminology meant, its purpose became clear and they were satisfied with the further instructions.

When asked on how the further instructions could be improved, some commented on the fact that not many knew what a chromium browser was - however this does not hold major repercussions since this is mainly aimed towards Linux FireFox users, who would presumably know basic browser terminology (since this application is not compatible with FireFox). Else, some wanted the website to automatically shift the website to match the top of the further instructions when it was toggled on; others felt that it was completely unnecessary and wanted the toggle to be removed. This just goes to show how it is impossible to please all stakeholders, therefore a perfect balance is necessary.



The next question was linked to the responses many voiced in regards to their issues with the further instructions; overall many people were satisfied with the home page, however some were less satisfied due to the previous issues with the further instructions. However in regards to the actual design of the home page - and consequentially all other pages afterwards - stakeholders received the round buttons and visual interactiveness of the website amazingly, they loved how almost everything in the website reacted to their actions and loved the overall (to quote) ‘vibe’ of the website.



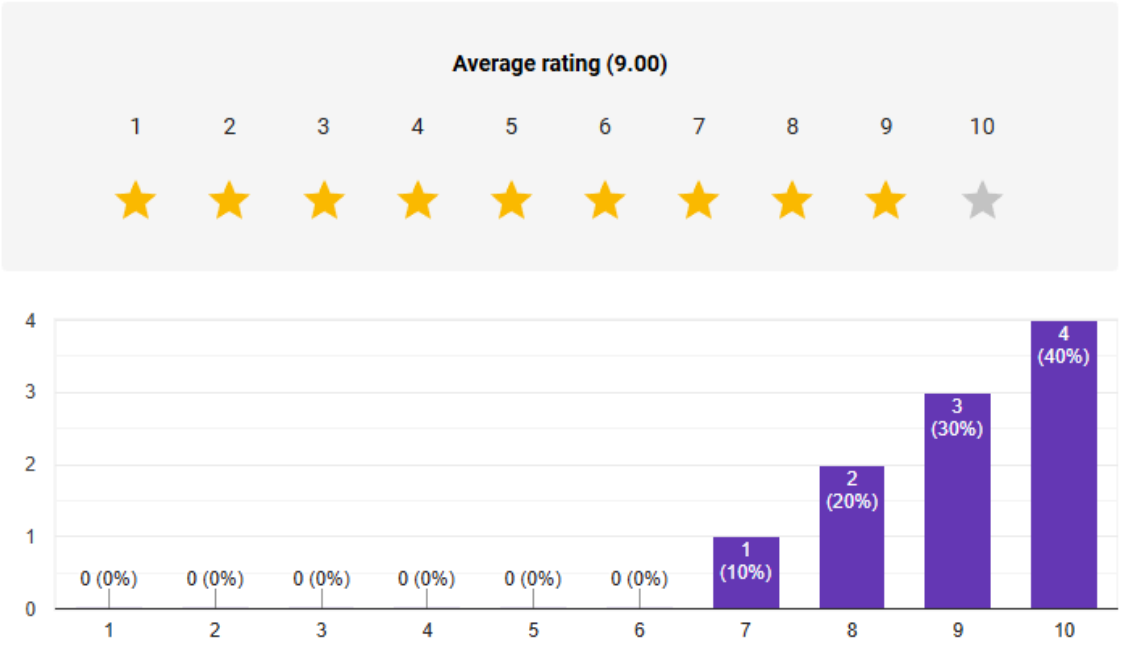
Then in regards to actually uploading images, people were extremely satisfied with the flexibility of being able to drag and drop, as well as all the file types that it could receive and with the overall robustness of the program.

Then when asked for any suggestions regarding the input page, some wanted the website to check an image beforehand and tell the user if it had no text. Else some wanted the further instructions tab to be in the input page itself so that the user did not need to flip between the two pages, otherwise another respondent suggested the ability to incrementally add images at a time - so instead of only being able to add images with a single drag and drop, have it so that you can drag and drop images one by one. This shows the vast array of ideas that can be implemented as future features, highlighting the benefits of an AGILE methodology, where constant stakeholder input is essential for an amazing product.

How does the transition from the input to the output page feel?

Copy chart

10 responses

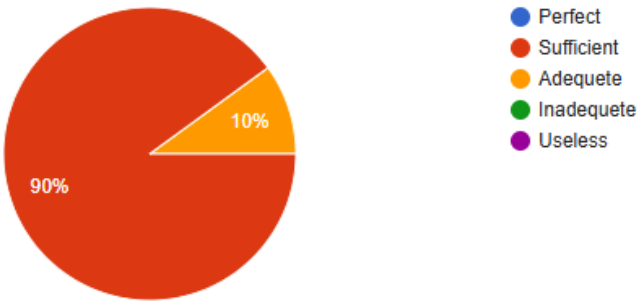


Many people were content with the transition from the input page to the output page, however some people disliked the fact that you needed to reload the page to see the final output; as well as the fact that sometimes the load times were especially slow (in regards to later testers when the model has been used up by quite a lot), however they recognised that it was a limitation in the project. Regardless this shows the impact of the limitation of needing to reload the page yourself, it causes the stakeholders to be unhappy with the result of the processing wait period.

Would you consider the transcribed text to be considerably accurate?

Copy chart

10 responses

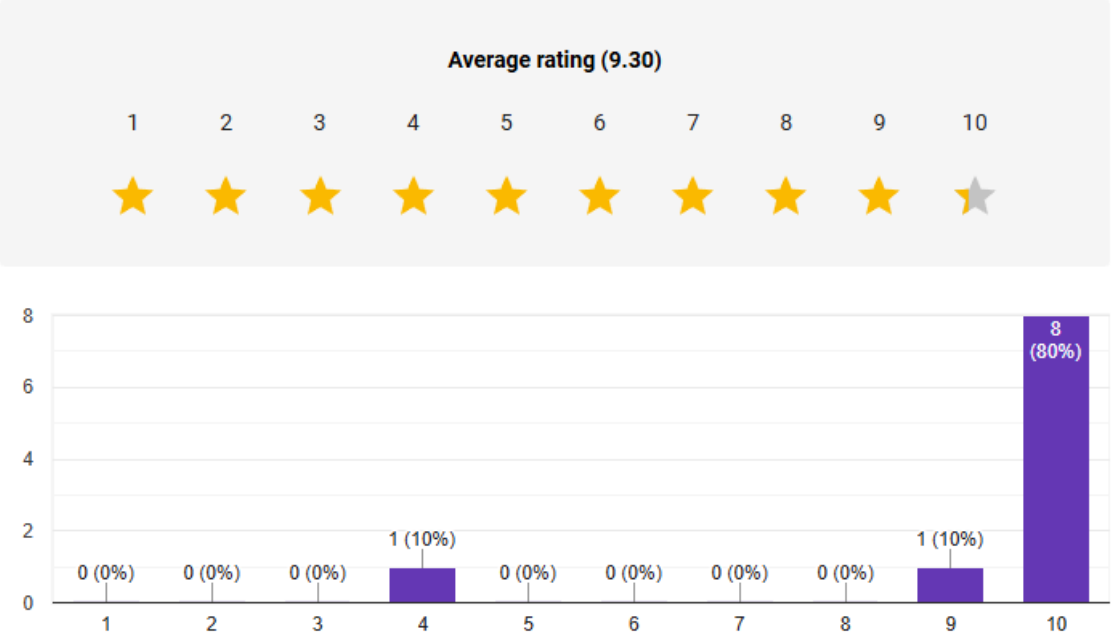


This question led to a very predictable answer, many thought that the model was sufficient at OCR with handwritten notes, which was the perfect response for an older Gemini model - showing the value of an older model, how it isn't completely obsolete. However this provides an amazing opportunity for future improvement, where upgrading to better models can greatly improve the result of the model to a near perfect response.

How would you rate the ability to export the transcribed text? (e.g. editable text field and multiple export options)

Copy chart

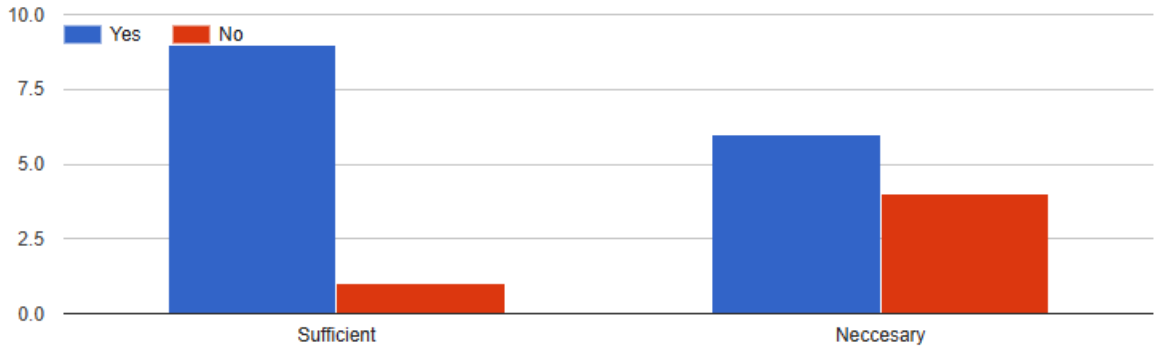
10 responses



Overall everybody loved the exploration features greatly, and it was the most fleshed out parts of the entire website. After questioning the one person who put a 4 in the survey, they said it was an accident. Which goes further to prove the incredible functionality of the exportation features.

Do you feel the addition of a zoom button for the text field text is neccesary or sufficient?

Copy chart

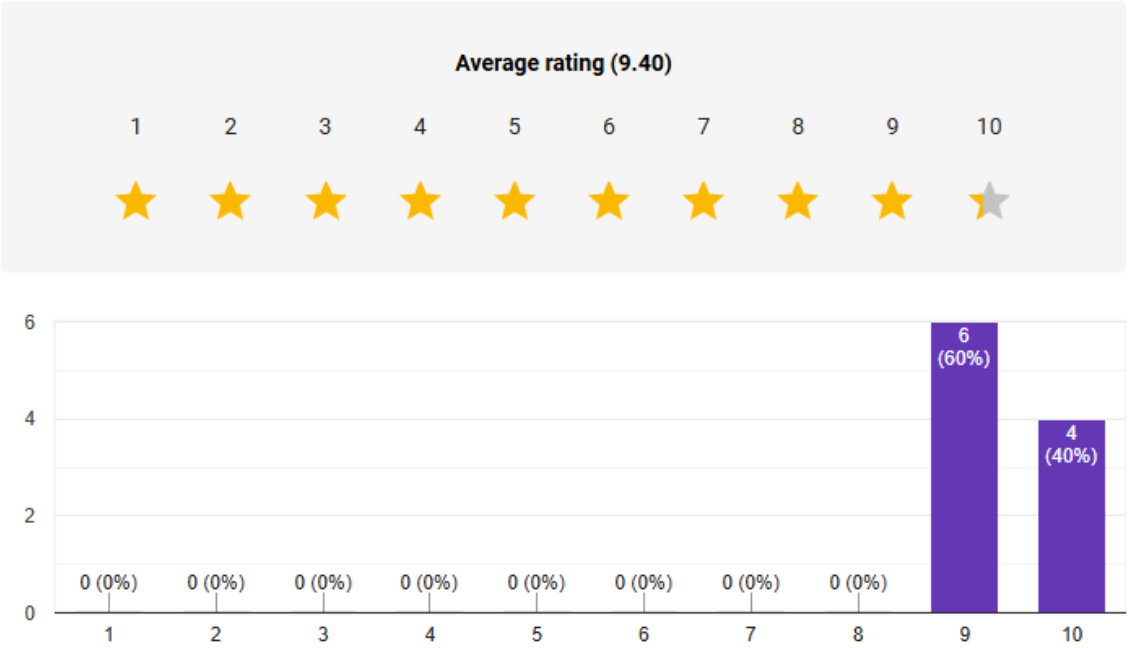


Another example of the success of the survey done at the end of cycle 2 in section 4.51, the vast majority of stakeholders thought that the zoom buttons were sufficient - even if less so thought it was not necessary, perhaps since they don't require a need for it to be necessary.

How user friendly and intuitive was this application?

Copy chart

10 responses

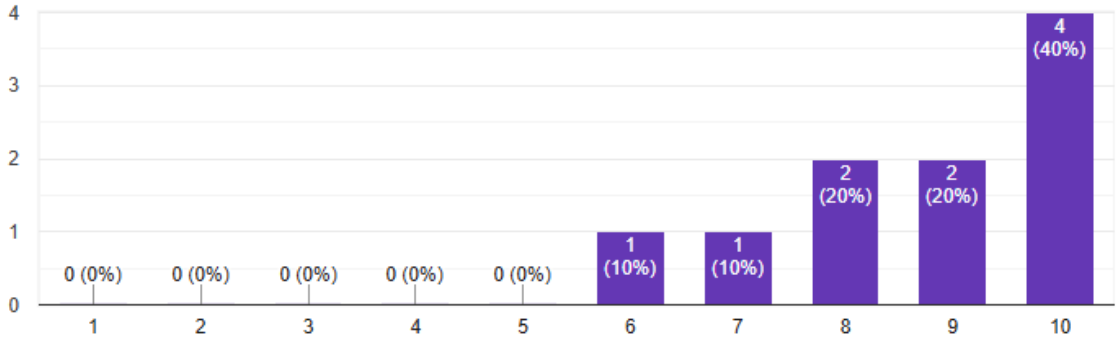


In terms of opinions on how friendly and intuitive the application was, everyone thought very highly of it. They loved how responsive everything was, it was very easy to follow with little to no confusion regarding everything and it accommodated almost anything for everyone. Additionally they loved the new colour scheme much more than the old one, many even commented on how nice the font looked.

On a scale of 10, would you recomend this to your friends/collegues?

Copy chart

10 responses



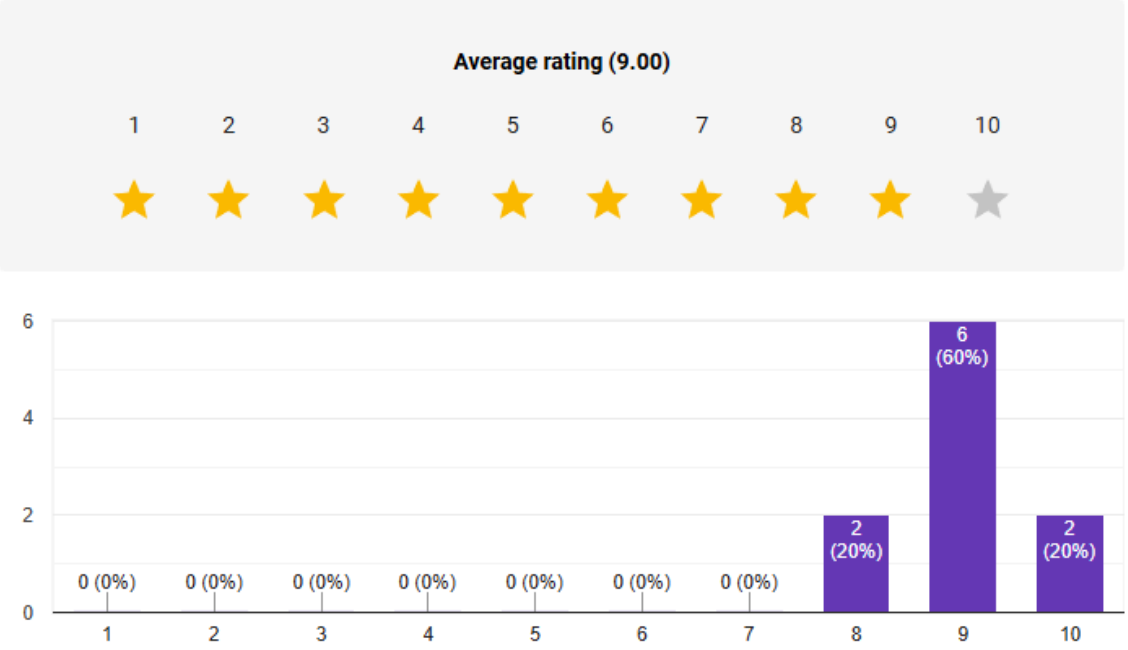
Then in terms of how much people would recommend it to their friends or colleagues; there was a wide variety of perspectives on how to think about this question. Some would reason that within the context that they were given, they would definitely recommend my application specifically if anyone they knew faced a similar situation. One girl even commented on how the previous night she needed to convert her handwritten history notes into digital text for her A level revision. Others - such as those more involved in mathematical subjects with not a lot of essay writing - commented that they themselves were unlikely to use a handwritten note

to text converter themselves, so they wouldn't normally recommend an application they wouldn't use themselves.

How would you rate the overall features of this application?

Copy chart

10 responses



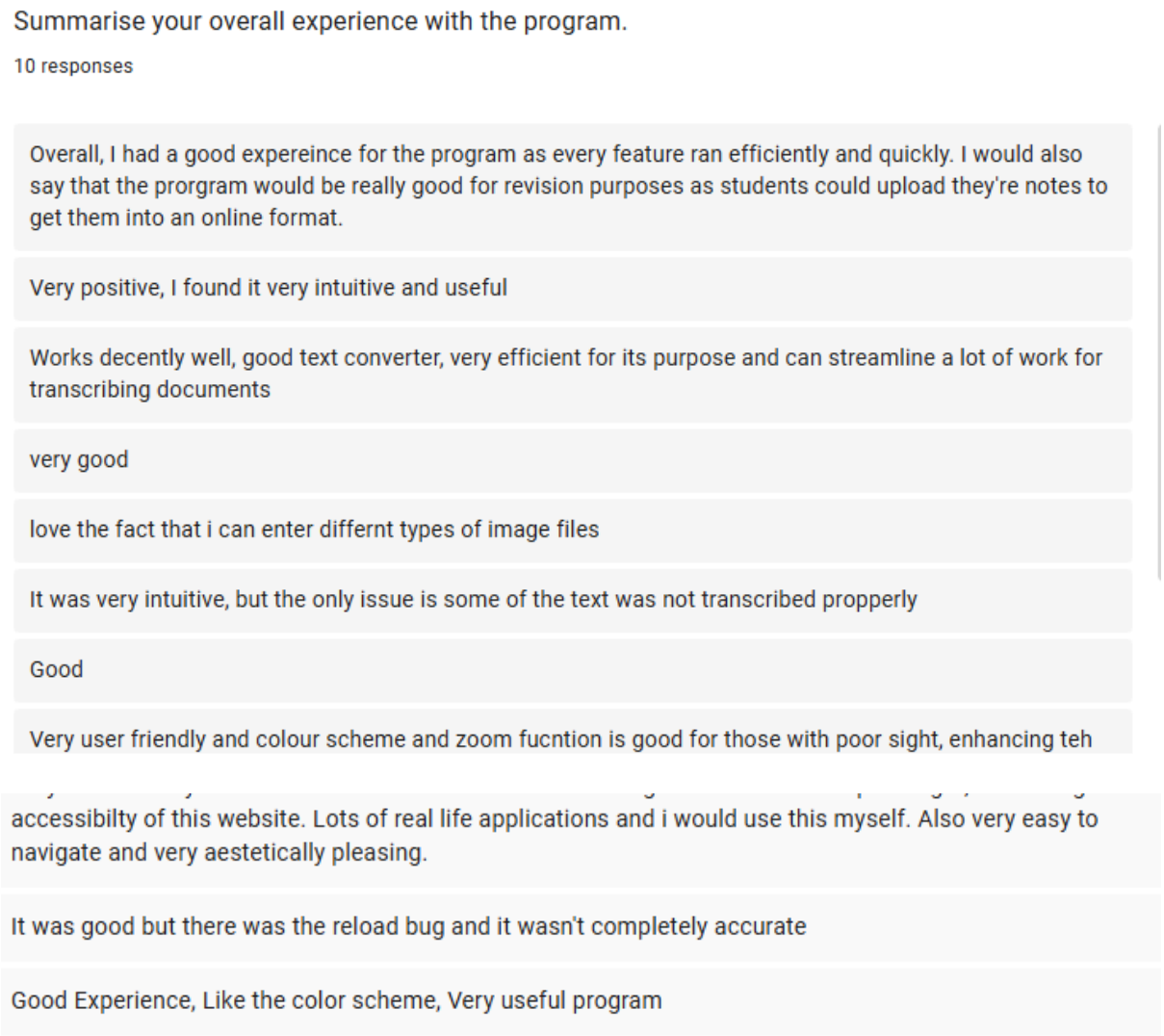
Furthermore, overall many people rated the features of this application to be very high, they were mostly satisfied with the end result of my project and it's capabilities, with many of the people that didn't score it as a ten were mainly dissuaded due to the current limitations of my solution - such as the manual reload.

Are there any improvements you would reccommend

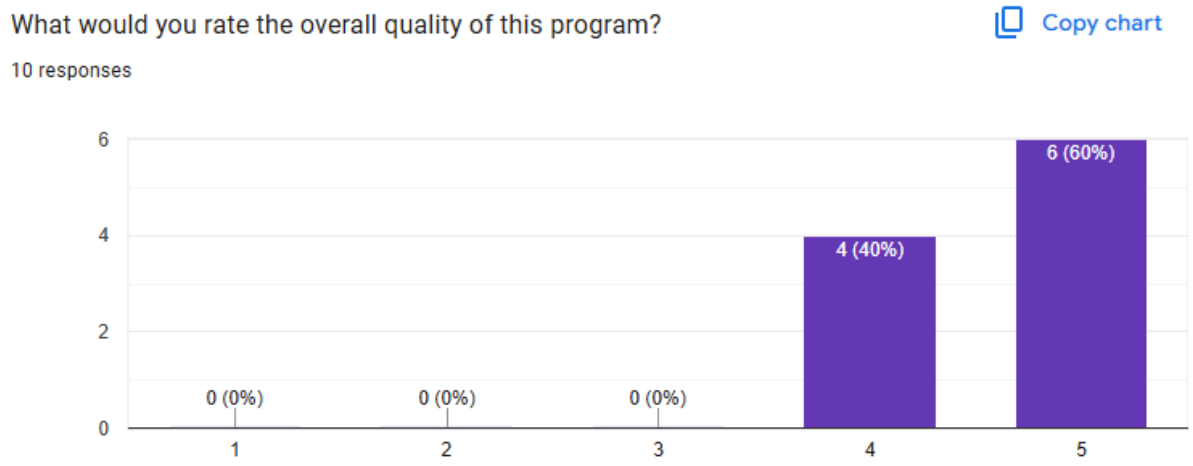
10 responses

- N/A
- In the future, you could try also implment picture recognition. Eg. for flowcharts so the original pictures formatt is preserved.
- Communicate the need to reload the page.
- allow downloading files on non chromium based browsers
- improve speed of processing images
- If given more time i think increased accuracy and printing it in the colour of the writing and demonstrating where it is highlighted in teh print
- Fix the bug

Though they had quite a few opinions on further improvements that they would love to see, giving me a multitude of new ideas I could implement within my future development.



Afterwards, all responses were mainly positive. Though some had a few gripes with the accuracy of the model and had problems with the reload bug, everyone mainly thought highly of the application.



And overall rated the quality of the program extremely highly.

In conclusion, stakeholders were impressed with the user interface and the majority of the frontend. They thought it was a useful and practical application that is quite ‘efficient for its purpose’. However, there were a few gripes on the reload bug that remained from my third cycle, along with a huge array of future improvements that everyone wanted to see accomplished. Therefore there were quite a few conflicting opinions from the form, hence since I can not fulfill the desire of the entire population of my stakeholders - I feel that I have done a more than sufficient job in creating a project that caters to the majority of my stakeholder’s needs.

6.3 Success Criteria Evaluation

In reference to the success criteria I outlined in section 1.4.6, I have evaluated the result of my attempts at adhering to my success criteria.

Front end - Website

Criteria	ID
Students and teachers need a clean intuitive user interface for ease of use, so they can start using the application straight away; such as through drag and drop/attach feature for images from their photo library or folders so that the UI is especially more convenient for phone users, which most students would fall under	1.4.6.1
To allow for flexibility, the users should be able to export their file into multiple different file types or be copied straight into their clipboard; as,students might want to paste multiple school notes into a single compiled digital document, while teachers might want a word file they can share with other teachers, overall by providing flexibility of exportation a good user experience is achieved, by accommodating to the various file types of that different users using this application might require	1.4.6.2
Lets the user edit the digital output before exportation; to correct any simple	1.4.6.3

mistakes made, or to create a finalised edit of the text before exportation, this simplifies the exportation process so that users don't need to export and then edit their text and then use it, when they can just do it in the text editor and then export the text	
Scalable onto mobile phones, should make sure the layout isn't distorted; as students would be more comfortable using a mobile phone, as that is most likely where their pictures are stored	1.4.6.4
Highlight uncertain characters or words for user review; helps students or teachers as they wouldn't want to waste much time proofreading, when instead the program can show them where there would be the highest error rates	1.4.6.5
Real-time (or near real time, roughly at least >=80% accurate) loading bars; as students prefer accurate representations of time due to normally lower attention spans	1.4.6.6
Display error messages if there was any issues, e.g. with uploading a file - file format doesn't work; helpful for all users to determine if this was a client or server error, so they don't waste time trying to get it to work when it was a server error, or so that they don't exit the program due to impatience which is common with the younger student demographic	1.4.6.7

Back end - Convolutional Neural Network

Criteria	ID
It auto spell checks the digital text before outputting it to roughly a >=80% accuracy; as teachers would want an emphasis on accuracy, especially if they are using the program for tests	1.4.6.8
It displays extracted digital text clearly, preserving formatting where possible, by targeting only the sections with text and digitising them so that at least if the image is not an essay esque document, there is still some form of formatting preserved so then it's easier for the user to adjust their text	1.4.6.9
The CNN trained on diverse handwritten datasets, including my own sample sets to reduce bias in the model, this will improve accuracy of the results leading to better user satisfaction	1.4.6.10
The CNN model is robust when handling variations in handwriting styles, extraneous image noise, and inconsistent lighting through good training data sets; this will make the model more flexible, which will greatly increase the type of image inputs the model can process. This will elevate the user experience as users won't need to retake pictures due to bad quality, and will generally get better results due to this increased flexibility of handwriting style recognition	1.4.6.11
It is efficient, with a low latency for faster loading times which leads to less of a wait time for users; this is extremely beneficial for younger users who have lower attention spans, and for people who want to process a larger amounts of images	1.4.6.12

It is integrated effectively with the frontend, with clear endpoints for file upload, processing status and file retrieval so that the frontend and backend are completely in sync, to reduce the possibility of any bugs or glitches going unnoticed during testing (which then can frustrate the user causing them to lose faith in the program)	1.4.6.13

ID	Evaluation
1.4.6.1	The user interface has become a clean, intuitive and a responsive interface that has been backed by the opinions of stakeholders from the end of cycle 2's evaluation surbey in section 4.5.1 as well as the final stakeholder response in section 6.2. The interactability of the website, as well as the new brighter colour scheme is well received by the targeted stakeholders. Furthermore, this is further reinforced by the well received feature of the drag and drop functionality that many are extremely content with - this is proven within the survey performed in section 6.2. Overall this criteria has been fully met.
1.4.6.2	Stakeholders can not only copy the contents of the editable text field to their clipboard but also export it to the current most popular form of digital media: text files, word files and PDFs. As seen beforehand in the stakeholder survey in section 6.2, users are extremely content with the exportation section of the application. Therefore this criteria has been fully met.
1.4.6.3	Stakeholders can fully edit the contents of the editable text field, all of their edits are saved and properly handled when exported. As aforementioned, in section 6.2 the stakeholders are extremely happy with the results of the exporting side of the application. It gives them freedom and convenience in the form of human error correction. Furthermore the capability of users being able to resize the editable text field to any dimension of their choosing is a phenomenal feature - inbuilt to the textarea tag - that continues to bolster the flexibility and convenience stakeholders can face when editing or simply perusing the returned response of their images. This criteria has been fully met.
1.4.6.4	This criteria has not been tested at all, since I have not been able to deploy this website on a mobile device. Though I have HTML code that should ensure that this criteria is met, currently it is uncertain whether this criteria has been accomplished due to the lack of evidence. Therefore, it is worth it to look over the mobile implementation of the website within the future development plans for the website.
1.4.6.5	Criteria has not been met, there has not been enough time within the development cycles to fully implement this criteria. Therefore this is a prime example of a feature that could be added within the future development section, by testing various modified versions of the prompt sent to the Gemini mode.
1.4.6.6	Criteria is partially met, since I reasoned beforehand that it would be

	extremely extraneous to create an accurate loading bar. Therefore a fake loading bar would be the only solution - in terms of loading bars - therefore I thought that a fake loading bar that would freeze at 99 percent (if a request took too long) would be far more frustrating than a wait message (which only came after the thought of an eternal throbber that would throb till a response was received, but was foiled due to the adapted pipeline of my application). This could be further touched upon in the further development section, by figuring out a way to accurately measure the Gemini API call time. This can be seen in any of the testing videos in section 6.1.
1.4.6.7	Criteria is partially met, as seen in section 6.1 when testing with various sorts of image test data, if the data is invalid and error is thrown at the user. However the only error message not currently working, is the out of bounds (overexceeding in size) error for when the file size is too large. Given more time this logic bug could easily be rooted out within the future developments section, but currently there was no time left to even explore what was causing the bug.
1.4.6.8	Criteria partially met by instead of autocorrecting a translated text, the editable text fields's spell check feature is enabled. Therefore if the user wants, they can correct any misspellings in their work; since if a teacher was marking a student's work and SPaG was important, it would make sense to include all of the spelling mistakes - therefore in essence it satisfies both primary stakeholders, making this criteria basically fulfilled; however within future developments this could become a checkbox for users to toggle based on tier needs. This can be seen in any of the testing videos in section 6.1.
1.4.6.9	Criteria partially met, it mostly ignores any diagrams or figures as seen in section 6.1, in the test case of the doodled image and the one with the mathematical figure. Furthermore as seen with the style test, it also preserves format by detecting: bullet points, dashes and numbered points; however it struggles mainly with double margins. A common staple for many mathematical students, therefore in the future development section it might be worth it to include the ideas of expanding to better models with prompts that make the model acknowledge double margins.
1.4.6.10	Criteria is partially met, since I have moved on from a locally run large language model to run my optical character recognition for my handwritten, to Gemini, I have not had the opportunity to personally train the model myself. However since this is a model trained by Google, with billions of dollars invested on this process, I trust the vast resources that Google has expended for this model - compared to what I could muster.
1.4.6.11	Criteria is fully met, as seen in section 6.1 in the style test, the model is superb at performing optical character recognition on a multitude of handwriting types, colours, stroke types, written media types, etc. (already mentioned in section 6.1 within the style test). Though it does have some flaws due to the age of this model, and how it is currently a free tier model, overall it's performance is

	sufficient so far.
1.4.6.12	Criteria is fully met, compared to the latency experienced from the Chandra model, the Gemini model 3.27b is exceptionally fast comparatively. Moreover, on its own it is also a force to be reckoned with, immediate responses to images with short phrases, to a minute or two latency time for multiple highly dense images packed with sentences (as seen in the tests done on the normal data in section 6.2).
1.4.6.13	Criteria is fully met, there are clear endpoints with perfect communication between the frontend and backend - which leads to perfect redirection and fetched responses. As seen in the tests performed in section 6.1.

6.4 Final Product Description

Overall my project has truly become a base model for an application that can truly benefit my target stakeholders to transcribe their handwritten notes into digital text. I have implemented a wide array of usability features from my design section and features that I have simply implemented due to circumstances that have popped up during my development section - by following an AGILE project flow I have been able to effectively tackle all of these unforeseen circumstances and I have consequently iterated over my program to adapt to these changes. For example, some changes that I have had to make during the course of development includes: needing to change the entire frontend persona, including its colour scheme and font (due to negative reception from the stakeholder survey I took in section 4.5.1, which after its improvement was received much well by the stakeholders in section 6.2 - making it a fully effective usability feature after its improvement) ; adding zoom in and out buttons for the editable text field for less tech literate stakeholders (which was appreciated quite a lot by the stakeholders, as seen in section 6.2, many thought of it as a sufficient usability feature while a few even declared it to be necessary for the entire application - allowing it to be a fully effective usability feature). Both of which were the consequence of an unplanned but useful stakeholder survey at the end of cycle 2, to ensure that the project was veering towards a path that was suitable for stakeholders.

Furthermore, I have had to constantly adapt the backend of this application to ensure that it was realistic to execute and test the application; such as going through the process of cycling through various different methods for optical character recognition for handwritten notes, from ChandraOCR (which I could not realistically execute, test or even integrate within my project), EasyOCR (which produced lackluster optical character recognition capabilities that I did not deem fit for my target stakeholders) and finally to an API call to Gemini (which was a free, accurate and effective solution that translated handwritten notes to a sufficient level, an opinion shared by my stakeholders in section 6.2, making it an effective adaptation that enabled a fully realised OCR translation for stakeholders) - which made the need for my image preprocessing file to be obsolete. However this change proved to be fruitful for the overall development of my project, providing a free but effective solution.

Otherwise, aspects that I stuck with from my design phase, included the rounded designs I previously envisioned - though I improved my original templates for my different project

pages; their effectiveness have always been unparalleled due to the good reception of the basic frontend design, commented upon by users in both stakeholder surveys of 6.2 and 4.5.1. Furthermore I also developed a sense for including a lot of visual interactiveness and responsiveness within the frontend to appeal to student stakeholders; a feature many appreciated, again in both aforementioned surveys.

However, this project did lead to a few minor incomplete features - discussed in further limitations. Which did not massively impact the outcome of my solution but led to a few complaints regarding the output page.

In conclusion, my project is a sufficient solution for the problem of needing to digitise handwritten notes. It uses a model that has billions of dollars invested in it, with an amazing page dedicated to exporting the users digitised text - an area my application is unparalleled in, in comparison to my previously analysed competitors (Adobe Scan and Pen to Print).

6.5 Maintenance

My codebase is extremely modular, with a multitude of classes and subroutines within the frontend and backend code. Furthermore, for every function in the backend, I have attached individual python docstrings to further improve the maintainability of my codebase. In the long term the only issues with maintainability (besides fixing preexisting bugs or adding new features) is ensuring that the current model of Gemini has not deprecated and that the set API key is functioning properly and has not reached its max limit. Else, in terms of implementing potential improvements, changes or current limitations; due to the modular nature of my entire project, it would be extremely easy to continue to develop this application. However a key limitation with future maintenance is that any external developers would need to use their own personal Gemini API key, by signing into their google AI devspace account. Therefore if the developer is not able to get an API key or even have access to a device that can not import the latest version of the fastapi, pillow, google’s gemini model, pathlib and uvicorn libraries, they won't be able to work on this project. This also extends to any inability in being able to download and run the Live Server extension within VS Code, since it is a crucial library needed for testing my website live.

6.6 Future Developments

If I were to continue this project in the future, I would firstly fix the previous limitations that were present within my solution. For example, it currently requires the user to manually reload the page before the proper Gemini response can be seen in the editable text field of the output page, furthermore its validation test for ensuring the file size of the uploaded file does not exceed five megabytes, is broken. Therefore I would prioritise fixing those first.

Then I would continue to complete the leftovers from my success criteria from 1.4.6. I would add a feature where uncertain words could be communicated to the user - such as by being highlighted - this would help the user to identify any sections of the transcribed text where they can divert their attention to make any manual corrections required.

Furthermore I would figure out a way to test this application within a mobile device, to ensure that all of its scaling and functionality is properly preserved. This would also be the best move to satisfy my primary stakeholder - students - as a phone would be their primary form of using this application, due to its convenience and their normal habits of simply relying on their phone a lot.

Additionally, I would attempt to figure out a way to implement a real time progress bar. Such as by timing the api calls to the Gemini model, relative to the size of each file and the number sent. I could create an algorithm that would then predict the amount of time the model would take and create a loading bar in the output page, which would then phase out when it reaches 100 percent. However, not only would I need to ensure that the progress bar is accurate - to justify the trouble of making it - but I also need to ensure that it will not constantly freeze when it hits 99 percent and the api call still hasn't been received by the backend.

Then, another feature that would be beneficial is to add a checkbox in the input page, so that users can toggle whether they want their processed text to be post processed with grammar, spelling and punctuation correction. Since many teachers might want to retain the inaccuracies in the text to mark for SPaG (if transcribing student test papers), while in other use cases it would save stakeholders a lot of effort and time by removing a lot of manual error correction.

Next, in accordance with the post development stakeholder survey I took in section 6.2, I would love to implement more features that are directly suggested by the target stakeholders. Such as: fixing the codebase’s incompatibility with non-chromium browsers, to allow for FireFox and other non-chromium browser users to effectively interact with my application; evolve upon the simple string response from the model to a response that also detects other formatting within the image, such as flowcharts, tables, etc. and therefore could return markdown text with proper formatting - given a newer model and prompt are also provided; furthermore the previous point can be further expanded upon by improving the capabilities of the editable text field and the model to be able to display highlighted, striked out and even coloured text in accordance to the inputted images.

Next, I would like to properly host this website in hopes of being able to get it a proper SSL certificate so that it can utilise HTTPS. This would not only make the website more secure but also make it more trustworthy for users.

Moreover, the final future development I would like to implement would be to test the model more deeply. Figure out the ideal number of files the model could be sent (and the maximum file size it could handle) for the perfect limit for the users. Furthermore I would love to find better models than the current 3.27b, which could potentially outperform it while not being too costly. This would increase the overall accuracy of the program, and allow for even more complicated prompts to be sent to the newer models, increasing the effectiveness of my solution.

6.7 Limitations

Firstly I am a singular person in my final year of A-levels, with A-Level computer science programming knowledge and Year 13 duties, Head Prefect duties along with a further EPQ subject. Therefore I do not have an ample amount of time I am able to dedicate to this project, this makes it imperative that I prioritise the features I need to implement.

Computationally, there are many limitations that still plague my program. For example, it can not prevent users from inputting images that are larger than five megabytes (the mandated amount). Furthermore there is no autoreload button within the output page, therefore once you are waiting with the message: ‘Please wait…’, you are stuck there until you reload the page to get your new response.

Additionally, the use of the free tier with Gemini causes the program to reduce its accuracy significantly as you reach the maximum API requests for your API key. This would prove to be problematic if many users were to use this program to transcribe a vast amount of documents. It would cause a huge backlog that would burn through the requests per minute I have access to within a given day.

Finally, due to my limited experience, the frontend of my website will never be up to the same standards as rival competitors that the vast majority of my stakeholders would encounter. I am not a professional web designer, so my CSS skills will be clearly lacking.